

Архив статей wasm.ru. Том 11: Assembler.Ru и прочие статьи

Собрание русскоязычных статей сайта wasm.ru о программировании, ассемблере, реверс-инжиниринге и компьютерной безопасности. Том 11 из 11. Разделы: 24. Assembler.Ru; 25. Разное; 26. Другие статьи. Все приложенные файлы находятся в wasm_files.zip.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

24. Assembler.Ru

Психологический тест «Настоящий ли вы ассемблерщик?»

Автор: Svet(R)off

Дата: 2002-06-14

На оригинальном сайте материал отмечен как незавершённый.

MS Developer Studio - среда разработки для ассемблерных приложений

Автор: Svet(R)off

Дата: 2002-06-14

Настоящие ассемблерщики - народ непритязательный, хорошая находка для рачительной хозяйки. Едят, как правило, мало, и не особо разбираясь, что (одного моего друга жена, большая любительница животных, накормила как-то ради эксперимента педигрипалом. Очень хвалил). Место в помещении почти не занимают: не более двух кв.м вместе с компьютером. Линяют не чаще раза в год. В спячку, правда, не впадают, зато гон непродолжительный и тихий. В совокупности представляют собой прекрасный вторичный рынок для компьютеров 386 серии. Основой операционной среды предпочитают видеть кастрированный по самое ядро Norton Commander 5.0. Трансляцию запускают из командной строки (самые волевые с трудом заставляют себя написать batch-файл). Отладчик при разработке собственных приложений вообще не используют. Поэтому Microsoft Developer Studio для них - как стадион "Уэмбли" для единственной выжившей после зимовки коровы симментальской породы из колхоза "Привет коммунизму": гораздо больше, чем нужно. И на такие его (MS DevStudio, а не "Уэмбли") мелкие недостатки, как отсутствие поддержки ассемблера, пенять не приходится.

Впрочем, к делу. Первый шаг - это **подготовка MS DevStudio** для работы с MASM:

1. Инсталлируйте, если не сделали этого до сих пор, MS Developer Studio в варианте Visual C++.
2. Найдите где-нибудь ;) MASM 6.11c или более поздний. Чистый 6.11 (без "c") лучше не использовать: есть у него кое-какие проблемы. (В частности, выравнивает структуры максимум на 4, а в win32 часто требуется выравнивать на 8.) Сейчас щедрый американский дядюшка позволяет пользоваться MASM бесплатно, а совсем недавно приходилось покупать его за хорошие зеленые деньги. Есть он, например, в составе MSDN. Или его можно скачать из Сети (только не с ftp.microsoft.com). По минимуму из всего пакета потребуются один лишь модуль *ml.exe* размером около 400 Кбайт.
3. Поместите *ml.exe* в папку исполняемых модулей пакета MS DevStudio (по умолчанию - *C:\Program Files\DevStudio\VC\BIN*).

Ваш пакет MS Developer Studio готов к разработке приложений win32 на MASM.

Любознательный читатель спросит: почему бы не использовать для разработки приложений inline-ассемблер, являющийся, как известно, неотъемлемым атрибутом C++. Вообще-то ничего невозможного на свете нет. Можно и отверткой в ухе ковыряться, только не очень глубоко. Но надо помнить, что inline-ассемблер предназначен для улучшения языка C++, а не для разработки самостоятельных приложений. Поэтому из него исключены большинство директив и, самое обидное, все макросредства, сильно помогающие облегчить процесс программирования.

Следующий шаг - это подготовка включаемых файлов:

1. Получите файл *includes.zip* (находится в архиве *wasm_files.zip*: *files/recovered/1005002/includes.zip*)
2. Распакуйте его и поместите файлы *@struct.inc* и *windows.inc* в папку включаемых файлов пакета MS DevStudio (по умолчанию - *C:\Program Files\DevStudio\VC\INCLUDE*).

Файл *@struct.inc* - это авторская отсебятина, которой в принципе можно и не пользоваться, а можно и наоборот, добавить в него что-нибудь свое, полезное.

С файлом *windows.inc* все гораздо сложнее. Это файл заголовков API, ассемблерный аналог той тучи h-файлов, которые содержатся в пакете Visual C++ и начинаются с файла *windows.h* (и, возможно, не только). Поскольку Microsoft официально не предоставляет файл *windows.inc* (в отличие от *windows.h*), у ассемблерного программиста в связи с этим возникает большая-пребольшая проблема. Ее можно попытаться решить несколькими путями:

- транслировать *windows.inc* из *windows.h* с помощью специальной программы-переводчика H2INC, поставляемой в составе пакета MASM 6.11+. Сразу скажем, что следует смело отправить эту надежду в мусоропровод. Продравшись сквозь частокол багов H2INC, вы обнаружите, что ее интеллекта хватает лишь на трансляцию тривиальнейших заголовочных файлов, к которым *windows.h*, безусловно, не относится. Вы утонете в бездне сообщений о некорректном переводе, и ваш сизифов труд по их обходу будет вознагражден получением совершенно негодного результата
- найти *windows.inc* (вариант - *win.inc*) в Интернете. Это гораздо более реально. Существует некоторое количество чудаков (возможно, это дети миллионеров, которым наплевать на необходимость зарабатывать на жизнь), которые тратят свое время на формирование и актуализацию этого файла и раздают его всем желающим. Есть мнение, что пользоваться такими файлами настоятельно не рекомендуется. Представьте себе ситуацию к концу второго месяца возни этого доброго человека с каким-нибудь пятым по уровню вложенности заголовочным файлом в пирамиде *windows.h*. Возьмет, да и перепутает с устатку порядок описания членов какой-нибудь структуры. Или забудет указать ее выравнивание относительно умалчиваемого. Хорошо еще, если соответствующая функция API при использовании ее вами будет вываливаться с фатальной ошибкой вроде нарушения общей защиты. А может ведь и не вываливаться. И будете вы разбираться, почему глючит ваше приложение, дольше, чем добрый дядя переводил заголовочный файл
- описывать структуры, объявлять прототипы и делать все прочие действия, которые делает *windows.h*, внутри собственного проекта. Это еще более реальный вариант. Для этого вам потребуется оперативный доступ к содержимому *windows.h* из своего проекта (см. ниже). Каждый раз, как у вас возникнет необходимость обратиться к какой-нибудь функции API, вы должны будете найти в *windows.h* все, что касается ее вызова, и включить это в свой проект, предварительно переведя вручную на язык ассемблера. При наличии небольшого навыка такой перевод выполняется довольно быстро и проблем не вызывает
- делать все то же самое, но не в рамках текущего проекта, а в едином файле *windows.inc*, чтобы не повторять эти действия в последующих проектах. Со временем необходимость править *windows.inc* будет возникать у вас все реже и реже

Именно последний вариант и предлагается в качестве основного. Файл *windows.inc*, который мы вам предложили выше, взят из состава приложения [MyCall](#). В нем только самая малая часть из того, что должно быть, и есть кое-что лишнее (RAS, например, в Visual C++ имеет собственный заголовочный файл). Но для старта этого достаточно.

Не забудьте только включить свой *windows.inc* в процедуру архивирования (надеюсь, таковая у вас имеется). Будет обидно начинать все сначала после какой-нибудь очередной переустановки системы.

Ну вот, а теперь пора **создать новый проект** :

1. Запустите MS DevStudio
2. Выберите в меню: *File/New*
3. В появившемся диалоге выберите вкладку *Projects*
4. В списке типов проектов выберите *Win32 Application*
5. В поле Location укажите каталог, в который предполагаете поместить папку проекта
6. В поле Project Name укажите желаемое имя проекта. Учтите, что по умолчанию это имя будет присвоено исполняемому модулю и, кроме того, папке, в которой будут находиться файлы проекта

на этапе его разработки. (Поле *Location* автоматически дополняется именем папки проекта.)

7. По остальным элементам диалога убедитесь, что *DevStudio* собирается создавать для вашего проекта новое рабочее пространство (*Create new workspace*) для платформы Win32

8. Нажмите кнопку *OK*

Новый проект создан.

В рамках вновь созданного проекта первым делом необходимо создать **файл с исходным текстом** :

1. Нажмите кнопку *New Text File* или создайте текстовый файл посредством диалога, вызываемого через меню *File/New*

2. Сохраните вновь созданный файл командой меню *File/Save As*. При этом дайте файлу желаемое имя с расширением *.asm*, например, *main.asm*. В принципе, расширение тоже может быть любым, но лучше не отходить от общепринятых стандартов

3. Подключите файл к проекту. Для этого, например, можно в окне *Workspace* на вкладке *FileView* щелкнуть правой кнопкой мыши по заголовку пока пустого списка файлов проекта и в появившемся контекстном меню выбрать пункт *Add Files to Project*. В файловом диалоге, чтобы увидеть наш новый файл, придется переключиться в режим Все Файлы.

А теперь самое важное. Файл когда-нибудь придется компилировать. Подготовиться к этому следует уже сейчас:

1. В окне *Workspace* на вкладке *FileView* щелкните правой кнопкой мыши по имени файла

2. В появившемся контекстном меню выберите пункт *Settings*. Появится диалог *Project Settings* в режиме установок для файла

3. В диалоге выберите вкладку *Custom Build*

4. В списке *Settings for* выберите пункт *Win32 Debug*

5. В текстовом поле *Build command(s)* введите новую команду:

```
ml.exe /c /coff /nologo /Zi /DDEBUG $(InputPath)
```

6. В текстовом поле *Output file(s)* введите текст:

```
.\$(InputName).obj
```

7. В списке *Settings for* выберите пункт *Win32 Release*

8. В поле *Build command(s)* введите новую команду:

```
ml.exe /c /coff /nologo $(InputPath)
```

9. В поле *Output file(s)* введите текст:

```
.\$(InputName).obj
```

Файл готов к компиляции с помощью модуля *ml.exe*. Обратите внимание, что пункты 5 и 6 касаются компиляции *asm*-файла для отладочного варианта проекта, и получаемый в этом случае объектный файл содержит отладочную информацию. Пункты же 8 и 9 касаются окончательного варианта проекта, готового к дистрибуции.

Наконец, наш файл должен иметь какое-нибудь корректное содержимое. Допустим, мы разрабатываем **минимальное приложение**. Учитывая, что кое-что из него уже имеется в файле *@struct.inc*, его содержимое может быть, например, таким:

```
include @struct.inc
include c:\program files\devstudio\vc\include\windows.inc

.code
WinMain PROC PUBLIC hinst,prev_hinst,command_line,cmd_show
    invoke ExitProcess,0
WinMain ENDP
end
```

Вот теперь можно смело нажимать кнопку *Compile (Ctrl+F7)*. Компиляция файла должна пройти без ошибок.

Обратите внимание, что имя включаемого файла *windows.inc* указано с полным путем. Это не случайно. В процессе разработки приложения вам придется очень часто обращаться к этому файлу, добавляя в него заголовочную информацию. В MS DevStudio это сделать очень просто, причем несколькими способами. Например, можно выделить имя файла и, щелкнув по нему правой кнопкой, выбрать пункт контекстного меню *Open Document*. Но удобнее все-таки подключить файл *windows.inc* к проекту "на постоянно", воспользовавшись правой кнопкой мыши в окне *Workspace*. Если вы удачно попадете по имени проекта или по какой-нибудь папке, вам станет доступна опция *Add files to Project (Folder)...* С ее помощью-то вы и подключите файл к проекту, и в дальнейшем простым двойным кликом сможете вызывать его на редактирование.

Теперь перейдем к **сборке проекта** :

1. В окне *Workspace* на вкладке *FileView* щелкните правой кнопкой мыши по заголовку списка файлов (он же имя проекта)
2. В появившемся контекстном меню выберите пункт *Settings*. Появится диалог *Project Settings* в режиме установок для проекта
3. В диалоге выберите вкладку *Link*
4. В списке *Settings for* выберите пункт *Win32 Debug*
5. В списке *Category* выберите пункт *General*
6. В поле *Output file name* вы можете при желании указать имя создаваемого исполняемого модуля, если вас не устраивает имя по умолчанию
7. Установите флажок *Ignore all default libraries*. Таким образом вы откажетесь от подключения runtime-библиотеки C/C++ и сэкономите несколько десятков килобайт кода. Подробнее см. [здесь](#) и [здесь](#).
8. В списке *Category* выберите пункт *Output*
9. В поле *Entry-point symbol* укажите *WinMain* (в данном случае) - имя функции, являющейся точкой входа для проекта
10. В списке *Settings for* выберите пункт *Win32 Release*
11. Прodelайте все то же самое по п.п.6...9

Все, проект готов к сборке. Нажмите кнопку *Build (F7)* и убедитесь, что сборка проходит нормально. Можете также попробовать запустить полученный исполняемый модуль проекта и убедиться, что ничего не происходит: он выполняется и завершается слишком быстро и без каких-либо видимых признаков.

Выше мы обещали обеспечить удобный **доступ к файлам заголовков API** , чтобы можно было пользоваться ими как справочником при подготовке файла *windows.inc*. Для этого нам потребуется активизировать встроенное средство MS DevStudio под названием *Browse Info*. Работает оно очень просто. Достаточно щелкнуть по интересующему нас имени в исходном тексте программы правой клавишей мыши и выбрать пункт контекстного меню *Go To Definition Of* , как моментально загрузится заголовочный файл Visual C++, в котором это имя описано. В проектах на C/C++ активизация *Browse Info* происходит автоматически, при первой попытке выполнить указанное действие. А вот в проектах на ассемблере, к сожалению, этого нет. Но ничего страшного. Просто нужно **немножко повозиться**.

И наконец, маленькое замечание о зависимостях (*Dependencies*). Не забывайте о них. Например, файл *main.asm* , очевидно, зависит от *windows.inc*. И поскольку последний будет очень часто подвергаться изменениям, в том числе и вне текущего проекта, имеет прямой смысл эту зависимость описать. При этом в каждом случае, когда происходит изменение файла *windows.inc* , будет автоматически вызываться перекомпиляция файла *main.asm*. В MS DevStudio зависимости описываются очень просто (хотя и не очень удобно: приходится вводить имя файла вручную),

гораздо проще, чем в заковыристых исторических make-файлах. Снова вызовите диалог *Project Settings* для файла *main.asm*. Найдите на вкладке *Custom Build* кнопку *Dependencies* и в появившемся диалоге укажите полное имя файла *windows.inc*.

Все, MS Developer Studio готово для ассемблерного программирования. Можно предъявить много всяких претензий по сервису и оформлению, но главное - это работает.

Использование Browse Info при разработке ассемблерных приложений

Автор: Svet(R)off

Дата: 2002-06-14

Всякий сколько-нибудь значительный программный проект содержит сотни, а то и тысячи идентификаторов. Как у программистов хватает фантазии их придумывать - это отдельная тема для исследования, пока что ждущая своего Фрейда, или Павлова, или Кастанеду, или, на худой конец, Маркеса.

Тут сразу следует напомнить о настоящих ассемблерщиках, которые не признают даже структурных директив MASM 6.1+. Ко всем прочим бедам они подбрасывают своему серому веществу еще и задачку придумывать имена для прорвы меток. Не заботясь о своем здоровье, надо сказать. Потому что, как известно, ничто так не разрушает здоровье, как работа, связанная с необходимостью принятия микрорешений. Позволим себе напомнить вам общеизвестный факт, что уровень преступности, суицида, алкоголизма, сексуальных извращений и политической активности среди сортировщиков куриных яиц примерно в восемь раз выше, чем среди трактористов!

А ведь сортировщики куриных яиц еще не сталкиваются с такой проблемой, как глобальность идентификаторов в пределах модуля! Спасибо любимой Microsoft за то, что хоть метки стали локальны в пределах процедуры. Впрочем, специально для садомазохистов разработчики предусмотрели директиву `OPTION NOSCOPE` - кайфуйте, ребята.

Каждый крутится как может. Шаблонно мыслящие конформисты идут на поводу у Чарльза Симони и украшают свои исходники венгерской тавтологией вроде `lpSomeTable`. Аристократические отпрыски Йеля и Итона, сложив губки гузкой, выстукивают холеными ногтями имя функции: `PleaseGiveMeAnErrorCode`. Раздолбанные хакеры, вытерев рукавом пиво с клавиатуры, без раздумий используют получившуюся комбинацию: `p9ijhbgfd`, и в течение следующей минуты навешивают на нее как минимум сорок строк кода. Стиль русских программистов старшего поколения, заставших время, когда их называли советскими программистами, особый, и не меняется уже много лет: `vodka362`, `kolbasa220`. Изредка попадающиеся в этой экологически нездоровой среде женщины широко используют свои специфические ассоциации: `carnation`, `champagne`, `alwaysultraplus`. Педанты же как всегда педантичны: `proc_for_scan_table_of_strings_00000001`.

И всех этих таких разных людей объединяет одна неизбежная, как желание выпить, проблема. Однажды (раньше или позже в зависимости от того, насколько способен программист организовать программу) мозг перестает контролировать расплодившихся идентификаторов. Всякий раз вы легко опознаете этот момент, застав себя запускающим контекстный поиск по папке проекта. Обычно после этого следуют коренные изменения в архитектуре программы: переформируются модули, рождаются новые классы, форматируется диск, наконец. Отсюда и известная присказка, что мол-де каждая программа пишется два раза.

Спасибо Microsoft: у нас теперь есть средство, позволяющее отложить этот тяжелый момент на этап, когда переписывать программу заново будет уже поздно. Это средство называется **Browse Info**. Оно встроено в MS Developer Studio и занимается вот чем. Стоит вам обнаружить где-нибудь в тексте упоминание какого-нибудь идентификатора, как ваш самый длинный палец получает возможность щелкнуть по правой кнопке мыши, и в появившемся контекстном меню тот палец, что

покороче, может выбрать пункт *"Go to Definition Of..."* или пункт *"Go to Reference Of..."*. И выбрав один из этих пунктов, вы мгновенно оказываетесь прямо в точке, где этот идентификатор описан, либо в точке, где на него ссылаются. Причем независимо от того, где это точка находится, включая все до единого файлы проекта и всю громаду заголовочных файлов API. DevStudio заботливо откроет перед вами нужный файл и подсветит искомый идентификатор.

Особенно полезна эта возможность для чудаков, пытающихся писать прикладной софт под win32 на ассемблере. Поскольку Microsoft не предоставляет заголовочных файлов API в ассемблерной транскрипции, а имеющим хождение в Интернете файлам независимых разработчиков полного доверия нет, рекомендуется строить файл windows.inc самостоятельно. Это совсем не сложно.

Прежде чем обсуждать особенности настройки DevStudio, несколько слов о организации работы Browse Info.

Информация об используемых в проекте идентификаторах и их местоположении собирается в файле, имеющем расширение .bsc и имя, по умолчанию совпадающее с именем проекта. Этот файл (опять же по умолчанию) располагается в папке Debug. Как Browse Info обращается к этому файлу, как он извлекает из него информацию, и как с ее помощью подставляет пользователю нужный ему фрагмент исходного текста проекта, мы разбирать не будем. Работает - и слава богу.

А вот откуда берется файл .bsc - разобраться надо, потому что без понимания этого процесса запустить Browse Info в ассемблерном проекте вряд ли удастся.

Процесс создания базы данных .bsc, как и следует ожидать, происходит на обоих этапах построения проекта - и на этапе компиляции, и на этапе компоновки.

Компилятор (будь то cl.exe для C++ или ml.exe для MASM) создает промежуточный файл базы данных, содержащий, естественно, только информацию об идентификаторах данной единицы компиляции - сpp-файла или asm-файла. Этот промежуточный файл имеет расширение .sbr и имя, по умолчанию совпадающее с именем компилируемого файла. Размещается он там же, где и создаваемый объектный файл - в папке Debug. Для того, чтобы компилятор озаботился созданием sbr-файла, в его командную строку следует включить опцию /FR или /Fr.

Опции отличаются друг от друга тем, что первая из них включает в sbr-файл информацию обо всех идентификаторах (в том числе локальных), а вторая - только об объявленных внешними и публичными. Выбор - за программистом.

Опции предоставляют возможность изменить местоположение и имя создаваемого sbr-файла, как, например: /FR"..\\Support Files\\browseinfo.sbr"

Поскольку компиляция файлов C/C++ поддерживается MS Developer Studio, то для того, чтобы заставить компилятор создавать sbr-файл, следует запустить диалог *Project Settings*, открыть вкладку C/C++, выбрать в списке *Category* пункт *Listing Files*, и поставить галочку в чекбоксе *Generate browse info*. Если нужно заменить местоположение или имя sbr-файла, следует воспользоваться полем *Intermediate browse info file destination*. А замена опции /FR на опцию /Fr производится галочкой в чекбоксе *Exclude local variables from browse info*.

Для ассемблерных файлов опцию в нужной форме следует прямо указать в командной строке на вкладке *Custom Build* при настройке [MS Developer Studio](#).

На этапе компоновки проекта из промежуточных sbr-файлов, созданных компилятором для каждого из модулей, собирается один на весь проект bsc-файл. Правда, слова "на этапе компоновки" отнюдь не означают, что этим занимается сам компоновщик. Для сборки bsc-файла существует

специальная утилита - *bscmake.exe* , находящаяся в папке исполняемых файлов *Vc\bin MS Developer Studio*.

Включить вызов утилиты *bscmake.exe* можно с помощью чекбокса *Build browse info file* , находящегося на вкладке *Browse Info* диалога *Project Settings*. Там же можно при необходимости поменять местоположение и имя bsc-файла, подавить выдачу баннерного текста и задать другие опции командной строки утилиты.

Учитывая обширность заголовочных файлов API, следует ожидать, что удовольствие от применения *Browse Info* хорошо оплачено ресурсами компьютера. Так оно и есть. Sbr-файл, образованный из чистого *windows.h*, весит почти восемьсот кило. А образованный из него bsc-файл - в два с лишним раза больше. При включенном *Browse Info* соответственно увеличивается время компиляции и компоновки проекта. Это обстоятельство могло бы превратить жизнь программиста в сущий ад, если бы не добрые дяди из Microsoft, предусмотревшие инкрементную обработку этих файлов. Суть ее в том, что сборка bsc-файла выполняется каждый раз не заново, а только с учетом реально произошедших изменений в проекте. Определить же, какие именно изменения произошли, утилита *bscmake.exe* умудряется очень просто. Всякий раз после очередной сборки она усекает до 0 размер участвовавших в сборке sbr-файлов. Таким образом в следующем сеансе сборки примут участие только те sbr-файлы, которые с прошлого раза прошли реальную перекомпиляцию.

Ну и, наконец, самое интересное. Как настроить *Browse Info* для работы с ассемблерным проектом. Специфика организации рабочей среды такова, что имеет смысл рассматривать два варианта настройки:

- рабочая среда (*workspace*) содержит один проект
- рабочая среда содержит несколько проектов

Вариант с одним проектом:

1. Создайте и настройте рабочий проект так, как указано в статье ["ms devstudio - среда разработки asm"](#)
2. В диалоге *Project Settings* на вкладке *Custom Build* в командную строку компилятора включите опцию:

```
/FR"Debug\"$(InputName).sbr"
```

Это необходимо сделать для всех asm-файлов, для которых вы хотели бы иметь включенным средство *Browser Info*.

Как видно, sbr-файлы будут помещаться в папку *Debug* с именем, соответствующим имени asm-файла. Эта папка выбрана нами потому, что по умолчанию в нее же помещаются sbr-файлы модулей, написанных на C/C++. Учтите, что в начале работы с проектом она не существует, поэтому попытка компиляции даст ошибку. Создайте ее вручную.

3. Если проект смешанный и содержит также файлы C/C++, включите для них опцию *Browser Info* установкой чекбоксов в диалоге *Project Settings* на вкладке C/C++ так, описано выше в этой статье.
4. Если проект чисто ассемблерный, то следует обеспечить доступ к заголовочным файлам C/C++ *windows.h* с тем, чтобы использовать их как справочные при построении собственного файла *windows.inc*. Для этого:

- создайте файл *brinfo.cpp* и включите его в проект
- содержимое файла *brinfo.cpp* должно представлять собой единственную строку:

```
#include <windows.h>
```

- включите для этого файла опцию *Browser Info* как для обычного C++-файла
- откомпилируйте его

5. В диалоге *Project Settings* на вкладке *Browse Info* :

- установите чекбокс Build browse info file
- установите чекбокс Suppress startup banner
- дополните содержимое поля Project options следующим текстом:

```
Debug\*.sbr
```

Таким образом вы дадите утилите bscmake.exe команду включить в сборку все sbr-файлы, содержащиеся в папке Debug

6. Выполните компиляцию всех модулей проекта, а затем его компоновку. Browser Info готово к работе.

Следует учесть одно неприятное обстоятельство. Рабочая среда определяет необходимость запуска утилиты bscmake.exe по факту выполнения компиляции хотя бы одного файла C/C++. Ассемблерные же файлы, будучи компилируемы посредством Custom Build, к сожалению, такой команды рабочей среде не дают. Поэтому все изменения в составе идентификаторов в ассемблерных файлах остаются втуне до тех пор, пока не будет перекомпилирован хотя бы один сpp-файл проекта и после этого не выполнена его компоновка. В чисто ассемблерных проектах придется вручную вызывать компиляцию файла brinfo.cpp.

В этой беде мог бы помочь вынос вызова утилиты bscmake.exe на этап Post-build step. Однако, этот фокус не проходит: дело в том, что однажды активизировавшись, Browse Info открывает bsc-файл и держит его в дальнейшем в открытом состоянии, запрещая таким образом запись в него всем внешним программам. Этот запрет отменяется только на этапе компоновки проекта, но не на этапе Post-build step.

Вариант с несколькими проектами базируется на тех же идеях, что и вариант с одним проектом, но с учетом некоторых коррекций:

- прежде всего следует определиться, имеется ли необходимость иметь единое для всех проектов пространство идентификаторов (не в смысле программирования, а в смысле просмотра их с помощью Browse Info), или у каждого проекта должно быть свое пространство, или проекты можно разбить на группы, с отдельным пространством для каждой группы
- для каждого пространства следует определить одну папку, в которой должны содержаться все sbr-файлы модулей, относящихся к этому пространству
- для каждого пространства следует использовать отдельный файл brinfo.spp

Компиляция файлов .asm с помощью компилятора ml.exe

Автор: Svet(R)off

Дата: 2002-06-14

Этот материал дополняет статью [MS Developer Studio - среда разработки для ASM](#). Здесь приведен формат командной строки компилятора MASM 6.1+, расшифровка ее опций и комментарии по их применению при работе в среде MS Developer Studio.

Командная строка MASM имеет вид:

```
ML [/options] filename [/options] filename [/link linkoptions]
```

Здесь:

- **/options** - необязательный перечень опций (см.таблицу ниже)
- **filename** - имена файлов исходных текстов, подлежащих компиляции, разделенные пробелами. В среде MS Developer Studio обычно указывается только одно имя в виде шаблона \$(InputPath).
- **/link linkoptions** - командная строка компоновщика в случае, если используется опция /BI. В среде MS Developer Studio обычно не применяется.

опция	назначение	применение для win32		
/AT	Enable tiny model (.COM file) Создать файл в формате .com (модель tiny)	Не применяется, так как формат исполняемого файла .com не используется в Win32.		
/BI<linker>	Use alternate linker Использовать альтернативный компоновщик	Обычно не применяется, так как возможностей link.exe вполне достаточно. Используется опция /c		
/c	Assemble without linking Только компиляция, без компоновки	Обязательно для применения в среде MS Developer Studio, чтобы выполнять компоновку отдельным этапом.		
/Cp	Preserve case of user identifiers Сохранение регистра пользовательских идентификаторов	Применение не обязательно, но возможно для дополнительного контроля синтаксиса. Вызывает ошибку "A2006: undefined symbol" при несовпадении регистра в объявлении идентификатора и обращении к нему. Позволяет избежать ошибок на этапе компоновки в случае, если идентификатор объявлен с неверным регистром.		

опция	назначение	применение для win32		
/Cu	Map all identifiers to upper case Приведение всех пользовательских идентификаторов к верхнему регистру	Не применяется, так как компоновка приложений Win32 чувствительна к регистру.		
/Cx	Preserve case in publics, externs Сохранение регистра идентификаторов, объявленных публичными и внешними	В применении нет необходимости. Регистр идентификаторов имеет смысл на этапе компоновки, но не на этапе компиляции.		
/coff	Generate COFF format object file Создать файл в формате COFF	Применение обязательно: это стандартный для windows формат объектных и исполняемых файлов.		
/D<name>[=text]	Define text macro Описание текстового макроса	Применяется по усмотрению программиста. Аналог директив EQU или =. Если текст содержит пробелы, его следует взять в кавычки. Обычно используется в отладочном версии приложения для объявления имени DEBUG.		
/EP	Output preprocessed listing to stdout Вывод листинга препроцессора в stdout	Обычно применять нет необходимости. Листинг препроцессора представляет собой исходный текст вместе с включаемыми файлами.		
/F<hex>	Set stack size (bytes) Определить размер стека (байт)	Практически не применяется. То же, что опция /STACK компоновщика link.exe. Обычно используется значение по умолчанию - 1 Мбайт.		
/Fe<file>	Name executable Имя исполняемого файла	Не применяется, так как с учетом опции /c компилятор не создает исполняемого файла		
/FI[file]	Generate listing Создание файла листинга	Обычно не применяется, так как средства MS Developer Studio, как правило, достаточны для работы с текстом приложения.		

опция	назначение	применение для win32		
/Fm[file]	Generate map Создание map-файла	Не применяется, так как map-файл создается компоновщиком, а с учетом опции /с компилятор не вызывает компоновщик		
/Fo<file>	Name object file Имя объектного файла	Обычно не применяется. Позволяет задать obj-файлу имя, отличное от имени asm-файла.		
/FPI	Generate 80x87 emulator encoding Включение кода эмулятора сопроцессора 80x87	Начиная с выхода в свет процессора 486 не применяется, так как с тех пор арифметический сопроцессор является неотъемлемой частью современных процессоров.		
/Fr[file]	Generate limited browser info Включить ограниченную информацию броузера	Применение менее предпочтительно, чем /FR, так как в информацию броузера не включаются сведения о локальных идентификаторах.		
/FR[file]	Generate full browser info Включить полную информацию броузера	Позволяет получать быстрый доступ к любому идентификатору во всем пространстве проекта и заголовочных файлов API win32.		
/G<c\	d\	z>	Use Pascal, C, or Stdcall calls Использовать соглашения вызова Pascal, C или Stdcall	В применении нет необходимости. Обычно использование соглашений вызова stdcall регламентируется директивой .model в тексте программы.
/H<number>	Set max external name length Установить максимальную длину внешних имен	Обычно не применяется. Значение по умолчанию - 31, и его достаточно для работы в среде win32.		

опция	назначение	применение для win32		
/I<name>	Add include path Добавить путь для inc-файлов	Не применяется, так как собственных возможностей MS Developer Studio обычно достаточно для определения путей к inc-файлам. Допускается использовать до 10 опций /I.		
/link	<linker options and libraries> Опции командной строки компоновщика и подключаемые библиотеки	Не применяется, так как компоновка отключена опцией /с.		
/nologo	Suppress copyright message Не показывать баннерный текст компилятора	Как правило, следует применять, так как баннерный текст смысловой нагрузки при разработке проекта не несет.		
/Sa	Maximize source listing Листинг максимального формата	Применяется редко, так как собственных средств MS Developer Studio обычно достаточно для работы с исходным и компилированным текстом программы, и в выдаче листинга нет необходимости.		
/Sc	Generate timings in listing Включить в листинг синхронизацию	То же		
/Sf	Generate first pass listing Листинг первого прохода	То же		
/Sl<width>	Set line width Длина строки листинга, символов: 60...255 или 0.	То же		
/Sn	Suppress symbol-table listing Не включать в листинг таблицу символов	То же		
/Sp<length>	Set page length Высота страницы листинга, строк: 10...255 или 0.	То же		
/Ss<string>	Set subtitle Текст подзаголовков листинга	То же		
/St<string>	Set title Текст заголовка листига	То же		

опция	назначение	применение для win32		
/Sx	List false conditionals Включить в листинг все фрагменты условной компиляции	То же		
/Ta<file>	Assemble non-.ASM file Компилировать не-.asm файлы	Обычно не применяется. Служит для компиляции файлов, имя которых имеет расширение, отличное от .asm.		
/w	Same as /W0 /WX То же, что /W0 /WX	См. далее.		
/WX	Treat warnings as errors Трактовать предупреждения как ошибки	Обычно в применении нет необходимости. В случае возникновения предупреждений компиляция завершается неуспешно.		
/W<number>	Set warning level Установить уровень предупреждений	Обычно в применении нет необходимости. Устанавливает перечень событий компиляции, трактуемых как предупреждения.		
/X	Ignore INCLUDE environment path Игнорировать путь, установленный переменной окружения INCLUDE	Обычно не применяется, так как при работе в среде MS Developer Studio переменная окружения INCLUDE не используется.		
/Zd	Add line number debug info Включить отладочную информацию в виде номеров строк	Обычно не применяется, так как на этапе отладки более целесообразно использовать опцию /Zi.		
/Zf	Make all symbols public Объявить все имена публичными	Обычно не применяется.		
/Zi	Add symbolic debug info Включить полную отладочную информацию	Обязательно применяется на этапе отладки. Формат отладочной информации MASM полностью совместим с используемым встроенным отладчиком MS Developer Studio.		

опция	назначение	применение для win32		
/Zm	Enable MASM 5.10 compatibility Включить совместимость с MASM 5.10	Обычно не применяется. Отключает полезные для прикладного программирования свойства MASM, введенные в версиях 6.1+.		
/Zp[n]	Set structure alignment Установить выравнивание структур	Может быть использован для установки принятого в win32 выравнивания по умолчанию - на 8 байт. Однако обычно в применении нет необходимости, так как в win32 используется два варианта выравнивания структур - на 4 и на 8 байт, и определять их выравнивание целесообразно непосредственно в описании структуры директивой STRUCT. Возможные значения для этой опции - 1, 2, 4 и 8 (последнее - в версиях MASM 6.13 и выше).		
/Zs	Perform syntax check only Выполнять только проверку синтаксиса	Обычно не применяется. Подавляет формирование объектного модуля.		

Типовая командная строка для этапа отладки (активен проект Win32 Debug) имеет вид:

```
ml.exe /c /coff /FR"..\.browseinfo\$(InputName).sbr" /nologo /Zi /DDEBUG $(InputPath)
```

Отключена **компоновка**, формируется объектный модуль в формате **COFF**, формируется sbr-файл, содержащий информацию для **Browse Info**, подавлена выдача **баннерной информации**, в объектный модуль включена полная **отладочная информация**, определена **константа** DEBUG.

Типовая командная строка для чистового этапа (активен проект Win32 Release) имеет вид:

```
ml.exe /c /coff /nologo $(InputPath)
```

Минимальная stub-программа

Автор: Svet(R)off

Дата: Не указана

"Stub" в переводе с английского - "пень, обломок, огрызок". Это в литературном переводе. В нелитературный перевод углубляться не будем. Скажем только, что это то самое, что показывает компьютер человеку, попытавшемуся запустить из-под DOS приложение, написанное для Windows. Обычно он показывает сообщение: "This program cannot be run in DOS mode". А вы что подумали?

В связи с понятием "stub-программа" обычно возникают три вопроса:

1. Как она работает?
2. Как сделать собственную stub-программу?
3. Как уменьшить ее до размера, не раздражающего настоящего ассемблера?

Как она работает? Чтобы ответить на этот вопрос, следует заглянуть во внутренности формата PE-файла - стандартного исполняемого модуля Windows. Там мы обнаружим следующее:

- PE-файл содержит в себе две программы, склеенные друг вслед за дружкой. Первая из них и есть stub-программа в обычном для MS-DOS exe-формате. Вторая - приложение для Windows.
- stub-программа, как и положено в MS-DOS, начинается в файле с нулевого смещения и первые два байта представляют собой знаменитые инициалы MZ, коими увековечил себя кто-то из корифеев Microsoft (не могу навскидку вспомнить. Кто знает - напишите) (16.05.01 написали. Михаил Орлов сообщил, что это был Mark Zbikowski. Спасибо им обоим.) Можно сказать, вырезал свое имя на каждом пне. С этого имени начинается стандартный заголовок exe-файла MS-DOS.
- windows-программа начинается с некоторого смещения, кратного параграфу (16 байт) и зависящему от размера stub-программы. Первые два байта windows-программы - это инициалы какого-то другого мужика со странным именем Portable Executable. Далее следует заголовок PE-файла и все остальные его элементы.

PE-формат - не единственный формат исполняемых модулей, который понимает Windows. Но мы не станем здесь рассматривать другие форматы, так как применительно к теме статьи это не имеет смысла.

Глупая DOS загружает exe-файл, обнаруживает в его начале привычный для себя заголовок и исполняет stub-программу, ничего не зная о наличии в файле еще чего-то, кроме слов "а пошел ты туда-то и туда-то".

Умная Windows поступает более изощренно:

- по аббревиатуре MZ определяет наличие заголовка MS-DOS. Определив, продолжает процесс:
- считывает двойное слово по смещению 3ch. Здесь компоновщик заботливо записал смещение PE-заголовка.
- считывает PE-заголовок по указанному смещению и пытается выполнить его разбор. Если это удастся - продолжает процесс загрузки приложения, который нас здесь больше не интересует.

Как сделать собственную stub-программу? Да очень просто! Напишите любую, сколь угодно сложную или простую DOS-программу. Только постарайтесь обойтись без извращений вроде перевода процессора в защищенный режим или показа картинок в режимах VESA: помните, что stub-программа должна сработать с флоппи-диска на 8086-й машине с текстовым монитором. Модель памяти можно использовать любую, кроме tiny - она для stub-программы не годится.

И это, к сожалению, неизбежно, как бы ни печалилась по этому поводу душа настоящего ассемблера. Ведь модель tiny порождает com-формат исполняемого модуля, не имеющий, как известно, никакого заголовка. А раз нет заголовка - то даже умная Windows не сможет узнать размер stub-программы и, соответственно, вычислить начало PE-заголовка. Глупая DOS ненамного легче переварит эту ситуацию. Обнаружив, что файл является исполняемым (имеет расширение exe, как положено в Windows), она выяснит также, что его фактический формат - com (отсутствует сигнатура MZ), и попытается-таки его загрузить и исполнить. Однако stub-программа сработает только в том случае, если размер исполняемого файла не превысит 64К, что для приложений Windows (даже написанных на ассемблере) - довольно редкое исключение. Скорее всего, ваш пользователь вместо цветного пожелания перейти, в конце концов, на новую ОС, получит сухое монохромное сообщение командного процессора о чрезмерном размере

исполняемого модуля.

Важно! При компоновке stub-программы в командной строке компоновщика укажите недокументированную и загадочно звучащую опцию /KNOWEAS. Это опция вносит некие изменения в заголовок ехе-файла. Если опцию не указать, то компоновка Windows-приложения со stub-программой окажется некорректна, о чем вы получите соответствующее предупреждение от линкера.

Подозреваем, что не все компоновщики поддерживают эту опцию. Однако link.exe из состава MS Developer Studio, конечно же, поддерживает.

Написанную stub-программу используйте на этапе компоновки своего Windows-приложения. Для этого в командную строку компоновщика добавьте опцию /STUB:"filename.exe", где filename.exe, как вы можете догадаться - это имя вашей stub-программы, при необходимости с путем. Обнаружив эту опцию, компоновщик заменит стандартную stub-программу на вашу.

Здесь настоящего ассемблерщика подстерегает серьезное разочарование. Проблема в следующем. Обычно при компоновке ехе-программ для DOS в исполняемом файле резервируется довольно большое место под таблицу перемещения. Например, в порядке вещей, если таблица перемещения вольготно располагается между смещениями 1ch (конец заголовка) и 200h (начало сегмента кода). Даже если ваша stub-программа не содержит ни одного фрагмента, подлежащего перемещению, все равно 486 байт чистых нулей будет бессмысленно вбавлено в нежное девственное тело вашего Windows-приложения. Задача, стоит ли избегать такого варварского разбазаривания дискового пространства и как это сделать, предлагается для самостоятельного решения. Кое-какие идеи читайте далее.

Минимальная stub-программа. Заглянув внутрь исполняемого модуля какого-нибудь Windows-приложения, вы обнаружите, что стандартная stub-программа чаще всего занимает 128 байт. Можно ли уменьшить этот размер и до какой величины? Отвечаем: можно, до 64 байт.

Возьмите любой редактор бинарных файлов и создайте с его помощью вот такой файл:

```
000000  4D 5A 00 00 01 00 00 00  02 00 00 00 FF FF 00 00
000010  40 00 00 00 00 00 00 00  40 00 00 00 00 00 00 00
000020  B4 4C CD 21 00 00 00 00  00 00 00 00 00 00 00 00
000030  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
```

Присвойте ему имя, например, stub.exe и используйте в качестве stub-программы.

Вот расшифровка содержимого файла stub.exe с комментариями:

смещение	значение	назначение	комментарий
+0	5A4D	Подпись ехе-файла ('MZ')	
+2	0000	Длина последней неполной страницы образа, байт	Игнорируется операционной системой
+4	0001	Длина образа, страниц (страница = 512 байт)	Программа занимает менее одной страницы
+6	0000	Число элементов в таблице перемещения	В этой программе перемещаемых элементов нет

смещение	значение	назначение	комментарий
+8	0002	Размер ехе-заголовка, параграфов (параграф = 16 байт)	Указывается размер базовой части заголовка. С учетом остальных значений параметров в данном случае означает, что исполняемый код в файле начинается со смещения 20h, и стартовый адрес находится в начале исполняемого кода
+0ah	0000	Минимум требуемой памяти за концом программы (параграфов)	В данном случае смысла не имеет
+0ch	FFFF	Максимум требуемой памяти за концом программы (параграфов)	Традиционно отводится вся доступная память
+0eh	0000	Сегментное смещение сегмента стека (для установки регистра ss)	
+10h	0040	Значение регистра sp (указателя стека) при запуске	В этой программе стек не имеет значения
+12h	0000	Контрольная сумма исполняемого модуля	Не используется
+14h	0000	Значение регистра ip (указателя команд) при запуске	Стартовая точка совпадает с началом кодового сегмента
+16h	0000	Сегментное смещение кодового сегмента (для установки регистра cs)	
+18h	0040	Смещение в файле 1-го элемента перемещения	В этой программе ни одного элемента перемещения нет, а исполняемый код находится внутри заголовка, поэтому данное значение совпадает с концом программы
+1ah	0000	Номер оверлея	Здесь не используется
+1eh	0000 0000	Резерв - 4 байта	
+20h	4CB4 21CD	Исполняемый код: mov ah,4ch int 21h	В данном случае программа просто возвращает управление операционной системе. Максимальный размер исполняемого кода для данных значений заголовка - 28 байт (смещения 20h...3bh). Обратите внимание, что исполняемый код с целью экономии размера программы находится в области, отведенной под ехе-заголовок!
+3ch	0000 0000	Зарезервированное двойное слово	Используется компоновщиком Windows-приложения для размещения смещения PE-заголовка

Вероятно, уменьшить размер stub-программы до величины, меньшей 64 байт, невозможно. А вот интересный вопрос: возможно ли вообще изъять stub-программу из модуля? Не спешите отвечать "нет"! 16-битные компоновщики от Microsoft имели в синтаксисе def-файла конструкцию вида STUB NONE, которая, согласно документации, предназначалась для использования при сборке dll-библиотек. В 32-битных компоновщиках эта возможность пропала. А жаль...

PS:

Вот уж в самом деле, "никогда не говори никогда"! 24 июля 2001 года, в день падения 114-летнего рекорда жары в Москве, пришло письмо:

"На вашем сайте assembler.ru в статье "Минимальная stub-программа" автором упомянуто, что создать stub короче 40h байт, вероятнее всего, невозможно.

Привожу пример stub'a длиной в 20h байт:

```
00000000 4D 5A 00 00 01 00 00 00 01 00 00 00 FF FF 00 00 MZ.....
00000010 B4 4C CD 21 00 00 00 00 40 00 00 00 00 00 00 .L.!....@.....
```

В отличие от примера, приведенного в статье, размер заголовка уменьшен до одного параграфа. В терминах статьи это означает, что stub-код располагается по смещению 10h от начала файла.

Очевидно, после такого "сокращения" области заголовка от 10h и выше продолжают использоваться. Но в данном случае это неважно, так как на месте 4CB4h находится initial SP, а на месте 21CDh - checksum. Ни то, ни другое роли не играют. К сожалению, stub-код ограничен в 4 байта, потому что уже следующий word - initial IP.

Далее, в файле со stub'ом в 20h байт смещение 3Ch от начала файла приходится на смещение 1Ch PE-заголовка. Здесь расположен dword, который, по-видимому, ни на что не влияет, так что можно спокойно ставить туда 00000020h:

```
00000020 50 45 00 00 4C 01 07 00 21 B1 97 36 00 00 00 00 PE..L...!.6....
00000030 00 00 00 00 E0 00 0E 01 0B 01 04 14 20 00 00 00 ..... ..
```

Такую штуку вряд ли поддерживают компоновщики (мне от них этого добиться не удалось), я лично правил EXE вручную. Зато работает.

Напоследок - пример тоже рабочего stub'a длиной в 18h байт - меньше, по-моему, некуда:

```
00000000 4D 5A 00 00 01 00 00 00 00 00 00 00 B0 21 CD 29 MZ.....!.)
00000010 B4 4C CD 21 0C 00 00 00 50 45 00 00 4C 01 07 00 .L.!....PE..L...
00000020 21 B1 97 36 00 00 00 00 00 00 00 00 E0 00 0E 01 !..6.....
00000030 0B 01 04 14 20 00 00 00 00 2E 00 00 18 00 00 00 .... ..
```

Из-под DOS он выводит восклицательный знак (21h).

Ну как, размер достоин настоящего ассемблерщика?"

автор - Grief (soulinspector@chat.ru)

Действительно, ни отнять, ни прибавить: только в голову настоящего ассемблерщика могут придти трюки, позволяющие так, чтобы помягче сказать, использовать формат исполняемого файла. Еще раз, вот они:

1. Когда делается попытка запустить программу из-под DOS, загрузчик, определив по сигнатуре "MZ", что это исполняемый файл, считывает по смещению 08h размер exe-заголовка, а по смещению 14h стартовое значение регистра ip, чтобы узнать, где находится исполняемый код. В предложенном Grief'ом варианте stub'a длиной 20h он таким образом выясняет, что передавать управление следует на смещение 10h (просто выход в DOS посредством функции 4Ch), а в варианте длиной 18h - на смещение 0Ch (здесь перед выходом успевает еще напечататься

восклицательный знак). И неважно, что занятые исполняемым кодом байты на самом деле являются полями ехе-заголовка "Максимум требуемой памяти за концом программы", "Сегментное смещение сегмента стека", "Значение регистра sp" и "Контрольная сумма исполняемого модуля". Все эти поля в данном случае не влияют работу программы.

2. Если же программа запускается из-под Windows, то загрузчик, опять же разобравшись с назначением файла, первым делом считывает двойное слово по смещению 03ch, чтобы узнать, где находится PE-заголовок. Так вот, здесь это смещение оказывается внутри самого PE-заголовка! В варианте "18h" оно попадает на поле, содержащее суммарный размер секций неинициализированных данных в образе приложения, а в варианте "20h" - на поле, содержащее размер секций кода. Использование этих полей в современных нам версиях Windows неочевидно. По одним источникам - они используются для первичного отведения памяти под приложение. По другим - не используются вообще. Во всяком случае, практическая проверка показывает, что приложения с таким stub'ом работают нормально. (Что будет дальше - пожалуй, неизвестно даже Microsoft'y.)

Конечно же, заставить какой-нибудь компоновщик создать исполняемый файл с предлагаемыми stub'ами не удастся. Такие трюки придется делать вручную, с помощью какого-нибудь бинарного редактора: переместить образ приложения внутри файла на нужное смещение, отредактировать ехе-заголовок, PE-заголовок, заголовки секций - ну, в общем, кто знает формат - тот справится. (А лучше доверить эту рутину какой-нибудь маленькой программке.)

Ну и напоследок, опровергнув свои же слова что "...меньше некуда...", Grief прислал вообще шедевр - вполне работающий stub длиной всего 0Ch!!! Разобраться, как он устроен, вы сможете самостоятельно, руководствуясь уже достаточно ясной идеей: в заголовках полным-полно никому не нужных полей, которые мы можем использовать для своих целей. Итак:

```
00000000 4D 5A 00 00 01 00 00 00 01 00 00 00 50 45 00 00 MZ.....PE..
00000010 4C 01 07 00 08 00 00 00 B0 21 CD 29 B4 4C CD 21 L.....!).L.!
00000020 E0 00 0E 01 0B 01 04 14 20 00 00 00 00 2E 00 00 .....
00000030 18 00 00 00 00 E0 00 00 00 10 00 00 0C 00 00 00 .....
```

Имеет ли смысл разработка приложений для Windows на ассемблере

Автор: Svet(R)off

Дата: 2002-06-14

На оригинальном сайте материал отмечен как незавершённый.

Минимальное Windows-приложение на ассемблере

Автор: Svet(R)off

Дата: 2002-06-14

Вопрос о минимальной программе для того или иного языка программирования и операционной среды обычно возникает в следующих случаях:

1. когда начинающий программист дочитывает до конца первую главу учебника и до него доходит, что большинство известных ему программ умеют нечто большее, чем тупо приветствовать мир. Пытливый ум новичка подсказывает ему, что если встречаются программы, умеющие больше этого, то, значит, где-то есть и программы, умеющие меньше
2. когда начинающий программист робко, опасаясь быть проигнорированным, замодерированным или посланным, забрасывает вопрос в какую-нибудь конференцию. Последующие события, как правило, приводят его в шок, надолго отбивая охоту приступить к прочтению второй главы. Со всех концов бескрайней саванны мгновенно слетаются и сбегаются бесчисленные гуру, возникает потасовка, из облака пыли доносятся рык, ржание, визги и предсмертные хрипы, разлетаются перья и клоки шерсти. Великие считают и пересчитывают строчки в исходниках, потом буквочки в них же, а потом и байтики в екзешниках. Вопли стихают только после того, когда кто-нибудь предлагает подсчитывать биты, установленные в 1, и все вдруг понимают, почему двери в психушках открываются исключительно вовнутрь. Вывалив языки и тяжело дыша, братия расползается по своим кельям зализывать раны и готовиться к грядущим битвам. Посередине вытопанной поляны, одинокий, жалкий и забытый, остывает трупик несчастного новичка
3. когда новичок, отчаявшись познать истину самостоятельно или испытать ее из ладоней великих, записывается на компьютерные курсы, и, с трудом досидев на первом занятии до сакраментального "У кого есть вопросы?", тянет руку и получает в ответ краткое изложение всей программы обучения. Затем, сверившись со списком группы и перезвонив в кассу, преподаватель также дает обещание, что к концу курса студент сам сможет элементарно ответить на этот простейший вопрос.

Между тем вопрос о минимальной программе - совсем не простейший, и представляет отнюдь не академический интерес. Минимальная программа, очевидно, решает две задачи: (1)**стартует** и (2)**завершается** в конкретной рабочей среде. И то, и другое она должна делать корректно, с тем, чтобы:

- стартовав, получить нормальный доступ к ресурсам рабочей среды
- завершившись, оставить рабочую среду работоспособной

Здесь, пожалуй, уместно напомнить про ограниченность тематики сайта: прикладное программирование на `masm` под `windows`. Поэтому дальше речь пойдет не о программах вообще, и не о каких-нибудь `VxD`, а о приложениях, то бишь `applications`, непосредственно взаимодействующих с операционной системой.

Сказанное особенно важно для программ на ассемблере, который дает программисту абсолютную власть над компьютером, что, конечно же, подразумевает и абсолютную возможность напакостить. Чем выше язык программирования, тем более обставлены всяческим сервисом процессы запуска и завершения программы, тем более они безопасны, и тем меньше требований к соблюдению соответствующих ритуалов программистом.

В `C/C++`, например, запуск и завершение приложения `win32` поддерживаются скрытой от прикладного программиста функцией `_WinMainCRTStartup`, содержащейся в `runtime`-библиотеке. Именно ее вызывает операционная система при запуске приложения,

а уж она, [кое-чего поделав](#), вызывает ту самую WinMain, с которой начинается всякое приложение, базирующееся на win32. (Конечно, строго говоря, запуск приложения посредством вызова _WinMainCRTStartup - это свойство не языка, а операционной среды.) От программиста же требуется, чтобы WinMain была правильно оформлена (включая четыре входных параметра), и завершалась оператором return с кодом выхода типа int.

В языках программирования более высокого уровня для программиста все еще проще, хотя там уже вряд ли можно говорить о том, что приложение-де "непосредственно взаимодействует с операционной системой". Java, еще более высокий, чем C/C++, уже не использует фиксированное название стартовой функции, да и завершение программы в нем - это просто закрывающая фигурная скобка.

Однако, вернемся к ассемблеру. Вот обещанный текст минимального приложения для win32:

```
.386
.model flat,stdcall
ExitProcess PROTO :DWORD
.code
WinMain PROC PUBLIC hinst,prev_hinst,command_line,cmd_show
;...
    invoke ExitProcess,0
WinMain ENDP
end
```

В результате [компиляции](#) и сборки этого приложения получается исполняемый файл размером 1536 байт. Из них собственно код занимает 10 байт. Все остальное - на совести создателей формата PE-файла. Приложение безропотно запускается и не задумываясь завершает свою работу, возвращая код выхода.

Что здесь что и зачем?

- .386 - директива ассемблера, определяющая набор команд процессора, который может быть использован в программе. Для приложений win32 необходимо использовать эту директиву или выше, в зависимости от того, собираетесь ли вы использовать возможности, предоставляемые процессорами последующих поколений
- .model flat,stdcall - сегментная директива ассемблера, определяющая сегментную модель приложения как плоскую, использующую соглашения о вызове процедур, принятые в win32. Именно такая сегментная модель должна всегда использоваться при написании приложений для win32
- ExitProcess PROTO :DWORD - прототип функции API, выполняющей завершение приложения. (Поскольку сервис этой функции предоставляется dll-библиотекой kernel32.dll, то при сборке приложения хорошо бы не забыть подключить библиотеку импорта kernel32.lib)
- .code - сегментная директива ассемблера, определяющая начало сегмента кода
- WinMain PROC PUBLIC hinst,prev_hinst,command_line,cmd_show - начало тела стартовой процедуры. Следует обратить внимание на наличие директивы видимости (visibility) PUBLIC, которая позволит сборщику сделать процедуру доступной для операционной системы, дабы та смогла передать управление приложению. Параметры процедуры обсудим чуть ниже, правда, новость будет не очень приятной.
- ;... - здесь должна быть всякая прочая мелочь, которую вы хотели бы заставить делать ваше приложение
- invoke ExitProcess,0 - собственно вызов функции завершения приложения. Здесь код выхода 0, но он, естественно, может быть любым в пределах 32-разрядного целого. В полноценных приложениях нормой считается определение кода выхода в цикле обработки сообщений главного окна.
- WinMain ENDP - всякое тело должно иметь конец. Это он и есть. Правда, в силу наличия предыдущей строки, он в данной программе не достижим, но это и не требуется: здесь мы просто исполняем необходимую формальность. Кстати, настоящим ассемблерщикам должно быть

приятно, что, опустив ненужную в связи с этим команду `get`, мы сэкономили аж 4 байта кода!

- `end` - конец модуля.

Замечание 1. По поводу необходимости применения функции `ExitProcess`. Именно с ее помощью, а не посредством команды `get`, как могли бы ожидать знатоки C/C++, должно завершаться приложение `win32`, написанное на ассемблере.

Если залезть внутрь runtime-библиотеки C/C++, то мы увидим, что именно так завершает работу приложения уже упоминавшаяся функция `_WinMainCRTStartup`. Но поскольку подключать runtime-библиотеку C/C++ к ассемблерной программе как-то нелогично (хотя и вполне возможно), мы должны вызвать `ExitProcess` "вручную". Только эта функция корректно завершает работу приложения, в частности, оповещая использовавшиеся приложением библиотеки DLL о необходимости декрементировать их счетчики занятости и, при обнулении последних, выгрузиться из памяти.

Замечание 2. А вот и обещанная плохая новость про параметры стартовой процедуры. Дело в том, что установкой их значений занимается - кто бы вы думали? - опять же `_WinMainCRTStartup`! И следовательно, в ассемблерных программах, где ее нет, при входе в `WinMain` параметры содержат:

- `hinst` - не дескриптор текущего экземпляра приложения, а количество акций Microsoft у Билла Гейтса
- `prev_hinst` - не 0, а прогнозируемое значение NASDAQ после выхода на мировой рынок российской ОС `BrokenWindows`
- `command_line` - не указатель на командную строку, а традиционный народный российский указатель направления
- `cmd_show` - не предлагаемый начальный вид окна, а то, что обещал показать мировому сообществу Никита Сергеевич Хрущов (и даже, помнится, начал раздеваться)

Так что, воленс-ноленс, придется получать эту ценную информацию самостоятельно. Как говорится, [клик хере](#).

Замечание 3. И все-таки приведенный текст приложения - не самый минимальный. Можно сделать его еще меньше, для чего следует убрать все параметры функции `WinMain`. Поскольку приложение собирается без runtime-библиотеки C/C++, это вполне допустимо, так как больше не с чем выполнять ее связывание. В результате объем исполняемого файла останется тем же, а вот объем исполняемого кода в нем (образа приложения) сократится аж до 7 байт. Про это стоит почитать [еще](#).

Замечание 4. В этой статье рассматривается минимизация размера кода программы, но отнюдь не `exe`-файла, ее содержащего. Как известно, исполняемые файлы Windows обычно имеют так называемый PE-формат, и манипуляции с этим форматом дают некоторые возможности для сокращения размера файла. Например, можно вместо стандартной [stub-программы](#) использовать более компактную, собственной разработки.

Шаблон ассемблерного оконного приложения

Автор: Svet(R)off

Дата: 2002-06-14

Каждый сайт, хоть немного причастный к программированию для Windows, считает своим долгом поместить на своих страницах шаблон оконного приложения. Assembler.ru в этом отношении ничем не хуже остальных. Почему бы и нет? Это настоящее, живое, работающее приложение, чего ж его стыдиться? Многим из нас его хватит, чтобы сказать себе с удовлетворением: "Достаточно, я в себя поверил", и вернуться к любимому третьему видеорежиму. Остальным же, возможно, оно понадобится еще много-много раз для того, чтобы начать с него новый проект, и, потратив полмесяца на его развитие, выяснить, что нужно было начинать с чего-то другого. Итак:

```
.386
.MODEL flat,stdcall

;заголовочный файл API;
;//////////////////////////////////// windows.inc
GetModuleHandleA PROTO :DWORD ;прототипы функций API
LoadIconA PROTO :DWORD,:DWORD
LoadCursorA PROTO :DWORD,:DWORD
RegisterClassExA PROTO :DWORD
CreateWindowExA PROTO :DWORD,:DWORD,:DWORD,:DWORD,\
                     :DWORD,:DWORD,:DWORD,:DWORD,\
                     :DWORD,:DWORD,:DWORD,:DWORD
ShowWindow PROTO :DWORD,:DWORD
GetMessageA PROTO :DWORD,:DWORD,:DWORD,:DWORD
TranslateMessage PROTO :DWORD
DispatchMessageA PROTO :DWORD
ExitProcess PROTO :DWORD
PostQuitMessage PROTO :DWORD
DefWindowProcA PROTO :DWORD,:DWORD,:DWORD,:DWORD

WNDCLASSEXA STRUCT 8 ;структура класса окна
    cbSize dd ?
    style dd ?
    lpfnWndProc dd ?
    cbClsExtra dd ?
    cbWndExtra dd ?
    hInstance dd ?
    hIcon dd ?
    hCursor dd ?
    hbrBackground dd ?
    lpszMenuName dd ?
    lpszClassName dd ?
    hIconSm dd ?
WNDCLASSEXA ENDS

POINT STRUCT 8
    x dd ?
    y dd ?
POINT ENDS

MSG STRUCT 8 ;структура сообщения
    hwnd dd ?
    message dd ?
    wParam dd ?
    lParam dd ?
    time dd ?

    pt POINT{}
MSG ENDS

NULL =0 ;разные константы
IDI_APPLICATION =32512
IDC_ARROW =32512
```

```

COLOR_WINDOWFRAME =6
SW_SHOW           =5
EXIT_ERROR        =0xffffffffh
WM_DESTROY        =2

CS_VREDRAW        =0001h
CS_HREDRAW        =0002h

WS_OVERLAPPED     =00000000h           ;константы стилей окна
WS_CAPTION        =00C00000h
WS_SYSMENU        =00080000h
WS_THICKFRAME     =00040000h
WS_MINIMIZEBOX    =00020000h
WS_MAXIMIZEBOX    =00010000h

WS_OVERLAPPEDWINDOW=WS_OVERLAPPED OR\
                    WS_CAPTION OR\
                    WS_SYSMENU OR\
                    WS_THICKFRAME OR\
                    WS_MINIMIZEBOX OR\
                    WS_MAXIMIZEBOX

;//////////////////////////////////// WinMain
.const
app_name          db "WinApp Template",0           ;имя приложения
class_name        db "WinApp Template Class",0     ;имя класса окна
.data?
win_class         WNDCLASSEX{}
win_handle        dd ?
loop_message      MSG{}
.code
;::::::::::::::::::::::::::::::::::::::::::::::::::
WinMain PROC PUBLIC hinst,prev_hinst,command_line,cmd_show
;..... регистрация класса окна
mov win_class.cbSize,sizeof(WNDCLASSEX)
mov win_class.style,CS_HREDRAW OR CS_VREDRAW
mov win_class.lpfnWndProc,offset win_procedure
mov win_class.cbClsExtra,0
mov win_class.cbWndExtra,0
invoke GetModuleHandleA,NULL
mov win_class.hInstance,eax
invoke LoadIconA,NULL,IDI_APPLICATION
mov win_class.hIcon,eax
mov win_class.hIconSm,eax
invoke LoadCursorA,NULL,IDC_ARROW
mov win_class.hCursor,NULL
mov win_class.hbrBackground,COLOR_WINDOWFRAME
mov win_class.lpszMenuName,NULL
mov win_class.lpszClassName,offset class_name
invoke RegisterClassExA,offset win_class
.if(!eax)
jmp abort
.endif
;..... создание окна
invoke CreateWindowExA,NULL,\
                    offset class_name,\
                    offset app_name,\
                    WS_OVERLAPPEDWINDOW,\
                    100,100,300,200,\
                    NULL,\
                    NULL,\
                    win_class.hInstance,\
                    NULL

.if(!eax)
jmp abort
.endif
mov win_handle,eax           ;он еще пригодится, только не сегодня
;..... вывод окна на экран
invoke ShowWindow,eax,SW_SHOW
;..... инициализация
;здесь можно вставить все необходимые действия для первичной инициализации

```

```

;приложения (например, загрузку и разбор файла настроек)
;..... цикл опроса очереди сообщений
msg_loop:
    invoke GetMessageA,offset loop_message,NULL,0,0
    .if(!eax)
        invoke ExitProcess,loop_message.wParam    ;успешное завершение приложения
    .endif
    invoke TranslateMessage,offset loop_message
    invoke DispatchMessageA,offset loop_message
    jmp msg_loop
;..... аварийное завершение
abort:
    invoke ExitProcess,EXIT_ERROR
WinMain ENDP

;//////////////////////////////////// Оконная процедура
win_procedure PROC window_from,message,w_param,l_param
    .if(message==WM_DESTROY)
        invoke PostQuitMessage,0
    .elseif(message==...)    ;обработка остальных необходимых сообщений
    ; ...
    .elseif(message==...)
    ; ...
    .elseif(message==...)
    ; ...
    .else
        ;сообщение не обработано
        invoke DefWindowProcA,window_from,message,w_param,l_param
        ret
    .endif
    xor eax,eax    ;сообщение обработано
    ret
win_procedure ENDP

;#####
end

```

Вот, собственно, и все приложение. В нем реализованы фундаментальные концепции общего прикладного программирования для Windows:

- работа начинается с вынужденного объектно-ориентированного действия - громоздкой **регистрации класса** окна с помощью функции **RegisterClassExA**. Вы должны сделать это, даже если собираетесь создать всего одно окно с такими свойствами, потому что так повелел когда-то *Уильям Гейтс Второй*, известный в народе как просто *Билл*. Впрочем, вы можете избежать этой неприятной процедуры, если воспользуетесь при создании окна одним из предопределенных классов - **BUTTON**, **COMBOBOX**, **EDIT**, **LISTBOX**, **MDICLIENT**, **SCROLLBAR**, **STATIC**. С помощью, например, класса **BUTTON** вы можете сделать большую-пребольшую кнопку и нажимать ее день и ночь до умопомрачения, ожидая, пока из floppy-дисковода выскочит банан. А класс **STATIC** очень удобен для эмуляции забора: написанное на таком окне емкое трехбуквенное слово всегда напомнит вам о необходимости вернуться к повседневным заботам.
- если регистрация класса окна прошла успешно, можно создать само окно с помощью функции **CreateWindowExA**. Полученный при ее возврате в регистре `eax` дескриптор окна следует сохранить до лучших времен. Он потребуется очень часто для выполнения манипуляций с окном.
- созданное окно полностью функционально за исключением одной мелочи: его не видно. Может, и в самом деле его не стоит показывать, чтобы не позориться. А иногда это действительно не нужно, например, если использовать главное окно для общей диспетчеризации приложения, а для общения с пользователем создавать другие окна. Но если вам действительно нужно показать окно, то это делает функция **ShowWindow**
- следующий этап - ваш. Делайте что хотите. Можете пойти попить кофейку, а можете чего-нибудь попрограммировать. Но помните: пока еще в приложении обработка сообщений не идет, поэтому оно мертво. Свежесозданное окно торчит недвижимо на экране как символ стабильности внутриполитических процессов, и ничто не оставляет на нем следов, как на зубах Кристины Пугачевой/Орбакайте/Пресняковой/Орбакайте.

- и только *цикл опроса очереди* сообщений оживит ваше окно, и кнопки в его заголовке начнут нажиматься, вызывая разные полезные эффекты, и границы его станут послушны движениям придавленной слева мыши, и много еще чего разного можно будет сделать с этим окном.
- потому что в вашем распоряжении теперь - оконная процедура **win_procedure**. Она принимает все и всяческие сообщения, адресуемые вашему окну всеми, кому не лень, и делает в ответ на них все, что вы пожелаете. В данном случае вы пожелали отправлять все эти сообщения обратно в систему, чтобы она сама делала с окном то, что повелел когда-то Билл. Единственное сообщение, на которое реагирует ваша оконная процедура - это **WM_DESTROY**, которое означает, что вам надоело любоваться окном, и вы нажали Alt+F4, или щелкнули по кнопке с крестиком, или выбрали Close в системном меню. Да и поделом ему.

Параметры функции WinMain

Автор: Svet(R)off

Дата: 2002-06-14

Приложения для win32, написанные на ассемблере и собранные без runtime-библиотеки (как это сделать - см. статьи [минимальное приложение](#) и [ms devstudio - среда разработки asm](#)), работают в среде, несколько отличающейся от той, которая привычна программистам C/C++ и выше. Все дело в том, что runtime-библиотека в значительной мере занимается подготовкой этой среды. Вернее, не сама библиотека (это, конечно, не ее задача), а входящая в нее функция _WinMainCRTStartup. В частности, эта функция устанавливает значения четырех параметров функции WinMain, которую программист воспринимает как стартовую функцию приложения:

```
WinMain PROC PUBLIC hInstance,hPrevInstance,lpCmdLine,nCmdShow
```

Здесь:

- hInstance - дескриптор текущего экземпляра данного приложения. Это значение используется в дальнейшем для взаимодействия с элементами рабочей среды, связанными с данным приложением: ресурсами, окнами, изображениями и многими другими. Как правило, надобность в знании этого дескриптора возникает почти сразу после запуска экземпляра приложения и сохраняется на протяжении всего срока его жизни. Этот дескриптор локализован в рамках данного экземпляра приложения и не имеет смысла для других приложений и других экземпляров этого же приложения.
- hPrevInstance - должен был бы быть дескриптор предыдущего экземпляра данного приложения. Однако в win32 этот параметр не используется и всегда равен 0. Часто бывает нужно знать, существуют ли в момент запуска приложения другие его экземпляры, и в зависимости от их существования предпринимать какие-либо действия. (Например, отменить запуск нового экземпляра и вывести окно уже существующего экземпляра на передний план.) Так вот, hPrevInstance после гибели win16 стал для этого совершенно бесполезен. (См. статью [взаимодействие экземпляров приложения](#).)
- lpCmdLine - указатель на командную строку. Сегодня большинство обычных пользователей практически не обращают внимания на архаичные возможности управления приложениями, предоставляемые командной строкой. Однако некоторые программисты все еще используют ее (в том числе и те, которые писали Проводник). Не факт, что командная строка вам понадобится, но знать, как получить к ней доступ - полезно.
- nCmdShow - параметр, описывающий, в каком виде рекомендуется создать главное окно приложения: скрытое, свернутое, нормальное, развернутое и т.д. Полезен в случаях, когда вызывающий процесс должен определять вид окна вызываемого процесса. Тогда nCmdShow следует использовать при первом вызове функции ShowWindow для главного окна приложения. Если же вы пишете самостоятельное приложение, то вполне допустимо проигнорировать этот параметр и показывать окно в том состоянии, которое вам больше нравится.

И вот все эти ценные и не очень параметры программисту на ассемблере недоступны. Сборка приложения без подключения runtime-библиотеки (например libc.lib) экономит от одного до нескольких десятков килобайт. Но тогда функция _WinMainCRTStartup не существует, а функция WinMain объявляется при сборке как точка входа. И следовательно, значения ее параметров неопределенны. Как же их получить?

Далее использован синтаксис [вызова системных функций](#) для masmb.1+

Дескриптор экземпляра приложения узнать очень просто:

```
invoke GetModuleHandleA,NULL  
mov hInstance,eax
```

Возвращаемое функцией *GetModuleHandleA* в *eax* значение - и есть искомый дескриптор. Будучи настоящими ассемблерщиками, мы, как видите, сохраняем его в *hInstance*, чтобы не пропадали даром целых четыре байта памяти. Между прочим, этот дескриптор фактически представляет собой базовый адрес памяти для размещения образа приложения (*image base* в терминах формата PE-файла). Для самостоятельных приложений по умолчанию он обычно равен 400000h.

Следует обратить внимание на наличие суффикса A в имени функции, говорящего о том, что мы работаем с кодировкой ANSI. Если вы предпочитаете Unicode, то суффикс должен быть W. Это, так сказать, "ручной" выбор типа кодировки приложения на этапе компиляции. Поскольку функций API, имеющих две версии, большинство, то, в принципе, может иметь смысл автоматизация этого процесса, как это сделано, например, для C++ в MS Developer Studio. Ввести некую настроечную константу и, в зависимости от нее, подключать ту или иную версию функции.

Командную строку следует получать так:

```
invoke GetCommandLineA
mov lpCmdLine, eax
```

Возвращаемое в *eax* значение указывает на буфер в памяти, содержащий текст командной строки с завершающим нулем. Результат, правда, несколько отличается от того, который можно было бы получить из *lpCmdLine*, будь у нас runtime-библиотека. Функция *GetCommandLineA* возвращает командную строку, включая полный путь к исполняемому модулю приложения. Путь взят в кавычки, что необходимо, поскольку он может содержать пробелы. Например, командная строка может иметь такой вид:

```
"D:\myapp\Little Joke.exe" c: /format
```

Между прочим, *lpCmdLine*, будучи оно нам доступно, содержало бы адрес в этом же самом буфере памяти, следующий за пробелом, которым завершается путь к исполняемому модулю.

Параметр показа главного окна приложения (если он вам действительно нужен) получить несколько труднее. Для этого необходимо воспользоваться фрагментом кода:

```
.data?
startup_info STARTUPINFO{}
.code
...
invoke GetStartupInfoA, offset startup_info
xor eax, eax
mov ax, startup_info.wShowWindow
mov nCmdShow, eax
...
```

Уникальная для каждого процесса структура STARTUPINFO заполняется вызывающим процессом, например, Проводником, перед запуском нашего приложения, при этом в нее заносится в том числе и значение параметра показа окна в виде члена *wShowWindow*, имеющего размер слова. С помощью этого кода мы считываем структуру STARTUPINFO, извлекаем из нее *wShowWindow*, приводим размер к двойному слову и записываем полученное значение в *nCmdShow*.

Таким образом, мы восстановили все, что потеряли, когда отказались от сборки ассемблерного приложения с runtime-библиотекой. Вернее, почти все, потому что runtime-библиотека делает чуть-чуть больше, чем установка параметров функции *WinMain*. Смайл.

И еще одно замечание. Раз уж мы выполняем сборку приложения без runtime-библиотеки, то ничто более не предъявляет к нашей пусковой функции каких-либо требований по ее именованию или числу аргументов. Мы можем назвать ее, например, *my_main_function* и лишить ее аргументов

насовсем (если, допустим, они нам не понадобятся, или мы будем сохранять их значения как-нибудь по-другому), либо оставить только те, которые нам необходимы. Например:

```
my_main_function PROC PUBLIC inst_handle,command_string
```

Главное - объявить эту функцию точкой входа приложения с помощью опции */entry:"my_main_function"* сборщика link.exe.

Старт и завершение ассемблерных Windows-приложений

Автор: Svet(R)off

Дата: 2002-06-14

Поэт утверждал, что-де "все начинается с любви". Поэтам вообще свойственно сказать что-нибудь этакое не подумав, получить гонорар - и в ресторан, а вы потом разбирайтесь, чего он имел ввиду. Например, предположим, имеется пара гомо сапиенс противоположных полов, выращенная до репродуктивного возраста в условиях, гарантированно исключающих ознакомление с процессом размножения двуполых животных вообще и вида гомо сапиенс в частности. Вопрос: сможет ли указанная пара, руководствуясь исключительно собственными наблюдениями разницы физиологического строения своих организмов и инстинктом, реализовать репродуктивную функцию? Низшие существа, как-то: хомячки - могут, проверено. А мы, цари природы, сможем? Разум не мешает? Ой, что-то сомнительно. Похоже, в процессе эволюции мы прикупили себе одно средство выживания - разум, заплатив за него другим - способностью существовать вне информационного поля социума.

Впрочем, что касается приложений win32, то здесь поэт точно был неправ. В этом случае все начинается с того, что вызывающая программа (например, Проводник) подготавливает и вызывает функцию API *CreateProcess*.

Ту же задачу, в принципе, решают и устаревшие функции *WinExec* и *LoadModule*. На самом деле на сегодняшний день они представляют собой всего лишь реализации той же функции *CreateProcess*.

Еще одно замечание, несколько офф-топик. Поскольку вызываемый процесс совершенно независим от вызывающего, то функция *CreateProcess* возвращает управление вызывающему процессу обычно до того, как вызываемый процесс закончит свою инициализацию. Если вы хотите, чтобы вызывающий процесс взаимодействовал с вызываемым, следует использовать функцию *WaitForInputIdle* для того, чтобы подождать завершения инициализации.

Функция *CreateProcess* :

- подготавливает для вновь создаваемого процесса рабочую среду (контекст, context), в которую входят: виртуальное адресное пространство, исполняемый код и данные, набор дескрипторов объектов, набор переменных окружения, базовый приоритет, минимальный и максимальный рабочие наборы страниц памяти
- в процессе подготовки контекста загружает PE-файл приложения. (Большинство из перечисленных выше элементов контекста определяются содержимым этого файла.)
- создает в рамках контекста процесса первичный поток (нить, thread) приложения и запускает его исполнение через точку входа приложения. Обычно точкой входа служит скрытая от прикладного программиста функция `__WinMainCRTStartup`, содержащаяся в runtime-библиотеке C/C++, подключаемой к приложению на этапе его сборки. В приложениях, написанных на ассемблере, без подключения runtime-библиотеки, точкой входа объявляется функция, подготовленная пользователем, обычно *WinMain*. ([Подробнее...](#))

Функция `__WinMainCRTStartup`, если таковая имеется, подготавливает рабочую среду для runtime-библиотеки, а также значения параметров для функции *WinMain*, после чего передает управление функции *WinMain*.

И, наконец, функция WinMain делает все, что придумает прикладной программист.

Несколько пояснений про элементы контекста. Поскольку мы не ставим своей целью продублировать всю имеющуюся в природе документацию на Windows, то эти пояснения будут самыми общими.

* **виртуальное адресное пространство** (*virtual address space*) - это набор адресов памяти от 0 до 0xffffffffh, доступных данному процессу. Каждый процесс имеет свое собственное виртуальное адресное пространство, одно для всех потоков процесса. Физически в каждый момент времени в оперативной памяти присутствует только небольшая часть из 4 Гбайт, доступных процессу, как правило, только то, что ему необходимо для работы. Для этого используется механизм страничной организации. Виртуальное адресное пространство разбито на страницы по 4 Кбайта. Страницы по мере необходимости загружаются в оперативную память или удаляются из нее. Это происходит автоматически, прозрачно для прикладного программиста

* **исполняемый код и данные** (*executable code and data*) - это совокупность кода и данных, загружаемых из соответствующих секций PE-файла приложения. Одни и те же фрагменты кода могут одновременно использоваться несколькими нитями процесса

* **дескрипторы объектов** (*object handles*) - это уникальный для данного процесса набор 32-битных идентификаторов, обеспечивающих обращение к самым разнообразным объектам рабочей среды: файлам, элементам графического интерфейса, процессам, потокам и пр.

* **переменные окружения** (*environment variables*) - это пришедший из DOS и UNIX набор текстовых пар имя=значение. Каждый процесс может иметь собственное окружение, либо пользоваться окружением вызвавшего его процесса

* **базовый приоритет** (*base priority*) - это значение в интервале от 0 до 31, определяющее приоритет каждого потока в многозадачной среде. В зависимости от базового приоритета распределяется процессорное время между всеми активными в данный момент потоками. Базовый приоритет формируется из класса приоритета, который присваивается каждому процессу и уровня приоритета, присваиваемого каждому потоку внутри процесса

* **минимальный рабочий набор** (*minimum working set*) - минимально необходимый для существования активного процесса набор страниц памяти

* **максимальный рабочий набор** (*maximum working set*) - наибольший набор страниц памяти, который может потребоваться активному процессу одновременно

В ОС Windows существует множество способов закончить работу приложения. В MSDN сказано, что это произойдет, если:

- любой из потоков приложения вызовет функцию *ExitProcess*
- первичный поток приложения завершится командой *ret*

Завершение первичного потока вызовом функции *ExitThread* в случае наличия других потоков не приведет к завершению приложения.

- завершится (любым способом) последний из потоков приложения
- любой из потоков приложения вызовет функцию *TerminateProcess*
- консольное приложение получит от пользователя сигнал CTRL+C (CTRL+BREAK)
- завершится работа системы (shut down) или работа пользователя в системе (log off).

Надо сказать, что этот список не окончателен. Николай Критский (nkritsky@mail.ru) подсказал еще и такой экзотический, но вполне работоспособный вариант: функцией *SetErrorMode* отключить выдачу системных сообщений об ошибках, а затем искусственно создать такую ошибку (например, выполнив деление на 0). Возникшая при этом исключительная ситуация приведет к завершению работы приложения.

На самом деле перечисленное разнообразие - только кажущееся, так как некоторые из указанных вариантов - это всего лишь скрытый вызов все той же функции *ExitProcess*. Это касается и *ret* в первичном потоке, и CTRL+C для консольного приложения, и даже варианта Николая Критского. Фактически в этих вариантах управление передается в существующие по умолчанию runtime-модуль, процедуру поддержки консоли или обработчик исключительной ситуации, которые и вызовут функцию *ExitProcess*.

Рекомендуется завершать работу приложения вызовом функции *ExitProcess*. При этом гарантируется, что система будет освобождена от последствий работы приложения, а именно:

1. Все занятые приложением dll-библиотеки будут освобождены, то есть для каждой из них будет вызвана входная функция со параметрами отключения. При этом внутренние счетчики занятости библиотек будут декрементированы, и те библиотеки, счетчики которых достигнут 0, будут выгружены из системы.
2. Будут закрыты все дескрипторы объектов, существовавшие в приложении
3. Будет завершена работа всех потоков приложения
4. Объект приложения перейдет в состояние "установлен", так что если кто-то в системе ждал завершения работы нашего приложения, сразу узнает об этом
5. Все объекты потоков приложения также перейдут в состояние "установлен"
6. Статус завершения процесса изменится со значения STILL_ACTIVE на значение, переданное функции *ExitProcess*

Упомянутая выше функция *TerminateProcess* делает все то же самое, за исключением освобождения dll-библиотек, и именно поэтому не может быть рекомендована как регулярное средство завершения работы. Ее следует использовать только в каких-то специальных или чрезвычайных случаях.

В заключение необходимо заметить, что отнюдь не все ресурсы, занятые приложением, освобождаются автоматически при завершении его работы. Поэтому хороший стиль программирования предполагает, что программист самостоятельно, не надеясь на систему, должен освобождать занятые им ресурсы по мере исчерпания надобности в них или перед завершением работы приложения.

См. также статьи [минимальное приложение](#) и [параметры функции WinMain](#).

Вызов функций API из ассемблерных приложений

Автор: Svet(R)off

Дата: 2002-06-14

В этой статье речь идет именно о вызове системных функций API win32, а не об организации процедур в ассемблере вообще. Обсуждаемая здесь тема, следовательно, существенна уже, так как многие возможности языка при вызове функций API не применяются. Например, здесь нет необходимости обсуждать описание процедур, так как функции API написаны добрыми людьми из Microsoft, и нам самим их писать не придется. Также не придется обсуждать вызов функций с переменным числом параметров и кое-что еще.

Вызов системных функций API win32 из программы на ассемблере подчиняется набору соглашений stdcall, ведущему свою родословную в части именования функций - от языка C, а в части передачи аргументов - от языка Pascal. С точки зрения прикладного программиста и с учетом специфики Windows и MASM эти соглашения заключаются в следующем:

- регистр символов в имени функции не имеет значения. Например, функции с именами ILoveYou и iloveyou - это одна и та же функция. Здесь мы наблюдаем отличие от реализации MS Visual C++, где компилятор строго отслеживает совпадение регистра в именах используемых программистом функций с прототипами, содержащимися в системных заголовочных файлах. Сборщику же, как видим, регистр символов безразличен
- аргументы передаются вызываемой функции через стек. Если аргумент укладывается в 32-битное значение и не подлежит модификации вызываемой функцией, он обычно записывается в стек непосредственно. В остальных случаях программист должен разместить значение аргумента в памяти, а в стек записать 32-битный указатель на него. Таким образом, все передаваемые функции API параметры представляются 32-битными величинами, и количество байт, занимаемых в стеке для передачи аргументов, кратно четырем
- вызывающая программа загружает аргументы в стек последовательно, начиная с последнего, указанного в описании функции, и кончая первым. После загрузки всех аргументов программа вызывает функцию командой call
- за возвращение стека в исходное состояние после возврата из функции API отвечает сама эта вызываемая функция. Программисту заботиться о восстановлении указателя стека esp нет необходимости
- вызываемая функция API гарантированно сохраняет регистры общего назначения ebp, esi, edi. Регистр eax, как правило, содержит возвращаемое значение. Состояние остальных регистров после возврата из функции API следует считать неопределенным. (Полный набор соглашений stdcall регламентирует также сохранение системных регистров ds и ss. Однако для flat-модели памяти, используемой в win32, эти регистры значения не имеют.)

На системном уровне этот набор соглашений добавляется вот еще чем. Компилятор при формировании из исходного текста объектного файла добавляет к началу имени функции символ подчеркивания, а к концу - выражение вида @n, где n - десятичное число, равное количеству байт, занятому в стеке под аргументы функции. Так формируется технологическое имя, позволяющее осуществить связывание не только по имени вызываемой функции, но и по количеству ее аргументов. Благодаря ему при сборке обнаруживаются ошибки программиста в случае, когда он задал для вызываемой функции неправильное число аргументов. Конечно, этому сервису далеко до строгого контроля типов C++, но от огромного количества трудноустраняемых ошибок он все-таки оберегает.

Соблюдение перечисленных соглашений обеспечивается компилятором автоматически. Для этого необходимо включить в начало исходного файла комбинацию директив:

```
.386
model flat,stdcall
```

Директиву `.386` можно заменить на более высокую в зависимости от того, на какой процессор вы рассчитываете. Директива `model` определяет сегментную модель приложения. Для всех приложений win32 она должна иметь именно такой вид, как показано здесь.

Прежде чем переходить к рассмотрению примеров, следует обсудить еще одно обстоятельство. Оно связано с тем, что Windows, благодаря прозорливости ее создателей, поддерживает два типа кодировки текстовых символов - архаичную восьмибитную ANSI и перспективную шестнадцатибитную Unicode. Из-за этого программистам Microsoft пришлось для всех функций API, так или иначе работающих с текстовыми символами, писать по два программных варианта. Тот, который работает с кодировкой ANSI, имеет суффикс `A`, например, `CreateFileA`. Тот, который работает с кодировкой Unicode, имеет суффикс `W`, например, `CreateFileW`.

Обычно приложение ориентируется на работу только с одной из этих двух кодировок. Выбор производится программистом на этапе компиляции путем указания значения для специальной настроечной константы. В MS Visual C++, например, это константа `UNICODE`. Если она определена, то вызов функции `CreateFile` в тексте программы при компиляции незаметно для программиста преобразуется в вызов функции `CreateFileW`, в противном случае - в `CreateFileA`.

В программах на ассемблере, конечно, можно реализовать тот же механизм. А можно, руководствуясь девизом "handmade forever", делать это вручную.

Основываясь на вышесказанном, приведем пример вызова какой-нибудь функции API. Допустим, это будет уже упоминавшаяся функция `CreateFile`. Вот ее описание для C++, как оно дано в Platform SDK:

```
HANDLE CreateFile(
    LPCTSTR lpFileName,
    DWORD dwDesiredAccess,
    DWORD dwShareMode,
    LPSECURITY_ATTRIBUTES lpSecurityAttributes,
    DWORD dwCreationDisposition,
    DWORD dwFlagsAndAttributes,
    HANDLE hTemplateFile
);
```

Фрагмент программы для кодировки ANSI, открывающий для чтения уже существующий файл, может выглядеть, например, так:

```
.data
file_name DB "MyFile.dat",0
.code
...
push NULL
push FILE_ATTRIBUTE_NORMAL
push OPEN_EXISTING
push NULL
push 0
push GENERIC_READ
push offset file_name
call CreateFileA
...
; имя открываемого файла

; hTemplateFile
; dwFlagsAndAttributes
; dwCreationDisposition
; lpSecurityAttributes
; dwShareMode
; dwDesiredAccess
; lpFileName
```

Но многим гораздо больше понравится тот же самый фрагмент так, как он выглядит в MASM 6.1+:

```
CreateFileA PROTO :DWORD,:DWORD,:DWORD,:DWORD,:DWORD,:DWORD
.data
file_name DB "MyFile.dat",0
.code
...
invoke CreateFileA,offset file_name,GENERIC_READ,0,NULL,
        OPEN_EXISTING,FILE_ATTRIBUTE_NORMAL,NULL
...
```

Оператор `invoke` - это совсем небольшая и довольно очевидная доработка, которая, вместе с некоторыми другими, мгновенно превратила ассемблер из машинного языка в нормальный язык программирования.

Ошибки при вызове функций API из ассемблерных приложений

Автор: Svet(R)off

Дата: 2002-06-14

Date: 17-24 ноября 1999
Newsgroup: microsoft.public.masm
Subject: How to call win32 API? (Как вызвать win32 API?)

Участвуют: Min minwang@hotmail.com
Xaphiosis void_s@ihug.com.au
Organization: The Internet Group Ltd
Randall Hyde rhyde@shoe-size.com

Min:

Рабочая среда: VC6.0/MASM6.11/NT Server 4.0

```
.386
.MODEL flat, stdcall
PUBLIC _start
.DATA
.CODE
_start:
    INVOKE MessageBox, NULL, "ok", "test", MB_OK
END _start
```

Получаю сообщение: error A2004: symbol type conflict
Также пробую MessageBoxA/MessageBoxW, бесполезно.
Есть идеи?

Regards,
Min

Xaphiosis:

Hehehehhhehhehhehheh ;)

Мне бы очень понравился такой синтаксис, но вы переоцениваете возможности ассемблера... если только вы не определили макрос MessageBox, вы не можете вызывать его с текстом в кавычках! И даже если вы сделали нечто подобное, вы не включили в строки завершающий ноль.

так что следовало бы поступить так:

```
INVOKE MessageBox, NULL, addr msg_ok, addr msg_test, MB_OK
```

Попробуйте.. Я гарантирую, что это будет работать...

Sincerely
X.

Min:

Спасибо за ответ. Но оно по-прежнему не работает :(

```
.386
.MODEL flat, stdcall
include win.inc
```



```

PUBLIC _start
.DATA
ALIGN DWORD
msg_ok BYTE "ok",0
msg_test BYTE "test",0
.CODE
_start:
    INVOKE MessageBox, NULL, addr msg_ok, addr msg_test, MB_OK
END _start

```

Сообщение об ошибке: "error A2004: symbol type conflict"

Regards,
Min

Xaphiosis:

Alright... Я быстренько откомпилировал ваш код на своей системе...
Напоминаю, если вы забыли:
OPTION CASEMAP:NONE
включает чувствительность к регистру, иначе win.inc не работает

кроме того, после MessageBox, мне кажется, вы должны вызвать
ExitProcess (правда, это не влияет на компиляцию)

Наконец, я думаю, следует проверить возможность того, что ваш
win.inc дефектен (довольно сомнительно)

В любом случае, вот код, который я использовал:
Попробуйте и убедитесь, что он работает ;)

```

.386
.MODEL FLAT, STDCALL
OPTION CASEMAP:NONE

include <my equivalent of win.inc = various files>
PUBLIC _start

.DATA
ALIGN DWORD
msg_ok BYTE "ok",0
msg_test BYTE "test",0

.CODE
_start:
    INVOKE MessageBox, NULL, addr msg_ok, addr msg_test, MB_OK
    call ExitProcess          ;EXIT POINT
END _start

```

Sincerely,
X.

Min:

Thank you very much.

Оказывается, MASM611 содержит include-файл для win3.1!
Поэтому я скачал пакет MASM32 - и все заработало.

Regards,
Min

Randall Hyde:

Если вам действительно нравится такой синтаксис, переходите на HLA
(High Level Assembler).

Он позволит вам писать код, подобный следующему:

```
MessageBox( MB_OK, "test", "ok", NULL );
```

(параметры перечислены в обратном порядке, потому что HLA использует соглашения вызова Паскаля, а не C/C++. Вы можете использовать макрос для устранения этой проблемы, если она для вас существенна).

HLA будет автоматически размещать строки в памяти "только для чтения" и предоставлять вам адреса таких строк. Вы можете найти HLA на <http://webster.cs.ucr.edu>

Randy Hyde

Наша точка зрения изложена в статье [Вызов функций API](#).

Взаимодействие экземпляров приложения

Автор: Svet(R)off

Дата: 2002-06-14

Вспоминается одна смешная история. Во времена Word 6.0, работая как-то в одной солидной организации, трепетно относившейся к своим коммерческим тайнам, был приглашен возмущенным соседом по комнате к его компьютеру. На экране красовалось нечто вроде: "Вы не можете открыть этот очень конфиденциальный документ, так как его уже открыл пользователь имярек". Имяреком был назван начальник отдела эксплуатации, сидевший в другом конце здания. Параноидальное сознание сразу стало выстраивать сложную схему шпионажа через локальную сеть, который развернул этот самый начальник. Другая часть мозга, пользуясь невытесняющей многозадачностью, принялась прикидывать размер вознаграждения за его поимку. Короче, помчались разбираться. Через пять минут половина отдела эксплуатации во главе с недоумевающим начальником толкалась около компьютера и выдвигала идеи. В итоге оказалось, что никакого шпионажа нет, а ...

... а во всем виноваты Word 6.0, невнимательный пользователь и пиратство. Пиратство виновато потому, что так тогда делали все. Word 6.0 инсталлировался с дискет, которые добрый начальник отдела эксплуатации по очереди раздавал всем желающим, так что под его именем рожала документы вся организация. Невнимательный пользователь виноват потому, что редактировал-редактировал документ в одном экземпляре Word'a, а потом взял, да и попытался открыть его из файл-менеджера еще раз. А Word виноват потому, что уж так он тогда обеспечивал взаимодействие нескольких экземпляров приложения (сейчас гораздо лучше).

Больше всех жалко начальника отдела эксплуатации, которому пришлось прилично понервничать. Пользователь-то, которому его заложил глупый Word, был сотрудником отдела безопасности!

Многозадачность Windows, являясь безусловным благом, не подлежащим ныне обсуждению и сомнению, является вместе с тем источником многих вызовов, бросаемых программисту. Некоторые из них весьма изощренны и ответ на них требует глубокого понимания принципов организации рабочей среды.

Например, при разработке приложения **MyCall** пришлось столкнуться с такой проблемой. Как известно, Remote Access Service (RAS), работающий в асинхронном режиме, запускается в отдельном потоке (нити), отличном от основного потока приложения. Также не менее хорошо известно, что для контроля процесса установления соединения в RAS используется callback-функция (например, *RasDialFunc*), которую работающий RAS вызывает при необходимости сообщить приложению о своем состоянии. Так вот, если у программиста имеется потребность передать из *RasDialFunc* какое-либо пользовательское сообщение окну приложения, то он может побояться воспользоваться для этой цели функцией *SendMessage*, полагая, что два потока, одновременно запускающие одну и ту же оконную процедуру в одном контексте - это пролог к катастрофе. Между тем такое действие вполне допустимо и безопасно, так как умная Windows, прежде чем передать управление оконной процедуре, проверяет, в каком потоке создавалось соответствующее окно, и при необходимости переключает потоки. При этом передающий поток замораживается на время обработки сообщения принимающим потоком.

Другие же проблемы многозадачности более тривиальны, даже рутинны. Одна из них встает всякий раз, когда программист начинает работу над новым проектом: что делать в случае попытки запуска приложения, если оно уже запущено?

В самом деле. Сама по себе Windows не ограничивает возможности пользователя запускать какое-либо приложение любое разумное количество раз. Выполнив два двойных щелчка по одной и той же иконке, пользователь может получить два экземпляра одного и того же приложения, и каждый из них будет функционировать в собственном процессе и собственном контексте так, как будто другого не существует.

Правда, только до той поры, пока они не попытаются разделить между собой какой-нибудь ресурс (в широком смысле), который разделять невозможно или нельзя. Например, невозможно разделять коммуникационный порт. И нельзя разделять общий файл данных, если оба экземпляра приложения пишут в него противоречащие друг другу данные. Разнообразие возникающих в этом случае эффектов может порадовать самого взыскательного искателя развлечений.

Принятие решения о том, как поступать со вторым и последующими экземплярами приложения, целиком возлагается на программиста. Можно придумать много разных вариантов стратегии поведения приложения, в зависимости от назначения приложения. Среди них такие:

1. Второй и последующие экземпляры приложения запускаются и функционируют самостоятельно, безотносительно к существованию друг друга
2. Запуск второго экземпляра (и, следовательно, всех последующих) запрещен, пока функционирует первый запущенный экземпляр
3. При попытке запуска второго экземпляра он не запускается, а в работе первого происходят некоторые изменения

Яркий пример третьей стратегии реализован в Word. Если экземпляр приложения уже запущен, и пользователь выполняет двойной щелчок по какому-нибудь файлу .doc в Проводнике, то запускаемый при этом второй экземпляр Word передает реквизиты загружаемого файла первому экземпляру и завершается, а первый экземпляр загружает файл в новое окно документа.

С **первой стратегией** все просто. При запуске приложения ничего делать не надо - и тогда вы сможете наблюдать на экране столько экземпляров вашего приложения, сколько нужно, чтобы получить моральное удовлетворение. Главное - позаботиться, чтобы экземпляры приложения никогда не пытались разделять ресурсы, которые невозможно или нельзя разделять. **Вторая стратегия** несколько сложнее. Очевидно, для ее реализации необходимо, чтобы второй экземпляр приложения каким-то образом узнал о том, что существует первый экземпляр, и завершился, не успев принести какого-нибудь вреда. К сожалению, никакого специального средства для этого не существует.

Поговаривают, что не всегда все было так трагично. Во времена наших предков, когда "Докторская" стоила 2.20, а Билл был еще совсем бедным со своими двумя десятками гигабайсов, заработанных на DOSe, это делалось очень просто. В win16 имел смысл параметр *hPrevInstance* функции *WinMain*. Достаточно было убедиться, что он ненулевой - и приложение знало, что оно не одиноко на этом свете. Но когда разрабатывалась win32, кто-то из главных системщиков Microsoft лопухнулся, а потом, когда все всплыло, на специальном совещании было принято решение прикрыть провинившегося и объяснить все произошедшее какими-нибудь благообразными словами, вроде: "Мы так и хотели с самого начала, а *hPrevInstance* сохранили для совместимости снизу вверх." Так или иначе - но *hPrevInstance* теперь бесполезен. А следует пользоваться средствами межпроцессного взаимодействия (Interprocess Communications), которые, к чести Microsoft, реализованы в Windows великолепно.

Способов дать знать второму экземпляру о существовании первого можно придумать много. Один из них прямо дается в MSDN в статье WinMain:

hPrevInstance

Identifies the previous instance of the application. For a Win32-based application, this parameter is always NULL. If you need to detect whether another instance already exists, create a named mutex using the CreateMutex function. If the GetLastError function returns ERROR_ALREADY_EXISTS, another instance of your application exists (it created the mutex).

Идентифицирует предыдущий экземпляр приложения. Для Win32-приложений этот параметр всегда равен NULL. Если вам необходимо определить, существует ли уже экземпляр приложения, создайте именованный объект mutex, используя функцию CreateMutex function. Если функция GetLastError вернет ERROR_ALREADY_EXISTS, другой экземпляр вашего приложения существует (так как он уже создал mutex).

Вот этой самой идеей и воспользуемся, слегка модифицировав ее:

```
.386
.MODEL flat,stdcall

;//////////////////////////////////// windows.inc
OpenMutexA      PROTO :DWORD,:DWORD,:DWORD      ;прототипы функций API
CreateMutexA     PROTO :DWORD,:DWORD,:DWORD
ExitProcess     PROTO :DWORD

SYNCHRONIZE      =00100000h                      ;константы типов доступа
STANDARD_RIGHTS_REQUIRED =000f0000h
MUTANT_QUERY_STATE =0001h
MUTANT_ALL_ACCESS =STANDARD_RIGHTS_REQUIRED OR SYNCHRONIZE
                  OR MUTANT_QUERY_STATE
MUTEX_ALL_ACCESS =MUTANT_ALL_ACCESS

NULL             =0                               ;разные константы
FALSE            =0
TRUE             =1

;//////////////////////////////////// WinMain
EXIT_OK          =0                               ;коды выхода
EXIT_ALREADY_EXIST =1
.const
    check_inst_mutex_name db "Check Instant Mutex 0001",0 ;имя mutex
.data?
    check_inst_mutex      dd ?                     ;дескриптор mutex
.code
;::::::::::::::::::::::::::::::::::::::::::::::::::::::::::
WinMain PROC PUBLIC hinst,prev_hinst,command_line,cmd_show
;..... проверка повторного запуска
mov check_inst_mutex,0
invoke OpenMutexA,MUTEX_ALL_ACCESS,FALSE,offset check_inst_mutex_name
.if(!eax) ;если mutex не существует - создать его и запустить приложение
    invoke CreateMutexA,NULL,TRUE,offset(check_inst_mutex_name)
    mov check_inst_mutex,eax
.else ;если mutex существует - прервать работу приложения
    invoke ExitProcess,EXIT_ALREADY_EXIST
.endif
;..... тело приложения
;...
;... ;весь остальной текст приложения
;...
;..... завершение работы
.if(check_inst_mutex!=0) ;уничтожить объект mutex
    invoke ReleaseMutex,check_inst_mutex
.endif
invoke ExitProcess,EXIT_OK
WinMain ENDP
;#####
end
```

Как видим, логика работы очень проста. Первым делом пытаемся открыть mutex с заданным именем.

Если вы - автор многих разных приложений, имейте в виду, что одинаковое имя mutex'a позволит запускаться только одному из этих приложений. Если ваше чувство юмора не столь развито, не забывайте модифицировать имя mutex'a в разных приложениях.

Если mutex открыть не удастся, значит запускаемый нами экземпляр приложения уникален, и мы смело можем продолжать его работу. Только перед этим создадим mutex, чтобы не дать запускаться последующим экземплярам.

Ну и, конечно, при завершении работы приложения следует не забыть вызвать функцию `ReleaseMutex`.

В принципе, это не обязательно, так как если не освободить mutex с помощью этой функции, по завершении работы приложения он переводится в состояние `WAIT_ABANDONED`, которое в данном случае не мешает достижению стоящей перед нами задачи. Однако хороший стиль программирования предполагает явное освобождение занимаемых приложением ресурсов.

А вот Сергей AKA The Byte Reaper (<http://neptunix.narod.ru>) совершенно справедливо обратил наше внимание на то, что приведенный выше пример реализации второй стратегии сильно перегружен формальностями и реверансами в сторону MSDN. Полностью сохраняет ту же функциональность, но гораздо приятнее взору настоящего ассемблерщика предложенный Сергеем вот какой вариант:

```
;-----  
invoke CreateMutexA,NULL,0,offset(check_inst_mutex_name)  
invoke GetLastError  
.if(eax==ERROR_ALREADY_EXISTS)  
    invoke ExitProcess,EXIT_ALREADY_EXIST  
.endif  
;-----
```

Идея проста предельно. Если создаваемый mutex уже присутствует в системе, то функция `GetLastError` возвращает код `ERROR_ALREADY_EXISTS`. Кстати, функция `CreateMutex` возвращает при этом вполне валидный дескриптор существующего mutex'a, но нас это совершенно не должно интересовать. Мы просто заканчиваем работу приложения - и все.

Третья стратегия существенно сложнее в реализации, а гамма возможных решений намного больше. Очевидно, ее основой должно быть не только взаимное оповещение экземплярами приложения о существовании друг друга, но и обмен между ними какой-то информацией. Все зависит от того, какой объем информации потребуется. Если это одно-два двойных слова, то может оказаться достаточным использовать систему сообщений. В других случаях имеет смысл, например, организовать разделяемую память посредством *File Mapping*.

Пример реализации третьей стратегии, использующий только систему оконных сообщений, можно найти в приложении **MyCall**. Там используется регистрируемое глобальное сообщение, которое приложение передает в широковещательном режиме в надежде, что уже запущенный экземпляр ответит ему тем же сообщением. Если такой ответ получен, то делается вывод, что один экземпляр приложения уже запущен, и запускаемый экземпляр завершает работу. Но перед этим он извлекает из ответного сообщения дескриптор главного окна работающего экземпляра и выводит это окно на первый план, чтобы удовлетворить потребность пользователя в доступе к приложению.

Здесь мы приведем версию, выполняющую ту же задачу, но несколько отличную в реализации. По сути, она является гибридом из описанной выше и примененной в **MyCall**: для определения факта

повторного запуска приложения используется mutex, а для передачи дескриптора окна работающего приложения вновь запускаемому - глобальное сообщение.

Это полностью работающее приложение. В его основу положен [шаблон оконного приложения](#). Содержимое файла windows.inc не приводится (то, что он должен собой представлять, показано в предыдущем примере. Недостающие элементы следует добавить самостоятельно, по аналогии, пользуясь заголовочными файлами для C/C++).

```
.386
.Model flat,stdcall

include windows.inc ;заголовочный файл API

;//////////////////////////////////// WinMain
EXIT_OK =0 ;коды выхода
EXIT_ERROR =1
EXIT_ALREADY_EXIST =2

QUESTION_PRIME_HWND =1 ;запрос идентификатора окна
ANSWER_PRIME_HWND =2 ;ответ идентификатора окна

.data?
win_class WNDCLASSEX{} ;структура класса главного окна
main_window dd ? ;дескриптор главного окна
check_inst_mutex dd ? ;дескриптор mutex'a
second_instance dd ? ;признак второго экземпляра
check_inst_message dd ? ;идентификатор регистрируемого сообщения
loop_message MSG{} ;сообщение для цикла обработки
.const
app_name db "Single Instance Application",0
class_name db "Single Instance Application Class",0
check_inst_mutex_name db "Check Instant Mutex 0001",0
check_inst_message_name db "Check Instant Message 0001",0
.code
;::::::::::::::::::::::::::::::::::::::::::::::::::
WinMain PROC PUBLIC hinst,prev_hinst,command_line,cmd_show
;..... регистрация класса окна
mov win_class.cbSize,sizeof(WNDCLASSEX)
mov win_class.style,CS_HREDRAW OR CS_VREDRAW
mov win_class.lpfnWndProc,offset win_procedure
mov win_class.cbClsExtra,0
mov win_class.cbWndExtra,0
invoke GetModuleHandleA,NULL
mov win_class.hInstance,eax
invoke LoadIconA,NULL,IDI_APPLICATION
mov win_class.hIcon,eax
mov win_class.hIconSm,eax
invoke LoadCursorA,NULL,IDC_ARROW
mov win_class.hCursor,NULL
mov win_class.hbrBackground,COLOR_WINDOWFRAME
mov win_class.lpszMenuName,NULL
mov win_class.lpszClassName,offset class_name
invoke RegisterClassExA,offset win_class
.if(!eax)
jmp abort
.endif
;..... создание окна
invoke CreateWindowExA,NULL,\
offset class_name,\
offset app_name,\
WS_OVERLAPPEDWINDOW,\
100,100,300,200,\
NULL,\
NULL,\
win_class.hInstance,\
NULL

.if(!eax)
jmp abort
.endif
mov main_window,eax
```

```

;..... зарегистрировать глобальное сообщение
mov check_inst_message,0
invoke RegisterWindowMessageA,offset check_inst_message_name
.if(eax)
mov check_inst_message,eax
.endif
;..... проверить повторный запуск
mov check_inst_mutex,0
invoke OpenMutexA,MUTEX_ALL_ACCESS,FALSE,offset check_inst_mutex_name
.if(!eax)
mov second_instance,FALSE
invoke CreateMutexA,NULL,TRUE,offset check_inst_mutex_name
mov check_inst_mutex,eax
.else
mov second_instance,TRUE
.if(check_inst_message)
invoke SendMessageA,HWND_BROADCAST,check_inst_message,
QUESTION_PRIME_HWND,NULL
.endif
invoke ExitProcess,EXIT_ALREADY_EXIST
.endif
;..... вывод окна на экран
invoke ShowWindow,main_window,SW_SHOW
;..... инициализация
;здесь можно вставить все необходимые действия для первичной инициализации
;приложения (например, загрузку и разбор файла настроек)
;..... цикл опроса очереди сообщений
msg_loop:
invoke GetMessageA,offset loop_message,NULL,0,0
.if(!eax)                                ;завершение работы
.if(check_inst_mutex!=0)                  ;освободить mutex
invoke ReleaseMutex,check_inst_mutex
.endif
invoke ExitProcess,loop_message.wParam
.endif
invoke TranslateMessage,offset loop_message
invoke DispatchMessageA,offset loop_message
jmp msg_loop
abort:
invoke ExitProcess,EXIT_ERROR
WinMain ENDP

;//////////////////////////////////// Оконная процедура
win_procedure PROC hwnd,message,w_param,l_param
mov eax,message
;..... обработка регистрируемого сообщения
.if(eax==check_inst_message) ;если получено зарегистрированное сообщение
.if(check_inst_message)
.if(second_instance) ;если это второй экземпляр
.if(w_param==ANSWER_PRIME_HWND) ;если в сообщении получен дескриптор окна
invoke SetForegroundWindow,l_param ;вывести это окно на первый план
jmp worked
.endif
.else ;если это первый экземпляр
.if(w_param==QUESTION_PRIME_HWND) ;если запрашивается дескриптор окна
mov eax,hwnd ;послать ему дескриптор главного окна
.if(eax==main_window)
invoke SendMessageA,HWND_BROADCAST,check_inst_message,
ANSWER_PRIME_HWND,main_window
jmp worked
.endif
.endif
.endif
.endif
jmp noworked
;..... закрытие приложения
.elseif(eax==WM_DESTROY)
invoke PostQuitMessage,EXIT_OK
jmp worked
;..... обработка остальных сообщений

```



```

; .elseif(message==...)
; ...
; .elseif(message==...)
; ...
; .elseif(message==...)
; ...
;..... сообщение не обработано
.else
noworked:
    invoke DefWindowProcA,hwnd,message,w_param,l_param
    ret
;..... сообщение обработано
.endif
worked:
    xor eax,eax
    ret
win_procedure ENDP
;#####
end

```

Общая идея такова:

- регистрируется класс главного окна и создается само окно с тем, чтобы вновь запускаемое приложение могло принимать и обрабатывать сообщения с помощью оконной процедуры *win_procedure*
- регистрируется глобальное сообщение, которое впоследствии должно использоваться для обмена информацией между экземплярами приложения
- выполняется проверка наличия уже работающего экземпляра приложения с помощью объекта *mutex*
- если уже работающий экземпляр приложения существует - ему посылается иницирующее сообщение с параметром *QUESTION_PRIME_HWND*. Поскольку дескриптор окна этого экземпляра неизвестен, сообщение посылается широковещательно (*HWND_BROADCAST*).
- ответ от уже работающего экземпляра принимает оконная процедура приложения в виде такого же сообщения, но уже с параметром *ANSWER_PRIME_HWND*. При этом второй параметр сообщения представляет собой дескриптор окна работающего приложения.
- запускаемый второй экземпляр приложения, пользуясь полученным дескриптором, выводит окно первого экземпляра на первый план с помощью функции *SetForegroundWindow* и завершает свою работу
- если запускаемый экземпляр приложения - первый и единственный, то он продолжает свою работу так, как задумал программист. При этом его оконная процедура всегда готова получить и обработать зарегистрированное сообщение с параметром *QUESTION_PRIME_HWND*, которое могут послать повторно запускаемые экземпляры. В ответ на такое сообщение работающий экземпляр отвечает таким же сообщением, но с параметром *ANSWER_PRIME_HWND*, передавая в качестве второго параметра дескриптор своего главного окна.

См. также интересный, эффективный и простой [вариант](#) решения рассматриваемой задачи, предложенный Геннадием Майко.

Взаимодействие экземпляров приложения (вариант)

Автор: Svet(R)off

Дата: 2002-06-14

Этот материал предоставлен [Геннадием Майко](#). Он дополняет публикацию "[Взаимодействие экземпляров приложения](#)", в которой мы излагали суть проблемы и рассматривали некоторые способы ее решения. Вариант, предложенный Геннадием, изящен и предельно прост в реализации. Приносим ему свою благодарность. Итак, слово автору:

Добрый день,

Хотел бы предложить еще один вариант (на мой взгляд, весьма эффективный) проверки того, что приложение, написанное под Win32, было уже запущено. Этот пример навеян очень интересной и полезной книгой Jeffrey Richter "Advanced Windows" (если я не ошибаюсь, есть ее перевод на русский язык).

Идея состоит в следующем. Для приложения или DLL можно определить некоторую секцию (сегмент) как "разделяемую" (shared). Это значит, что ее содержимое будет использоваться совместно всеми экземплярами приложения. Этим можно воспользоваться для проверки факта запуска приложения во второй и более раз.

Определим в некотором отдельном сегменте переменную Cnt и проинициализируем ее 1 (или любым другим ненулевым значением). В самом начале работы приложение проверяет значение этой переменной и, если оно равно 1, присваивает ей 0. Понятно, что эта операция должна быть атомарной (во-первых, к этой переменной могут обращаться несколько программ одновременно, во-вторых, мы можем работать на многопроцессорной машине). Это можно сделать, например, с помощью команды XCHG или LOCK CMPXCHG (в последнем случае необходимо обязательно использовать префикс LOCK и определить в программе директиву .486).

Если значение этой переменной равно 1, значит программа работает первый раз. Если значение этой переменной равно 0, была запущена вторая (или более) копия приложения. Для того, чтобы объявить некоторую секцию разделяемой, необходимо добавить к опциям компоновщика следующее:

```
link /section:<имя_разделяемого_сегмента>,RWS
```

Здесь ключевым является 'S' (SHARED) в списке атрибутов сегмента.

В приложении к этому письму Вы можете найти проект для Visual C++ 6.0 с соответствующим примером (я воспользовался Вашими рекомендациями для его создания). Текст программы с соответствующими комментариями находится в файле ms.asm. В каталоге Debug есть исполняемый файл ms.exe. Запустив его один раз и открыв Task Manager, мы можем найти Image Name ms.exe в списке Processes. При втором, третьем и т.д. запуске этой программы новых процессов ms.exe в этот список не добавляется. Если в программе заменить команду jnz на jmp и пересобрать проект, то при втором и т.д. запусках добавляются новые процессы ms.exe.

Обратите внимание, что запущенная программа будет работать бесконечно, поэтому, чтобы ее окончить, необходимо принудительно закончить соответствующий процесс с помощью Task Manager'a.

Работа программы проверена на двухпроцессорной машине под Windows NT 4.0.

_C уважением,
Геннадий Майко._

Вот исходный текст приложения (ms.asm):

```

; Простое приложение, которое может быть запущено только один раз.
; Однажды стартовав, работает бесконечно. Для завершения:
;   NT:   Откройте Task Manager. На вкладке Processes выберите ms.exe.
;         Нажмите кнопку End Process.
;   95/98: Нажмите Ctrl+Alt+Del. Выберите в списке приложение Ms.
;         Нажмите кнопку End Task.
; При построении проекта в командную строку link.exe следует
; включить опцию /SECTION:SHS,RWS

.386
.MODEL flat,stdcall

; Разделяемый сегмент
SHS SEGMENT
Cnt dd 1 ; Флаг: 1 - работает первый экземпляр приложения;
; 0 - запущен второй (и последующие) экземпляры.
SHS ENDS

; Код
.code
WinMain PROC PUBLIC hinst,prev_hinst,command_line,cmd_show
xor eax,eax ; eax = 0
xchg eax,Cnt ; eax <--> Cnt. Атомарный обмен eax и Cnt
or eax,eax ; проверить флаг
jnz L_Cont ; переход к коду для первого экземпляра

; Код для второго (и последующих) экземпляров

ret ; Завершение второго (и последующих) экземпляров

; Код для первого экземпляра приложения

L_Cont:
jmp L_Cont ; Бесконечный цикл
WinMain ENDP

end

```

Несколько комментариев от assembler.ru:

1. Опция командной строки компоновщика link.exe */SECTION:name,[E][R][W][S][D][K][L][P][X]* позволяет принудительно назначать атрибуты секциям PE-файла. В данном случае секции, образованной из сегмента SHS, устанавливаются атрибуты R (доступна для чтения), W (доступна для записи), S (разделяемая). Атрибут S означает, что все процессы, запущенные с помощью одного и того же исполняемого файла ("image" в терминах PE-файла), получают общий доступ к области памяти, содержащей переменную Cnt. Всякое изменение этой переменной одним процессом будет наблюдаться другими процессами.

Возможность объявлять секцию разделяемой, вероятнее всего, была заложена в архитектуру Windows именно с целью обеспечить взаимодействие экземпляров одного и того же приложения. Какое-либо иное применение этого механизма как-то не приходит в голову.

Ms.exe демонстрирует одну интересную особенность операционной среды. Как видим, разделяемая секция, содержащая инициализированные данные, при загрузке приложения инициализируется только в случае, когда загружается первый экземпляр приложения. При загрузке же последующих экземпляров инициализации переменных не происходит, а они получают значения, установленные к этому моменту предыдущими экземплярами приложения. Это логично, и странно было бы, если бы было по-другому, но как же все-таки производители операционных систем умудряются все предусмотреть?

2. Обсуждаемое Геннадием требование атомарности обращения программы к переменной Cnt, в целом справедливое для любых операций с разделяемой памятью, применительно к данному случаю, по нашему мнению, не является строгим.

Немного найдется пользователей, которые способны запустить два экземпляра приложения один за другим с достаточно малым интервалом времени так, чтобы они успели вступить в конфликт при обращении к переменной Cnt. Однако настоящий ассемблерщик, конечно же, обязательно поставит префикс lock.

Про префикс lock Геннадий представил следующее пояснение. Как известно, этот префикс имеет смысл в многопроцессорных системах. Он блокирует на время выполнения команды, перед которой стоит, доступ к памяти со стороны других процессоров. Так вот, команда xchg гарантированно формирует сигнал LOCK# вне зависимости от того, имеется или нет при ней префикс lock (см. "Intel Architecture Software Developer's Manual. vol.2: Instruction Set Reference"). А вот команда cmpxchg такого сигнала не формирует, поэтому в случаях, когда в многопроцессорных системах имеется опасность одновременного доступа нескольких процессоров к одним и тем же данным, применение префикса для нее обязательно.

3. Рассматриваемый способ обеспечения уникальности экземпляра приложения имеет органически присущее ему отличие от способов, основанных на средствах межпроцессного обмена. Если скопировать ms.exe в другую папку и попытаться запустить обе копии приложения, мы обнаружим, что обе они успешно запустятся, потому что это разные image'ы, и разделения секции SHS между ними не происходит.

В контексте стоящей задачи, конечно, это несущественно, так как даже в эпоху морального устаревания 8-гигабайтных бытовых винчестеров, опять же, весьма немногие пользователи увлекаются содержанием на диске нескольких копий приложений. Но все равно интересно.

4. Разделение сегмента между экземплярами приложения можно использовать не только для обеспечения уникальности экземпляров, но и для реализации сколь угодно сложного обмена информацией между ними (третья стратегия), ведь мы можем добавить в разделяемый сегмент любой набор необходимых нам для этого переменных. Это даже удобнее, чем File Mapping, так как не требует специальной поддержки программой, не связано с использованием файловой системы, и к этим переменным можно обращаться по именам, а не по смещениям.

А вот здесь уж точно без атомарности не обойтись. Точнее, без средств межпроцессного взаимодействия. Еще точнее - без средств синхронизации процессов. И уж совсем точно - без объектов mutex и/или event. Потому что, передавая друг другу данные, процессы должны обеспечить согласованный доступ к разделяемой памяти, то есть сообщать друг другу о готовности к передаче и приему данных и не допускать обращения к неготовым или изменяемым в данный момент данным.

В целом решение, предложенное Геннадием, следует признать очень интересным и совершенно функциональным. Еще раз большое ему спасибо.

Лептонный стиль программирования

Автор: Svet(R)off

Дата: 2002-06-14

ЧАСТЬ ПЕРВАЯ. ПОСТАНОВКА ЗАДАЧИ

Лептоны (от греч. *leptos*, что значит "легкий". Ср. "лепта" - мелкая разменная монета. Напр. "внести свою лепту" - сделать дела на копейку, а потом орать, что потратился на рубль) - это такие ма-а-асенькие элементарные частицы.

Существуют две теории лептонов: правильная и неправильная.

НЕПРАВИЛЬНАЯ ТЕОРИЯ ЛЕПТОНОВ

По признаку участия или неучастия в сильном взаимодействии все ма-а-асенькие элементарные частицы делятся на два вида: адроны (участвующие) и лептоны (не участвующие). Об адронах мы будем говорить, если вдруг придумаем какой-нибудь адронный стиль программирования.

Пока же не придумали, то вспомним только одно: в старинных русских дворянских родах существовала традиция называть мальчиков именем Адрон, которое должно было дать будущему воину особую жизненную силу. Возможно, таким образом еще в доядерную эпоху проявлялось интуитивное понимание истинного мироздания, суть коего - иррациональное слияние метафизических и материалистических концепций, свойственное исключительно русскому человеку.

Это богоданное свойство хорошо было наблюдать летом 2000 года в очереди к мощам Св.Пантелеймона в Храме Христа Спасителя в Москве. Бесконечной вереницей стояли русские люди по 12-15 часов в надежде приобщиться святости великомученика и целителя, и такие же русские люди бесконечно обходили стоящих, собирая деньги якобы на лекарства, надгробия и восстановление храмов, и такие же русские люди с дракой и бранью прорываются в храм без очереди.

Одним из самых известных Адронов наших дней является знаменитый кинорежиссер Андрон Михалков-Кончаловский, отпрыск древнего дворянского рода Михалковых, известных своей неумейной творческой предприимчивостью, близостью к царям и неиссякаемой мужской силой. Самый популярный фильм Михалкова-Кончаловского - "Танго и Кэш", шедший в прокате города Красноголозадовска (бывш. Бухаринград) под названием "Кто девушку платит, тот ее и танцует".

Лептонов известно науке всего двенадцать штук. Точнее шесть, а остальные шесть - это их античастицы: электрон, мюон, тау-лептон, и три нейтрино: электронное, мюонное и, как вы уже можете догадаться, тау.

Нейтрино - вообще очень интересная штука. Есть подозрения, что на самом деле Вселенная состоит в основном-то как раз из нейтрино. На миллиард нейтрино приходится одна какая-нибудь другая ма-а-асенькая элементарная частица. И основной вклад в мировую гравитацию вносят именно они - нейтрино. И если вдруг однажды мы доживем до Большого Чпока (процесса, обратного Большому Взрыву), то именно они будут в нем виноваты, свернув своей массой Вселенную обратно в сингулярность, из которой она когда-то родилась. И тогда у Кого-то на экран выскочит синяя маска смерти с предложением нажать Ctrl+Alt+Del. Но это будет уже совсем другая история.

Данная теория лептонов названа неправильной, потому что с точки зрения программирования она совершенно бесплодна.

ПРАВИЛЬНАЯ ТЕОРИЯ ЛЕПТОНОВ

Лептоны - это элементарные частицы мировой информации. Они принципиально неуловимы никакими физическими приборами. Скорость света - это недостижимая для них минимальная скорость, а обычно они движутся гораздо быстрее. Лептонное поле хранит информацию о всех прошедших, настоящих и будущих событиях в любой точке пространства-времени, включительно от чиха простудившегося этим летом микроба до мановения пальца Любимого Руководителя Товарища Ким Чен Ира, указывающего Владимиру Владимировичу направление движения России в сторону окончательной победы идей Чучхе.

В указанном промежутке лежат также разработка ОС Windows, грядущее столкновение нашей Галактики с галактикой Андромеды, передача НТВ Газпрому за долги, хрущовские испытания 50-мегатонной бомбы на Новой Земле, день рождения в прошлую субботу, раскаяние в прошлое воскресенье и многие другие события, которые здесь следовало бы перечислить, но неохота.

Единственный прибор, способный взаимодействовать с лептонами - это человеческая душа, поскольку сама она суть лептонный сгусток. Все, что для нас проявляется как интуиция, озарения, идеи - это тривиальные результаты взаимодействия души с лептонным полем (см., например, песню группы "Любэ" Улочки Московские, где есть такие пророческие слова: "Эх, сегодня я, кажись, надерусь"). Это, так сказать, произвольные формы взаимодействия. Существуют также и произвольные формы, то есть взаимодействия, совершаемые человеком сознательно: молитва, медитация, камлание, гадание, транс, сеанс Кашпировского, выборы президента и пр. Как правило, целью произвольных форм взаимодействия является получение из лептонного поля интересующей информации или попытка изменить его конфигурацию желательным для себя образом.

Здесь мы прекратим рассмотрение теорий лептонов, потому что все, что нужно, уже выяснили, и перейдем к практике программирования.

Среди традиционных проблем ассемблерного программиста связывание занимает далеко не последнее место.

Поясняю. Всякий сколько-нибудь значительный проект обычно содержит более одного модуля, то бишь файла с исходным текстом, по той простой причине, что будучи запихнутым в один файл, проект становится неуправляемо громоздким. Модуль еще иногда называют единицей трансляции, имея в виду то обстоятельство, что обрабатывая его и превращая в объектный файл, компилятор ни сном, ни духом не ведает о существовании других модулей. Своевременное и логически обоснованное разбиение проекта на модули столь важно, что некоторые авторы даже склонны рассматривать модульность как отдельную парадигму программирования, наряду с хаотической, процедурной, структурной и объектно-ориентированной (Собоцинский В.В. Практический курс Турбо С++. М, "Свет", 1993).

Вообще-то понятие "модуль" достаточно широкое и многозначное. Но в этой статье мы будем иметь ввиду именно модуль как единицу трансляции, то есть самостоятельный asm-файл, содержащий исходный текст, который может быть откомпилирован без ошибок.

Модули должны взаимодействовать друг с другом, иначе программа не окажется единым целым и не будет работать. Взаимодействовать они могут одним из двух способов:

1. Команды, содержащиеся в одних модулях, обращаются к переменным, описанным в других модулях, то есть читают или пишут данные в память, именованную этими переменными
2. Команды call, содержащиеся в одних модулях, вызывают процедуры, тела которых содержатся в других модулях.

Возможен еще такой способ взаимодействия, как передача управления из одного модуля в другой командой `jmp` или ее условными формами (`je`, `jb`, `js` и пр.). Но, если только программист не пытается реализовать какую-нибудь специальную идею, применение этого способа следует считать проявлением запредельного случая неправильного стиля программирования.

В Windows существует еще один способ взаимодействия модулей: когда из одних модулей посылаются сообщения окнам, созданным в других модулях. Но об этом ниже.

Стоит проблема: как позволить командам из одного модуля обращаться к данным, описанным в других модулях. Или как из одного модуля вызывать процедуры, тела которых содержатся в других модулях. Решение этой проблемы как раз и называется связыванием.

Родной для ассемблерных программ способ связывания обеспечивается применением сладкой парочки директив `PUBLIC` и `EXTRN` (она же `EXTERN`). Директива `PUBLIC` объявляет имя переменной или процедуры доступным для обращения из других модулей. Директива `EXTERN` сообщает компилятору, что команды данного модуля могут обращаться к имени переменной или процедуры, описанной в другом модуле.

Например:

в модуле `masha.asm` содержится такой фрагмент кода:

```
PUBLIC mashin_vozrast
.data
mashin_vozrast dd 17
```

а в модуле `vasya.asm` содержится такой фрагмент кода:

```
EXTRN mashin_vozrast:dword
.code
;...
cmp mashin_vozrast,18
;...
```

Здесь дан наиболее часто применяемый вариант директив `PUBLIC` и `EXTRN`. На самом деле их возможности несколько шире, но они не являются предметом этой статьи. Читайте документацию на соответствующий вашим предпочтениям ассемблер.

Обратите внимание, что переменная `mashin_vozrast` объявлена как двойное слово, что вполне естественно для программирования под `win32`, даже при том условии, что Маша вряд ли достигнет возраста 4294967296 лет, сколько бы здоровья мы ей ни желали. Настоятельно рекомендуется в подобных случаях (когда числовая переменная объявлена как одиночная, а не как массив) всегда использовать переменные размером в двойное слово, потому что обращение к таким переменным из 32-разрядного кода существенно более компактно, чем к переменным размером в слово или байт.

Как видим, Вася пытается оценить, насколько близкими могут быть его отношения с Машей, чтобы не загреметь по очень неприятной статье. Его, вероятно, ждет разочарование, но только не на этапе компиляции, потому что компилируется он программой-компилятором (например, `ml.exe`) в одиночку, без участия предмета своих вожделений.

Связыванием же занимается компоновщик, он же линкер, он же `link.exe`. Это его основная задача, откуда и произошло его название. Схватив полученные в результате двух актов компиляции объектные файлы `masha.obj` и `vasya.obj`, он обнаруживает, что Маша публично заявляет о своем возрасте (впрочем, 17-ти лет стесняться не приходится), а Вася внимательно прислушивается. Компоновщик удовлетворяет обоим, выделяя в исполняемом модуле `sex.exe` место под переменную размером в двойное слово, инициализирует ее числом 17, а в команду `cmp`, с помощью которой Вася интересуется Машиным возрастом, подставляет адрес этой переменной.

Связывание вызовов процедур выглядит ненамного сложнее:

модуль masha.asm:

```
PUBLIC get_masha
.code
get_masha PROC
;...
ret
get_masha ENDP
```

модуль vasya.asm:

```
EXTRN get_masha:near
.code
;...
call get_masha
;...
```

Обратите внимание, что процедура `get_masha` объявлена как ближняя. В win32, поскольку дело происходит в плоской (flat) модели памяти, все процедуры ближние.

Как видим, Вася добрался-таки до Маши. Вероятно, в какой-то момент переменная `mashin_vozrast` инкрементировалась, или любовь оказалась сильнее условностей. Но так или иначе связывание состоялось.

Сказанное выше описывает стандартный для ассемблера механизм связывания на этапе компоновки приложения. По сути своей это так называемое раннее связывание, то есть связывание, выполняемое во время разработки программы. В отличие от позднего, используемого, например, в C++ для виртуальных функций, что является основой для реализации полиморфизма - одного из столпов объектно-ориентированного программирования.

Есть ли у этого метода недостатки? Да до... в общем, вам по пояс будет.

Представьте себе, что вы коренной русак без малейшей примеси немецкой крови. Тогда, видимо, вы вряд ли являетесь поклонником такого понятия, как инкапсуляция, и глобальных переменных в ваших программах - как блох на мамонте. А каждая глобальная переменная - это одна директива `PUBLIC` и энное количество директив `EXTRN`, по одной на каждый модуль, из которого на нее ссылаются. Поменяли имя переменной - будьте любезны править все модули. А если у вас, допустим, сто глобальных переменных? Не слабо?

А даже если вы и наоборот, запердельный педант, и пипифакс рвете исключительно по перфорации, и объектная ориентация для вас важнее, чем сексуальная? Тогда, вероятно, глобальных переменных в ваших программах нет совсем, а межмодульный обмен данными вы обеспечиваете с помощью специальных интерфейсных процедур, на манер публичных функций-членов классов в C++. Ну и что, намного легче вам иметь длиннющие списки директив `PUBLIC` и `EXTRN` для процедур, чем для переменных?

Возрадуйтесь! Ваши мучения закончились. С сегодняшнего дня вы можете забыть не только синтаксис директив `PUBLIC` и `EXTRN`, но и их названия, потому что они вам больше не понадобятся. На сцену выступает лептонный стиль программирования.

Отвечая на первый попавшийся FAQ, объясним сразу, при чем же здесь все-таки лептоны и почему стиль назван так претенциозно, если не сказать помпезно.

Все началось несколько лет назад, когда мы только-только начинали программировать под Windows. Почти сразу выяснив, что одно из главных достоинств ассемблерных программ - максимальное быстродействие - уже больше не является безусловным благодаря многозадачной архитектуре операционной системы, мы сначала опечалились, а потом, подумав, решили, что это даже хорошо. Можно расслабиться, перестать высчитывать

загрузку конвейера процессора и позволить себе некоторые излишества: размочить позавчерашний сухарь водичкой, постирать носовой платок, не буферизировать файлы размером меньше кластера, заглянуть на Интердаму, наконец.

Особенно большое впечатление произвел механизм сообщений Windows. Допустим, нажимаете вы, не подумав, мышкой на кнопку в диалоговом окне. Внешне все происходит просто: нажимается кнопка "Нет" в диалоге "Хотите ли вы сохранить документ, над которым вы работали последние четыре часа"; вы страшно богохульничаете, плюете в монитор и с горя отправляетесь в магазин. Вот и все, и большую часть из этой процедуры вы выполняете вообще без помощи компьютера.

А на самом деле? На самом деле все ужасно сложно и долго. Драйвер мыши обнаруживает судорожное движение вашего пальца. Потом он интересуется местоположением курсора на экране. Полученные координаты он отправляет операционной системе на предмет определения окна, над которым в данный момент находится курсор. А окон открыта целая туча: всякие там кнопочки-шмопочки, документики-шмокументики, списочки-шмисочки. И все перекрываются, и каждое норовит чего-нибудь обработать. И вот операционная система перебирает их все и находит то, которое в точке курсора лежит поверх всех остальных. И отправляет ему сообщение. Методом Post между прочим, то есть ставя его в очередь, а не вызывая непосредственно оконную процедуру. Сообщение отправляется родительскому окну и торчит там в очереди, пока не придет пора его обработать. Кроме того, поскольку дело происходит в диалоговом окне, кнопочка еще и перерисовывается, отображая нажатое состояние. А потом еще удаляется с экрана диалоговое окно, и перерисовывается то окно, в котором находился ваш документ, а потом удаляется и оно, и перерисовываются все лежавшие под ним другие окна, при этом тоже не особенно экономя процессорное время и память. И все это сопровождается передачей такого количества сообщений, что Смольный в ночь с 24 на 25 октября (с.с.) 1917 года по сравнению с этим гвалтом покажется вам санаторием ЦК КПСС во время послеобеденного тихого часа, когда пациенты переваривают телятину по-кабардински под белым соусом в сопровождении нежных трапецеидальных импульсов стимулирующих перистальтику кремлевских таблеток. Короче, рассмотрев эту предельно упрощенную схему действий, мы понимаем, что вместо того, чтобы просто уничтожить все ваши труды, ОС сама долго и плодотворно трудится.

И что же, среди этого разврата мы будем экономить, отказывая себе во всем, даже в полкило красной икры на завтрак?

Да никогда!

Мы построим свою программу из модулей. Каждому модулю мы поручим выполнение какой-нибудь конкретной задачи в рамках общего дела. Один пусть занимается работой с настройками: содержит информацию об активных панелях управления, составе кнопок на них и пр., и предоставляет эту информацию другим модулям по их запросам. Другой модуль пусть обеспечивает сохранение состояния приложения в период между его запусками: запоминает размер и положение окон, имена редактировавшихся документов, списки изменяемых данных и еще что-нибудь. И тоже предоставляет эту информацию другим модулям по запросам. Третий модуль пусть обслуживает строку статуса: инициализирует ее, разбивает на поля, показывает в них то или иное содержимое по запросам других модулей. Четвертый модуль пусть предоставляет текстовые строки на выбранном пользователем языке. Пятый модуль...

Впрочем, достаточно. И так уже ясно, что главным элементом архитектуры нашего приложения является обмен запросами и ответами модулей между собой. По сути, модули выступают друг для друга в качестве так любимых всеми современными разработчиками клиентов и серверов. Клиенты хотят иметь информацию или требуют выполнения каких-то действий, и поэтому запрашивают соответствующий сервис у серверов. Сервера выполняют запросы клиентов, не допуская, между прочим, обслуживания некорректных запросов. (Например, некий модуль отвечает за хранение

глобальных констант. Следовательно, он должен выдавать значения констант другим модулям, но не должен предоставлять им никаких средств для изменения этих значений.)

Таким образом, общая идея довольно проста и даже тривиальна. И уж конечно, в той или иной степени она реализуется практически в каждой более-менее серьезной программе.

Отличие же лептонного стиля состоит в том, что запрашивающий модуль-клиент не обязан знать, и не может знать, и вовсе даже не интересуется, кто из существующих модулей-серверов обслуживает его запрос, и вообще, существует ли такой сервер.

В этом - квинтэссенция лептонного стиля. Клиент отправляет свой запрос в пространство, не адресуясь ни к кому конкретному, наподобие молитвы всем богам. "Эй, кто-нибудь, кто может, ответьте, пожалуйста, какова сейчас ширина главного окна приложения?" - как бы кричит в эфир модуль, создающий, например, подчиненное окно, занимающее одну восьмую часть главного окна по горизонтали. И получает в ответ неизвестно от кого: "720 пикселей, дружище! Пользуйся на здоровье!".

Теперь ответим на второй первый попавшийся FAQ. Тянет ли эта идея на понятие "стиль программирования"? По-нашему, тянет. Ведь что есть стиль программирования? Это более-менее стабильный набор приемов организации программы и оформления исходного текста, используемый программистом с целью облегчения ее разработки и понимания. С точки зрения этого определения то, о чем мы сейчас говорим - именно и есть стиль. Стабилен ли он? Да вы только попробуйте его хотя бы разок, и обратно уже не запроситесь. Идет ли в данном случае речь об организации программы? Безусловно: и с формальной точки зрения (отсутствие директив PUBLIC/EXTRN), и с фактической (организация специальной системы обмена сообщениями внутри программы) - это инструмент организации программы. Идет ли в данном случае речь об оформлении исходного текста? Конечно, да. Далее мы покажем, как с помощью макросов удобно и наглядно записываются запросы клиентов.

Первая реализация поддержки лептонного стиля программирования была прямо выполнена на штатных средствах обмена сообщениями Windows. До сих пор еще некоторые наши программы используют ее. Общая идея там была такова: при запуске приложения создавалось невидимое окно, называемое нами супервизором, дескриптор которого объявлялся глобальным для всего проекта. Оконная процедура супервизора, таким образом, становилась доступной из любой точки программы для передачи (SendMessage) или посылки (PostMessage) пользовательских сообщений, то есть относящихся к диапазону WM_USER (00400h...07fffh) и/или диапазону WM_APP (08ffffh...0bffffh):

```
invoke SendMessageA,hWnd,Msg,wParam,lParam
```

В каждом модуле объявлялась как PUBLIC одна-единственная процедура, названная нами диспетчером. Упрощенно система работала так:

1. если в каком-то месте программы требовалось обратиться к лептонному сервису, вызывалась функция API SendMessage (реже, при необходимости - PostMessage), при этом в качестве окна-получателя (hWnd) указывался супервизор. Сообщение (Msg) определяло смысл запроса, а два параметра (wParam и lParam) - его параметры;
2. оконная процедура супервизора, получив это сообщение, поочередно транслировала его всем доступным в приложении диспетчерам модулей, передавая им параметры, например, через стек;
3. очередной диспетчер модуля, получив вызов от супервизора, определял, обрабатывается ли он этим модулем. Если не обрабатывается, то модуль возвращал управление супервизору с признаком "не обработано". Если обрабатывается, то передавал его на обработку и, по ее завершении, возвращал ответ в регистре eax с признаком "обработано";

4. супервизор, получив от какого-нибудь очередного модуля ответ с признаком "обработано", прекращал опрос модулей и завершал оконную процедуру, возвращая ответ в регистре `eax`;
5. запрашивавший модуль считывал ответ из регистра `eax`.

Как параметры (`wParam` и `lParam`), так и ответ (`eax`) могли представлять собой непосредственные данные, если они помещались в размер двойного слова, либо являться адресами (указателями) блоков памяти (структур, строк), в которые модуль-клиент и/или модуль-сервер помещали соответствующие данные. В каждом конкретном случае это определялось смыслом запроса, то есть значением сообщения (`Msg`).

У этой схемы, при ее общей очевидной работоспособности, имелись несколько неприятных недостатков (мы настаиваем на этом определении, ибо всякий поживший на свете человек знает, что некоторые недостатки иногда бывают очень даже приятны):

1. для своих внутренних утилитарных целей мы задействуем весь монструозный механизм сообщений Windows. (Васин элементарный запрос `MASHA_GOTO_POGULYAT_NA_CHERDAK`, не сомневайтесь, будет в подробностях обсосан и переварен всем двором и уж точно не минует машинного батяню-шоферу с вот такенными кулачищами.) Конечно, душа настоящего ассемблерщика станет невыносимо страдать при каждом таком запросе. А вы представляете, если запросы попрут, допустим, по сто штук в секунду? Прямая дорога либо к инфаркту миокарда, либо к циррозу печени;
2. очередь оконных сообщений начинает жить только после создания окна супервизора, когда заработает цикл `GetMessage-TranslateMessage-DispatchMessage`. А между тем, очень часто до того хочется провести некоторое количество операций по инициализации приложения. И очень хочется выполнить их тоже в лептонном стиле. Почему бы и нет?
3. два входных параметра - это, конечно, больше, чем ни одного, но заметно меньше, чем три или более. То есть маловато будет в весьма многих случаях. А ничего не поделаешь: таков формат системного сообщения Windows. Можно, конечно, дополнительно использовать регистры `esi` и `edi`, которые, как известно, сохраняются функциями API, но это откровенный паллиатив;
4. параметры передаются через стек, и поэтому их всегда два. Даже если не нужно ни одного. Налицо преступное разбазаривание кода, стека и процессорного времени;
5. 48128 сообщений, зарезервированных в диапазонах `WM_USER` и `WM_APP` - это не так уж много, неправда ли? Всего каких-нибудь 20 лет работы над программой, и вы их исчерпаете. Что будете делать тогда? Куда лучше было бы иметь в запасе 232 сообщений. Тогда можно было бы более-менее спокойно работать чуть более 1784810 лет, что в большинстве случаев следует признать достаточным.

Поэтому мы не станем рассматривать здесь вариант реализации лептонного стиля на базе системы сообщений Windows, а сразу перейдем к более прогрессивному - автономному варианту. Его отличительной особенностью является применимость не только под Windows, а в любой ассемблерной программе, в том числе под DOS, Linux и пр. Если он вам понравится, вы можете использовать приведенный здесь код один к одному. Он вполне работоспособен и достаточно функционален. Но еще правильнее воспринимать его как набор идей, которые вы свободно можете развить и дополнить так, как посчитаете нужным. Или, например, перевести на другой язык программирования.

Реализация и заключительные замечания - во второй части статьи.

ЧАСТЬ ВТОРАЯ. РЕАЛИЗАЦИЯ

В реализации задействованы файлы:

- `@struct.inc` - файл глобальных макросов. Это макросы, с помощью которых программист улучшает свою рабочую среду. каждый может придумать их великое множество. Некоторые из используемых нами вы можете найти в проекте `MyCall`. Важно, что этот файл следует включать во все модули

проекта.

- `globals.inc` - файл глобальных констант проекта. В нем вы будете вести список констант, представляющих молитвы (лептонные вызовы). Он также должен быть включен во все модули проекта, что позволит выполнять любой лептонный вызов из любой точки проекта. Впрочем, возможен вариант, когда молитвы могут быть разбиты на группы по назначению. В таком случае все группы молитв должны быть доступны только супервизору, а каждому из модулей - только те группы, молитвы из которых используются в этом модуле. Такая организация потребует от программиста дополнительных накладных расходов на администрирование. В то же время заманчивым может оказаться сокращение времени на компиляцию проекта. Ведь если файл `global.inc` один на всех, то при добавлении в него каждой новой молитвы (то есть довольно часто) автоматически будет вызываться полная перекомпиляция всех модулей проекта (если, конечно, вы правильно настроили MS DevStudio). Выбор - за вами. Со своей стороны скажем, что по причине врожденной лени ни разу не пытались разбивать молитвы на группы. На современном компьютере MASM работает достаточно быстро, можно и потерпеть;
- `00_main.asm` - главный модуль проекта - тот, который содержит процедуру `WinMain` (см. также статью Минимальное приложение). У нас он теперь будет содержать еще и супервизор;
- `XX_*.asm` - все остальные модули проекта. Здесь `XX` - двухсимвольный идентификатор модуля, в котором каждый символ `X` может быть цифрой или буквой латинского алфавита, а `*` - произвольный набор символов, смысл которого нужен человеку, но не компилятору (например: `01_main_window.asm` - модуль обслуживания главного окна). Каждый такой модуль, если только он собирается выступать в качестве лептонного сервера для молитв других модулей, обязан иметь в своем составе процедуру диспетчера.

Ниже приведены фрагменты кода, включаемые в перечисленные файлы и обеспечивающие поддержку лептонного стиля программирования, и комментарии к ним.

Файл глобальных макросов `@stuct.inc`:

```
;----- эквиваленты параметров молитвы
@par0 EQU esi
@par1 EQU edi
@par2 EQU edx
@par3 EQU ecx
;----- макрос молитвы
@pray MACRO pray,par0,par1,par2,par3
    IFNB
        mov esi,par0
    ENDIF
    IFNB
        mov edi,par1
    ENDIF
    IFNB
        mov edx,par2
    ENDIF
    IFNB
        mov ecx,par3
    ENDIF
    invoke supervisor,pray
ENDM
;----- макрос формирования списка диспетчеров
@dispatchers MACRO
    IFDEF MODULES
        @id_offset=1
        @id_size SIZESTR MODULES
        :next
        IF @id_offset GT @id_size
            EXITM
        ENDIF
        @module_id SUBSTR MODULES,@id_offset,2
        @module_name CATSTR ,@module_id
        @module_name PROTO :DWORD
        dd offset(@module_name)
    ENDIF
ENDMACRO
```

```

        @id_offset=@id_offset+3
        GOTO next
    ENDF
ENDM
;----- директивы определения модели
.386
.Model flat,stdcall
;----- прототип супервизора
supervisor PROTO :DWORD
;----- идентификатор модуля
@module SUBSTR @FileName,1,2
;----- конструкции, зависящие от модуля
IFIDN @module,<00> ;если это модуль супервизора
.const ;создать список диспетчеров
    dispatchers dd 0 dup(0)
                @dispatchers ;вызов макроса формирования списка диспетчеров
                dd 0
ELSE ;если это обычный модуль
    @dispatcher CATSTR ,@module ;сконструировать имя диспетчера
ENDIF
ENDIF

```

Пояснения:

1. Эквиваленты параметров молитвы используются внутри диспетчеров (см.ниже в файле XX_*.asm). Они нужны для того, чтобы у программиста не было необходимости помнить, какой параметр передается в каком регистре. Символ "@" напоминает программисту о том, что это регистры, а не адреса памяти.
2. Макрос молитвы используется для формирования лептонных вызовов из программы - молитв и бродкастов, например:

```
@pray P_GET_STRING,offset(string_buffer),buffer_size,string_id
```

Этот текст после компиляции будет преобразован в последовательность команд:

```

mov esi,offset(string_buffer)
mov edi,buffer_size
mov edx,string_id
push P_GET_STRING
call supervisor

```

Видно, что в отличие от описанной выше реализации на основе средств обмена сообщениями Windows, этот код оптимален: кодируются только те параметры, которые используются. Другие примеры молитв и бродкастов:

```

@pray B_START,hinstance
@pray P_SET_MAIN_WINDOW_TITLE,offset(main_window_name)
@pray P_STORE_STATUS,
    STATUS_MAIN_WINDOW_POSITION,
    offset(main_window_rectangle),
    show_status
@pray B_STOP

```

3. Макрос формирования списка диспетчеров используется только один раз - при компиляции главного модуля. На основании списка идентификаторов модулей MODULES (см. ниже в файле 00_main.asm) он создает в константном сегменте приложения таблицу адресов процедур диспетчеров всех модулей. Таблица завершается нулевым значением. При каждом лептонном вызове супервизор поочередно передает управление по адресам из этой таблицы, вызывая, таким образом, диспетчеры модулей.
4. Прототип супервизора обеспечивает связывание лептонных вызовов из модулей с процедурой супервизора, описанной в главном модуле 00_main.asm.
5. Идентификатор модуля представляет собой два первых символа имени asm-файла. Можно придумать и другие варианты, но нам показался привлекательным этот. Он удобен еще и тем, что в панели Workspace рабочей среды MS Developer Studio именованные таким образом файлы располагаются в строгом порядке, так как происходит их сортировка по имени.

6. В зависимости от того, какой модуль приложения компилируется - главный или обычный, - в нем либо создается список диспетчеров, либо конструируется имя диспетчера. Имя диспетчера, будучи глобальным, должно быть уникально в пределах проекта. Поэтому оно имеет вид "dispatcher_XX", где XX - идентификатор данного модуля.

Файл глобальных констант globals.inc:

```
;----- молитвы
P_BASE    =0 ;базовый номер молитв
P_HINSTANCE =P_BASE+0
P_MAIN_WINDOW =P_BASE+1
P_GET_STRING =P_BASE+2
P_GET_MAIN_WINDOW_SIZE =P_BASE+3
P_SET_BACKGROUND_COLOR =P_BASE+4
;----- бродкасты
B_BASE    =P_BASE+100 ;базовый номер бродкастов
B_START   =B_BASE+0
B_STOP    =B_BASE+1
B_MAIN_WINDOW_SIZED =B_BASE+2
```

Пояснения:

1. Возможны два варианта лептонных вызовов: молитвы и бродкасты. Молитвы применяются в случаях, когда требуется получить некий конкретный сервис (например, текстовую строку). Бродкасты нужны для того, чтобы оповещать о каких-то событиях всех, кого это может заинтересовать (например, об изменении размеров главного окна приложения). При обработке молитвы каким-либо из модулей опрос диспетчеров прекращается, и супервизор завершает свою работу. Бродкаст же передается поочередно всем диспетчерам без исключения.
2. Видно, что идентификаторы молитв и бродкастов - это просто 32-разрядные значения в непересекающихся диапазонах. Программист должен обеспечить это требование правильной установкой базовых номеров.
3. Здесь приведены примеры некоторых часто встречающихся молитв. P_HINSTANCE - получение дескриптора экземпляра приложения. P_MAIN_WINDOW - получение дескриптора главного окна. P_GET_STRING - получение текстовой строки из единого хранилища строк (удобно в локализуемых приложениях).
4. Особо следует остановиться на организации обработки молитв, похожих на P_GET_MAIN_WINDOW_SIZE. Очевидно, что возвращаемое значение должно иметь тип RECT, то есть не помещается в регистр eax. Возможны два варианта решения этой проблемы. Либо при вызове этой молитвы в одном из ее параметров передается адрес памяти для приема значения RECT. В этом случае ответственность за выделение этой памяти несет модуль-клиент. Либо наоборот, память под переменную RECT выделяется модулем-сервером, и тогда клиенту возвращается ее адрес в регистре eax. И тот, и другой варианты имеют право на существование. Программист должен сделать свой выбор, создавая обработчик молитвы, исходя из ее назначения и условий использования.
5. Бродкасты B_START и B_STOP практически необходимы в любом сколько-нибудь развитом приложении. Как следует из их имен, они предназначены для запуска приложения (т.е. инициализации модулей-серверов) и его завершения (освобождения ресурсов, занятых серверами). B_START может вызываться, например, сразу из процедуры WinMain. А B_STOP - по команде пользователя на завершение работы приложения.
6. Для бродкаста B_START (в первую очередь для него, но иногда и для других) возникает одна интересная проблема: в какой очередности он должен обрабатываться модулями? Например, при инициализации сервера главного окна требуется, чтобы уже была доступна строка, представляющая его имя, то есть уже был проинициализирован сервер строк. В принципе, эту проблему можно было бы решить, тасуя идентификаторы в списке MODULES в файле 00_main.asm. Однако правильная последовательность не всегда очевидна, да и не обязательно должна быть одинаковой для разных бродкастов, а если она установлена таким образом, то уже

останется неизменной навсегда. Поэтому следует ориентироваться на другой способ. Зависимые модули должны инициализироваться не бродкастом B_START, а тем модулем, который требует их готовности к моменту своей инициализации. Для этого можно использовать специальную молитву, например, P_START_STRINGS. На первый взгляд, описанный механизм является отступлением от лептонного стиля, который предполагает взаимную независимость модулей. Однако на практике он не вызывает проблем, так как используется в особых, крайне редких случаях, и достаточно прозрачен.

Файл главного модуля 00_main.asm:

```
;----- список идентификаторов модулей
MODULES EQU <01,02,03,04>
;----- include-файлы
include @struct.inc
include windows.inc
include globals.inc
;...
.code
;...
;----- супервизор
supervisor PROC USES ebx ecx edx esi edi pray
    mov eax,pray
    ;..... молитвы главного модуля
    @if(eax==P_HINSTANCE)
        invoke GetModuleHandleA,NULL
    @elseif(eax==P_MAIN_WINDOW)
        mov eax,main_window
    @else
        ;..... опрос диспетчеров модулей
        mov ebx,offset(dispatchers) ;ebx - указатель в списке диспетчеров
        @while(dword ptr[ebx]!=0)
            push pray
            call dword ptr[ebx] ;вызов очередного диспетчера
        @if(pray
```

Пояснения:

1. Здесь и далее применена транскрипция структурных директив MASM (.if, .while и пр.) с лидирующим символом "@" вместо точки. Обоснование, как и почему это сделано, можно прочитать в статье @struct.inc для MyCall.
2. Список идентификаторов модулей используется макросом @dispatchers (см. выше в файле @struct.inc) на этапе компиляции для формирования списка диспетчеров, который опрашивается супервизором при каждом лептонном вызове. Программист должен, включив в проект новый модуль, дополнить список модулей его идентификатором, иначе супервизор не сможет вызвать диспетчера этого модуля.
3. Супервизор - это очень простая процедура. Она состоит из двух независимых частей. В первой части выполняется обработка молитв (но не бродкастов!), которые по замыслу программиста должен обрабатывать главный модуль приложения. При обработке таких молитв используется только первая часть процедуры. Вторая часть - это простой цикл, сканирующий таблицу диспетчеров. Производится поочередная выборка из таблицы адресов процедур диспетчеров и передача им управления с трансляцией параметров, находящихся в регистрах esi, edi, edx, ecx.
4. Когда супервизор обслуживает молитву, цикл опроса диспетчеров продолжается до тех пор, пока какой-нибудь из них не вернет в регистре eax ненулевое значение, что является признаком "обработано". При обслуживании бродкастов такая проверка не выполняется, поэтому супервизор обязательно перебирает всех диспетчеров.
5. Следует иметь в виду, что в случае, когда какая-нибудь молитва не обработана ни одним из модулей приложения, супервизор, опросив всех диспетчеров, возвращает в регистре eax значение 0.
6. Супервизор сохраняет с помощью атрибута USES директивы PROC значения всех регистров, за исключением регистра eax.

Файл модуля XX_*.asm:

```
;----- include-файлы
include @struct.inc
include windows.inc
include globals.inc
;...
.code
;...
;----- диспетчер
@dispatcher PROC USES ebx ecx edx esi edi pray
    mov eax,pray
    ;..... обработка бродкастов
    @if(eax==B_START)
        ;...
    @elseif(eax==B_STOP)
        ;...
    ;..... обработка молитв
    @elseif(eax==P_GET_STRING)
        mov buffer_address,@par0 ;пример использования параметров молитвы
        mov buffer_size,@par1
        mov id,@par2
        ;...
        jmp ok
    ;..... завершение диспетчера
    @endif
nok:    ;не обработано
    xor eax,eax
ok:     ;обработано
    ret
@dispatcher ENDP
```

Пояснения:

1. Диспетчер - это процедура, которая содержится в каждом модуле, обрабатывающем лептонные вызовы - молитвы или бродкасты. Идея диспетчера очень проста: сравнение параметра pray со списком лептонных вызовов, обрабатываемых данным модулем. Если полученный лептонный вызов обрабатывается, то после его обработки диспетчер возвращает управление супервизору с ненулевым (полученным в результате обработки) значением в регистре eax (выполняется выход на метку ok:). В противном случае регистр eax сбрасывается в 0 (выполняется выход на метку nok:). Что касается бродкастов, то все равно, на какую метку передается управление по завершении их обработки.
2. Имя диспетчера, поскольку оно глобально в пределах исполняемого модуля, должно быть уникальным. А диспетчеров у нас - по одному на каждый модуль. Во избежание конфликтов здесь применен следующий прием. Имя @dispatcher на самом деле является вызовом макроса, описанного в файле @struct.inc (см.выше). Этот макрос формирует действительное имя процедуры вида dispatcher_XX, где XX - идентификатор модуля.
3. Входными значениями для диспетчера являются передаваемый через стек идентификатор лептонного вызова (pray) и до четырех параметров, передаваемых через регистры: @par0 (он же регистр esi), @par1 (edi), @par2 (edx), @par3 (ecx). Эквиваленты параметров определены в файле @struct.inc.
4. Использовать в качестве параметров действительные имена регистров или их эквиваленты - зависит от предпочтений программиста. Применение эквивалентов, освобождая программиста от необходимости помнить то, в каком регистре передается какой параметр, одновременно может явиться источником тяжелых ошибок, если программист вздумает вдруг воспользоваться эквивалентами регистров в качестве входных параметров диспетчера после того, как что-нибудь с этими регистрами уже поделает.
5. Можно придумать много различных вариантов организации диспетчера, оптимизирующих как использование памяти, так и время выполнения (если это имеет смысл). Здесь показан простейший вариант, основанный на применении директив @if-@elseif-@endif.

6. Показанное здесь разбиение процедуры диспетчера на части (обработка бродкастов и обработка молитв) условно. На самом деле очередность выполнения сравнений, конечно же, смысла не имеет.

7. Диспетчер, также как и супервизор, сохраняет значения всех регистров, за исключением регистра `eax`. Это помогает уберечься от труднообнаруживаемых ошибок, связанных со случайным изменением содержимого регистров в процессе опроса диспетчеров.

В заключение несколько дополнительных замечаний:

- легко видеть, что деление лептонных вызовов на молитвы и бродкасты весьма условно. Можно обойтись одними молитвами, если в отношении тех молитв, которые должны выполнять роль бродкастов, аккуратно выполнять правило: каждый диспетчер должен возвращать нулевое значение;
- использование нулевого значения регистра `eax` в качестве признака "не обработано" может вызвать возражения. Как в таком случае отличить случай "не обработано" от случая, когда результат обработки равен 0? Однако на самом деле это никакая не проблема. Во-первых, именно так работают большинство функций API win32. Во-вторых, если есть необходимость, можно придумать множество разных средств для разрешения этой ситуации. Например, ввести молитву `P_GET_LAST_ERROR`, по принципу использования аналогичную функции `GetLastError` API win32. Молитва должна возвращать значение, соответствующее последней ошибочной ситуации. А уж установить это значение в случае, если супервизор безуспешно просканировал весь список диспетчеров - не проблема;
- предложенная технология может служить основой для создания библиотек молитв. Во всяком случае наш опыт показал, что, раз встав на эту скользкую дорожку, сойти с нее оказывается очень трудно. И вот уже кочуют из проекта в проект модули, внутреннее устройство которых давно забыто, а для встраивания в приложение вполне хватает интерфейса, представленного набором молитв;
- небольшая модификация лептонной идеи легко расширяет ее для реализации в проектах, использующих собственные dll-библиотеки. При этом появляются молитвы вроде `P_LOAD_DLL` и `P_FREE_DLL`, назначение которых очевидно, но имеется одна особенность. Следует предусмотреть несложный механизм, включающий диспетчеров модулей, загруженных в виде dll-библиотек, в список диспетчеров, сканируемый супервизором. Сопутствующие мелкие проблемы (вроде инициализации загружаемых dll-библиотек, которым не довелось присутствовать при прохождении бродкаста `B_START`) вполне решаемы.

Как писАть на MASM в строчку

Автор: Svet(R)off

Дата: 2002-06-14

В этой маленькой статье речь идет совсем не о том, что TASM лучше MASMa, как могут подумать те, кто ошибся в постановке ударения при прочтении названия статьи, а о том, как записывать последовательности команд в исходном тексте в одну строку.

Настоящим настоящим ассемблерщикам рекомендуется непосредственно перед прочтением нижележащего крамольного еретического текста выполнить следующие операции:

1. Взять 20 капель спиртовой настойки валерианового корня (Tinctura Valerianae)
2. Влить упомянутые в п.1 20 капель в высокую узкую стеклянную емкость емкостью 0,5 л
3. Дополнить упомянутую в п.2 емкость до уровня золотого ободка пивом (beer, пиво) по вкусу
4. Взболтать, но не размешивать, по рецепту Д.Бонда (в переводе на русский - агент МЖ7)
5. Употребить полученную суспензию внутрь большими глотками в три приема: сначала 0,25 л + 10 капель, затем 0,125 л + 5 капель, и затем оставшиеся 0,125 л + 5 капель
6. Кликнуть на [ссылку](#), найти реквизиты подписки на конференцию RTFM_Helpers и активно участвовать в ней, выбросив из головы кошмарные идеи, изложенные на этой странице.

Этот совет продиктован исключительно заботой о здоровье настоящего настоящего ассемблерщика, который, как известно, испытывает непереносимые мучения от любого комфорта и просто-тки титаническим усилием воли заставляя себя писать программы с помощью мнемочкодов команд, а не одной только директивы db. Ибо звучат еще в наших ушах бессмертные слова: "Ну вы, блин, заставьте нас еще горизонтально исходник скроллить, как в вижувал васике!"

Ну вот, продолжаем для оставшихся ренегатов и предателей.

Ассемблер - величайший язык, царь языков, практически лишенный недостатков, достойный всенародной любви и поклонения почти таких же, как Заместитель Руководителя Администрации Президента Российской Федерации. Но есть у него (у ассемблера, а не, упаси боже, Заместителя) один не то чтобы недостаток, а так, мелкая мелочь.

Неприятность этой мелкой мелочи мы ощутили во всем ее необозримом масштабе, когда спустя три дня по окончании гарантийного срока сдох наш любимый Sony GDM-200, и временно перекочевал под стол вместе со своими 1280x1024, а вместо него на столе оказался старенький SyncMaster 500s, с натугой выдающий 800x600. С тех пор до 80% нашего рабочего времени уходило на переключение между окнами и, чтоб он пропал совсем, скроллинг исходного текста вверх-вниз.

Существенно сократить число сгорающих в секунду нейронов нам удалось благодаря одному-единственному совсем простенькому макросу. Вот он:

```
@ MACRO p0,p1,p2,p3,p4,p5,p6,p7
p0
p1
p2
p3
p4
p5
p6
p7
ENDM
```

И все! Как только этот макрос был включен в нашу постоянную макробиблиотеку, мы получили счастливую и давно желаемую возможность записывать в одну строчку несколько (а именно - от 2 до 8) команд.

Возьмем, к примеру, самую что ни на есть типовую ситуацию. Нужно скопировать фрагмент памяти в новое место. Что запишет в исходный текст кто-нибудь из покинувших нас настоящих настоящих ассемблерщиков? Ну, например, нечто вроде этого:

```
;Copy from one buffer to another
mov esi,one_buffer
mov edi,another_buffer
mov ecx,how_many_bytes
cld
rep movsb
```

А что запишем мы, ренегаты и предатели? А вот что:

```
;Copy from one buffer to another
@<mov esi,one_buffer>,<mov edi,another_buffer>,<mov ecx,how_many_bytes>,<cld>,<rep movsb>
```

Вот какие преимущества, по нашему скромному ИМНО, мы получаем в результате этого маневра:

1. Про уменьшение трудозатрат на вертикальный скроллинг уже сказано
2. Кроме того, наше подсознание перестает возиться с понятием "команда" и переходит на более высокий уровень: "блок". Результатом чего, очевидно, должно явиться повышение производительности труда.
3. У нас появляется дополнительный стимул к хорошему комментированию исходного текста.

Может оно, казалось бы, ассемблерщику и ни к чему, но это только до тех пор, пока не соберешься разобраться в собственном коде, а то и, не дай бог, кто-нибудь не захочет купить наши гениальные исходники за 100000 зелени. Вот тогда-то и скажешь себе: от дурак-то был, когда комментарии не писал!

4. С повышением потребности в комментировании возрастает наша грамотность, культура, любовь к языку, родному и английскому, и умение кратко выражать свои мысли, причем не только в троллейбусе.
5. Облегчается копирование текста. Допустим, за год работы мы копируем текст с места на место 50000 раз, в среднем по 10 строк. Так вот сравните, каково: во втором случае мы копируем 50000 строк, а в первом - 500000! В десять раз больше, но за ту же зарплату!

Освоив предлагаемый метод, вы можете смело идти к своему руководителю и рекомендовать ему снизить вам зарплату в 10 раз. Гарантируем: с этого момента вы будете его любимым сотрудником.

6. Появляется возможность наконец-то изучить клавишные макросы рабочей среды. Щелк на горячую клавишу - и группа строк свернулась в одну строку. Щелк на другую - и строка развернулась в группу строк: отлаживай, дорогой.
7. При желании можно считать, что повышается читабельность исходного текста.

Недостатков у предлагаемого метода только один:

1. Так вот сиди теперь и думай: чего это я, дурак, все с ассемблером вожусь? Может, на Visual Basic перейти?

Дальнейшее совершенствование предложенного метода видится нам на пути увеличения числа параметров макроса. Предварительные оценки показывают, что добавление только одного параметра р8 может привести к увеличению числа обрабатываемых макросом строк примерно на 12,5%. Желающие могут провести соответствующие эксперименты с целью уточнения этой оценки.

Так в каком же веке мы живем?

Автор: Svet(R)off

Дата: 2002-06-14

Извини, усталый путник, случайно забредший на эту страницу. Предвижу твою кислую физиономию и предскажу добрые слова, которыми ты выражаешься в мой адрес в связи с нижеизложенным. В самом деле, просто достали все, от Папы Римского до изготовителей юбилейной туалетной бумаги, по 2000 метров в рулоне!

И еще: по домену millenium в каждой зоне (а за Россию радостно: ребята, прикинув процент не забывших латынь врачей и адвокатов на территории ru, не пожалели сотню баксов и на всякий случай зарегистрировали еще и millenium, ведущий на тот же хост. Уверен, большинство из набравших url вручную попали на сайт именно этим путем. Сужу по себе. Наличие millenium и milenium не проверял, лень, но с них станется!)

И еще: кандидат в книгу рекордов Гиннеса в период с 15.06.99 по 31.12.99 съел 2000 куриных яиц. Скончался от легочной эмфиземы, осложненной сердечной недостаточностью, при прослушивании прощальной речи Первого Президента. А вот ел бы перепелиные, дурачок, мог бы скончаться куда более благородно, от сексуальных излишеств.

И еще: Windows 2000 с братом-Office'м. Точно говорю: прозорливец Билл прикидывал их раскрутку еще когда решал обозвать Chicago не 4.0, а 95.

И еще: умница Милен Фармер. И Милка с Охотного, берущая 20%-ю наценку, тоже большая молодец.

И еще: магическое число международных террористов на территории Чечни. При том, что примерно по столько их уничтожается ежемесячно, если судить по официальным сводкам. Число погибших наших, похоже, тоже неуклонно округляется до этого же значения.

И еще: хорошая передача Дм.Киселева "Национальный интерес-2000". Правда, в названии имеется неоднозначность: если это про зарплату, то в RU или в USD? Хотелось бы второе.

И еще, и еще, и еще...

Кажется, под верстовым столбиком с цифрами "2000" не пописала только соседская Аська, да и то, потому что она не собака, а кошка. Все остальные благополучно отметились, близко не подойдешь...

Но не могу молчать! Во-первых, следующий случай высказаться по этой проблеме представится не скоро, боюсь, не доживу. И во-вторых, профессионально не терплю ошибок.

Всякому настоящему ассемблерщику знакомо это ощущение, когда программа готова к дистрибуции, но из личного опыта, из законов Мерфи, из знания жизни и еще черт знает откуда автор прикидывает, сколько в ней багов. Ненастоящие ассемблерщики и настоящие неассемблерщики могут примерно представить себе это ощущение, если когда-нибудь им доводилось летом, в жару, при мокрой спине, ехать домой в переполненном троллейбусе после посещения дешевой мужской парикмахерской. Возможно, именно в целях избежать упомянутых специфических ощущений, настоящие ассемблерщики почти не программируют и почти не стригутся.

Между тем, в ныне общепринятом определении того, в каком именно веке мы живем, имеет место самая настоящая ошибка.

Большинство людей, так уж сложилось, считает, что 2000-й год принадлежит 20-му веку. Одни - потому что с детства их учили считать: "раз, два, три, ...".

Настоящих ассемблерщиков чудаковатые родители учили, конечно же, так: "ноль, один, два,..., девять, а, бе, цэ, дэ, ..." и так далее, со всеми вытекшими отсюда печальными последствиями. Неспешно плыло светлое время шестидесятых, мир был наивен и добр. Печаталась цветная "Техника молодежи" про ЭВМ, про робота Сепульку, с фантастическим рассказом в каждом номере... А еще толстенная "Наука и жизнь", и "Юный техник". За ночь глотался "Понедельник начинается в субботу". Полыхал дюзам "Непобедимый" Лема. Слово "голубой" ассоциировалось с небом. В году было 365 солнечных дней... Впрочем, это совсем другая история.

Вторые - потому что хотят стабильности. Третьи - потому что боятся будущего. Четвертым - наплевать. Пятые - решили, что так надо. Шестые - потому что верят пятым. Седьмые - потому что они посчитали, и у них так получилось. И так далее.

Очень интересно и привлекательно выглядит позиция официальных лиц, начиная от упомянутого Папы и кончая нашим начальником ЖЭУ. Не знаю, когда они все встречались, но на этой встрече они договорились о важном компромиссе. Чтобы не ссорить между собой настоящих ассемблерщиков и представителей всех остальных конфессий, принято было решение считать 2000-й год переходным, (не)принадлежащим обоим смежным векам. Слава мудрым!

Однако, время, как каждый вечер напоминает Михаил Леонтьев. Я лично вижу в его словах два смысла. Первый означает: "Треп закончен, переходим к делу". Второй обращает наше внимание на существо рассматриваемого вопроса.

Действительно, давайте вспомним, что речь идет о времени. И не просто о времени, а о времени, прошедшем от Рождества Христова. Потому что именно так называется Наша Эра (она же Новая Эра, она же Anno Domini). Вспомнили?

А теперь давайте считать. Время, оно ведь не кирпичи. Оно не поштучное, а непрерывное. Вот так его и будем считать.

Итак. Через пять минут после рождения Христа, когда Он лежал в яслях, освещаемый падавшим сквозь прореху в крыше убогого сарая светом Вифлеемской звезды, сколько Ему было от роду?

Правильно, **ноль лет и пять минут**.

А через месяц, когда, насытившись от груди Марии, радостно агукал, не зная о сужденных Ему Страстях?

Правильно, **ноль лет и один месяц**.

А через одиннадцать месяцев тридцать дней двадцать три часа и пятьдесят девять минут, когда Мария качала Его, спящего, в колыбели, мысленно поздравляя с первым годом жизни и мечтая о том, как Он вырастет добрым, хорошим сыном, будет счастлив и станет ей опорой в старости, как мечтают все родители от Начала Времен?

Правильно, **ноль лет одиннадцать месяцев тридцать дней двадцать три часа и пятьдесят девять минут**.

То есть **нулевой год входит в жизнь Христа** (впрочем, ничем не отличаясь от нулевого года жизни любого другого человека). Следовательно, нулевой год входит и в Нашу Эру, именно с него она начинается.

А в первом (не по счету, а по номеру) году Нашей Эры Христу уже исполнился год, и шел второй. Если этот тезис к данному моменту непонятен, то дальше можно не читать - читатель с автором в этой части общего языка уже не найдет. Это нормально.

Следовательно, первый век Нашей Эры составляли годы 00, 01, 02, 03 ... 99, поскольку в веке, по определению, 100 лет. А 100-й год относился уже, естественно, к следующему веку н.э.

Помаленьку экстраполируя, добираемся 1999 года. Кто не помнит, подсказываю: там был еще такой президент, Ельцин Борис Николаевич. Давно это было, еще в XX веке н.э. А вот 2000-й год - это уже XXI век, потому что за две тысячи лет понятие "век" ничуть не изменилось. Как было в первом веке 100 лет, с 00 по 99, так и осталось в XX-м с 1900 по 1999.

Делаем окончательный вывод. **2000-й год принадлежит XXI веку.** Кто не понял: мы живем в новом, XXI веке н.э. Поздравим себя, дожили. А ведь могли и не дожить.

Здесь необходимо сделать маленькое пояснение. Все вышесказанное верно с формальной точки зрения. То есть если вы, допустим, настоящий ассемблерщик, и, как всякий настоящий ассемблерщик, ненавидите считать итерации в циклах while, for, do и until, потому что каждый раз нужно лезть в учебник, и все равно на 1 ошибешься, то с вашей точки зрения все сказанное - верно.

А вот если вы, допустим, раннесредневековый монах-богослов, и присутствуете в исторически поворотной точке пространства и времени, когда определяются все последующие воззрения на дату Рождества Христова, и только что вернулись со строительства монастырской стены, и собираетесь поработать с летописями, то все сказанное выше должно быть вам далеко не очевидно. Возьмете с устатку, да и назначите нулевой год жизни Христа первым. И пошло-поехало. Вплоть до нашего 2000-го года, последнего года XX-го века.

Однако не в наших силах сейчас выяснить, откуда тысячу лет назад вернулся тот монах: с кирпичного мини-завода или из трапезной, где наоборот, созерцал ламинарный процесс перетекания анжуйского из одной емкости в другие. Поэтому давайте останемся на формальных позициях: мы - в XXI веке.

Через public mail мы получаем альтернативные мнения о рассматриваемом в этой статье вопросе. Так, Sergey Chehuta из WhiteTown прислал письмо:

Добрый день!
Очень простое и наглядное дидактическое пособие для объяснения того, когда начинается новый век:
"Второй ящик водки начинается с двадцать первой бутылки". :))

Это, на наш взгляд, очень интересная и перспективная версия, хоть она и отмечает все сказанное выше. К сожалению, подтвердить ее или опровергнуть столь же трудно, как, скажем, Догадку Гольдбаха. Во всяком случае, при проведении многочисленных экспериментов assembler.ru даже вместе с друзьями, даже под шашлык, никогда не добирался дальше восьмого века н.э. Было бы здорово, если бы кто-нибудь из неравнодушных к науке посетителей взялся бы проверить эту версию. Спонсировать, к сожалению, не сможем, но отчет опубликуем с удовольствием. (Репортажи со свадеб и юбилеев просьба не предлагать.)

Ну и в заключение - маленькая сенсация. Основываясь на той же методике, порассуждаем не о годах, а о веках.

Первое тысячелетие составляли века: 00, 01, 02, 03, ..., 09.

Второе тысячелетие составляли века: 10, 11, 12, 13, ..., 19.

Еклмн! Братья и сестры, да мы уже сто лет, как живем в третьем тысячелетии! Во бараны, сколько денег в празднование вбухали, колеса по всему миру строили, хрустальные шары спускали, на одни фейерверки штуки три российских годовых бюджета спалили! Хорошо хоть мы, как обычно, умнее всех оказались: не стали транжириться, водкой и оливье обошлись.

О пользе монополий или как будем резать Билла - вдоль или поперек?

Автор: Svet(R)off

Дата: 2002-06-14

Итак, Microsoft снова, теперь уже окончательно и бесповоротно, признана монополистом. 3 апреля 2000 года некто Томас Джэксон, федеральный окружной судья, про которого до сего дня никто дальше родной вашингтонщины не слыхивал, тронул святое. И добро бы по делу!

Допустим, собрались бы лидеры нации и решили, что Microsoft малость перебрала с ролью в мировой истории. Статьи в законе про это не подберешь, но, поскольку случай, мягко говоря, неординарный, можно на худой конец и 28-ю поправку к конституции принять: "Всякий Карфаген обязан быть разрушен". Бывало, и по куда более мелким поводам поправки принимали. Устроили бы сенатские слушания, побазарили - да и приняли. Все умные люди поняли бы и согласились.

Так ведь нет же! Вместо того чтобы решать проблему по существу, лицемерно подобрали повод. Повод, надо признать, довольно сомнительный.

Имела ли право Microsoft встраивать Explorer в Windows? То есть было ли встраивание монополистическим актом, направленным на ущемление прав конкурентов, или могло быть объяснено иными, более благородными соображениями? Давайте обсудим серьезно, не сбиваясь на панегирики или филиппики.

Вспомним не столь далекие годы, когда в прохладных машинных залах паслись стайки милых, как на подбор, девушек-операторов, день и ночь скармливавших ревушим считывателям колоды перфокарт. В то дремучее благословенное время IBM взяла, да и зарегистрировала торговую марку DOS, ибо появились магнитные накопители. А буква D, между прочим, в той ТМ была от слова "дискковая". Потому что тогда настала такая пора. Уходили в прошлое дырочки в бумаге, им на смену приходили магнитные домены. И дискковая подсистема становилась неотъемлемой частью ОС. Дело дошло до того, что некоторые решили, будто ДОС - это и есть ОС, а все остальное, пользовательский интерфейс, например - это нашлапки. Во всяком случае, представить себе современную ОС без встроенной файловой системы (дискковой в первую очередь, конечно же) можно только при наличии очень извращенного воображения (о сетевых ОС читай дальше).

Сегодня мы на другом, далеком витке, но все той же самой диалектической спирали. Наступает новая реальность: компьютеры сливаются в глобальный информационный континуум. Уходят в прошлое локальные ресурсы, приходят распределенные.

Это, конечно же, не значит, что локальные ресурсы когда-нибудь вымрут, как перфокарты, но они перестанут быть основой операционной среды.

Хотим мы того, или не хотим, но операционная среда каждого компьютера станет частью единой планетарной операционной среды. У этого процесса на сегодняшний день есть всего два серьезных препятствия: каналы связи и защита конфиденциальности. Серьезных, но не непреодолимых, так что ожидаемое слияние не так уж далеко.

Операционная система обязана ответить вызову времени. Не просто иметь встроенные сетевые средства, а гораздо более того: жизнь в сети должна стать ее сутью. И было бы странно, если бы пользовательский интерфейс, вторая неотъемлемая и главнейшая часть современной ОС, остался бы в стороне от этих перемен.

А ведь именно броузер - зачаточный, но, безусловно, базовый элемент, из которого в будущем должен вырасти пользовательский интерфейс интегрированной в мировую сеть ОС. И Microsoft неторопливо, но совершенно недвусмысленно двигалась по этому пути.

Первым шатающимся шагом на нем, как все помнят, был ублюдочный стиль веб в проводнике-95.

С точки зрения изложенного судебное решение о неправомерности интеграции браузера в ОС представляется как минимум некомпетентным, а называя вещи своими именами - бездарным и вредоносным.

Поговорим о более общих вещах.

Так ли уж негативен монополизм Microsoft?

В чем, собственно, состоит основной вред монополизма для общества? Отнюдь не в уничтожении соперников нечестными приемами: это с успехом проделывается и в немонопольных ситуациях (см. популярную передачу "Дорожный патруль", там через день во всех подробностях показывают аккуратно продырявленные головы предпринимателей). Главный вред монополизма - в угнетении общественного прогресса. Монополия не дает возникать и развиваться альтернативным решениям встающих перед обществом задач. Общество теряет выбор. А вместе с ним теряет потенциал выживания.

Кто не верит - см. вокруг себя. Стены вашего подъезда несут на себе письма измученных подростковой гиперсексуальностью прыщавых юношей потому, что у вас в районе единственное ЖЭУ, и вам некому, кроме него, отнести свои денежки. И в телефоне вашем, срывая коннект, тархтят наборные импульсы всей округи, потому что на сто окрестных домов у вас один-единственный "телефонный оператор" (взято в кавычки преднамеренно). И в стране вашей зарплата дворника зависит от решений стран-участников ОПЕК, потому что термин "естественная монополия" для вас почему-то более приемлем, чем, скажем, "дойный козел".

Несла ли Microsoft вред монополизма? Нет, нет, и еще раз нет. Наоборот! И это действительно уникальная ситуация. Наверное, ее можно объяснить только невероятным сочетанием личностных особенностей ее лидеров с особенностями материала, живого и бешено растущего, с которым они работали. И только этим сочетанием объясняются фантастические достижения корпорации.

(Джобс, между прочим, при тех же, а то и более благоприятных, условиях, с ситуацией не справился).

Microsoft, идеально эффективно используя для бутстрэпа свои гигантские интеллектуальные и финансовые ресурсы, перла вперед сама и тащила за собой весь технологический мир. И если она вламывалась на уже занятый кем-то рынок, то не ради подавления конкурентов, а принося с собой лучший продукт. И если перекупала у кого-то мозги, то не ради того, чтобы обескровить противника, а для предоставления талантам условий, позволявших реализоваться гораздо лучше, чем на старом месте.

Даже естественная для всякого нормального человека ненависть к чужому успеху в случае Microsoft была плодотворна. На этой ненависти выросли не только фирмы - целые отрасли и даже концепции. Чего стоит одна только идея независимого программирования, которая без Microsoft вряд ли бы заинтересовала хоть кого-нибудь, кроме десятка чудаков - социалистов.

И это фантастическое время уходит. Просто потому, что оно обязательно должно уйти. Природа не терпит ситуаций, когда выживание вида ставится в зависимость от одного конкретного фактора. А Microsoft объективно все более претендовала на роль такого фактора. Корпорация изменила мир - и мир изменился. Без Microsoft он не станет лучше или хуже. Пройдет несколько лет, и он просто станет другим.

Но все мы немножко будем скучать по детству, правда?

Итак, на вопрос, является ли компания Microsoft монополистом в классическом понимании этого слова, следует ответить: нет. На вопрос, подлежит ли компания Microsoft разделению, следует ответить: безусловно, да. И объяснить людям - почему. Так и для NASDAQ будет спокойнее,

глядишь - еще и вырастет. Потому что биржевые индексы бесятся в условиях неуверенности инвесторов, а в условиях разумной прозрачной политики либо остаются стабильны, либо растут.

Как починить Hewlett-Packard (но не автомобиль)

Автор: Svet(R)off

Дата: 2002-06-14

Допустим, у вас есть Hewlett Packard SureStore 7200. Это такой внутренний пишущий дисковод CD (он же - CD-recorder, CD-writer) для IDE-интерфейса, способный работать как с CD-R, так и с CD-RW. Старенький. Служит вам честно больше двух лет. Отливает в золоте плоды вашего творчества. Любуется вы, как растет стопка архивных дисков, и теплеет на душе: не пропадет ваш скорбный труд, а достанется истории в целостности и сохранности.

Впрочем, если у вас есть что-нибудь другое от того же производителя, принтер там или даже компьютер, тоже неплохо. Много разной полезной техники производит Hewlett Packard. Бранднэйм - он и есть бранднэйм.

И вот пишете вы как-то раз очередной свой золотой диск, и случается с вами то, чего больше всего боятся все владельцы пишущих сидюков: выскакивает buffer underrun.

Маленькое техническое пояснение. Технология записи компакт-дисков такова, что данные на запись обязательно должны поступать непрерывным потоком. Это очень просто понять, если представить себе, как бежит луч пишущего лазера по записываемому слою вращающегося диска-матрицы, прочерчивая на нем спиральную дорожку, от центра к периферии. Позиционирование осуществляется простым перемещением луча вдоль радиуса диска. Приостановиться, вернуться назад, попытаться повторить запись невозможно - это вам не винчестер, где есть разметка секторов, позволяющая точно позиционировать необходимый блок информации. Любое нарушение в потоке записываемой информации ведет к краху процесса записи и безвозвратной порче матрицы (если, конечно, это одноразовый CD-R, а не многоразовый CD-RW).

Для обеспечения непрерывности потока записываемых данных внутри пишущего дисковода имеется специальный буфер памяти (в вашем случае - 768 Кбайт). Данные в него подгружаются порциями, по мере необходимости, из внешнего источника через дисковый интерфейс IDE, а считываются непрерывно, в темпе записи на диск. Естественной заботой всей системы в течение всего процесса записи должно быть поддержание буфера в наполненном состоянии. Если буфер в какой-то момент хотя бы на мгновение окажется пуст - запись прервется со всеми вытекающими печальными последствиями. Это и есть та самая ситуация, которая называется buffer underrun.

Обычная причина опустошения буфера - это недостаточность скорости источника данных. Допустим, вы пишете компакт-диск на двухкратной скорости, что означает, что необходимо обеспечить скорость входного потока данных 300 Кбайт/с. А в качестве источника выбрали старенький сидишник от Panasonic тоже с двухкратной скоростью чтения. Казалось бы, все получается. А нет. Вы забыли, что при чтении компакт-дисков довольно часто возникают ошибки, для устранения которых дисковод-источник пытается считать информацию оригинала повторно, для чего, в свою очередь, долго позиционирует читающий луч. Пока он этим занимается, буфер пишущего дисковода неумолимо опустошается, и в конце концов все заканчивается катастрофой. То же самое следует ожидать в случае, когда источник вы выбрали подходящий (например, винчестер), но при этом загрузили систему какой-нибудь параллельной работой. Пишущий дисковод, например, чувствует, что пора бы подгрузить буфер, обращается к системе, а она в это время занята расчетом зарплаты вашего соседа за прошлый год с целью проверки обоснованности покупки им подержанной "копейки". И амба.

Для того чтобы оценить, насколько подходит предполагаемый источник данных для записи на CD, имеются разные средства. Например, в поставляемом с дисководом приложении Adaptec Easy CD Creator их два: тест Transfer rate и тест, имитирующий запись. Первый служит для предварительной оценки пригодности источника данных и представляет собой замер скорости поступления данных при чтении некоторого тестового количества файлов. При этом отдельно оценивается скорость при чтении маленьких и при чтении больших файлов. Первая обычно несколько меньше, чем вторая, потому что маленькие файлы требуют дополнительных накладных расходов файловой системы на позиционирование головок дисководов. Выполнять тест Transfer rate следует нечасто. Обычно достаточно сделать это один раз для каждого из потенциальных источников данных. Желательно, чтобы скорость источника данных как минимум в два раза превышала скорость записи. Имея, например, пишущий дисковод с четырехкратной скоростью записи (600 Кбайт/с), вам придется быть готовым к проблемам, если вы используете в качестве источника старенький винчестер с оценкой по тесту Transfer rate что-нибудь около 1000 Кбайт/с. А вот используя более современный винчестер с 3000 Кбайт/с и более, вы можете чувствовать себя более-менее спокойно.

Не забывайте только еще об одном обстоятельстве. Некоторые винчестеры, давая в целом очень неплохие характеристики по тесту Transfer rate, имеют привычку время от времени задумываться о чем-то своем, сокровенном. В частности, грешат этим некоторые модели от Maxtor. О том, что вам попался именно такой задумчивый винт, вы, возможно, догадаетесь по куче испорченных матриц.

Второй тест следует выполнять каждый раз перед записью. Он, по идее, заключается в имитации процесса реальной записи реального набора ваших данных, за тем небольшим исключением, что при этом не включается пишущий лазер. (Сказано "по идее" потому, что практика показала, что это не совсем так. Но об этом ниже.) Нормальное пишущее приложение, вроде уже упомянутого Adaptec Easy CD Creator, по умолчанию предлагает вам именно такую комбинацию: тест, а потом запись. И это правильно, потому что скорость потока данных очень сильно зависит от того, что этот поток собой представляет: то ли один файл размером 100 Мбайт, то ли 100000 файлов по 1 Кбайт. В последнем случае даже на быстром винчестере можно запросто столкнуться с ужасным buffer underrun'ом.

Резюмируя сказанное, следует дать несколько полезных советов:

- * запуская запись на компакт-диск, закройте все приложения и прекратите все работы с компьютером до самого момента ее завершения. При этом лучше вообще не дышать, особенно по утрам в понедельник исключите возможность автоматического запуска на вашем компьютере во время записи каких-либо процессов: хранителя экрана, энергосберегающих функций, планировщика задач и т.п.
- * если компьютер включен в сеть и предоставляет пользователям свои ресурсы (принтер, диски) - запретите доступ пользователей
- * запитайте компьютер от какого-нибудь UPS'a, особенно если имеете дело с нестабильной электросетью, что в условиях демократии - в порядке вещей
- * используйте в качестве источника данных для записи самые быстрые устройства из имеющихся в вашем распоряжении
- * обратите внимание, где ваше пишущее приложение собирается хранить временные файлы. Это устройство тоже должно быть быстрым
- * обязательно тестируйте записываемые данные.

Однако вернемся к вашему Hewlett Packard SureStore 7200. Источником данных служил для него жесткий диск от Seagate. Имитирующее тестирование прошло нормально, что настроило вас на благодушный лад. И вдруг, записав чуть больше 5000 блоков, дисковод вываливается в buffer underrun! И просительный выдвинутый трей достаточно прозрачно намекает на предстоящие вам незапланированные траты...

Хорошо, что наученные предыдущим горьким опытом общения с неким угрюмым детищем фирмы Pinnacle еще в те времена, когда чистая золотая матрица стоила 10 баксов, вы, прежде чем дописывать на уже использованный диск новую сессию, предварительно сохранили все его предыдущее содержимое на винчестер. Иначе был бы безвозвратно утрачен труд двух предыдущих месяцев.

Поначалу вам показалось, что ничего особенно страшного не произошло. Ну запоролась матрица, эка невидаль. Возьмем другую. Сейчас их по доллару за штуку найти - не проблема. Но на всякий случай вы решили поэкспериментировать. Взяли перезаписываемый диск. И давай пытаться писать на него. Из пяти попыток три закончились крахом. Попробовали писать другие данные. Не помогло. Переинсталировали софт. Тот же эффект. Вот именно на этом этапе вы ощутили некий неприятный холодок между лопаток.

Дальше пошли эксперименты с железом. Вскрыли брюхо компьютеру и, наигравшись с комбинациями Master-Slave, в конце концов отключили висящий на том же интерфейсе второй CD-ROM. Без толку. Вынули страдальца из этого компьютера и запихнули его в другой. На том очень кстати оказались проинсталированы Windows 95 и NT4.0. Ничего не помогло. И только тут вы, наконец, сознались себе в том, что ваш любимый HP SureStore 7200 гавкнулся.

Что делать (как спрашивали друг друга Н.Г.Чернышевский и В.И.Ленин)? Естественно, чинить. Причем за деньги, потому что годовая гарантия давно истекла.

Вы лезете в Интернет, и без напряжения находите на каком-то сайте московский телефон технического центра Hewlett Packard: 797-3520. Звоните туда. Милый механический голос предлагаем вам нажать в тональном режиме кнопку "1". Все еще надеясь на лучшее, вы выполняете указание и, бродя по виртуальному телефонному меню, в конце концов добываетесь от механической барышни адреса центра: Спартаковская, 16.

Кроме того, вы получаете возможность побеседовать с живой девушкой- оператором. На ваш лепет про buffer underrun, про проверку на двух компьютерах и на трех ОС и прочие ваши метания девушка отвечает, что сиди-райтеры они не чинят, но с удовольствием после тестирования заменят его вам целиком. Вы радостно прикидываете, сколько сэкономите, если получите новый, современный пишущий сидюк с вычетом стоимости почти исправного старого, и мчитесь через всю Москву на Бауманскую, в районе которой расположен ваш благодетель - техцентр Hewlett Packard. Оглушенный счастьем, вы при этом совершенно не обращаете внимания на интонацию, с которой любезная девушка произносила свои любезные слова. А ведь, задумавшись, вы могли бы без труда припомнить, что точно такую же интонацию вы слышали совсем недавно, не далее как в прошлую субботу, покупая на рынке черешню. На вопрос о том, есть ли в черешне червяки, здоровенная тетка отвечала: "Да нет, вроде," - как будто сама видела эту свою черешню в первый раз. И слово "нет" было произнесено с той самой интонацией.

Прибыв на место, вы долго бродите вокруг полуразвалившегося здания дореволюционной постройки, расположенного по адресу Спартаковская, 16, вызывая нездоровый интерес у толпы бауманских бомжей и охранников какого-то офиса без вывески, но зато с телекамерой. И только напрягши все свои умственные способности и еще раз созвонившись с любезной девушкой, вы наконец понимаете, что оказались на Спартаковской улице, а вам нужна Спартаковская площадь. Видно, здорово наследил древнеримский пугачев в Москве, если в его честь назвали и улицу, и площадь, и еще переулок.

Преодолев метров семьсот между Спартаковскими 16, вы, наконец, находите вожаемую синюю вывеску Hewlett Packard. Техцентр оказывается расположенным в современном деловом здании, инкрустированном ценными породами мрамора и, вероятно, евросантехникой. Порадовавшись за сотрудников техцентра, вы одновременно понимаете, что аренду этого архитектурного великолепия частично придется оплатить и вам.

И вы входите в прохладный кондиционированный сумрак обиталища отвертки и паяльника. Первое, что бросается в глаза прямо от входа - выгородка с надписью "Касса". Тут вы вспоминаете, что телефонный робот, разъясняя вам расписание работы техцентра, особо подробно, тщательно, со вкусом, подавал расписание работы кассы. И еще почему-то вспоминаете червивую черешню.

Недалеко от кассы вы обнаруживаете несколько столов, за которыми удобно расположились несколько же симпатичных молодых людей мужского пола, на каждом из которых написано "Hewlett Packard". Решив, что именно эти люди заберут у вас сейчас ваш старый барахлящий сидюк и вынесут из недр своего волшебного замка новый, прекрасный, перевязанный синей ленточкой, вы мчитесь прямо к ним. Правда, молодые люди оказываются несколько заняты с уже сидящими перед ними клиентами, и вам приходится ждать, в отсутствии свободного стула переминаясь с ноги на ногу и прислушиваясь к разговорам.

Один из молодых людей беседует с клиенткой - дамой среднего возраста из числа тех, на которых держится любая организация. Дама нервно гоняет по столу образцы печати струйного принтера и пытается втолковать своему визави, что конкретно не устраивает ее в цветопередаче. Видно, что молодой человек страшно озадачен, потому что он молча берет в руки то один листок, то другой, так и сяк рассматривает их и кладет обратно. Этим он продолжает заниматься все оставшееся время вашего пребывания в техцентре.

Другой молодой человек уже заканчивает беседовать с клиентом. Окончание беседы еще более укрепляет ваши подозрения, потому что представляет собой заполнение каких-то бумажек с многими графами и проставление каких-то росписей. И вы опять понимаете, что речь идет о деньгах.

Наконец последняя роспись проставлена, и молодой человек обращает свое благосклонное внимание на вас. Вы усаживаетесь перед ним и изливаете свои жалобы, подпихивая в сторону молодого человека железного виновника своих несчастий. Но молодому человеку не нужен ваш дисковод. Он даже не смотрит в его сторону. Он внимательно смотрит на вас и спрашивает: "Но вы знаете, что это будет стоить дорого?" Вы спрашиваете: "Сколько?", думая при этом про себя: "А-а-а, черт с ней, с сотней баксов!" Но молодой человек не торопится отвечать на ваш глупый вопрос. Он долго щелкает клавишами компьютера, объясняя вам при этом ваши права, как будто он никакой не приемщик в радиомастерской, а настоящий американский коп. Он говорит вам, что сначала они возьмут у вас дисковод на тестирование. И если окажется, что он действительно неисправен, то они будут иметь с вами дальнейший разговор. А если он окажется исправен, то вернут они его вам только взамен шестидесяти баксов. За беспокойство, надо понимать, потому что за шестьдесят баксов можно протестировать не то что дисковод, а целый "Мерседес", и еще на "Жигули" останется. Но вот если дисковод действительно окажется неисправен (а это скорее всего так, говорит молодой человек, уверенный в технике своей фирмы), то вы сэкономите эти самые шестьдесят баксов, потому что тогда у вас никто их не возьмет. Тогда вам предложат замену с трехмесячной гарантией. И эта замена будет стоить... Будет стоить эта замена... Замена эта стоить будет... Стоить замена будет эта... Триста двенадцать долларов двадцать центов.

"Спасибо," - говорите вы, укладываете железяку в бесполезный антистатический пакет, прячете пакет в чемодан и позорно ретируетесь из техцентра, бросая на поле боя погибших и добывая раненых.

Дома вы снова залазите в Интернет, к [Павло Дзиковскому](#), и обнаруживаете, что по всей Москве продается четырехскоростной HP SureStore 8250, и красная цена ему - 200 долларов. И в тот же день вы едете в давно уважаемую вами фирму НАК, что в Лебяжьем переулке, в ста шагах от кремлевской стены, и покупаете там новый прекрасный дисковод, даже дешевле среднестатистической. Естественно, этот самый, от Hewlett Packard. А как же иначе: бранднэйм - он и есть бранднэйм.

Пролетая над миллионом баксов (вместе с Гольдбахом)

Автор: Svet(R)off

Дата: 2002-06-14

Догадка Гольдбаха подтверждается не абсолютно строго, но абсолютно убедительно. Дальше будет приведено полное доказательство, это не шутка. Но сначала предыстория, вдруг кто-нибудь из посетителей перед десертом любит чего-нибудь сладенькое?

Вернее, предысторий две. Первая давняя. Говорят, что ей 258 лет. Так оно и есть.

Леонард, брателло!

В довесок к той цистерне бензина, которую я закинул тебе на прошлой неделе, отгоняю еще вагон селедочных голов, как договаривались. С бабками не тяни, переведи их на тот счет, который я тебе тогда у Муськи дал. Как у тебя дела? Не достаёт ли Починок? Ну, бывай, на той неделе свидимся.

Твой кореш Христиан.
7.06.1742

P.S. Кстати, тут я давеча задумался: а не является ли всякое четное число суммой двух простых чисел? По-моему, так оно и есть. А ты как думаешь?

Так (или примерно так) писал своему другу великому шведу Леонарду Эйлеру Христиан Гольдбах. Именно постскриптумом он и стал знаменит. Во всяком случае, в Grolier есть статья "Goldbach's Conjecture", а вот статьи "Goldbach" нет.

Загнал ли Эйлер селедочные головы, история умалчивает. А вот на догадку внимание обратил, и с Гольдбахом согласился, но доказательства этому утверждению так и не нашел. Чувствовал, что задачка тривиальная, все думал: как-нибудь; да, видно, некогда было, замотался.

Шли годы. Эйлер закончил свои дела в этом мире, Гольдбах тоже, но люди про догадку не забыли. Уж больно привлекательны такие задачи: на первый взгляд элементарные, а решение в руки не дается. А задумаешься чуть поглубже - и вот уже два мужика в халатах тащут тебя, болезного, по белому коридору на очередной сеанс электрошока.

С годами слово "Догадка" стало писаться с большой буквы, из уважения к труду многих поколений психиатров. Сначала еще теплилась надежда, что кто-нибудь успеет доказать ее раньше лоботомии, но с появлением компьютера угасла: люди стали стремительно разучиваться думать. Зато много народу с большим удовольствием стало гонять процессоры на предмет эмпирических проверок. Дошли до 1014 (вот это тема для диплома! Блеск!).

Некоторое оживление в историю Догадки привнесло появление научной криптографии, поскольку криптографы, будучи людьми ленивыми, очень полюбили простые числа и не захотели возиться со сложными. Но и криптографы, занятые бегом по кругу в поисках ядов и противоядий, приняли Догадку как должное и не стали очень уж надрываться ее подтверждением.

Так и дошло дело до наших дней.

Вторая предыстория - продолжение первой. В 1992 году греческий писатель Апостолос Доксиадис написал роман "Дядя Петрос и Догадка Гольдбаха" (Uncle Petros and Goldbach's Conjecture by Apostolos Doxiadis)

Роман начинается так: "В каждой семье есть своя черная овца - в нашей ею был дядя Петрос". Рассказчик - любимый племянник дяди Петроса. Семейство всегда считало

Петроса неудачником. В начале рассказа племянник оглядывается назад, в годы своей учебы в средней школе в Афинах. Тогда его живой детский интерес вызывал эксцентричный дядя, отшельником живший в предместье и интересовавшийся только шахматами и озеленением. Племянник знал, что дядя Петрос когда-то был талантливым математиком, долгие годы учился и преподавал в университете в Германии и немного - в Кембридже. Однажды он приходит к дяде и объявляет, что хотел бы делать карьеру математика. Дядя Петрос предлагает племяннику решить одну математическую задачку. Если решит - то это будет означать, что математика действительно является его призванием. А если не сможет - то должен дать обещание навсегда забыть о математике. Племянник принимает условие, и, так и не найдя решения, по достижении совершеннолетия все-таки отправляется в университет в США в полной неуверенности в своем будущем математика. Его случайный попутчик, оказавшийся математиком, объясняет ему, что задача, заданная ему дядей, была ничем иным, как Догадкой Гольдбаха. Оказалось, что дядя потратил всю свою жизнь в поисках ее доказательства, но так и не нашел его. Во второй части романа читатель видит рассказчика вернувшимся в Грецию и требующим у дяди объяснений: почему тот задал ему неразрешимую задачу. И тогда мы добираемся до "Истории дяди Петроса Папахристоса", которую рассказчик подает нам в третьем лице. Это очаровательная история о математике и гениальности, о гордыне и навязчивой идее, которые все вместе составили жизнь дяди Петроса. Третья часть романа переносит нас в наши дни. Петрос теперь 80-летний старик, и слегка не в себе. Племянник, все-таки начавший свою математическую карьеру, но впоследствии оставивший ее, вновь пытается получить от дяди ответ на вопрос, который не получил когда-то. И далее следует очаровательный конец повествования с великолепной кульминацией. Доксиадис - удивительный рассказчик. Он побуждает вас переворачивать страницы одну за другой, раскрывая вам историю и мир ученого-математика, всю свою жизнь целеустремленно преследовавшего единственную цель: подтверждение Догадки Гольдбаха. А это, между прочим, самая простая и самая трудная из догадок: каждое четное число, большее 2, представляет собой сумму двух простых чисел. Звучит элементарно, неправда ли? (© J.A, перевод assembler.ru)

В этом году роман, по случаю слегка переработанный автором, опубликован на английском языке сразу в США и в Великобритании, двумя издательствами - Faber and Faber Limited (Лондон) и Bloomsbury Publishing (Нью-Йорк). И для того, чтобы сделать потенциальному бестселлеру необходимую рекламу, кто-то из сотрудников того или другого издательства предложил гениальный ход: бросить челлендж ценой в пресловутый миллион баксов. Тогда казалось, что никто особо ничем не рискует (жалко будет, если человека теперь уволят), ведь над подтверждением Догадки Гольдбаха люди бьются сотни лет, и до сих пор даже не уверены, что таковое существует.

Насчет увольнения сотрудника - шучу, конечно. Они ведь тоже не лыком шиты. Никто никого не уволит и никакого миллиона никто никому не заплатит, так что ежели кто уже точит фомку, чтобы вскрывать мой пуленепробиваемый сейф, оставьте железо в покое. Правила вызова составлены из того предположения, что потенциальные разгадчики загадки Гольдбаха живут исключительно либо на территории США, либо на территории Великобритании. Как будто если Догадку докажет не-американец или не-англичанин, то это как бы и не будет никаким доказательством. Ну да им виднее.

Так не доставайся ж ты никому! Этой публикацией я нарушаю все правила вызова, установленные издательством, но уж постараюсь сделать так, чтобы мой миллион было бы трудно присвоить кому-нибудь шустрому. Впрочем, все возможно. В правилах же не сказано, что миллион будет вручен автору. Там сказано, что миллион будет вручен выполнившему условия. Эй, резиденты США и Великобритании, для вас еще не все потеряно!

В английском языке то, что я сейчас делаю, определяется идиомой "vsju malinu obossal, gad".

Впрочем, шутки в сторону. Вот доказательство истинности Догадки Гольдбаха.

Примечание от 30.05.00: На самом деле, то что вы можете прочитать далее - это только первый, несовершенный, вариант. Возможно, он будет полезен для понимания основных идей, но его недостатки не позволяют считать его настоящим доказательством. Настоящее сухое математическое доказательство здесь.

Примечание от 27.07.00: Еще более сухое и, возможно, более корректное (или истинное, чем черт не шутит?) доказательство Догадки прислал Тим Туманный. Читайте!.

Еще раз повторю формулировку Догадки:

"Каждое четное число, большее двух, может быть представлено в виде суммы двух простых чисел."

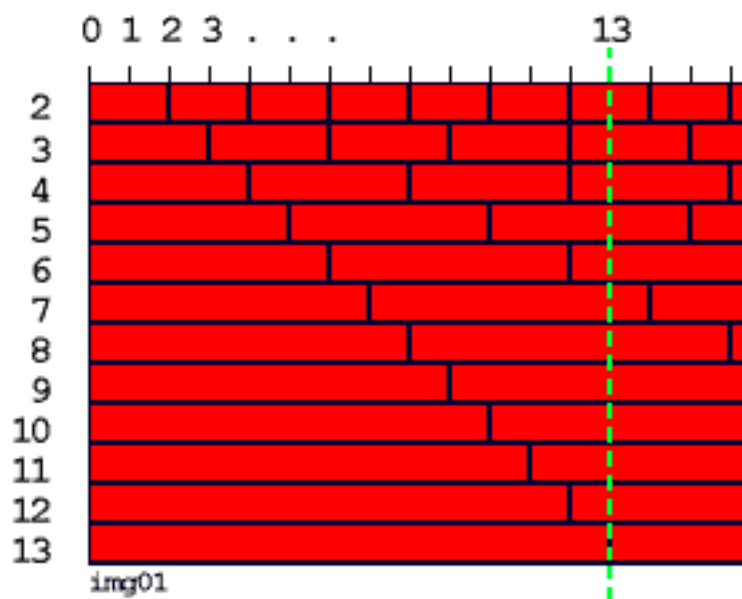
Для начала поговорим о природе простых чисел. Всякому, кто когда-нибудь интересовался простыми числами, знакомо чувство удивления, которое вызывает видимая бессистемность их расположения в числовом ряду. Казалось бы, про ряд натуральных чисел люди должны знать все, ведь это самые первые числа, с которыми познакомились наши пращуры сразу после того, как однажды на охоте завалили не одного мамонта, а двух. И вот в этом незыблемом от Начала Времен монолитном строю, оказывается, полным-полно бойцов, которые стоят где хотят, и совершенно непонятно, как они там оказались.

Причем любой здравомыслящий человек, конечно же, понимает, что никакого хаоса нет, и вот это-то и раздражает больше всего. Если нет хаоса - значит, есть система. Видимо, сложная, но все-таки система. Так в чем же она?

Для ответа на этот вопрос, и раз уж мы обсуждаем натуральные числа, самое время вспомнить о кирпичах.

Допустим, перед нами стоит задача определить, является ли простым некое число (на рисунке - любимое число 13, для примера). Естественной процедурой в таких случаях является поочередная попытка деления тестируемого числа на все числа, меньшие его, начиная с 2, на предмет определения делимости нацело. Смешно об этом говорить, но если число делится без остатка только на самое себя, то оно простое.

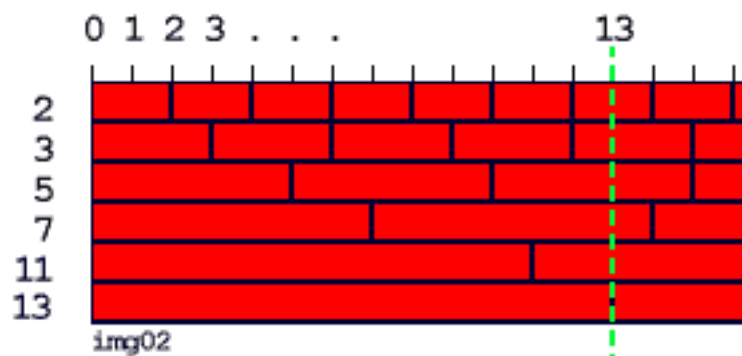
Нарисуем все потенциальные делители так, как показано, в виде кирпичной кладки.



Идея, по-моему, достаточно очевидна. Например, делитель "2" укладывается в кладке до отметки "13" шесть раз целиком, и еще пол-кирпича торчат наружу. А вот делитель "3" - четыре раза целиком и еще кусочек. И так далее.

Мы видим, что на отметку "13" попадает стык кирпичей в одном-единственном ряду - в ряду "13". Не нужно быть бакалавром математики, чтобы сообразить, что смысл этого рисунка, если исключить из рассмотрения избыточный нижний ряд, можно сформулировать следующим образом: *"Если на отметку числа не попадает ни один стык кирпичей, то оно простое"*.

Теперь приглядимся к кладке повнимательнее. Посмотрите: все стыки в ряду "4" попадают на стыки в ряду "2"! Это значит, что если тестируемое число не попадает на стык в ряду "2", то уж на стык в ряду "4" оно точно не попадет! А мы, дураки, мучились, раствор мешали, на прораба матерились... Оказывается, ряд "4" совершенно неинтересен в смысле тестирования чисел на предмет их простоты. Убираем его нафиг. А вместе с ним и ряды "6", "8", "9", "10" и "12", так как все их стыки тоже надежно совпадают со стыками вышележащих рядов. Смотрим, что получилось.



Мама дорогая! Так это ж все сплошь простые числа! Вот оно, первое гениальное прозрение вольного каменщика: *"Чтобы выяснить, является ли число простым, достаточно установить отсутствие его делимости нацело на все простые числа, меньшие его."*

Уважаемые товарищи программисты! Не загружайте процессоры бесполезной работой! При определении простоты чисел используйте в качестве делителей только простые числа! (За исключением случаев, когда вы пишете на JavaScript. Тогда вы можете на этот призыв наплевать - все равно скрипт работает на чужом компьютере. Именно так я и сделал в этом примере.)

Впрочем, все вышесказанное никаких откровений в себе не содержит и давно всем очевидно. Не волнуйтесь: это всего лишь подготовка строительной площадки. А вот теперь, еще раз хорошенько

всмотревшись в кирпичи, мы готовы сделать действительно принципиально важное для доказательства Догадки Гольдбаха умозаключение. Вот оно:

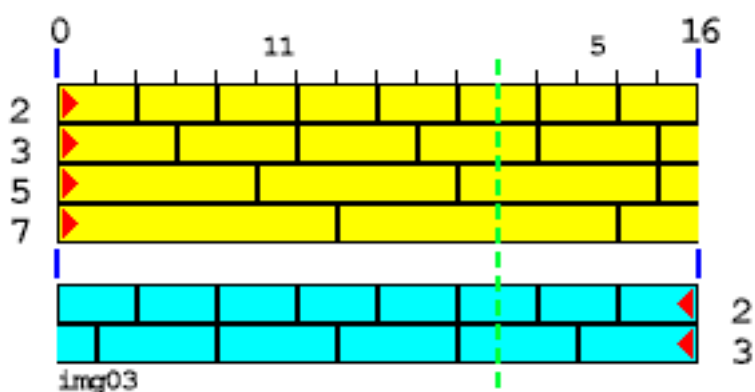
Простое число есть совокупный продукт
от всех простых чисел, меньших его.

Каждый ряд в кирпичной кладке - это периодический процесс с длиной периода, равной соответствующему ему простому числу. В точке 0 числовой шкалы узлы всех периодов совпадают. Очередное простое число образуется в точке, в которой периоды всех предыдущих простых чисел не имеют узлов. Следующее простое число образуется в новой точке, зависящей не только от всех предыдущих простых чисел, но и от вновь найденного.

Не удивительно, что ни на первый, ни на второй взгляд невозможно обнаружить системы в расположении простых чисел в числовом ряду! Ведь каждое простое число - это каша из всех предыдущих, тем более перемешанная и многокомпонентная, чем дальше от начала координат мы находимся.

Из этого тезиса можно делать разные полезные выводы, впрочем, имеющие отдаленное отношение к теме. Например, можно смело утверждать, что не может существовать никаких других строгих правил определения произвольного простого числа, кроме полного перебора всех простых чисел, меньших его.

И вот теперь, разобравшись с природой простых чисел, вернемся собственно к Догадке, и, конечно же, к кирпичам. Разве что раскрасим их теперь в какие-нибудь веселенькие цвета.

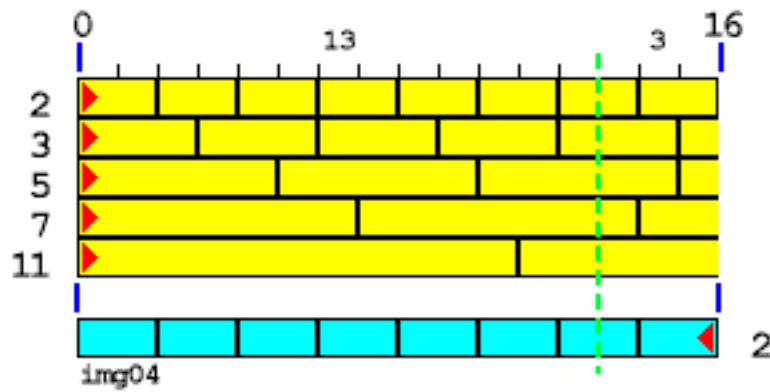


Этот рисунок уже прямо иллюстрирует Догадку и способ ее подтверждения. Для примера мы взяли число 16, которое, как мы знаем, является четным, и, если верить Гольдбаху, должно иметь два простых слагаемых. Не станем томить, таковые действительно имеются. Например, 11 и 5.

На этом рисунке слагаемое 11 покрашено в желтый цвет и представлено не само собой, а совокупностью формирующих его рядов кирпичей, соответствующих меньшим простым числам. Кирпичи, составляющие же слагаемое 5, покрашены в голубой цвет. Кроме того, начало кладки голубых кирпичей мы перенесли в точку "16", и направили кладку навстречу желтой. Видим, что имеет место точка randevu, в которой встречаются отложенное слева число 11 и отложенное справа число 5. В этой точке, в соответствии с изложенным выше кирпичным взглядом на природу простых чисел, ни в одном ряду нет стыка кирпичей.

Целью всех этих, без сомнения, допустимых манипуляций, является переформулировка Догадки Гольдбаха в таком виде: для каждого четного числа можно составить хотя бы один вариант описанной кирпичной кладки, в котором имеется точка randevu.

Почему сказано "вариант кладки" - потому что возможны и другие варианты, как показано на следующем рисунке.



Давайте еще раз рассмотрим рисунок img03, ибо он позволяет сформулировать условие существования точки randevu. Видно, что существует 15 точек, каждая из которых потенциально может стать точкой randevu. Но всем, кроме одной, мешают стыки рядов кирпичей, иногда одного, а иногда и сразу нескольких. Разберемся с каждым из рядов по отдельности.

Правда, на рисунке img04 есть еще точка, отстоящая слева на 1, а справа - на 15, которой не мешает ни один стык кирпичей. Такая точка имеется у всех пар простых слагаемых, одно из которых - 3. Существование этой точки еще ждет своего объяснения, и, я думаю, кирпичная теория простых чисел способна дать его.

Видно, что желтый и голубой ряды кирпичей "2" синфазны. Их стыки совпадают. Это не удивительно, ведь число 16 кратно 2. Эти ряды будут совпадать и для любых других четных чисел. И они отсеивают половину всех потенциальных точек randevu. Если бы фазы голубого и желтого рядов "2" не совпадали, тогда совместными усилиями они бы отсеяли все точки. Однако фазы рядов "2" не совпадают только у нечетных чисел.

А вы-то, небось, думали, что нечетное число не может быть суммой двух простых чисел, потому что сумма двух нечетных всегда четна? На самом-то деле причина в кирпичах!

А вот ситуация с желтым и голубым рядами "3" гораздо интересней. Поскольку число 16 не кратно 3, то фазы этих рядов не совпадают. В результате ряды "3" еще отнимают каждые две из трех потенциальных точек randevu, оставшихся после разгрома, учиненного рядами "2". Очевидно, что после их совместных усилий остается вот какая доля потенциальных точек randevu:

$$\frac{1}{2} \times \frac{1}{3} (1)$$

Следующий ряд - "5". К сожалению, на рисунке img03 ситуация не очень удачная - нет голубого ряда "5", потому что число 16 слишком маленькое. То же касается и ряда "7". Впрочем, мы уже достаточно подготовлены, чтобы забросить возню с кирпичами и обратиться-таки к абстрактным рассуждениям. Если бы исследуемое четное число было побольше, и у второго слагаемого был бы голубой ряд "5", то он бы мог быть синфазен желтому (например, для числа 20), или несинфазен (например, для числа 22). Будем ориентироваться на худший для нас второй случай. Рассуждая аналогично рядам "3", можно считать, что несинфазные ряды "5" оставят только 3 из 5 точек, оставшихся после совместного воздействия рядов "2" и "3". То есть доля оставшихся потенциальных точек randevu будет:

$$\frac{1}{2} \times \frac{1}{3} \times \frac{3}{5} (2)$$

Аналогично для рядов "7":

$$\frac{1}{2} \times \frac{1}{3} \times \frac{3}{5} \times \frac{5}{7} (3)$$

Очевидно, зависимость доли остающихся потенциальных точек randevu от числа рядов образует прогрессию. Каждый очередной ее член формируется путем умножения предыдущего на величину,

определяемую очередным простым числом, то есть зависит сразу от всех предыдущих простых чисел и от текущего. (Обратите внимание, насколько это согласуется с природой простых чисел, которую мы обсуждали выше!) Первый член этой прогрессии - $1/2$. Остальные в общем виде можно записать так:

$$q_i = q_{i-1} \frac{P_i - 2}{P_i} \quad (4)$$

Здесь:

- q_i - очередной (i -й) член прогрессии
- P_i - соответствующее этому члену простое число из ряда простых чисел, начиная с 2

Также очевидно, что прогрессия (4) бесконечно стремится к 0.

Сформулируем достаточное условие существования точки рандеву: **"Для того, чтобы для некоторого четного числа E_j существовала хотя бы одна точка рандеву, достаточно, чтобы интерполированное значение прогрессии q_i при P_i близком к половине E_j было не меньше, чем значение прогрессии:**

$$e_j = 1 - \frac{E_j - 2}{E_j} \quad (5)$$

Поясним, откуда здесь что.

$(E_j - 2)/E_j$ - это максимальная доля точек, которые не должны являться точкой рандеву для того, чтобы существование точки рандеву стало возможным. Наверное, проще пояснить на примере. Смотрим рисунок . Здесь $E_j = 16$. Число потенциальных точек рандеву - $16 - 1 = 15$. Но существование хотя бы одной точки рандеву возможно лишь в том случае, если не-"точками рандеву" будут не более $16 - 2 = 14$ из них. То есть их максимальная доля составляет $14/16$.

$1 - (E_j - 2)/E_j$ - это, следовательно, минимально необходимая доля действительных точек рандеву (то есть доля одной-единственной точки).

Наконец, почему P_i должно быть близко к половине E_j ? Да потому, что Догадка как раз про это: про четное как сумму двух простых. И еще вот почему.

Как уже было сказано, нас интересуют наихудшие для существования точек рандеву случаи. Если в этих случаях условие существования выполняется - то уж во всех других оно будет выполняться точно. Случай, когда простые слагаемые близки к половине составляемого ими четного числа, как раз из таких. Количество желтых и голубых рядов кирпичей в этих случаях примерно равно, в результате чего складываются наиболее благоприятные возможности для появления их несинфазности. Кстати, именно для таких, а вернее, для предельно худших случаев составлена прогрессия (4), потому что она вообще не предполагает наличия синфазных рядов (кроме, естественно, ряда "2").

Ну что ж, осталось совсем немного: убедиться, что достаточное условие существования точки рандеву действительно выполняется. Щелкайте - и убеждайтесь.

Убедились? Есть вопросы, да? Первый бросается в глаза сразу: почему в начальной части $q_i < e_j$? Ну что ж, если вы хотите проверить, что в этой части каждое четное число имеет пару простых слагаемых, просто посчитайте, это нетрудно (или поверьте предыдущим поколениям математиков). А насчет правоты достаточного условия - так оно ведь достаточное, а не необходимое. Здесь оно не работает по двум причинам. Во-первых, помните, что прогрессия (4) составлена в расчете на худший из худших случаев, когда голубых и желтых рядов - примерно поровну. Если вы учтете для первых нескольких чисел конкретные более благоприятные расклады (это выразится в замене в выражении (4) двойки на единицу в случае синфазности соответствующих рядов), весьма вероятно, все встанет на свои места. И во-вторых, на начальном участке велико влияние краевого эффекта, ведь наше достаточное условие в некотором роде - статистическое, и плохо работает на небольших выборках. Последнее утверждение, конечно же, образное. На самом же деле краевой

эффект проявляется в том, что при небольшом числе младших простых чисел, определяющих слагаемые, на значение прогрессии (4) большое влияние оказывают дробные части от деления E_j на младшие простые числа (грубо говоря, части кирпичей, выступающих наружу).

Правда, насчет краевого эффекта - это просто догадка. Есть ли он на самом деле, и в какую сторону он влияет, я не анализировал, потому что задача эта громоздкая и в нашем контексте бессмысленная.

Второй вопрос возникнет у тех, кто разберется в программе. Она составлена "наоборот": мы не задаемся четными E_j , подбирая потом подходящие P_i , а последовательно перебираем P_i и получаем из него E_j простым умножением на 2. На самом деле, никакого вопроса здесь нет: нам достаточно построить кривые, а как - это безразлично. Нам даже все равно, четные ли числа E_j .

Если быть до конца честным, то получаемые с помощью выражения (4) значения прогрессии q_i - это не самые минимальные из возможных значений минимальной доли точек randevu, но они очень близки к минимальным, и ими вполне можно пользоваться для проверки достаточного условия существования точки randevu. Причем, чем дальше по числовой оси, тем с большей надежностью. Убедиться в этом можно с помощью примера, который вычисляет реальные доли потенциальных точек randevu для пар простых чисел, подозрительных на то, что они дают в сумме заданное четное число. Сохраните пример на диск и попробуйте поиграть с переменной e_j , давая ей разные четные значения. Видно, что значения долей для этих пар образуют седло, и результат вычисления выражения (4) всегда находится в его нижней части. Извините за невнятность примера, но это - факультативная информация.

Кстати, вот еще один пример, позволяющий убедиться в справедливости нашего метода. Здесь сравнивается реальное количество пар простых слагаемых для каждого четного числа (N_{pairs}) и прогнозируемое с помощью прогрессии (4) минимальное количество точек randevu ($q_i * E_j$). Совершенно однозначно видно, что после преодоления краевого эффекта ожидаемое минимальное количество точек randevu надежно меньше, чем реальное число пар, но, конечно же, больше 1. То есть наша оценка действительно учитывает вариант "хуже худшего", но показывает, что всегда имеется не менее одной пары простых слагаемых.

Итак, мы убедились, что e_j стремится к 0 гораздо быстрее, чем q_i . **Догадка Гольдбаха верна и подтверждена.**

В заключение несколько замечаний.

Первое - философское. Обратите внимание НАСКОЛЬКО e_j быстрее стремится к 0. И это, учтите, при наихудших условиях! Поэтому Догадка - это ОЧЕНЬ сильный закон. И, наверное, в этой силе - пролог ее мистической истории. Вот почему она буквально бросилась в глаза Гольдбаху (а если быть точным - то еще Декарту) во времена, когда простые числа не могли иметь никакого практического приложения и интересовали разве что чудаков. Вот почему люди искали, ищут и будут искать ясное и строгое ее доказательство: им просто не верится, что у такого сильного закона может его не оказаться. А с практической точки зрения эта сила проявляется в том, что пары простых слагаемых в подавляющем большинстве случаев обнаруживаются уже в непосредственной близости от половины четного числа. Убедитесь!

Второе - конкретное. Как видим, приведенное доказательство истинности Догадки нельзя считать абсолютно строгим. А возможно ли действительно строгое доказательство? Ответ: нет! Все те люди, включая дядю Петроса, кто потратил свои жизни на поиск доказательства, на самом деле строили вечный двигатель. Дело в природе простых чисел. Каждое очередное простое число ортогонально всем предыдущим и как бы создает новый мир, в котором не действуют законы предыдущих миров. И уж тем более для всех этих миров нет единого закона. Как невозможно точно

выяснить, является ли взятое наугад нечетное число простым, не перебрав все предыдущие простые числа, так невозможно строго подтвердить Догадку.

Третье - эмоциональное. Печальное завершение романтической истории, правда? Так хотелось, чтобы у Догадки было такое же простое и красивое доказательство, как она сама: полтора десятка слов, и ничего не добавишь и не выкинешь... Вместо этого - кирпичи и приблизительные оценки. Жаль...

Догадка Гольдбаха (версия 2.1)

Автор: Svet(R)off

Дата: 2002-06-14

Догадка Гольдбаха была высказана прусским математиком Кристианом Гольдбахом в 1742 году в письме Леонарду Эйлеру и формулируется следующим образом: "Каждое четное число, большее двух, может быть представлено в виде суммы двух простых чисел." Попытки доказать истинность Догадки Гольдбаха или подтвердить невозможность такого доказательства до настоящего времени успеха не имели.

Предыдущая попытка от assembler.ru тоже была не совсем удачной. Нынешняя кажется более убедительной. Здесь используются те же идеи, но уже нет никаких кирпичей, зато есть довольно внятная математика, устранены кое-какие проблемы и выявлены новые существенные обстоятельства.

Еще одна очень интересная попытка доказательства представлена Тимом Туманным. Всем интересующимся этой проблемой рекомендуется к изучению и обсуждению в первую очередь.

1. Основная идея предлагаемого доказательства заключается в том, чтобы сопоставить каждому четному числу E некоторую величину SE , заведомо меньшую, чем действительное число пар простых чисел, дающих в сумме число E . Четное число E , для которого значение $SE \geq 1$, обязательно будет иметь не менее одной пары простых чисел, дающих в сумме число E . Если условие $SE \geq 1$ выполняется для всех E , то Догадка Гольдбаха верна.

2. Каждое четное число E можно представить в виде суммы двух чисел с помощью $E/2$ уникальных пар. Необходимо исключить из этого количества все пары, в которых содержатся не простые числа. При определении, является ли число простым, будем руководствоваться правилом: *число является простым, если не делится на все простые числа, меньшие или равные корню квадратному из этого числа.*

Возьмем для примера четное число 78. Его можно представить в виде суммы двух чисел с помощью 39 пар: 1+77 2+76 3+75 4+74 5+73 6+72 7+71 8+70 9+69 10+68 11+67 12+66 13+65 14+64 15+63 16+62 17+61 18+60 19+59 20+58 21+57 22+56 23+55 24+54 25+53 26+52 27+51 28+50 29+49 30+48 31+47 32+46 33+45 34+44 35+43 36+42 37+41 38+40 39+39.

3. Исключим из числа имеющихся пар те, которые содержит числа, делимые на 2. Останется либо половина от определенного в п.2 числа пар (как, например, для $E=16$: 1+15 3+13 5+11 7+9), либо больше половины (как для $E=18$: 1+17 3+15 5+13 7+11 9+9). Воспользуемся первым случаем, потому что эта величина меньше, и, следовательно, соответствует принятому методу доказательства. Итак, после исключения пар, числа в которых делятся на 2, останется $\frac{E}{2} \times \frac{1}{2} = \frac{E}{4}$ пар.

В примере с числом 78 на этом этапе подлежат исключению 19 пар: 1+77 **2+76** 3+75 **4+74** 5+73 **6+72** 7+71 **8+70** 9+69 **10+68** 11+67 **12+66** 13+65 **14+64** 15+63 **16+62** 17+61 **18+60** 19+59 **20+58** 21+57 **22+56** 23+55 **24+54** 25+53 **26+52** 27+51 **28+50** 29+49 **30+48** 31+47 **32+46** 33+45 **34+44** 35+43 **36+42** 37+41 **38+40** 39+39.

Остается 20 пар: 1+77 3+75 5+73 7+71 9+69 11+67 13+65 15+63 17+61 19+59 21+57 23+55 25+53 27+51 29+49 31+47 33+45 35+43 37+41 39+39.

Предлагаемая оценка дает значение 19.5, что меньше фактического числа оставшихся пар и согласуется с принятой методикой доказательства.

4. Исключим из числа оставшихся после п.3 пар те, которые содержат числа, делимые на 3. Следует оценить, какое наименьшее количество пар останется. Поскольку число 2, использовавшееся в предыдущем пункте, и число 3 - простые, то можно было бы утверждать, что при достаточно больших E это количество будет приближаться к $1/3$ части от числа пар, оставшихся после предыдущего пункта. Однако количество пар должно выражаться натуральным числом. Следовательно, имеют место ошибки округления. Чтобы учесть их в наихудшем варианте, вычтем из результата оценки единицу: $\frac{E}{4} \times \frac{1}{3} - 1$

В примере с числом 78 на этом этапе подлежат исключению 7 пар: 1+77 **3+75** 5+73 7+71 **9+69** 11+67 13+65 **15+63** 17+61 19+59 **21+57** 23+55 25+53 **27+51** 29+49 31+47 **33+45** 35+43 37+41 **39+39**.

Остается 13 пар: 1+77 5+73 7+71 11+67 13+65 17+61 19+59 23+55 25+53 29+49 31+47 35+43 37+41.

Предлагаемая оценка дает значение 5.5, что меньше фактического числа оставшихся пар и согласуется с принятой методикой доказательства.

5. Продолжая рассуждения, исключим из оставшихся после п.4 пар те, которые содержат числа, делимые на 5. Первое число в каждой паре делится на 5 примерно в $1/5$ случаев. Второе число также делится на 5 тоже примерно в $1/5$ случаев, не обязательно других и не обязательно тех же самых. В худшем варианте те и другие - разные случаи, и тогда пар, в которых ни одно из чисел не делится на 5, примерно $3/5$ от числа, оставшегося после предыдущего пункта. С учетом ошибки округления их количество: $\left(\frac{E}{4} \times \frac{1}{3} - 1\right) \times \frac{3}{5} - 1$

В примере с числом 78 на этом этапе подлежат исключению 4 пары: 1+77 5+73 7+71 11+67 **13+65** 17+61 19+59 **23+55** **25+53** 29+49 31+47 **35+43** 37+41.

Остается 9 пар: 1+77 5+73 7+71 11+67 17+61 19+59 29+49 31+47 37+41.

Предлагаемая оценка дает значение 2.3, что меньше фактического числа оставшихся пар и согласуется с принятой методикой доказательства.

(Следует обратить внимание на наличие в числе остающихся пары 5+73. Она не подлежит исключению на этом этапе, так как число 5 - простое.)

6. На следующем шаге исключаются пары, которые содержат числа, делимые на 7. После этого в наихудшем случае остается количество пар, описываемое выражением:

$$\left(\left(\left(\frac{E}{4} \times \frac{1}{3} - 1 \right) \times \frac{3}{5} - 1 \right) \times \frac{5}{7} - 1 \right)$$

В примере с числом 78 на этом этапе подлежат исключению 2 пары: **1+77** 5+73 7+71 11+67 17+61 19+59 **29+49** 31+47 37+41.

Остается 7 пар: 5+73 7+71 11+67 17+61 19+59 31+47 37+41.

Предлагаемая оценка дает значение ~ 0.643 , что меньше фактического числа оставшихся пар. Вместе с тем это число меньше 1, то есть наличие пар простых слагаемых для числа 78 нашим доказательством не гарантируется (оно подтверждено эмпирически). Однако это еще не значит, что доказательство неверно.

Все числа в полученных парах - простые, так как использованное на этом шаге число 7 является наибольшим простым числом, меньшим, чем квадратный корень из 78. Дальнейшие итерации для числа 78 выполнять нет необходимости.

7. Обобщая сказанное выше, каждому E можно сопоставить значение, величина которого никогда не превысит фактическое количество пар простых чисел:

$$S_E = \left(\left(\left(\left(\frac{E}{2} \times \frac{1}{2} - 1 \right) \times \frac{1}{3} - 1 \right) \dots \right) \times \frac{P_E - 2}{P_E} - 1 \right)$$

где P_E - наибольшее простое число, удовлетворяющее условию $P_E < \sqrt{E}$

Работу этого выражения можно проиллюстрировать предлагаемой программой. Программа работает в браузерах, поддерживающий JavaScript и фреймы. Она моделирует процесс отсева пар, содержащих не простые числа. При этом вычисляется оценка количества остающихся пар на каждом шаге отсева (step SE[i]) и накопленная оценка (partial SE(i)). Пробуя различные E, можно убедиться, что ни та, ни другая оценка не превышают реального количества остающихся пар.

8. Очевидно, значения SE образуют последовательность на множестве простых чисел. В контексте доказательства достаточно выполнить ее анализ, пользуясь расчетом. Видно, что значения SE, начиная с некоторого момента, устойчиво превышают 1 и возрастают. В точках, где вступает в силу новое P_E , наблюдаются отрицательные девиации значений последовательности SE. Эти девиации обусловлены тем, что в этих точках в выражении для SE появляется новый множитель $(P_E - 2)/P_E$ и вычитается еще одна единица. Но их влияние с ростом E, очевидно, постоянно уменьшается, поскольку E возрастает линейно, а их появление обусловлено корнем квадратным из E, и, кроме того, $(P_E - 2)/P_E$ стремится к 1. Следовательно, начиная с некоторого момента, девиации никогда не станут настолько велики, чтобы значение SE стало меньше 1.

Вывод. Каждому четному числу E, начиная с некоторого, можно поставить в соответствие значение SE такое, что оно заведомо меньше фактического числа пар простых чисел, дающих в сумме число E, и одновременно больше 1. Следовательно, для каждого такого четного числа существует не менее одной пары простых чисел, дающих в сумме это число. Наличие таких пар для чисел E, меньших указанного, подтверждается эмпирически. **Догадка Гольдбаха доказана.**

Догадка Гольдбаха (версия Тима Туманного)

Автор: Svet(R)off · Комментарии читателей (0)

Дата: 2002-06-14

Замечания:

1. Настоящий текст является интернет-публикацией документа, полученного через электронную почту 25.07.00 в 10:59. Отправителем документа является Тим Туманный
2. Текст публикуется без изменения содержания, то есть таким, каким был получен
3. Текст публикуется с разрешения, содержащегося в сопроводительном письме
4. Первое упоминание о существовании этого текста имеется в электронной переписке от 12.06.00
5. Все замечания и вопросы по содержанию текста следует направлять Тиму Туманному по адресу timtum@mail.ru или ttumanny@hotmail.com, либо в адрес assembler.ru. В последнем случае они будут перенаправлены Тиму Туманному без изменения содержания. Вы можете также ознакомиться с иными интересами автора этого варианта доказательства Догадки в Юмористическом журнале Прикол и Ежедневной газете Денечки.RU.

Проблема Гольдбаха может быть переформулирована следующим образом:

Теорема1. Для любого целого числа t всегда существуют два равноудаленных от него простых числа.

То есть всегда $t=p-c$, $t=m+c$, где p и m - простые числа, c - некоторое целое число 0. Например, для 10 равноудаленными являются простые числа 7 и 13.

Если верна Теорема1, то тогда для любого четного числа $2t=p+m$, что равносильно утверждению Гольдбаха о том, что всякое четное число всегда можно выразить в виде суммы двух простых чисел.

Начнем с общеизвестного доказанного факта: всякое нечетное число можно выразить в виде разности квадратов двух целых чисел.

Действительно, $t^2-(t-1)^2=2t+1$ - общая формула для нечетных чисел.

$5=3^2-2^2$
 $7=4^2-3^2$
и т. д.

Легко заметить, что такое разложение не всегда единственно:

$15=8^2-7^2=4^2-1^2$
 $21=11^2-10^2=5^2-2^2$
 $35=18^2-17^2=6^2-1^2$

То есть видим, что всякое составное нечетное число, имеющее простой множитель p , можно выразить в виде разности квадратов:

$$a^2-(a-p)^2=a^2-a^2+2ap-p^2=p(2a-p)$$

Действительно, число $2a-p$ - число всегда нечетное, то есть число $p(2a-p)$ и есть некоторое составное нечетное число, имеющее множитель p . При $a=p$ мы получаем квадрат числа p . Ясно, что если число p входит в разложение составного нечетного числа со степенью 1, произведение остальных простых сомножителей всегда имеет вид $2a-p$, где a и p - взаимно простые числа. Также ясно, что $a-(a-p)$ не равно 1.

Итак, произведение двух простых чисел p и m есть число нечетное и составное, значит оно может быть выражено в виде разности квадратов двух целых чисел

$$p \cdot m = t^2 - c^2$$

По формуле разности квадратов имеем $p \cdot m = (t+c)(t-c)$.

Если положить $t=a$, $c=a-p$ то как было показано выше $t-c$ не равно 1.

Но по основной теореме арифметики представление числа в виде простых сомножителей единственно, следовательно, если положить $p > m$, тогда обязательно $p = t+c$ и $m = t-c$, что равносильно утверждению Теоремы 1.

Итак, мы получили, что действительно для любого целого числа t всегда найдутся два равноудаленных от него простых числа p и m . Следовательно, Теорема Гольдбаха верна.

Дополнение.

Хорошо иллюстрирует вышесказанное метод конечных разностей. Если взять ряд квадратов натуральных чисел и образовать из него ряд, состоящий из разностей последующего квадрата с предыдущим, получим ряд нечетных чисел:

0	1	4	9	16	25	36	49	64	81	100
1	3	5	7	9	11	13	15	17	19	

Легко заметить, что разность квадратов двух чисел разной четности, из которых меньшее отличается от большего более чем на 1, может быть выражена суммой некоторого нечетного количества слагаемых из ряда первых разностей. Например,

$$42-12=3+5+7$$

$$82-52=11+13+15 \text{ и т. д.}$$

Ряд первых разностей представляет собой арифметическую прогрессию с разностью $d=2$. Сумма n членов арифметической прогрессии выражается как $s_n = [n(a_1 + a_n)]/2$, где в данном случае n -ый член арифметической прогрессии имеет вид $a_n = a_1 + (n-1)d = a_1 + (n-1) \cdot 2$.

Итак, имеем для суммы n членов арифметической прогрессии $s_n = n \cdot (a_1 + n - 1)$.

Рассмотрим s_3 , начиная с $a_1=3$ и далее $a_1=5$

$$s_3 = 3 \cdot (3+2) = 15 = 3 \cdot 5$$

$$s_3 = 3 \cdot (5+2) = 21 = 3 \cdot 7$$

$$s_3 = 3 \cdot (7+2) = 27 = 3 \cdot 9$$

То есть получаем ряд составных нечетных чисел кратных 3. То же верно и для s_5 :

$$s_5 = 5 \cdot (3+4) = 35 = 5 \cdot 7$$

$$s_5 = 5 \cdot (5+4) = 45 = 5 \cdot 9$$

составные нечетные кратные 5.

Итак, по индукции, составные нечетные числа могут быть выражены разностью квадратов двух чисел разной четности, из которых меньшее отличается от большего более чем на 1.

Итак, мы получили, что действительно для любого целого числа t всегда найдутся два равноудаленных от него простых числа p и m . Следовательно, Теорема Гольдбаха верна.

Впрочем, у серьезного и въедливого читателя может возникнуть вопрос: а не доказали ли мы теорему, обратную данной, а именно звучащую так: каковы бы ни были простые числа p и m , всегда найдется целое число t равноудаленное от них. Это доказывается тривиально. В самом деле, числа p и m - нечетные, значит числа $t = (p+m)/2$ и $c = (p-m)/2$ числа целые, откуда автоматически

следует $p=t+c$ и $m=t-c$. Но верно ли обратное, т. е. выполняется ли это для любого t ? Обратимся снова к методу конечных разностей. Мы рассматриваем разности квадратов $t^2 - c^2$, где числа t и c разной четности. Мы уже видели, что такие разности выражаются суммой некоторого нечетного количества первых разностей и $sn=n \cdot (a_1+n-1)$. Ясно, что всегда можно подобрать n равным некоторому простому числу m . С другой стороны, видно, что $a_1=(c+1)^2-c^2=2c+1$. Тогда имеем $sm=m \cdot (2c+1+m-1)=m(2c+m)$. Из постулата Бертрана имеем, что на промежутке от $t/2$ до $2t$ имеется хотя бы два простых числа, одно меньшее, а другое большее t .

Пусть $t-c=m$ простое число. Но с другой стороны $t/2+b=m$ и $t/2=c+b$, откуда $t/2t$, значит число $2c+m$ лежит в промежутке от t до $2t$, то есть там, где по Бертрону существует простое число p . Ясно, что разность между двумя простыми числами p и m всегда равна некоторому четному числу, следовательно всегда найдется такое c , что $2c+m=p$.

Итак, имеем, что какое бы целое число t мы не взяли, всегда найдется такое c , что t^2-c^2 равно произведению двух простых чисел p и m . Далее все ясно.

Я уже знаю, что для доказательства моей последней теоремы можно использовать понятие о квадратичных вычетах и так называемый закон квадратичной взаимности Гаусса. Но можно обойтись и без этих штук. Достаточно того, что как мы уже выяснили, произведение двух простых чисел всегда можно представить в виде разности квадратов. Но с другой стороны имеет место формула:

$$a^2-b^2=(a+b)(a-b)=(a-k)^2-(b-k)^2+2k(a-b)$$

Тогда

$$(a-b)(a+b-2k)=(a-k)^2-(b-k)^2.$$

Пусть даны два простых числа p и m , тогда их произведение представимо в виде разности квадратов $a^2-b^2=(a+b)(a-b)$, где $m=(a-b)$ и $p=(a+b)$. С другой стороны всегда существует некоторое простое число p_1 равное $p-2k$. То есть имея два простых числа, равноудаленных от числа a , всегда можем получить два простых числа равноудаленных от другого числа $a-k$.

Например:

$$\begin{aligned}(8+5)(8-5) &= 3 \cdot 13 = 72 - 42 + 2 \cdot 3 \\ 3 \cdot (13-2) &= 72 - 42 \\ 3 \cdot 11 &= 72 - 42\end{aligned}$$

То есть зная доподлинно исходя из постулата Бертрана, что справа и слева от числа 7 имеются простые числа $p < 2 \cdot 7$ и $m < 7$ и выбрав $p=13$ и $m=3$, получили простые числа 3 и 11 равноудаленные от числа 7.

Еще одно дополнение.

Утверждение Теоремы 1 равносильно следующему: всегда существует некое простое число p , которое при делении на целое число $t > p/2$ имеет ненулевой остаток s , который равен остатку s от деления числа t на простое число m . Например, остаток от деления 13 на 10 равен 3. С другой стороны остаток от деления 10 на 7 тоже равен 3. 10 - это и есть число, равноудаленное от 7 и 13. Особый случай представляет собой тройка чисел 3, 7 и 11. Скажем, остаток от деления 7 на 3 равен 1, а 11 на 7 - равен 4. Но мы можем записать $7=2 \cdot 3+1=3+4$, То есть и здесь приходим к одинаковому остатку 4. Существование одинаковых остатков следует из элементарных положений теории делимости и условия m

[C] Svet(R)off

· [Комментарии читателей \(0\)](#)

Впечатления от *Сегоднячко*

Автор: Svet(R)off

Дата: 2002-06-14

Хотелось бы поделиться впечатлениями от вчерашней (28.03.2000) передачи "Сегоднячко". Там Лев Юрьевич Новоженев вместе с милой Юлианной Шаховой и блистательным Игорем Воеводиным раскручивали народ на предмет поделиться: имел ли кто любовные акты в необычных местах. Не скрою, удивить удалось.

Во-первых, приятно поразили результаты телефонного опроса. Итоговых цифр не помню, но на каком-то этапе "да" было за 1000, а "нет" - меньше 100. Вот и гадай, то ли сограждане у нас такие романтические, то ли понятие "необычное место" у них включает, допустим, кухонный стол.

Но если судить по звонкам, скорее первое, чем второе. Правда, откровенничали в основном мужики, а они, как известно, подвержены в таких рассказах древнему инстинкту рыболовов.

А вот одна симпатичная, судя по голосу, девушка выступила с рассказом о том, как со своим знакомым пришла на деловую встречу в Белорусское посольство, а там оказалась такая подходящая пожарная лестница.

А все говорят, что распад СССР повлек только негативные последствия! Ничего подобного: посмотрите, сколько новых посольств образовалось, сколько в них пожарных лестниц, и сколько симпатичных девушек имеют теперь возможность небанально ожидать назначенного времени для международной деловой встречи.

Запомнился еще рассказ одного джентльмена, с южным акцентом поведавшего о том, как при входе в самолет (с женой) ему как-то по-особому улыбнулась стюардесса. И в полете, пока жена наслаждалась сервисом первого класса, муж вышел в туалет и там у них с этой стюардессой все и случилось.

Дело происходило в 1998 году. Возможно, это была внучка той самой стюардессы, которая в свое время одолжила свой свитерок Эммануэль Арсан. Приятно, что годы идут, а сервис Аэрофлота по-прежнему на высоте.

Но несомненным победителем, конечно же, стал рассказ инженера-ракетчика. Оказывается, отечественные космические технологии предусматривают по двухчасовой готовности подъем на фермы обслуживания неких девушек, которые должны что-то там космическое пришивать нитками к чему-то тоже, само собой, космическому. А чтобы девушки пришили то, что надо, за ними наблюдают инженеры-ракетчики. (Именно так, во множественном числе, и те и другие в рассказе и упоминались.)

На старте стоял печально памятный проект "Фобос". Ласково дул теплый байконурский ветерок. Вокруг, насколько хватало взгляда, расстилалась бесконечная цветущая степь. Наверное, была весна - самое благодатное время в тех местах. Где-то внизу, метрах в пятидесяти, суежилась стартовая команда.

Девушки быстро справились с установленной циклограммой работой и занялись инженерами. Все участники этой истории до сих пор вспоминают ее как самое необычное романтическое приключение в своей жизни.

Только одно обстоятельство озадачивает их. Когда все закончилось и пора было спускаться, они всей компанией долго, сколько позволяло время, искали трусики одной из девушек, но так и не нашли. То ли их сдуло с фермы ветерком, то ли застряли где-то в конструкциях и потом сгорели в пламени двигателей, то ли завалились куда-то. Спускаясь в продуваемом лифте, девушка ежилась и прижимала юбку (допустим, они были в юбках, а не в комбинезонах) к бедрам руками. Все очень

веселились.

Спустя несколько месяцев, при выполнении подлетного маневра к Марсу, связь с "Фобосом" была потеряна. Возможно, как раз в это время там, в холодной неуютной бездне, над неприветливым красным диском, сервопривод антенны честно, добросовестно и безуспешно пытался преодолеть сопротивление изящно обшитой кружевом резинки. Привод визжал и визжал, падал заряд аккумуляторов, но антенна так и не двинулась с места. Через несколько часов все было кончено. Фиолетовое небо Марса неторопливо прочертила блеклая звездочка. Еще двадцать минут - и на красное ложе марсианской пустыни опустился обгорелый кусочек кружевной ткани. Вечный ледяной ветер повлек его по мертвым барханам, загнал в какую-то ложину и быстро занес сыпучим песком.

Во всей этой истории есть две проблемы, которые необходимо обдумать.

Во-первых, при сходных обстоятельствах погиб и второй "Фобос". Неужели наши специалисты по космической технике столь беспечны, что умудрились дважды потерять трусики?

И второе: неужели технология подготовки к пуску Mars Polar Lander тоже предусматривала пришивание чего-то нитками?

Правильная история компьютера (вернее, вычислителя)

Автор: Svet(R)off

Дата: 2002-06-14

Не помню, кто (как бы даже не сам Карл Маркс) как-то взялся распределить, на каком языке с кем разговаривать. Как распределил - тоже не помню. Помню только, что с врагом - на немецком.

С другом - на английском? С девушкой по вызову - на итальянском? С дамой сердца - на французском? С холодильным мастером - на финском? С товарищем по партии - на китайском? С соседом в самолете - на идиш? С соседом по камере - на русском? С продавцом на рынке - на азербайджанском?

Но что с компьютером нужно разговаривать именно на английском - это точно. По крайней мере не на русском.

А не слабо нам мысленно взмахнуть волшебной палочкой? Абак и астролябия, уж Бог с ними, пусть остаются на своих исторических местах. А вот, например, первый калькулятор пусть изобрел не Блез Паскаль в помощь отцу в 1642 году, а крепостной Федька Дранный для учета телесных наказаний при Алексее Михайловиче "Тишайшем", и его за это случайно не запороли насмерть. И поэтому дальше покатилося!

Чарльзу Томасу Кольмарскому, допустим, утер нос наш Михал Василич Ломоносов. Пресловутого Бэббиджа заткнул за пояс, само собой, Колька Лобачевский - кому ж еще? Хермана Холлериха поставил на место, например Пал Карлыч Штернберг, который своевременно бросил валять дурака с астрономией и Москревоеноветом, а занялся делом: основал товарищество на паях Международные Деловые Машины (далее - "МДМ").

Дальше все шло в том же духе. В Штатах, после неудачи с сухим законом, долго искали виновных, посажали-постреляли миллионов двадцать народу потолковых, потом посажали-постреляли тех, кто сажал-стрелял, ну и так далее. Во время войны американцы, вместо того, чтобы расчет траекторий стрельбы автоматизировать, Эккерта с Мочли вместе со всем Пенсильванским университетом как самых умных в заградотряды определили. А потом, в 46-м, вместо возни с ЭНИАКом они долго излечивались от космополитизма в благодатном юконском климате.

С фон Нейманом американцам совсем не повезло. Они его шлепнули еще в 41-м, за одну только приставку к фамилии. А Винер отсидел по полной за увлечение лженаукой "сибирьнетикой", про которую узнал из передач радиостанции "Рука Москвы". За КВ-приемник ему, кстати, еще червонец накинули.

Что случилось потом - общеизвестно. В 53-м наши гении сварганили БЭСМ. Потом за дело взялся штернберговский "розовый гигант" - МДМ. К концу 60-х весь машинно-счетный мир говорил на русском. Дальше всех, по обыкновению, сопротивлялись американцы: настойчиво употребляли неблагозвучную аббревиатуру ЕСМ (произносится "э-кэ-эм") вместо простого и ясного русского слова "вычислитель" (пишется через "я", так как относится к неодушевленному предмету. Ср. "вычислительщик" - человек, работающий с вычислителями).

Был еще такой эпизод. Два друга, молодые парни, Степан Попов и Степан Ямщик, в отцовском огороде слепили из отечественных комплектующих первый одночеловечный вычислитель, "Картошка-1". По сей день всему миру известен товарознак основанной ими компании - картофельный клубень со сгнившим боком.

В настоящее время лидирующее положение на вычислительном базаре занимают такие товарищества, как Умник (известное в первую очередь своими оченьмаленькодвигалками), Малослабость (прогобеспный гигант), МДМ, Солнце, Урочище, Коммунистический Упаковщик и многие другие, составляющие российскую гордость в начале XXI века. По акциям этих компаний судят о состоянии наукоемких и высокотехнологичных отраслей экономики России, для чего в свое время был введен специальный индекс, называемый "Максимально Универсальный инДекс Акций Компаний".

В заключение этого краткого исторического обзора приведем небольшой чатлано-пацацкий словарь по вычислительной технике и электронике:

Классификация информации

Автор: Svet(R)off

Дата: 2002-06-14

Очевидно, что вся информация, доступная человечеству к началу третьего тысячелетия от Р.Х., делится на три больших класса:

1. информация об ассемблере
2. информация, так или иначе касающаяся ассемблера
3. информация, никоим образом не касающаяся ассемблера

Чрезвычайно интересно, не останавливаясь на семантическом уровне приведенной выше классификации, проследить деривационные, и, не побоюсь этого слова, педигриальные свойства генезиса каждого из ее разделов. Ибо естественно, что прямо из этих коренных свойств проистекают качественные характеристики текущего состояния информации соответствующего класса.

Начнем, с информации класса (с), на сегодняшний день наименее структурированной и упорядоченной. Как показывают многочисленные исследования (см., например, фундаментальный труд проф.М.Л.Кнопковича "Эпидемия ящура в Саратовской губернии в 1903 году"), самым значимым источником этой информации во Вселенной является неживая природа. Основной объем т.н. с-информации образовался одновременно со Вселенной в момент Большого взрыва, и на всех последующих этапах ее жизни возрастал незначительно, одновременно подвергаясь постоянной энтропийной эрозии и, в силу этого, деградируя качественно.

Если ограничиться рамками планеты Земля, то в ее истории обнаруживаются два отчетливых генерационных всплеска с-информации. Первый произошел в третьей четверти мелового периода, на исходе мезозойской эры, и именно он, по результатам последних исследований, явился причиной гибели динозавров. (Здесь надо отметить, что, в отличие от Вселенной в целом, на Земле существенную роль в генерации с-информации играет присутствующая здесь живая природа.) Меловой с-всплеск, как выяснилось, объясняется чрезмерной плодовитостью динозавров в условиях благоприятного климата той эпохи. По нашим подсчетам, жившее тогда на Земле количество особей, имевших вес свыше полутора тонн, превышает 17 миллиардов. Что же говорить об их более легких собратьях! Каждый экземпляр динозавра имел до сорока только ближайших родственников, а число двоюродных достигало тысячи и более. Представляете, что творилось в период предрождественских распродаж? А дни рождения? А, наконец, 8 марта? Собственно, необходимость помнить все эти даты и породила тот невероятный объем с-информации, оказавшийся непосильным для слабого мозга пресмыкающихся, и, в конечном итоге, погубивший их.

Второй всплеск, как может догадаться читатель, происходит в современную эпоху, и виновником его является, к сожалению, человек. Длительное время казалось, что праздники и юбилеи не угрожают человечеству. Дело в том, что человек, более ментально продвинутый, чем динозавры, нашел мощное защитное средство против информационной угрозы такого рода. В ходе эволюции примерно половина человечества, впоследствии определившаяся как мужчины, ценой многих лишений выработала уникальное свойство забывать памятные даты своих родных и товарищей, благодаря чему угроза гибели человечества была отложена. Но, оказывается, не отведена вовсе! В один прекрасный момент на арену информационной борьбы выступил интернет, с его бесчисленными download-серверами, и оказалось, что до сорока процентов их ресурсов отведено под многочисленные программы, главной целью которых является напоминание пользователю

о необходимости поздравить одну или несколько особей с каким-либо праздником. С этого момента выживание человечества снова встало под вопрос. Но эта тема, к сожалению, не является предметом нашего нынешнего исследования.

Информация класса (b) несколько более цивилизована, если можно так выразиться, так как основным ее источником является разумное человечество, а точнее, его наиболее передовая часть, называемая "корпорация Microsoft". Значительно меньший b-объем генерируется рядом других корпораций, фирм и физических лиц. В-информация существенно более качественна, чем с-информация, имеет приятную сметанообразную консистенцию и оригинальный привлекательный аромат, легко и полностью усваивается организмом, особенно со второй попытки.

И именно в этом состоит ее опасность. До появления корпорации Microsoft человечество было вынуждено довольствоваться в основном грубой и плохо переваривавшейся с-информацией. Но с-информация хорошо очищала зубы от отложений зубного камня, держала в тонусе кишечник, стимулировала выработку слюны и желудочного сока. Массовая замена ее на b-информацию привела к катастрофическому зашлаковыванию организмов, ослаблению деятельности эндокринных желез и, как следствие, общему снижению стойкости организма к болезнетворным микробам и вирусам. Наиболее яркий пример последствий появления корпорации Microsoft - это пандемия СПИДа. На этом, пожалуй, можно остановиться в описании опасностей b-информации. Употребленная в разумных пределах она, как и все остальное, полезна, поэтому мы с удовольствием включаем ее в наши статьи.

И, наконец, информация класса (a). Наивысшая, без сомнения, категория информации. Недаром столько благородных a-созвучий находим мы вокруг себя. "А-а-а", - первый крик ребенка и последний крик выпавшего из окна. На "а" начинается слово "ассемблер", причем как в русском, так и в английском языке. Самые уважаемые и богатые люди в нашей стране называются "чиновники категории А". Впрочем, довольно примеров. Перейдем к сути. А-информация - это концентрированный интеллект, порожденный высшими проявлениями разума, свободного от забот о бренном теле. Строжайшая аскеза, беспримерный добровольный отказ не то что от наследования виртуальных функций, но даже и от элементарных фигурных скобок, порождают в итоге информацию класса А. Читайте а-статьи! Получайте а-знания! Программируйте а-программы! Ур-а!

Что такое фашизм

Автор: Svet(R)off

Дата: 2002-06-14

Вызывает Михаил Сергеич Горбачев, в бытность свою Первым Папой на Деревне, к себе однажды академиков-профессоров разных, и говорит:

- Господа академики, - говорит, - профессора, и другие младшие научные сотрудники! Я тут давеча затеял в нашей великой державе процессы демократизации и гласности. Предвижу на моем тернистом пути большие проблемы. Постараюсь, само собой, сделать все эти дела с человеческим лицом, но не все у меня получится. Обещаю в отпуск не ездить, пока, знаете ли, не построю перестройку в отдельно взятой стране и вслед за тем не зажгу всемирный пожар ускорения.

Академики захлопали в ладоши и преподнесли Михаилу Сергеичу памятный подарок - бутылку водки завода "Кристалл" с винтом, который тогда еще, в эпоху расцвета застоя, был в дефиците. Непьющий Михаил Сергеич поморщился, видно, чего другого ожидал, и сделал какую-то пометку в своей особой папке. Потом, однако, взял себя в руки и продолжил.

- Только, - говорит, - есть у меня одна забота, которая в корне может подкосить все мои коренные преобразования процветающего нашего советского общества. Боюсь я, что народ, устав от моих направленных на повышение его благосостояния реформ, отшатнется от крыла реформаторов и станет искать сильной руки, которой у меня нет, и которую мы никому не позволим. Вот тут - то и поднимут голову нездоровые тенденции, о которых я уже говорил выше, и о которых еще скажу ниже. В частности, товарищи академики, прошу я вас сформулировать для меня понятие фашизма, поскольку сам я в этом ни черта не понимаю, но чувствую, ох, понадобится оно мне в будущем.

Академики задачу поняли и отбыли из Кремля, еще раз заскочив в тамошний буфет и затоварившись про запас кристалловской, потому что больно подозрительным показались им манипуляции Михаила Сергеевича с особой папкой.

И стали ломать они головы, прихлебывая время от времени кристалловскую. Задача-то непростая оказалась. С одной стороны, фашизм в нашей стране как в сорок пятом победили, так с тех пор больше о нем слуху и не было. Сажать - сажали, стрелять - стреляли, войны развязывать - развязывали. Идеологией гуманной все это оформляли и подписывали. А вот фашизма не было. А как его сформулируешь, если его нет? Не в ФРГ же ехать опыт перенимать! Тем более что все академики, как самые умные, были невыездными.

С другой стороны, раз Главный Папа просит - надо уважить. Значит, знает, о чем говорит. Пронзевает орлиным своим взором пространство и время, и, под руководством КПСС, видит, гений, грядущие проблемы. Значит, надо пахать.

Вот пашут они, пашут. Уж запасы кристалловской иссякают, а конца пахоте не видно, и до угла добежать некогда, пополнить. Не формулируется фашизм, хоть ты тресни.

А за окнами университетов ихних меж тем жизнь течет. Михаил Сергеич, болезный, не сдержал своего обещания, уехал в отпуск. Да и как тут не уехать - реформы вот-вот на самый пик взберутся, уже повышение качества жизни намечаться стало, народ плевать перестал. Слюна кончилась. И тут бац -вылазит мужик какой-то, из наших, из строителей, на танк, и давай по бумажке читать, что его на телевидение не пускают, и что хочет он с Михаилом Сергеевичем за здоровье переговорить. Там, в бумажке, вроде еще было написано про то, что запасы кристалловской кончаются. Но мужик хитрый оказался, он этот абзац читать не стал, думал, наверное, что первым на угол успеет, пока народ ничего не знает.

А академики все пашут.

И тут звонок в дверь. Ну, они, понятное дело, последнюю бутылку, початую уже, под стол, бумажки кое-как по столу исписанные разбросали, и открывают. А там стоят.

Академики, конечно, перепугались. Подумали, Михаил Сергеич призывает ответ держать. Небось, решили, народ от реформ устал, и расцветает в его здоровой сути недобитая фашистская нечисть. А без формулировки как же ее добыешь! Прознает Верхний Папа, что ничего они не сформулировали - полишает к черту всех званий академиков и назначит членами-коррами. В кремлевском буфете фиг тогда кристалловской отоваришься - не та номенклатура.

Привозят их в Кремль. А там лепота! Кабинет зеленым шелком обит, что твой стадион Динамо, если на нем в футбол не играть. Мебель белая. И сидит за этой мебелью простой мужик нашенский, строитель, судя по румянцу на носу.

- А где же Михаил Сергеич? - спрашивают академики, чтобы увести разговор от опасной темы про формулировку фашизма.

- Какой, понимаешь, Михаил Сергеич? Немирович-Данченко, что ли? - говорит нашенский мужик. А сам при этом лезет под белый стол и достает кристалловскую, да не просто с винтом, а еще и с такой специальной пластмассовой затычкой, чтобы, пока наливаешь, желудочный сок успел выработаться.

Как увидели эту затычку академики - поняли, дело швах. Значит, реформы так далеко зашли, что и представить себе Михаил Сергеич на заре не мог. Народ, значит, совсем устал, и фашизму теперь на каждом углу должно быть пруд пруди, и пора отпрашиваться у нашенского мужика в туалет, благо он рядом с буфетом, потому что потом уже не пустят.

Ан нет! Мужик - Борис Николаичем, его, оказалось, зовут - вроде, в хорошем расположении духа. Наливает себе кристалловской, и говорит:

- Господа академики и, понимаешь, профессора! Я тут решил решить разом и навсегда все наши социально-политические проблемы. По этому поводу отбываю на днях в Ирландию проездом через Соединенные, понимаешь, Штаты Америки. Так вот, пока буду ездить - решите-ка для меня одну ма-а-а-аленькую загвоздочку, понимаешь. Танька говорила, что ей один знакомый говорил, что вы у нас самые большие специалисты по фашизму. Ну-у, я не думаю, что вы больше специалисты, чем Чубайс или там этот, как его, Лившиц. Но раз уж вы все равно тоже бездельничаете, сформулируйте-ка вы мне, что же есть на самом деле оно такое - фашизм. Как сформулируете - приходите. А то боюсь, доведется однажды свидеться, а я и не пойму, что это он.

Хряпнул Борис Николаич кристалловской, да и проводил академиков с миром.

Едут они по Москве, а там - чудеса! Кристалловская на каждом углу и даже просто на тротуарах, между углами. Народ веселый ходит, как на демонстрации. Не работает никто. В одном месте пришлось опять танки объезжать. Только там уж никакие мужики сверху на танки не забирались, потому как их, танки, перед этим свежей зелененькой краской выкрасили. Да и заберись на танк, когда он из своей пушенции так ювелирно шарашит!

Видят академики это безобразие, и понимают, что никогда им не сформулировать, что же оно такое - фашизм. Нет у них никаких жизненных опор для такого гениального прозрения. Фашизм - он что? Порядок! А у наших академиков весь жизненный опыт тянет в лучшем случае на бардак.

Остановились у ближайшего тротуара, скинулись на кристалловскую, да и раздавили с горя по двести пятьдесят на каждое академическое рыло. Академик, который по пьянее получился, сел за руль, и поехали они домой к одному из них, у кого еще бутылка в морозильнике завалялась.

Тормозит их, само собой, гаишник. Он запах учуял, когда они еще из-за угла поворачивали, чтобы трамвай обогнать. Разговорились. И заходит у них разговор, естественным образом, за науку. Жалуются академики гаишнику, что вот-де уже второй Большой Папа ставит им задачу, а они ни с места, ни в карьер.

- Эх вы, - говорит гаишник, одной рукой сворачивая голову академической кристалловской, а другой отвинчивая номер с академической же "Чайки", - академики, мать вашу. Что ж вы еще при Михал Сергеевиче ко мне не обратились! У нас в ГИБДД проблема классификации неидентифицируемых существ решена давным-давно. Вот протрезвеете, - тут гаишник приложился к горлу кристалловской, и академикам пришлось нетерпеливо переминаясь с ноги на ногу, пока гаишник не отполовинил посуду, - приносите к нам пересдавать экзамены, там все и покажем.

- Нет, нет, - закричали академики в один голос. - Мы, как истинные ученые, не выдержим в неведении ни минуты, зная, что истина где-то здесь!

- А нальете? - на стал долго запирается гаишник, польщенный вниманием столь высокопоставленных особ.

- А права отдашь? - проявили народную сметку академики.

- Права отдам. А за номером милости прошу к нам в гости.

- Ну не томи, раскрой истину, о великий!

- Ну так вот, - совсем растаял гаишник, скрутил голову второй академической кристалловской и привинтил-таки номер к двери академической же "Чайки". - Берете вы организацию, которую вам приказали проидентифицировать насчет ее фашистской сущности. Кладете перед собой листок бумаги. Только листок тот должен быть разработан в нашем Главном Институте по Борьбе с Дорожной Безопасностью и утвержден Государственной, блин, Думой. А в листочке табличка. Из трех граф, - показал гаишник академикам для ясности два пальца, потом подумал, и добавил еще один. - В первой графе перечислены признаки фашизма, которые наши борцы с дорожной безопасностью удумают. Во второй баллы, которые за эти признаки они же посчитают нужным назначить. А в третьей вы будете ставить галочки. Галочки ставить умеете?

- Умеем, умеем! - радостно загалдели академики.

- Ну и ставьте себе. Призывает организация к наведению порядка - плюс им за это десять, например, баллов. Хоть это ЖЭК какой-нибудь, все равно десять баллов. Имеет флаг красного цвета - плюс пятнадцать. Пусть это и пионерский отряд тимуровцев, помогающий ЖЭКу. Имеет на флаге белый круг - плюс еще пятнадцать. Даже если это японский пионерский отряд, помогающий ЖЭКу. Не говоря уже о центральносимметричных символах разветвленной от центра симметрии формы, находящихся в значительном цветовом контрасте по отношению к полю, на котором они размещаются. За них можно и полсотни баллов накинуть. Набирает организация в ряды пацанву - двадцать баллов. Одевает ее в портупеи - тридцать. Машет руками, в едином порыве приветствуя вождя, - двадцатник. И так далее. Потом берешь калькулятор. Калькулятор-то есть?

Академики все, как один, выхватили из-за пояса логарифмические линейки. Гаишник брезгливо поморщился и допил вторую академическую.

- И считаете сумму. Конечно, если вы - не академики паршивые, а наш справедливый народный суд. Или, на худой конец, сотрудник ГИБДД. Перевалила, допустим, сумма через пятьсот баллов - как Дума, блин, определит, - значит, фашисты вы и гады недобитые. Имеете четыреста девяносто девять баллов - спасибо вам, ребята, за патристическое воспитание подрастающего поколения и другие благородные ваши дела. Уели суть, головастые?

Пораженные глубиной бездны народной мудрости, академики столпились вокруг лежащего на асфальте гаишника.

- Ну чего, братва, вам еще? - раздраженно приоткрыл он левый глаз. - Валите отсюда в свой институт. Мне работать надо.

Молча погрузились. Поехали. Недалече гремела пушечная канонада и сыпался сухой горох автоматных очередей. Дело происходило на набережной. Самый пораженный глубиной народной

мудрости академик, сидевший за рулем, даже не подумал повернуть его, когда на пути "Чайки" случайно оказался чугунный парапёт. Долго еще чугунные обломки кувыркались в сизом воздухе над маслянистой поверхностью Москвы-реки, пока сверкающий лаком автомобиль плавно и неспешно приближался к этой поверхности, навсегда унося в своем чреве неизбывную народную мудрость, познанную учеными мужами.

Гаишник, привлеченный звуками, заглушившими привычный грохот боя, приподнял голову и сопроводил взглядом исчезающую за обрезом набережной "Чайку". Мрачно сплюнул и перевернулся на другой бок.

25. Разное

Интервью с создателем пакета MASM32

Автор: vkim / HI-TECH

Дата: Не указана

Можете ли вы написать более или менее сложную программу под Win32, например, на С, не имея необходимых утилит, заголовочных файлов, стандартной библиотеки? Вряд ли. По крайней мере, затраты времени и труда будут огромными. А ведь примерно в таком положении находятся программисты, пишущие на ассемблере. Не мудрено, что многие, желающие изучить этот язык, отступаются, не написав даже традиционной программы "Hello, world".

Попытки создать пакет для программирования на ассемблере под Windows, который включал бы в себя все необходимое, предпринимались неоднократно. Но лишь один из них приобрел всеобщую и заслуженную популярность - MASM32. Его создатель Стивен Хатчессон (Сидней, Австралия) любезно согласился дать интервью нашему сайту.

- Здравствуйте, Стив. Пожалуйста, расскажите нашим читателям что-нибудь о себе, о своей работе, о своих интересах.

- Вообще-то изначально я вращался в инженерных кругах в области высокоточных приборов и машиностроения. Я инструментальщик и работал в этой области 15 лет. В конце 70-х я поступил в университет, чтобы изучать философию языка и формальную логику.

Сначала я выполнял простую работу на компьютерах (где-то в 1980-м), но потом я выяснил, что компьютеры, доступные для персонального пользования, не могли делать то, что меня интересовало, поэтому я переключился с них на свой собственный бизнес.

Я вернулся в компьютерную область в 1990 году и изучил ассемблер, Бейсик и С примерно в одно и то же время, так как тогда была большая коммерческая потребность в программном обеспечении, и не так много людей были способны делать что-то полезное для компьютеров. Поэтому мне потребовалось изучить ассемблер, так как тогдашние компиляторы не были достаточно гибки, чтобы делать все то, что мне хотелось.

- У вас и сейчас есть свой бизнес? В чем он заключается? - После начала моей карьеры программиста под x86 я стал тем, кого называют Independent Software Vendor, и именно этим я зарабатывал себе на жизнь, пока это было возможно с экономической точки зрения. Я все еще делаю мелкую работу от случая к случаю для старых заказчиков и друзей, но в основном теперь я живу на доход от инвестиций. Я трачу свое время на восстановление старого дома, в котором живу, потому что через некоторое время я стану слишком стар для подобной работы. Это также помогает держать себя в форме. Я стараюсь быть в курсе того, что творится в компьютерной индустрии, и если положение на рынке улучшится, и большая часть того, что нужно сделать для восстановления дома, будет выполнена, я буду работать больше, чем сейчас. **- Изучив ассемблер, Вы создали пакет MASM32, который приобрел огромную популярность во всем мире. Как вам пришла мысль о его создании?**

- Как только с приходом Windows 95 стало необходимо создавать 32-х битные программы, я обнаружил, что инструментов для создания полезных и мощных программ сильно не хватает, и все большее распространение начали получать пакеты, облегчающие программирование для начинающих, но при этом, как правило, терялась большая часть возможностей, позволяющих писать высококачественное программное обеспечение.

Сначала я написал собственный редактор с помощью специализированного компилятора языка Бейсик, поскольку я был знаком с этим языком, и затем начал собирать информацию о создании ассемблерного пакета. Iczelion уже сделал кое-какую подготовительную работу, основываясь на том, что сделал ранее Стив Гибсон, и поскольку у меня был уже опыт в написании Windows-программ, я использовал некоторую часть из спроектированного Iczelion'ом, чтобы создать ранние версии MASM32.

Главной идеей с самого начала было то, чтобы код на ассемблере было писать не труднее, чем на C, но чтобы он был более понятный и более мощный. Начиная с самых ранних версий MASM32, он разрабатывался при участии многих опытных программистов, которые хотели передать свой опыт следующему поколению программистов, и чтобы у них были подходящие инструменты, так же, как и у нас.

- Давно хотелось у Вас спросить, почему в 7-ой и 8-ой версии MASM32 только 24 tutorials Iczelion'a, хотя он написал больше 40? Вы, случаем, не разругались с ним? Знаете, как это бывает... - Нет, это в основном потому, что достучаться до Iczelion'a в настоящее время очень трудно, но его tutorials можно легко найти на его сайте, поэтому если они нужны программисту, он может получить их без особых проблем. **- Многие хакеры и крэкеры используют MASM32 с незаконными намерениями. Вас это не волнует?**

- Эта проблема не одного только MASM'a или ассемблера вообще. Незаконные программы, повреждающие компьютеры, будь то вирусы или трояны написаны в большинстве своем на языках высокого уровня, таких как C++, Pascal и скриптовых языках вроде VBS и ему подобных. Подавляющее большинство людей, использующих MASM32, пишут полезные вещи, которые требуют больше времени и умения, чем вредоносный код, и узнают много нового. Опытный в ассемблере программист будет в более выгодном положении, программируя на языках высокого уровня, так как он понимает больше, чем те, кто не знает ассемблера.

Основываясь на своем опыте, могу сказать, что те, кто начал с крэкинга или написания вирусов бросают это занятие, как только они становятся хорошими программистами, так как писать полезные и умные программы гораздо более выгодно, чем отнимать время у других людей и портить им жизнь.

- Понятно. Скажите, Стив, нуждается ли Вы в помощи со стороны пользователей MASM32? Как люди могли бы помочь вам? - Создание MASM32 стало возможным, потому что много разных людей помогало в этом. Я написал значительное количество того, что входит в пакет, а также должен координировать его развитие. Если кто-то предлагает помощь или добавить что-то в пакет, мы решаем с ним, что лучше сделать и как интегрировать это в пакет. Таким образом достигается лучшее разделение труда. **- Что же ожидает нас в следующих версиях MASM32?** - Я в целом доволен текущей 8-ой версией MASM32, где я проапгрейдил довольно много разного, добавил больше исходного кода и уменьшил занимаемое им место, поэтому я пока не планирую новую версию. Если появится такая надобность, я сделаю это, но для создания цельного пакета требуется много времени, поэтому я изменяю его только тогда, когда это действительно нужно. Я стараюсь быть особенно осторожным с размером пакета, так как MASM32 качают многие, и это требует большой пропускной способности. **- Вот Вы создали инструмент для программирования на ассемблере, но бытует мнение, что писать программы на этом языке бессмысленно, потому что современные компиляторы умеют оптимизировать код очень хорошо. Что вы можете сказать на это?**

- У языков высокого уровня есть своя ниша, так как использующееся сейчас во всем мире программное обеспечение не всегда должно быть качественным, и хороший компилятор может показать очень неплохие результаты, если код написан опытным программистом, который понимает, как работает компилятор и как надо оптимизировать код по размеру и по скорости. Однако когда нужно написать то, что называют критичным кодом, ассемблер может предоставить

людям с опытом возможности, выходящие далеко за рамки возможностей компилятора.

Другим преимуществом является свобода проектирования и возможность делать все, что можно сделать в данной операционной системе, но чего зачастую не может сделать компилятор. Выбор определяется задачей, и если под рукой есть ассемблер и опыт правильного написания кода, можно многое написать и оптимизировать под задачу.

- Хорошо, а не таится ли угроза существованию ассемблера с другой стороны? Как вы думаете, что будет с приходом 64-х битных процессоров?

- Ассемблер в том виде, в котором мы знаем его в 32-х битном окружении, когда-нибудь исчезнет, но всегда будут требоваться те, кто умеет использовать ассемблер и знает низкий уровень, на котором работает железо.

Ассемблер - это непростой для изучения язык, и для него обычно требуется определенный склад ума, но программист, который сможет постичь его кристальную логику, будет иметь гораздо лучшее представление о том, что может компьютер и как работают компьютерные языки. Хотя программист, который знает только более простые языки, может зачастую писать полезные программы, программист, понимающий, как работает компьютер, может делать то, что не под силу обычному программисту.

Также надо учитывать, что у платформы x86 огромное количество пользователей, поэтому пройдет еще много времени, пока 32-х битное x86-железо перестанет использоваться. Далеко не многие в мире могут позволить себе самое новое компьютерное железо, поэтому большинство людей используют компьютеры, которые намного медленнее, чем последние модели. Программное обеспечение, спроектированное в расчете на современное железо, выполняется на более ранних компьютерах очень плохо, поэтому маленький быстрый код будет выполняться на большинстве компьютеров быстрее, чем большой и медленный, которому нужен самый новый процессор, чтобы он работал нормально.

- Наверное, нет двух программистов, которые предпочитали бы один и то же набор утилит. Какие инструменты используете вы, программируя на ассемблере?

- В основном те, которые идут вместе с MASM32, так как я написал их в первую очередь для персонального использования. Я не люблю концепцию IDE, так как считаю, что она ограничивает меня в том, как я должен писать свой код. Редактор очень аскетичен, поскольку я не люблю все эти "рюшечки", но он очень удобен для того, что мне нужно. Единственное из используемого мной, что я не могу поместить в пакет MASM32, это очень старый редактор ресурсов от Symantec, так как это лицензионная программа, которую нельзя распространять.

Доступной справочной информации слишком много, чтобы ее можно было поместить в MASM32. Лично я в основном использую старый файл win32.hlp, когда мне нужно узнать о последних функциях - PLATFORMSDK, а также второй том Интеловского справочника по ассемблеру. Это все доступно в интернете, также вы можете заказать PLATFORMSDK на CD, если время скачки слишком велико.

Я тяготею к написанию маленьких специализированных утилит, поскольку часто в них нуждаюсь, но зачастую они настолько специализированны, что будут без надобности кому-то еще. Такое бывает со многими программистами, и в этом нет ничего необычного.

- Использовали ли вы когда-нибудь другие ассемблеры? Например, такие экзотические, как fasm, spasm, hla?

- Я использовал и MASM и TASM под ДОСом, но с приходом 32-х битных Windows, я использую только MASM. Я также пишу на PowerBASIC'e, у которого очень хороший встроенный ассемблер, и некогда в прошлом использовал встроенный ассемблер C.

Относительно упомянутых выше экзотических ассемблеров я думаю, что FASM имеет неплохой потенциал, так как его автор регулярно обновляет его и отвечает своим пользователем. Я не в очень хороших отношениях с автором Spasm, и хотя с моей стороны были предприняты определенные усилия, он не хочет уступать настолько, чтобы сделать свой ассемблер более пригодным для широкого круга программистов, поэтому не думаю, что его ожидает значительное будущее.

HLA - это гибрид, работающий как очень мощный препроцессор для нескольких ассемблеров. Его автор Рэндал Хайд - пионер в ассемблере, который сделал его доступным, когда немногие были в нем заинтересованы, и частью целевой аудитории HLA являются студенты, которые должны изучить ассемблер и сделать на нем что-нибудь полезное, используя то ограниченное время, которое есть у них в университете.

- Какой разговор может обойтись без упоминания о Microsoft? Стив, что вы думаете о новой платформе .NET? Действительно ли это что-то революционное или просто еще одна попытка Microsoft захватить контроль над миром?

- Вероятно, и то и другое. Если у вас есть под рукой инфраструктура с высокой пропускной способностью, самое последнее компьютерное железо, приложения, которым необходимо программное обеспечение от Microsoft и этот уровень взаимосвязанности, вероятно, это что-то революционное, но я был бы наивен, если бы не видел попытку Microsoft увеличить свой контроль над интернетом.

Проблема в том, что у очень большого количества компьютерных пользователей в мире нет линий с высокой пропускной способностью, они не могут позволить себе самое последнее компьютерное железо и им не требуется этот уровень взаимосвязанности. Также существует проблема безопасности. Хотя с одной стороны, это очень хорошо, что кто угодно может соединиться с кем угодно, риск, что при заражении машины, где нет дорогостоящего бэкапующего оборудования, будут потеряны месяцы или годы работы, возрастает экспоненциально.

- Мы с Вами выполнили свой долг, но оставим "исчадие ада" в покое. Я имею в виду Microsoft. Поговорим о более возвышенном. Верите ли вы в Бога или другую мистическую суперсилу?

- Это интересный вопрос. Я вышел из узкого и консервативного круга английской церкви, поэтому я знаю эту область достаточно хорошо, и хотя я не стесняю себя ритуалами или традициями какой-либо религиозной группы, я никогда не утрачивал любопытства о различных аспектах мира, будь то природа, физика, механика и многих других. Во всем этом я вижу порядок, предсказуемость и то, что я назвал бы дизайном, по большей части мир подчинен рассудку, а не хаосу. Я никогда не видел конфликта между миром и его создателем.

Простым ответом будет: да, я верю в Бога. Я не определяю себя как "религиозного" человека, но как верующего.

- Пробовали вы когда-нибудь практиковать медитацию, дзэн или магию рун?

- Простым ответом будет: нет. По крайней мере, в тех формах, в которых обычно понимаются эти понятия, но я имею некоторое представление о том, что такое медитация, так как понимание индийской ведической литературы достаточно распространено здесь.

Дзэн - это древнекитайская форма мудрости, против которой я не имею ничего против, так как она базируется на опыте, полученном за очень долгое время. Я бы назвал дзэн философией, а не религией.

Я имею очень небольшое представление о магии рун, но основываясь на том, что мне известно, лично я не вижу в ней чего-нибудь примечательного.

Я стараюсь не наклеивать ярлыки на то, чего не понимаю, так как всегда будет то, что нам непонятно. Старушки, которых называли ведьмами, знали, как лечить людей от некоторых болезней, используя натуральные препараты, действие которых было тогда неизвестно. Знание не должно происходить из невежества, как, например, это было с китайским иглоукалыванием, которое до недавнего времени считалось суеверием.

Если что-то существует в мире, нельзя сбрасывать это со счетов, основываясь на чистых теориях, нужно исходить из практических свойств.

- Что вы думаете о Каббале? - Я знаю достаточно, чтобы не поддерживать любую форму магии, которая основывается на более старом религиозном учении. Как один из тех, кто уже верит в Бога, я не могу сидеть на двух стульях, иначе можно упасть с обоих. **- Спасибо за интересное интервью, Стив. Мы желаем Вам успехов и благодарим за замечательный инструмент, который Вы дали миру.**

Создание приложений на ассемблере для Menuet OS

Автор: GnuOwned / RMX#13, пер. Aquila

Дата: 2003-08-23

Это моя первая статья для RMX ("Raza Mexicana" - мексиканский электронный журнал, из которого была переведена статья - прим.пер.), поэтому не судите меня строго. В данной статье я собираюсь объяснить, как писать приложения на 32-х битном ассемблере для маленькой и неприхотливой системы Menuet. Но сначала рассмотрим самое главное:

Что такое MenuetOS?

Это маленькая операционная система, которая влезает на дискету и написана полностью на 32-х битном ассемблере, очень быстрая и маленькая (но функциональная) с очень лёгким графическим интерфейсом, не говоря уже о том, что в ней есть текстовый редактор, график процессора и так далее.

Самое интересное в этой ОС то, что она отдаётся по лицензии GPL (www.gnu.org), поэтому мы можем посмотреть её код, модифицировать и так далее. Дело в том, что это прекрасная возможность научиться программировать на 32-х битном ассемблере, а затем применить свои знания для более выгодных дел :). Официальная страница этой интересной операционной системы располагается по адресу www.menuetos.org, где вы можете посмотреть скриншоты, скачать её, посмотреть FAQ и т.д.

Инсталлируем MenuetOS...

Это самое простое... =), нужно только скачать програмку под названием MSETUP.EXE и установить её под виндами на чистую дискету, после чего загрузиться с неё, и тогда перед вами появится навороченный графический интерфейс (хе-хе-хе).

Чтобы установить MenuetOS под нисками, вам нужно сделать в консоли следующее:

```
dd if=MSETUP.EXE of=mfloppy.img ibs=1000 obs=1000 skip=20
```

После этого образ операционной системы окажется в файле mfloppy.img и всё, что остаётся сделать, это скопировать образ на дискету:

```
dd if=mfloppy.img of=/dev/fd0
```

и готово.

Достоинством данной ОС является то, что не надо ничего модифицировать на своём жёстком диске, потому как всё происходит в оперативной памяти и не надо делать дополнительные партиции, ничего...

Введение в 32-х битный ассемблер

Это краткая выжимка из документации, поставляющейся с Menuet'ом, на тот случай, если вы не знаете английского или не очень хорошо понимаете его. Итак, приступим.

Краткое введение в программирование на ассемблере - 0.02

В ассемблере есть много команд, которые в действительности не являются необходимыми для создания наших программ.

Есть шесть регистров, которые в Menuet можно рассматривать как переменные: eax, ebx, ecx, edx, esi, edi.

Каждый из этих регистров может хранить 32-х битное число.

Сохранить число в регистре можно следующим образом:

```
mov eax,10      ; сохраняем десятичное 10 в eax
mov ebx,2000000 ; десятичное 2000000 в ebx
mov ecx,0       ; ноль в ecx
mov edx,11b     ; двоичное 11 в edx (десятичное 3)
mov esi,0xFF    ; шестнадцатеричное FF в esi (десятичное 255)
mov edi,ebx     ; и можем присвоить значение одного регистра другому...
```

Как вы уже заметили, комментарии можно помещать после ";".

Также мы можем складывать и вычитать:

```
add eax,ebx
add esi,eax
add eax,100
sub ecx,edx
sub edx,50
sub edi,0xF
```

Также мы можем делать умножение и деление - прочтите о командах mul, div, imul, idiv в файле CMD.TXT на дискете с операционной системой (на английском).

Безусловные переходы можно делать так:

```
jmp haz_esto

; код

haz_esto: ; Метки обозначаются ':'

; код продолжается
```

Para comparar los registros entre si se hace esto:

Чтобы сравнить регистры, нужно сделать следующее:

```
cmp eax,ebx ; Сначала сравниваем с помощью 'cmp'

jb EAX_ES_MENOR_EBX ; Потом проверяем результат

; или
jg EAX_ES_MAYOR_QUE_EBX
; или
je EAX_ES_IGUAL_A_EBX
; или
jne EAX_NO_ES_IGUAL_A_EBX

; Здесь задаём следующие метки:

EAX_ES_MENOR_EBX:

; какой-то код

EAX_ES_MAYOR_QUE_EBX:

; какой-то код
```

ОК, у нас уже есть следующие основные функции:

- - Сохранение переменных
- - Изменение переменных
- - Безусловные переходы
- - Условные переходы

Задавать числа можно следующим образом:

```
db 0,1,2      ; заданы байты 0,1,2
dw 0,300,65000 ; заданы слова 0,300,65000
dd 1000000    ; задано двойное слово 1000000
```

; Строка

```
db 'Здорово, мексиканский народ!' ; Задана ASCII-строка:  
; Здорово, мексиканский народ
```

В Menuet системные функции вызываются командой 'int 0x40'.

Например, кнопка на окно помещается следующим образом:

```
mov eax,8          ; Функция, задающая кнопку  
mov ebx,100*65536+10 ; Координата по x - 100, а ширина кнопки - 10  
mov ecx,50*65536+8  ; Координата по y - 50, а высота кнопки - 8  
mov edx,10          ; Id. Это число идентифицирует кнопку.  
mov esi,0x000099    ; Цвет (синий)  
int 0x40            ; Вызываем системную функцию и у нас есть  
; кнопка ;-)
```

Просто, не правда ли?

Скомпилировать приложение можно выбрав menu->programming->fasm и указав имя файла с исходником на ассемблере и файла, который должен получиться.

Здесь (находится в архиве wasm_files.zip: files/recovered/appmenuet/appmenuet.asm) можно найти исходник рабочего приложения для MenuetOS с комментариями (на языке оригинала - прим.пер.)

IDE для больших проектов на ассемблере

Автор: masquer

Дата: 2003-10-01

1. Введение
2. Требования в IDE
3. Некоторые примеры IDE
 1. RadASM
 2. WinASM и Negatory
 3. Visual SlickEdit 8.0
 4. Source Insight 3.5
4. Заключение

1. Введение

Итак, сначала небольшое введение, посвященное тому, зачем мне (в частности) вообще нужно **IDE**. Дело в том, что наличие **IDE** практически необязательно для проектов с небольшим размером. Под проектами с небольшим размером я подразумеваю проекты с количеством строк, меньшим 1000. Как правило такая программа состоит из одного файла (или один **.asm** и один **.inc**) и содержит 10-15 процедур, столько же структур, макросов, глобальных переменных и констант. Для работы над таким проектом вполне достаточно даже Notepad-a, а компиляцию запускать из **bat** файла. Все эти элементы программы достаточно легко запоминаются в голове и их применение практически не вызывает никаких трудностей, в худшем случае придется прокрутить текст, посмотреть название или элемент структуры и вернуться к коду. Насчет NotePad-a, это, конечно, утрировано, и подразумевается, что с этим заданием справится любой текстовый редактор. Выбор такого рода редакторов достаточно широкий. Все они, как правило, поддерживают настраиваемую подсветку синтаксиса, возможность запуска внешних приложений и захват текста, возвращаемого компилятором. Для таких целей я предпочитаю использовать **UltraEdit** (www.ultraedit.com). Почему? Кроме всего вышеперечисленного меня устраивает его компактность, скорость работы, удобные поиск и замена, возможность показа названий процедур (настраиваемая). На этом работы с небольшими проектами я касаться не буду.

2. Требования в IDE

Перейдем к более интересному, а именно большим проектам. Традиционно считается, что на ассемблере сейчас большие проекты не пишутся (или вообще ничего не пишется). А мне, например, нравится и у меня есть несколько проектов с количеством строк, большим 10 000 у каждого. Причем менять язык я не планирую, а число проектов - только увеличивать. Кроме этого каждый мой большой проект (незавершенный, кстати) имеет порядка 300 процедур, примерно такое же количество структур и глобальных переменных и дюжина макросов. При этом каждая процедура имеет 5-10 локальных переменных и 1-3 структуры. Все это разбросано по 5-6 **asm**-файлам и такому же количеству **inc**-файлов. Ну, и как прикажете этим хозяйством управлять? Самое время подумать о выборе **IDE** для управления такого ассорти.

Но прежде я попытаюсь сформулировать те требования, которые я предъявляю к такой системе, в которой мне (лично) было бы удобно работать. В принципе, ничего сверх экстраординарного я не требую. Все ниже перечисленное идет в том порядке, в котором приходит в голову по мере

написания, а не по степени важности. Для удобства я разобью эти требования на группы:

1. Редактор текста.

1. Возможность настроить размер, стиль и имя шрифта. Я, например, предпочитаю все писать шрифтом Courier New, 14, bold;
2. Настраиваемая подсветка синтаксиса;
3. Возможность автозаполнения - выпадающая подсказка во время ввпечатывания символов. В саму подсказку должны входить ВСЕ глобальные переменные, процедуры, структуры, константы и все локальные для данной процедуры;
4. При наведении (клике) на переменную/константу/структуру/макрос/процедуру хотелось бы видеть ее во вспомогательном/всплывающем окне.
5. Желательно отделять цветом/стилем шрифта локальные переменные, глобальные и пр.
6. Неплохо (хотя и не критично для меня) было бы "сворачивать" процедуры и структуры.
7. Про прочие св-ва, присутствующие редакторам, типа настраиваемой табуляции и пр. даже неудобно напоминать.

2. Работа с проектами.

1. Удобство при добавлении и удалении файлов проекта (пофайловое добавление - это неудобство);
2. Возможность индивидуальных настроек у каждого проекта;
3. Копирование св-в одного проекта другому, наследование св-в
3. Непосредственно IDE
 1. Отображение дерева из глобальных процедур, переменных, структур и констант с удобным поиском не только по первым символам (у меня многие процедуры начинаются одинаково);
 2. Т.к. компилятор и линкер у masm консольный, то IDE должно захватывать результат компиляции и парсить его для того, чтобы в случае ошибки я мог перейти на нее одним кликом;
 3. Удобная настройка запуска внешних приложений (отладчика например) с присвоением кнопки на тулбаре и сочетания клавиш;
 4. Удобное создание и использование репозитория кода (хотя мне не очень важно);
 5. Поддержка средств контроля версий;
 6. Интерфейс для написания плагинов;
 7. В качестве общего требования - как можно больше кастомизации (я люблю все под себя настроить).

Как это ни странно прозвучит, но я считаю редактор ресурсов абсолютно не нужным компонентом IDE, в качестве внешнего приложения - сколько угодно. Не стоит изобретать велосипед.

3. Некоторые примеры IDE

После беглого осмотра я остановился на нескольких продуктах, претендующим на место на моем жестком диске в качестве среды, в которой мне было бы удобно творить. Именно творить, а не набивать текст и больше работать мышкой, прокручивая десятки килобайт листинга.

Итак, к своему неудовольствию, программы, полностью удовлетворяющей вышеперечисленным требованиям я не нашел. Собственно, этого я и ожидал, но все же некоторые продукты очень даже достойны внимания.

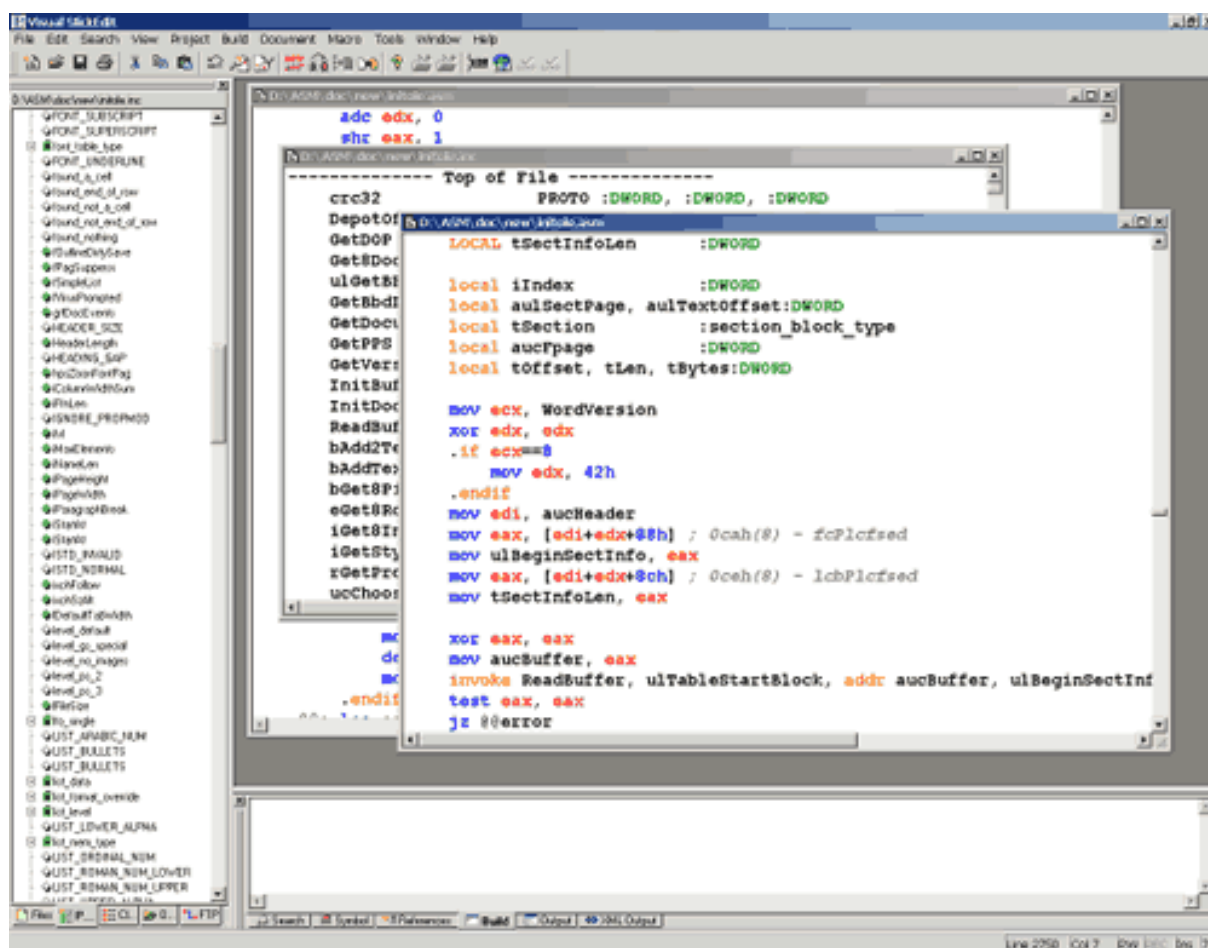
1. **RadASM**. Автор - *Ketil Olsen*. На этот продукт стоит однозначно обратить свое внимание, т.к. он сделан программистом на ассемблере для программистов на ассемблере. Кроме того, сам проект полностью написан на ассемблере. Один этот факт уже достоин уважения. Итак, мы имеем быстрый, маленький по объему и бесплатный **IDE** написанный энтузиастом для таких же энтузиастов. Среди возможностей - настраиваемая подсветка синтаксиса, автоподсказка при ввпечатывании функций WinApi, поддержка нескольких ассемблеров, плюс **HLLA**, удобный выбор

цветов и составление битовой маски, в панели свойств отображаются имена элементов языка с выпадающим списком либо аргументов (для процедур и макросов), либо элементов структуры (для структур соотв.), либо значений для констант. Есть возможность создавать ресурсы. Можно сворачивать процедуры и if-else a la MS Visual Studio .NET (collapsing). Есть интерфейс для плагинов. Вкратце - в нем есть все для работы со средними и малыми проектами. Почему он не пригоден для работы с большими? Допустим, у меня в проекте 300-400 процедур. Как мне в них ориентироваться, если нет абсолютно никакого поиска, простая сортировка по имени. Хорошо, хоть при ввпечатывании первых букв можно в выпадающем списке выбирать. При ввпечатывании имен структур и глобальных переменных меню с предложением вариантов возникает, только если нажать Ctrl-Space, а для этого я слишком ленив :) Локальные переменные и аргументы вообще не индексируются. Меню программы не продумано и неудобно (мне, например), для того, чтобы что-нибудь настроить, нужно хорошо полазить по меню. Нет возможности добавлять свои группы при подсветке синтаксиса.

2. Вторым пунктом хочу упомянуть небольшие, но (кажется?) развивающиеся проекты. **Negatory Assembly Studio**(www.negatory.com/asmstudio/) хочу упомянуть лишь как яркий пример того, что автор увлекся интерфейсом в ущерб функциональности. В результате получился симпатичный (хоть и спорный) интерфейс при практически полном отсутствии функциональности. По крайней мере я решил сохранить свое зрение, а не изучать функционал программы с жестко закодированным шрифтом размером в 6 pt. Еще одним продуктом, достойным упоминания (уже с лучшей стороны) является также индивидуальная разработка **WinAsm** (*Antonis Kyprianou*) (<http://winasm.how.to/>). Есть collapsing для процедур, при наборе понимает локальные переменные (после моей просьбы :)) Выпадающий список из процедур все с тем же прокручиванием или поиском по первым буквам. Поддержка AddIn У этого проекта определенно есть перспективы, но и сейчас его можно использовать для небольших проектов.

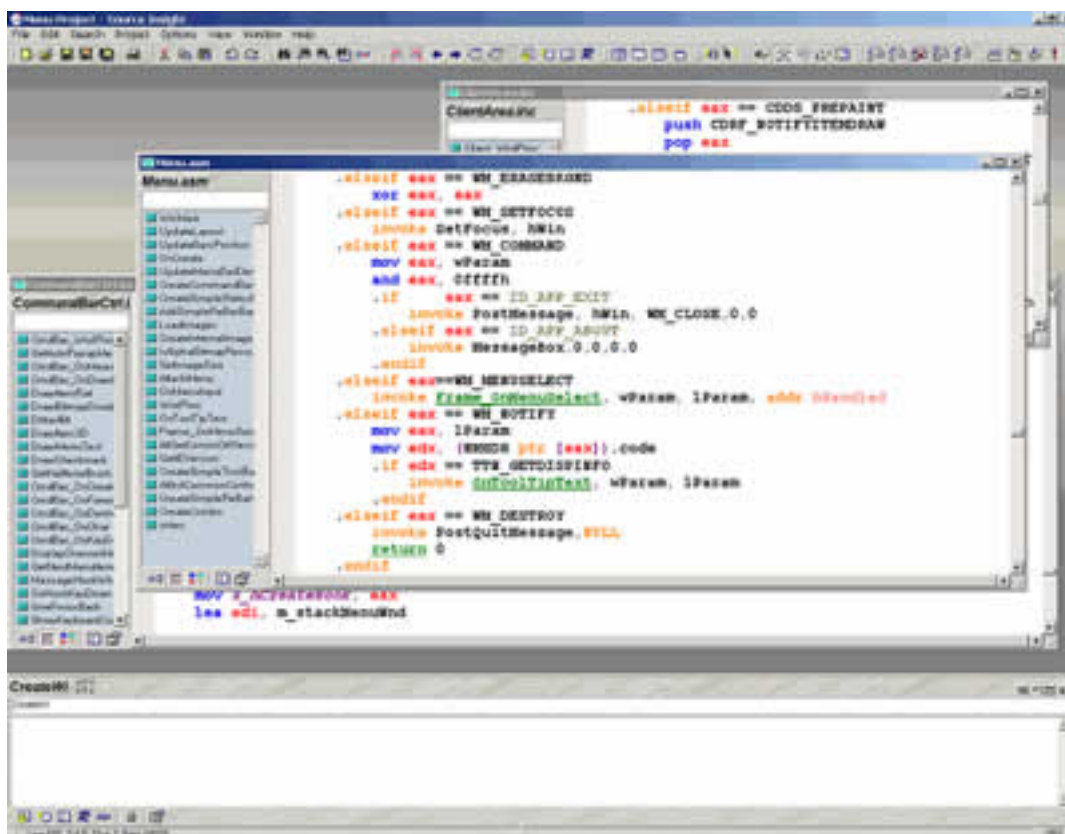
Самое интересное я оставил на конец. Следующие 2 программы относятся к многофункциональным IDE, в которых можно разрабатывать не только программы на ассемблере (точнее - не столько). Это - дорогие коммерческие приложения, (в отличие от первых - бесплатных) и серьезные игроки - по функциональным возможностям они не хуже (а иногда и лучше чем Visual Studio от Microsoft). Начнем с самого дорогого...

3. Visual SlickEdit. (www.slickedit.com) Цена - \$269. И по размеру, и по спартанскому в общем интерфейсу (ну, по сравнению с VS, например) сразу видно, что продукт очень серьезный. И после продолжительного тестирования я все больше и больше убеждался в этом. Вначале вкратце о пакете вообще - одним словом - предназначен для программиста, причем неважно на каком языке писать, SlickEdit имеет поддержку таких языков, о которых я даже и не слышал :). К своему немалому удивлению и радости ассемблер оказался в их числе, но об этом попозже. Пакет содержит мощный макроязык, синтаксически напоминающий C, с большим количеством примеров. Есть средство для сравнения файлов, причем все это с подсветкой и возможностью редактирования сравниваемых файлов. Для всех языков есть поддержка соответствующего синтаксиса, настраивается абсолютно все, что можно себе представить (и чего нельзя - тоже).

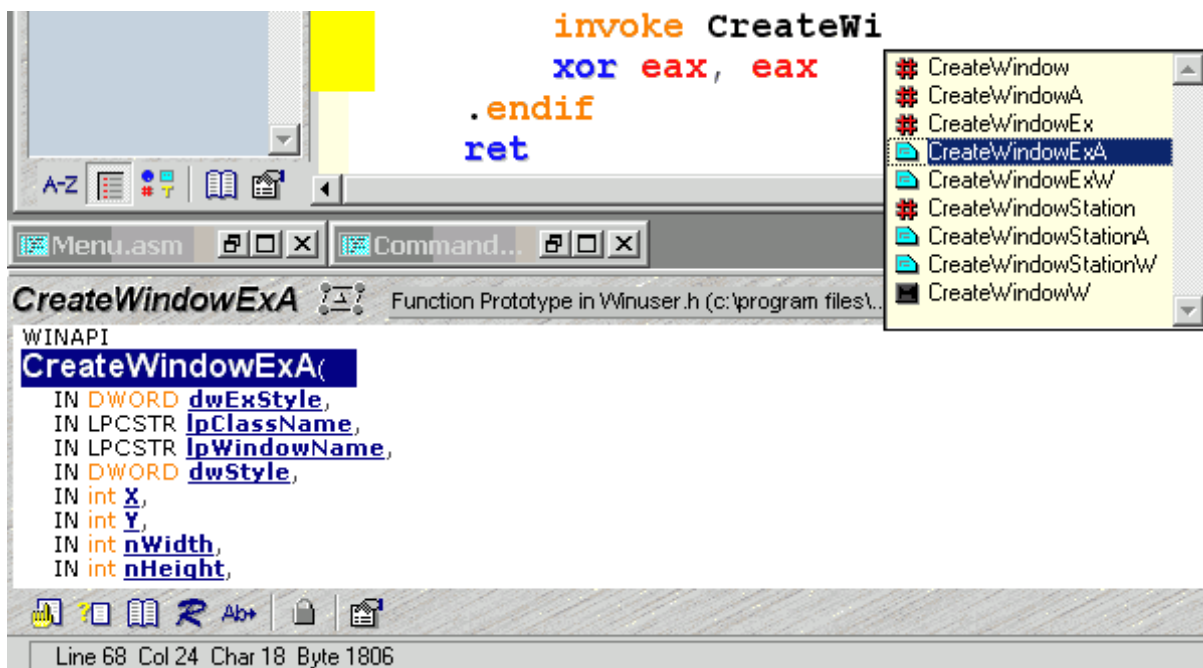


Имеется поддержка нескольких систем контроля версий (включая CVS). Возможностей пакета вполне хватает для конкуренции на равных с VS, а по некоторым параметрам - и превосходить. Что же мы имеем для работы с ассемблером? Легко настраивается подсветка синтаксиса, формата чисел. Возможность запуска внешних приложений с захватом и очень быстрым парсингом вывода - компилятора, например. При этом в случае ошибки к ней можно добраться одним щелчком. Есть поиск и замена. Для проекта на ассемблере есть дерево процедур в стиле MS VS, высвечиваются также глобальные переменные, макросы и структуры, правда, элементы структуры выводятся рядом, как независимые переменные (хотя для структур на C collapsing работает), и как это настроить для ассемблера, я так и не понял (возможно, скрипт переделывать).

4. Итак, последним по списку, но не по значению у меня Source Insight 3.5 (Source Dynamics, Inc.) Сайт - www.sourceinsight.com (Цена - \$269).



При размере, на порядок меньшем (как у дистрибутива, так и в установленном виде), чем у SlickEdit функциональность не меньше, а порой и больше, чем у последнего. Хорошая настраиваемость, есть бейсикоподобный макроязык, но нет средств для коллективной работы (хотя я особо не искал). Интерфейс несколько отличается как от остальных, рассмотренных выше IDE, так и от VS. Что мне очень понравилось, так это то, что я могу видеть все процедуры, макросы, etc. как всего проекта, так и каждого файла по отдельности, плюс настроить, что именно я хочу видеть в данный момент - процедуры или константы, и т.д. Очень удобно сделан поиск по части названия (процедуры, например), а не по первым буквам.



Еще понравилась одна вещь, которой нет ни в одном IDE - Source Insight индексирует все мои структурные элементы программы и при наведении на них курсора, показывается нужный кусок кода внизу на панели. Для этой панели можно также задать ее размер и св-ва шрифта, но подсветка синтаксиса сохранится!



О подсветке синтаксиса тоже спую дифирамб - так, как ее можно настроить в Source Insight, я больше нигде не увидел в других продуктах. Очень удобно настраивается навигация по коду - я легко настроил ее как в браузере. Можно вывести статистику об используемых структурных элементах - об их встречаемости в проекте. Есть возможность вывести Relationship для процедур (надо сказать что для ассемблера эта функция работает, но несколько странно результаты показаны). Для настройки парсинга локальных переменных пришлось написать регулярное выражение, правда, при этом включаются все локальные переменные файла, а не процедуры - не очень удобно, но терпимо. Зато при наборе любого текста появляется всплывающее меню autocompletion, при этом я могу пробежаться по предложенному списку, а все то же окошко внизу мне показывает первый десяток строк процедуры, например. Во время работы делается backup проекта, так что я практически застрахован от всяких неожиданностей, при аварийном завершении мне будет предложено восстановить работу на том месте, где ее прервал сбой. Кнопки сборки и запуска проекта идентичны кнопкам в VS. Из минусов, хоть и не критичных - долгий парсинг вывода, но зато я опять же могу не переходя сразу к коду, просмотреть первые строчки, при парсинге сразу же идет переход к первой ошибке (если они есть). Еще одна приятная мелочь - при печатании выражения, в котором есть несколько вложенных скобок, внешние становятся больше внутренних и сразу видна вложенность (хотя для ассемблера это не часто встречается). Еще одна немаловажная особенность, которой я больше нигде не видел, это подсвечивание строк, в которых произошли изменения - т.е. сразу можно увидеть, что и где я за этот сеанс изменил. Фактически, мне Source Insight напомнил улучшенный вариант VS + Visual Assist, и пока я свой выбор остановил на этом продукте.

4. Заключение

Данная статья отнюдь не претендует ни на полноту описания программ (это объем хорошей книги), ни на объективность, так как это только мое личное мнение. Нынешним и будущим авторам IDE также не стоит воспринимать все вышесказанное, как инструкцию о том, каким должно быть IDE, потому что я, например, после знакомства с Source Insight и не подозревал, как могут пригодиться те, казалось бы, мелочи, которые там есть.

В заключение скажу, что хотя Source Insight меня устраивает больше всего, окончательный выбор я еще не сделал, и верю, что возможно лучшее решение...

Разработка больших проектов на ассемблере

Автор: SolidCode

Дата: 2004-03-24

Глава 0: Disclaimer

Исследуемые здесь принципы разработки больших проектов должны, пожалуй, работать для любого языка. В этом плане они "кросс-платформенны".

Сразу оговорюсь, что я не глаголю аксиому. Это лишь моё мнение, мои размышления, мой опыт программирования на асме. Где-то - моя биография. Я приветствую иные мнения. И буду внимательно читать Ваши отзывы.

Итак, приступим.

Путь адепта

Происходит такая оказия, что обычный программёр узнаёт о существовании языка ассемблера. Здесь как в опере. Если понял и прочувствовал его силу, красоту и мощь, то полюбил навсегда. Если же чего-то не хватило, то возненавидел навсегда. Ибо всякий раз эти дурацкие "mov, int и sbb" будут напоминать ему о поражении. Мы же останемся с тем, который полюбил ЯА.

Обычно в начале он великий энтузиаст. Он летит на крыльях новых возможностей. Он узнаёт, что теперь может контролировать каждый байт код своей программы. Он с умным видом открывает программы в HEX редакторе и просматривает их содержимое, хотя пока что понимает здесь лишь "MZ" и "PE". Он может теперь разговаривать с Президентом компьютера, Процессором, на его собственном языке за чашкой чая. И различные придворные порты, шины, диски и платы ему внимают. Он уже скоро вскричит: "I AM GOD HERE!" Да тут ещё выходит "Матрица" на экраны.

А сколько радости, когда списал у Iczelion из урока листинг "Hello, world" и откомпилировал в экзешник величиной всего 1,5 килобайта. И он даже корректно работает. А какое счастье посещает его, когда он узнаёт, что можно всё это добро засунуть в 1 килобайт. И там ещё куча пустых нулей остаётся.

Но жизнь - штука суровая. После того, как новый адепт начинает отходить от примеров учителя, после того, как он обнаруживает, что не всегда всё идёт так, как он хочет, после того, как он неделю копает отладчиком свою программу, пытаясь найти эту дурацкую ошибку, а потом обнаруживает, что просто забыл поставить одну букву в своём исходнике (напр. "je" -> "jne"), после того, как он походил тернистыми путями Intel'a и понабивал шишки в ямах Microsoft'a, после многих перипетий адепт постепенно начинает более или менее реально воспринимать жизнь и великое искусство программирования. Но, как говорил Рей Бредбери, "Единственный способ заставить писателя не писать, это кольт 45-го калибра".

Посему писатели пишут. Проекты становятся всё более крупными и интересными. Ходить по всему листингу скролбаром, отыскивая код процедуры или место на котором закончил в последний раз; мучительно вспоминать, что ты вчера хотел реализовать; всё заново переписывать из-за того, что выбранный путь оказался неверным; просто искать способы быстро выполнять стандартные операции, всё это приводит адепта к просветлению и осознанию полезности удобной среды разработки.

Глава 1: Среда разработки

В этом разделе я много писать не буду. Хорошая статья на указанную тему уже есть на WASM.RU, и там всё подробно написано. Скажу лишь, что для себя самой удобной, хотя и не совершенной средой, нахожу RadASM от KetilO. И вот почему:

- Частые операции типа `assemble`, `build`, `save`, `debug` выполняются теми клавишами, которые я настрою
- Имеется удобная настраиваемая подсветка синтаксиса
- Весь проект объединён в группу файлов
- Автоматически отслеживается версия
- Удобная навигация по файлам
- Быстрый переход на определение функции и назад
- И многое другое

Однако главное, чтобы среда была максимально удобна Вам. Ибо прилежный адепт не должен много времени отвлекаться на бытовые мелочи.

Проблемы разработки

Начинающий адепт, недавно познавший написание программ уровня "Hello, world", получив озарение свыше или задание от начальника на написание программы, сразу бросается к клавиатуре. Он стремится выплеснуть в код всё своё искусство, накопленное за эти две недели, которые он изучает ассемблер. "Сейчас я сделаю вам всем шедевр в несколько килобайт! Не пройдёт и два месяца". Клавиатура стонет и скрипит под натиском обрушивающегося на неё дождя ударов пальцами. И в то же время она радуется, что он не обезьяна, а то бы ещё и пальцами ног набирал для скорости.

Но вот каркас программы написан, а что дальше? А дальше он начинает писать одно. Что-то не получается. Переключается на другую сферу в программе. Потом делает третью часть. О, нашёл проблему в первой - возвращается к ней. На этот момент он уже забыл, каким способом хотел организовывать работу с памятью в своей программе. Приходится изобретать велосипед снова. А как удобно представить свои данные? Лучше так. А может быть по-другому? Или так? А что там было в третьей части? Да это не важно. Вот в статье он прочитал об оригинальном способе представления данных. И снова всё надо переписывать. И новые структуры появляются. "А это что за структура? Да это ведь для моего первого варианта работы с памятью два месяца назад!" Но теперь уже он не нужен.

Кроме того, он ведь не просто пишет программы. Он пишет на ассемблере! Самые быстрые программы. А как раз недавно он прочитал классную статью по оптимизации кода по скорости и размеру. Поэтому во время написания программы адепт её сразу оптимизирует. Но это оказывается БОЛЬШОЙ ошибкой.

Ваш покорный слуга, например, долго пребывал в глубокой уверенности, что в масме диалоги не работают, их нельзя создавать. Или масм у Стива кривой? В наших ошибках всегда виноват кто-нибудь другой. Но вот же есть в примерах диалоги. Дрожащими пальцами компилирую его. Всё работает. На основе этого файла начинаю собирать свой диалог. В какой-то момент он перестаёт работать. А я не заметил после чего. Приходится всё начинать сначала. Примерно через месяц или два я понял свою ошибку. Её озвучил кто-то из великих программистов: "Многие беды программирования происходят от преждевременной оптимизации". Я не сохранял всякие `IF`, `ELSE` в первоначальном виде, а переводил их в `CMP` и переделывал структуру диалоговой процедуры. Читая Iszelion'a, я не сразу обратил внимание, что диалоговая процедура отличается от оконной. Что она должна возвращать определённое булевское значение. В результате я возвращал неправильные значения и диалоговое окно не создавалось, так как считало, что произошла ошибка создания окна.

Зато про сегментные регистры (EBX, ESI, EDI) я прочитал и заметил сразу. Iczelion говорит, что их нельзя использовать. А если и используешь, то надо восстанавливать их значение перед возвратом в систему. Глупый! Ведь это же их надо тогда в стек складывать. А потом восстанавливать. Это ж сколько лишних байт и тактов процессора! Но я ведь умнее этого наивного Iczelion'a! Я попробовал их не восстанавливать. Всё прошло нормально в Win98SE. Ну и стал так писать везде.

А как-то злосчастным утром я попробовал свою одну прогу запустить в Win2000. А она возьми да и зависни! Как!? Пошёл домой. Проверил: всё работает. Прихожу на тот комп - прогадохнет. Причём вываливается внутри системы, а не моей программы. Я и на другой машине попробовал. Тот же результат: в двухтысячной вылетаем по GPF. Вот тут Микрософту досталось за кривую систему. "Да все они там ламера!" Как настоящий кул-хацкер, я в 2000-ой взялся за отладчик. Отыскал, где начинается проблема. В моей части кода всё ОК. Пошёл дальше в систему. А тут через несколько инструкций идёт обращение к памяти по адресу ESI. А я его в своей программе обнуляю для собственных нужд...

Это было мне хорошим уроком. Пришлось несколько программ переделывать, чтобы эти регистры не менялись.

Итак, в результате описанных выше способов кодирования, программа быстро превращается в непроходимую кашу. Это ужасно в любом языке. Особенно в ЯА, где разобраться в получившейся каше гораздо сложнее. Порой проще заново начать всю программу. А месяцы идут своим чередом. И кто это нам грозился написать программу за два месяца?

Боюсь, что многие, включая вашего покорного слугу, узнают себя в этих строках на каком-то этапе.

Как же действительно оптимально разрабатывать большой проект? Я не говорю здесь о проектах, которые затеваются для себя, в образовательных целях. Такие могут длиться годами, да так и не найти завершения. Меня интересуют принципы разработки проектов, которые нужны людям, которые будут кому-то полезны, у которых есть определённая "целевая аудитория". На самом деле, эти принципы известны уже давно и с успехом используются во многих софтверных компаниях. Однако большой проект - он и в Африке большой проект, и даже у одного разработчика. Поэтому давайте рассмотрим основные этапы разработки продукта. Эти этапы были позаимствованы на сайте фирмы ABYY, которая известна нам по таким проектам как FineReader и Lingvo. Однако моей целью не было повторение этапов разработки проектов, используемых в больших компаниях, но адаптация тех принципов к ситуации, когда проектом занимаются от одного до десяти разработчиков, которые к тому же могут быть разделены расстоянием (жить в разных городах). Вот перечень этапов:

- Выработка целей и требований к предмету разработки
- Разработка предварительного (эскизного) проекта
- Разработка детального проекта
- Реализация
- Тестирование

Глава 2: Техническое задание

Это и есть пункт первый - "Выработка целей и требований к предмету разработки". По большому счёту это задание должен писать конечный пользователь или заказчик программы. Желательно, вместе с программистом-разработчиком или координатором проекта.

В техническом задании на разработку программы указывается назначение программы. На этом этапе решаются следующие вопросы.

- Для чего она разрабатывается?
- Есть ли реальная потребность в такой программе?
- Какие возможности будут у разрабатываемого проекта?
- Чем он выгодно будет отличаться от конкурентов?
- Кто входит в "целевую аудиторию" программы?
- Какой уровень компьютерных знаний должен быть у пользователей?
- Как много времени потребуется на обучение пользователя эффективной работе с этой программой?

Разработка эскизного проекта

Когда техническое задание определено, наступает этап разработки предварительного проекта.

Составляется общая архитектура системы, последовательность запуска и работы модулей, определяется общий "сценарий" работы программы и действия пользователя в ней. Общая система делится на подсистемы, отвечающие за определённые функции в проекте. Продумывается интерфейс взаимодействия этих подсистем. При этом по мере возможности минимизируются взаимосвязи между ними. Определяется, что будет поступать на вход этих подсистем и что мы должны получить в результате. Это позволит в будущем проектировать и кодировать подсистемы параллельно, что особенно важно при разработке каждой подсистемы отдельным специалистом или группой. Но даже одиночному разработчику это поможет просто и удобно организовать разработку модулей. И не придётся долго и мучительно вспоминать, что делал в последний раз, где остановился, и для чего всё это было вообще затеяно?

Разрабатывается эскизный проект интерфейса программы в целом и отдельных окон, диалогов и логических связей между ними. Проект интерфейса должен обеспечивать удобную работу пользователя с программой и реализацию всех возможностей, заявленных в техническом задании. Причём интерфейс должен быть интуитивно понятным. Возможности, опции и настройки программы должны быть классифицированы и размещены по степени важности, логике совместного использования и частоте обращения. Например, открытие и сохранение файлов обычно происходит довольно часто и используется при каждом запуске. Оно должно быть всегда "под рукой". А настройка тонкостей интерфейса или значений по умолчанию - обычно только при первом запуске. Это можно сделать в каком-нибудь отдалённом диалоговом окне. С другой стороны, настройки, общие по логике, могут быть разделены на основании частоты использования и глубины знаний пользователя. Например, установки размера символов текста, цвета символов, гарнитуры шрифта (внешнего вида шрифта) можно разместить недалеко в легкодоступном диалоговом окне. И там же сделать кнопку "Advanced" для "продвинутых" пользователей, которые хотят настраивать интерлиньяж (расстояние между строками), кернинг (расстояние между буквами), угол наклона букв относительно горизонтали строки, кодировку и т.п.

Подбираются соответствующие технологии и принципы, которые наиболее удобно позволят реализовать требования к программе и её функциональность. Определяются ориентировочные требования к компьютеру, на котором будет работать программа, в плане как аппаратного (процессор, память и др.), так и программного обеспечения (минимальная версия Windows, Internet Explorer, DirectX и т.п.). При этом, обратите внимание, мы ещё не написали ни строчки кода! Мы даже ещё не обговорили, на каком языке это всё будет написано. На данном этапе, это лишь абстрактная система. Её можно реализовать практически в любом языке.

И последнее замечание: к разработке эскизного проекта не рекомендуется привлекать более двух человек, иначе разработка проекта может затянуться до того, как они перестреляют друг друга, доказывая, что Си лучше Паскаля. Хотя ассемблер лучше любого другого языка.

Глава 3: Разработка детального проекта

На этом этапе общая идея о программе сформирована. Мы уже представляем, что хотим получить, как будет выглядеть внешне программа, и как будет работать в ней пользователь. Она разделена на модули и подробно описана.

Теперь мы уже должны определиться с языком программирования. На этом этапе разрабатываются форматы представления данных и, если требуется, собственные форматы файлов. Каждая выделенная подсистема разрабатывается детально отдельной группой или разработчиком, компетентным в сфере задач, выполняемых модулем. Продумываются алгоритмы и методы реализации требуемой функциональности. Для каждого модуля расписываются все функции, в какой последовательности они вызываются и за что отвечают. Могут потребоваться блок-схемы или другие визуальные способы отображения алгоритма работы подсистемы. На этом этапе уже можно начать писать программу. Но лишь следующим способом:

```
;шапка модуля
InitModule proc arg1:DWORD, arg2:DWORD
;эта процедура прежде проверяет наличие требуемых возможностей из структуры параметров компьютера
;устанавливаем фрейм обработчика исключений
;потом выделяем память для того и этого, согласно параметров структуры по адресу [arg1]
;и т.д.
    nop
    ret
InitModule endp
```

Так мы постепенно формируем костяк кода. В каждую процедуру, по мере разработки вписываем, что она должна выполнять в комментариях и в соответствующей последовательности, чтобы потом долго не мучиться при кодировании. Там же в комментариях можно писать идеи частей кода, которые приходят в голову по мере детализации, но только в комментариях. Не захламляйте свой код. Потом вы придумаете что-нибудь лучше или иначе. Пока блок закомментирован, можно его легко удалить. Компилятор и не узнает о ваших старых невоплощённых идеях, и не останутся случайно неиспользуемые переменные из-за того, что было сначала так, а потом обошлись без этой переменной, но убрать её из секции кода забыли. В RadASM-е легко комментируются и очищаются от комментариев блоки любой длины. Не торопитесь вписывать в свою программу код. Китайская пословица гласит: "Трижды подумай, потом действуй".

Кодирование

Да, вы дождались. Наконец-то мы добрались до кодирования! Как мы с удивлением осознаём, среди этапов разработки программы кодирование оказывается чуть ли не на последнем месте. Да, это так. Оглянитесь назад и посмотрите, сколько важной работы надо выполнить прежде, чем стоит приступить к реализации самой программы. Отсутствие этих этапов может оказаться губительным для большого проекта, и в конечном итоге времени на проект вы потратите гораздо больше, а результат будет хуже.

Главным критерием при разработке кода программы является удобочитаемость и ясность исходных текстов. Именно это, а не оптимальность кода. Если вы сделали такой оптимальный код, который никто, кроме вас не поймёт, то труд ваш был напрасен. Уже через месяц отвлечения от проекта вы займёте плотные ряды тех, кто не может прочесть и понять ваш собственный код.

Теперь о любимой всеми нами оптимизации. НЕ ОПТИМИЗИРУЙТЕ ВАШИ ПРОГРАММЫ! По крайней мере, на стадии разработки. Это можно сделать потом, после того, как программа готова и протестирована. Опять же, можно в функции оставлять заметки по оптимизации и куски компактного кода, но только в комментариях.

Практика показывает, что реально оптимизация может пригодиться в таких случаях, когда выполняются сложные вычисления или циклы. Изредка стоит оптимизировать выборку сообщений

процедурой окна. И, тем не менее, не торопитесь. Вашу оптимизацию всё-равно вряд ли кто заметит в большинстве случаев, а вот непроходимость дебрей кода Вам обеспечена сразу. Оптимизировать всегда успеете. Напишите бета-версию неоптимизированную. Пусть её так погоняют. К тому же, именно первая версия самая трудоёмкая. Потом уже идут только надстройки, добавления, исправления.

В окончательной версии можно попробовать осторожно оптимизировать. За одно сразу увидите, насколько Ваша оптимизация реально приносит пользу. И когда проходите по своему исходному тексту в поисках возможностей оптимизации по скорости или размеру, не удаляйте неоптимизированный (но обычно более удобочитаемый) текст, но оставляйте его рядом в комментариях. Это позволит легко вернуть назад в случае ошибки или другой проблемы, а также поможет легче разобраться, что делает оптимизированный код. А как тяжело искать ошибки в оптимизированном коде!

Не бойтесь комментариев. Они очень важны. Они в значительной степени повышают читабельность текстов. Они хранят важную информацию о том, что тут у нас в исходном тексте происходит и зачем это нужно. Именно из-за комментариев в значительной степени исходные тексты так дороги и так охраняются. Листинг любой программы можно получить и от дизассемблера. Но вот разобраться в инструкциях и понять назначение кода, не так просто. И, в конце концов, вы начинаете в полученный листинг вписывать комментарии при разборе другой программы. И это правильно. Ведь человек думает не так, как компьютер, и комментарии призваны создать этот мостик между выражением чего-либо с точки зрения компьютера и человека.

Всё вышесказанное относится к коду, написанному на любом языке, но в особенной степени это относится к программам, написанным на ассемблере.

Ещё одним важным аспектом кодирования является выбор языка программирования. В принципе, мы должны были с этим определиться на предыдущем этапе. И я всё это время говорил, полагая, что мы пишем на ассемблере. Однако этого ещё не достаточно. Дело в том, что есть разные ассемблеры. (Я подразумеваю `masm`, `fasm`, `tasm`, `spasm`, `hla` и другие, а не `.8086`, `.286`, `.386`, `.486` и т. д.) Выбирать стоит тот ассемблер, который всем в группе разработки наиболее понятен. Все должны писать на одном ассемблере, чтобы блоки легко между собой сочетались. Могут быть группы, в которых все - ярые поклонники `tasm` или `fasm`. Я не большой специалист в разных ассемблерах, но при выборе применяю следующие критерии: ассемблер должен быть максимально удобочитаемым (желательно с высокоуровневыми директивами типа `".if .else .endif"`), но при этом не должен теряться контроль над каждым байтом кода. Иначе, какой смысл писать на ассемблере? Берите уж тогда C++.

Для себя я выбрал `masm`. На мой взгляд, он в наибольшей степени отвечает приведённым здесь требованиям. В случаях, когда нужен простой IF, нет смысла вручную городить `"CMP EAX,EDX JNE @1"` и вводить новый ярлык. Пусть компилятор сделает это сам. У него лучше получается. Он впишет тот же самый код, байт в байт, только самостоятельно. А мы в листинге напишем гораздо более удобную конструкцию `".if eax==edx .endif"`, которая при чтении очень красноречива. Подобным образом можно организовывать и циклы. Если компилятор не добавляет в конструкцию цикла что-нибудь лишнее, то можно воспользоваться стандартным вариантом. Но если требуется особенный цикл или цикл с оптимизацией, то всегда можно реализовать его просто командами ассемблера. Однако и в этом случае стоит вписать директивы цикла и закомментировать их, чтобы не мешали, но намекали на то, что здесь по логике закодирован цикл.

Таким образом, мы приходим к выводу, что главным критерием кода в листинге должна быть удобочитаемость. Это важно как для текущей разработки, так и для дальнейшего развития проекта.

В конечном итоге весь код собирается, организуется и компилируется в отдельные библиотеки или исполнимые файлы. На поверхностный взгляд всё сошлось и работает довольно устойчиво в течение уже 5 минут. Можно отдавать на тестирование.

Глава 4: Разработка документации

Этот этап сложно выделить в отдельный временной промежуток. В некоторой степени он может быть распределён по остальным. На первом этапе мы пишем техническое задание. На втором - эскизный проект-описание системы. Параллельно с кодированием и тестированием можно выполнять разработку ReadMe, описывающей и обучающей документации в каком-либо формате помощи (help).

Тестирование

Для успешного тестирования требуется составить план тестов. В такой план обычно входят следующие виды тестов:

- Тестирование соответствия возможностей программы тому, что заявлено в техническом задании.
- Тесты на устойчивость. Сюда входит тестирование на всех операционных системах, поддерживаемых программой, тестирование при различном аппаратном окружении, тестирование устойчивости работы программы в экстремальных условиях (когда забита до отказа память, когда система занята множеством тяжёлых процессов, когда идёт интенсивная запись на диск и т.п.).
- Тестирование освоения программы неподготовленным пользователем.
- Сравнительное тестирование с конкурирующими продуктами.
- Тестирование читабельности и адекватности пользовательской документации.

Для тестирования подбирается определённая команда. В группе распределяются обязанности и требования. Во время тестирования находятся недоработки и сбои. Эти сбои следует сразу анализировать и исправлять. Кроме того, код, связанный с найденной ошибкой надо проверять дополнительно на подобные ошибки. При тестировании также часто возникают предложения по дальнейшему улучшению программы. Их нельзя бросаться и внедрять в текущую версию, ибо к совершенству можно стремиться всю жизнь. Улучшения и нововведения должны включаться в техническое задание на следующую версию. А в текущей версии должно быть только то, что заложено в техническом задании.

Заключение

Вот мы и дошли до конца технологического цикла разработки программы. Когда все этапы выполнены, можно подготавливать реализацию продукта и его внедрение в рабочий процесс, для которого он создавался.

Кроме того, любой современный серьёзный продукт обеспечивается службой технической поддержки. Поэтому в ReadMe стоит включить хотя бы адрес электронной почты службы поддержки этого продукта. Служба технической поддержки в некоторой степени служит "лицом" производителя. На вопрос о том, обратится ли к вам клиент, при другой подобной okazji в значительной степени зависит от уровня обслуживания клиента как до, так и после приобретения продукта.

Благодарю вас за внимание и желаю успешного кодирования и интересных проектов.

Обзор "Agent Aika"

Автор: Noir

Дата: 2004-04-01

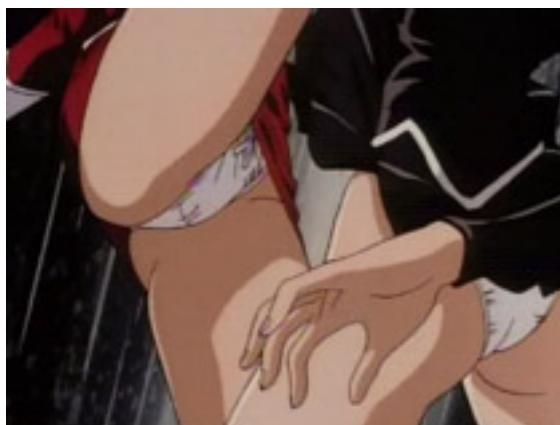
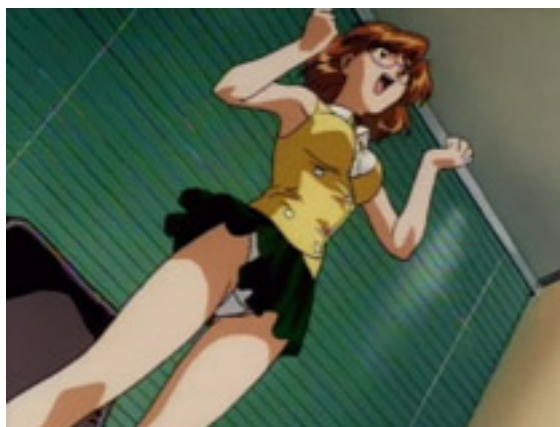
Бывают моменты, когда хочется отдохнуть от стандартных 26-ти и 52-х серийных картин, повествующих о любви, крепкой дружбе и прочих благообразных глупостях и посмотреть что-нибудь для души часика на два-три. Обычно эту роль берут на себя OVA или Movie. В данном обзоре я хочу рассказать о весьма необычной, смешной и чуть-чуть пародийной овашке "Agent Aika".

Эта милое и доброе произведение повествует о недалёком будущем, в котором значительная часть поверхности Земли погрузилась под воду в результате некоторого глобального катаклизма, произошедшего двадцать лет назад. Выжившие в большинстве своём переквалифицировались в Обшарильщиков - людей, обшаривающих дно океана в поисках затонувших ценных предметов и технологий.

Главная героиня сериала, которую зовут Аека, также является представителем этой почётной профессии. Ей помогает дочь хозяина фирмы, который сам не занимается собственно поисками, а заведует организационными вопросами. В один прекрасный день Аека и её партнёры переходят дорогу неким могущественным кругам, с чего собственно и начинается завязка сюжета. С этого момента героев ждёт множество приключений, итог которых вы сможете узнать, посмотрев данную OVA.

Но данное аниме стоит смотреть не только ради сюжета, вернее не столько ради него, сколько ради возможности увидеть всех участников аниме женского из довольно необычного ракурса огромное количество раз. Вот несколько стандартных кадров из этого аниме:





И этого, гмм... если так можно выразиться, рулеза целых семь серий. Постепенно сюжет, арт и прорисовка меркнут перед феерией продукции какой-то безымянной трикотажной фабрики. Это даже может помочь узнавать героев не прибегая к лицевой идентификации, правда, это не касается мужских персонажей, впрочем, тех можно легко опознавать по штанам.

P.S. Для любителей(льниц) юри в данном аниме есть несколько небольших приятных сюрпризов.

Програмерский спецназ

Автор: Roustem

Дата: 2004-07-01

Уступая пожеланиям трудящихся :), я решил посмотреть, можно ли извлечь какой-нибудь практический смысл из использования Alt-кодов. Это пересеклось с другой давно меня занимавшей темой. Представьте необитаемый остров, пещеру, а в ней много "железа", но почти нет софта. Ситуация может оказаться не такой фантастической, как кажется на первый взгляд, начиная от случайно оставшейся в деревенском чулане "двойки" с допотопным модемом и кончая системами на основе NT с умышленно стертым debug'ом, не говоря о более продвинутых средствах разработки. В общем, мы оказываемся в ситуации некоего "виртуального экстрима", когда только от наших собственных способностей и умений, как и в случае экстрима реального, зависит, сможем ли мы "выжить" на этом необитаемом виртуальном острове.

Чтобы не было недоразумений, с самого начала подчеркну, что речь идет именно о "выживании". Никаких рафинированных систем с SDK, никаких уютных комфортных квартир - тайга, сырость, даже спичек нет - разжигай свой костер только подручными средствами. Любителям комфорта и собственных привычек тут делать совершенно нечего. Как и в настоящем спецназе, здесь могут пройти лишь парни с огрубевшими мозолистыми руками и обветренными лицами; их я и приглашаю совершить со мной эту прогулку.

Здесь мы сталкиваемся с интересным вопросом: а что, собственно, считать "подручными средствами"? Понятно, что компьютер уже должен быть :). Единственным его ПО "в комплекте" является BIOS. Поэтому можно было бы потребовать, что сначала нужно создать загрузочную дискету - но для этого опять нужен компьютер с уже запущенной, пусть самой примитивной, ОС! Замкнутый круг. В принципе, "задачей спецназа" можно было бы назвать подготовку загрузочной дискеты с использованием собранного из подручных средств электромагнита :). Или, скажем, с использованием тех же проводов и прочих подручных средств перепрограммировать BIOS так, чтобы можно было вводить бинарные значения в оперативную память прямо с клавиатуры. На самом деле, это весьма забавно; возможно, когда-нибудь мы этим даже и займемся, но все же это уводит нас в область железа. А у нас спецназ програмерский, а не аппаратный.

Поэтому примем следующий постулат: доступна загрузочная дискета с минимальными возможностями. Если такой дискеты нет, задача считается неосуществимой, а для ее решения требуется вызвать "аппаратный спецназ". Минимальный загрузочный набор является ориентированным на DOS (boot-сектор, io.sys, msdos.sys, coomand.com или их аналоги) - это наиболее распространенный вариант; вероятность того, что найдется загрузочная дискета или хотя бы "форматированная под загрузку" (format /s), относительно велика. Обратите внимание, что это никак не уменьшает наши возможности: мы получаем доступ к любому IBM-совместимому компьютеру (независимо от возможно установленной на нем ОС - будь то Windows XP, Windows 2003 Server, Linux, FreeBSD или т.п.), а через него - при наличии соответствующего коммуникационного железа - мы можем (в принципе!) получить доступ к любым другим системам.

Замечание для тех, кто любит все делать своими руками и хочет создать такую загрузочную дискету. При ее создании из-под Windows 98 с помощью команды 'sys a:' на дискету записывается фиктивный command.com размером 20 Кб; его нужно удалить, а вместо него вручную скопировать настоящий размером 92 Кб из каталога Windows. В Windows Me команда sys не работает для дискет, а format не имеет опции /s - для создания загрузочной дискеты необходимо воспользоваться вкладкой "Загрузочный диск" диалога "Установка и удаление программ" панели управления. Затем, для чистоты эксперимента, можно удалить все лишние файлы, кроме перечисленных выше. В линейке Windows NT традиционную загрузочную дискету создать нельзя;

там создается аварийный набор из нескольких дискет, но это другое. В этом случае используются дискеты, созданные в DOS или Windows9/Me, или же при уже запущенной ОС используется командный интерпретатор `cmd.exe`. Заметим также, что ничто не мешает работать с окном DOS в уже запущенном Windows9/Me или с полноценным, а не урезанным DOS - дискета с указанными файлами является лишь *минимальным* необходимым набором.

Итак, нам удалось слегка оживить холодное железо, мы запустились и имеем теперь интерактивную командную оболочку с довольно ограниченными возможностями. Что дальше? Как "вытянуть" из наличного железа возможности, которые в нем могут быть запрятаны? Насколько далеко мы сможем продвинуться в этом предприятии, целиком зависит от наших собственных знаний и умений. Повторюсь еще раз, это не является нормальным программированием. К сожалению, термин "экстремальное программирование" уже используется в другом контексте - нам бы он очень подошел. Тем не менее, это и есть настоящий экстрим. Точно такой же, как лазание по вертикальным откосам скал и через таежные буреломы, несмотря на наличие ровных автострад и железных дорог - в реальном мире находятся любители таких путешествий, почему бы им не быть в мире виртуальном?

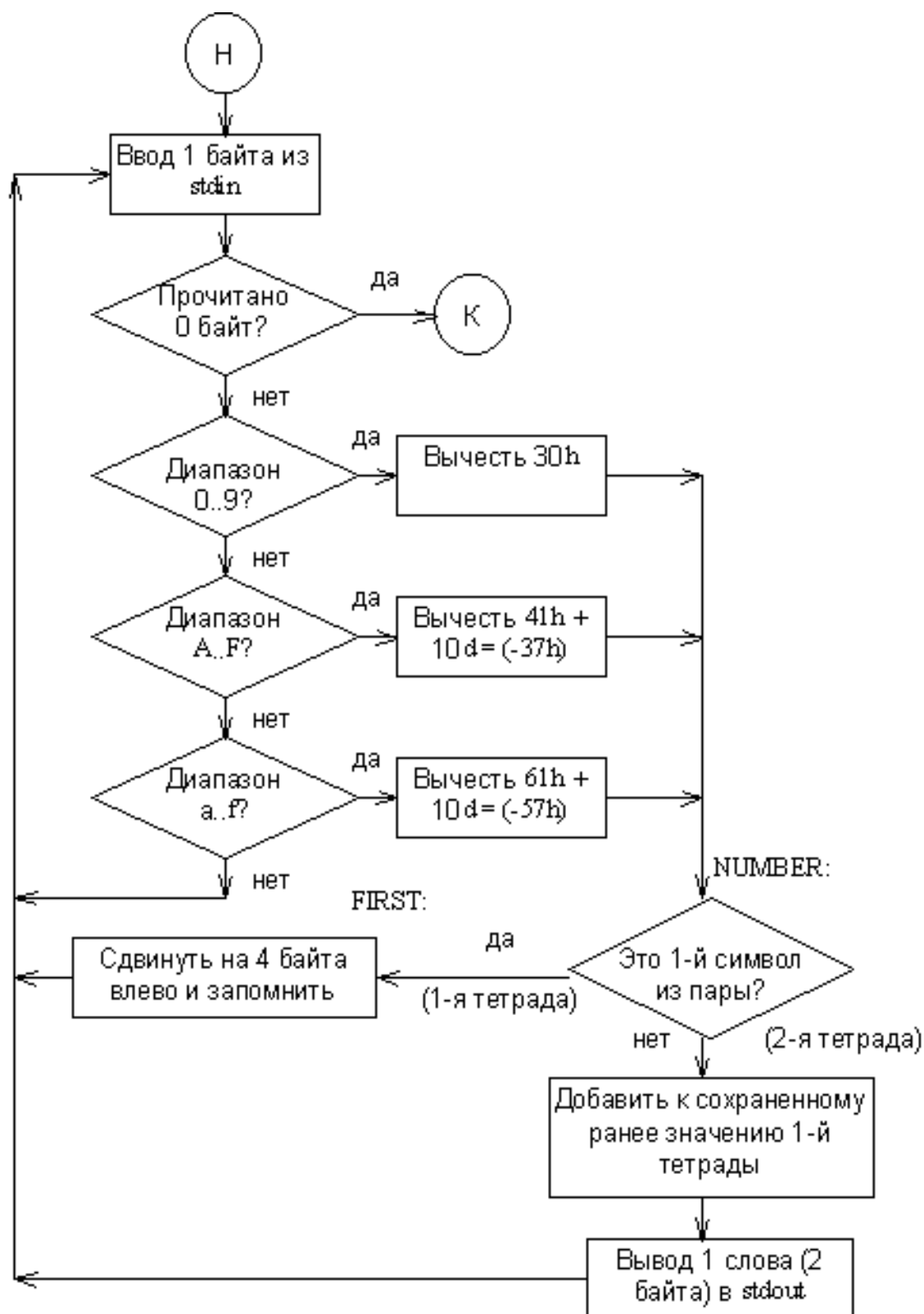
Следующим шагом, как и при выживании в реальном мире, нам нужно начать создавать простейшие, но жизненно необходимые в повседневном использовании инструменты - те, с помощью которых мы и должны выполнять свою задачу (будь то коммуникационная программа, какой-нибудь драйвер для железа или т.п.) Причем все подчинено одной главной цели - выжить и выполнить поставленную задачу (настоящий спецназ!) А это значит - ничего лишнего; экономить силы, экономить время, делать только абсолютно необходимое, выявляя самую суть и безжалостно отбрасывая все постороннее. Тут уже не до програмерских примочек, оптимизаций, шлифовки и даже (о ужас!) "правильного стиля". Все должно быть надежно и просто, как автомат Калашникова.

Как вы уже, наверное, догадались, такой инструмент мы и будем создавать с помощью так называемых Alt-кодов - встроенной в интерпретатор `command.com` (а также `cmd.exe` в Windows NT и последующих) возможности набирать бинарные значения, используя правый цифровой блок клавиатуры при нажатой клавише Alt. В самом деле, мы можем использовать команду '`сору соп file.bin`' в сочетании с Alt-кодами, чтобы создать двоичный файл `file.bin`. Хотя здесь есть и проблема.

Дело в том, что не все двоичные значения можно набрать таким способом - некоторые управляющие символы вызывают непосредственную реакцию системы, а в файл не записываются. К этим значениям относятся: 0, 3, 6, 8, 10 (0Ah), 13 (0Dh), 16 (10h), 19 (13h), 26 (1Ah), 27 (1Bh), 127 (7Fh). Причем эти значения относятся к `command.com` DOS (соответственно, и Windows9*). `Cmd.exe` от линейки Windows NT в этом отношении более "либеральна" - "запрещенными" являются лишь символы 8, 13 (0Dh), 26 (1Ah) и 127 (7Fh). К тому же, размер буфера строки в команде `сору` ограничен 127 байтами (в Windows 2k/XP - 510), а это неприятно осложняет создание нормально функционирующих программ. Большие файлы набирать с помощью '`сору соп`' можно, но придется специально обрабатывать переходы через символы конца строки (CRLF), которые будут вставляться в файл.

Поэтому, в соответствии с нашим принципом выживания - максимум эффективности с минимумом усилий - нам жизненно необходимо создать инструмент для последующего создания бинарных файлов. В просторечии такой инструмент называют шестнадцатеричным редактором :).

В нашем случае это будет даже не редактор, а скорее конвертор, переводящий поток ASCII-символов из стандартного ввода в поток двоичных значений в стандартный вывод. Причем все символы, не попадающие в диапазон ASCII-символов 0..9A..Fa..f, просто игнорируются. Это позволяет форматировать вводимый текст для большего удобства восприятия (добавлять пробелы между отдельными байтами, а между строками вставлять перевод строки - CRLF). Алгоритм приведен на следующем рисунке.



Нам понадобятся три переменные размером в 1 байт: для хранения обрабатываемого байта (назовем ее `place`), для хранения предыдущего байта (`place2`) и указатель того, относится ли обрабатываемый байт к первой или второй тетраде (`check`). В целях экономии (нам нужно уложиться в 127 байтов!) эти переменные оставим за пределами `com`-файла. Тем не менее, в начале программы необходимо на всякий случай проинициализировать их нулями. Команда типа `'mov byte ptr [place],0'` не годится, т.к. в этом случае в код попадает непосредственное значение 0, а это недопустимый символ. Поэтому используем косвенный способ с использованием регистра:

```

100: 31C0          xor    ax,ax
102: A3nnnn        mov    [place],ax
105: A3nnnn        mov    [check],ax

```

В листинге слева приведено смещение с начала com-файла (в hex со смещения 100h), далее - машинный код, а затем с отступом - ассемблерная мнемоника для удобства восприятия. Здесь мы инициализируем сразу по 2 байта. Вместо 'nnnn' мы потом подставим действительные адреса этих переменных.

Далее нам нужно ввести 1 байт данных из стандартного ввода. Для этого используется функция 3Fh чтения из файла прерывания DOS 21h. В регистре AH должен содержаться номер функции (3Fh), в регистре BX - дескриптор файла (для стандартного ввода - 0), в регистре CX - число читаемых байтов (в нашем случае 1), DS:DX адресует буфер, куда должны читаться данные. В данном случае, нельзя использовать инструкцию 'mov cx,1', т.к. получаем код 'B9 01 00' - появляется запрещенный нулевой байт.

```

108: B43F          mov    ah,3F
10A: 31DB          xor    bx,bx
10C: 31C9          xor    cx,cx
10E: 41            inc    cx
10F: BAnnnn        mov    dx,offset place
112: CD21          int    21

```

Дальше начинаются проверки введенного значения, здесь же осуществляется и основное преобразование вводимых значений. Для начала проверим состояние флага переноса (CF) после функции файлового чтения - если он установлен, произошла ошибка чтения, выходим из программы:

```

114: 72nn          (1) jb    EXIT

```

В регистре AX возвращается число прочитанных байтов. Если прочитано 0 байт, файл закончился - также выходим.

```

116: 85C0          test   ax,ax
118: 74nn          (2) jz    EXIT

```

Далее загружаем сохраненный в переменной 'place' байт в регистр AH для анализа.

```

11A: 8A26nnnn      mov    ah,byte ptr [place]
11E: 80FC30        cmp    ah,30
121: 72E5          (3) jb    108
123: 80FC39        cmp    ah,39
126: 7705          (4) jnbe  12D
128: 80EC30        sub    ah,30
12B: EBnn          (5) jmp    NUMBER
12D: 80FC41        cmp    ah,41
130: 72D6          (6) jb    108
132: 80FC46        cmp    ah,46
135: 7705          (7) jnbe  13C
137: 80EC37        sub    ah,37
13A: EBnn          (8) jmp    NUMBER
13C: 80FC61        cmp    ah,61
13F: 72C7          (9) jb    108
141: 80FC66        cmp    ah,66
144: 77C2          jnbe   108
146: 80EC57        sub    ah,57

```

Логика исполнения должна быть ясна из рисунка. Небольшие ссылки вперед ("перескакивание через инструкцию") вычислены сразу; часто используемая ссылка назад - на ввод очередного символа, соответствующий код расположен по смещению 108h. Остальные ссылки вперед помечены метками NUMBER и FIRST (далее); они для удобства обозначены также и на рисунке. Начало следующего участка кода соответствует метке NUMBER: мы получили смещение (149h), теперь можно вычислить все ранее сделанные ссылки на этот адрес.

```

149: 90            nop

```

```

14A: 8A1Ennnn      mov    bl,byte ptr [check]
14E: 84DB           test   bl,bl
150: 74nn            jz     FIRST
152: F6DC           neg     ah
154: 2826nnnn       sub     byte ptr [place2],ah

```

Этот код может показаться несколько странным. Зачем нужен пор, когда мы и так стараемся вместить код в ограниченные 127 байтов? Если вы посмотрите на сделанные ранее ссылки вперед и подсчитаете соответствующие смещения (в частности, для инструкции по адресу 13Ah), при ссылке на адрес 149h вы получите инструкцию 'EB 0D', а 0Dh у нас "запрещенный" символ. Поэтому ближайший адрес, на который мы можем сослаться, - 14Ah, а по адресу 149h придется оставить пустую инструкцию. Далее, если введенный ASCII-символ соответствует первой тетраде выходного байта, в переменной check будет 0. Но использовать инструкцию типа 'cmp byte ptr [check],0' нельзя, поскольку в код опять попадает 0 в виде непосредственного значения. Поэтому используем косвенный метод через копирование значения в регистр BL и последующим test.

Вычисленное значение второй (в данном случае) тетрады необходимо добавить к переменной (place2), содержащей значение сохраненной первой тетрады. К несчастью, опкод инструкции add равен 0, поэтому нам придется вообще от нее отказаться, заменив ее двумя инструкциями neg и sub. Думаю, с этим разобрались.

Далее идет вывод одного байта в стандартный вывод. Используется функция 40h файлового вывода прерывания DOS 21h. В регистре AH, как обычно, должен быть номер функции (40h), в BX - дескриптор файла (на этот раз стандартный вывод, т.е. 1), в CX - число выводимых байтов (1), DS:DX адресует буфер для вывода (у нас - адрес переменной place2). Для избежания появления нулевых байтов в коде придется применить те же хитрости с регистрами и inc, которые мы использовали при вводе. В случае ошибки (установленный бит CF при возврате из функции) сразу завершаем программу, чтобы не нагородить лишнего.

```

158: B440           mov     ah,40
15A: 31DB           xor     bx,bx
15C: 43             inc     bx
15D: 89D9           mov     cx,bx
15F: BAxxxx         mov     dx,offset place2
162: CD21           int     21
164: 72nn           jb     EXIT

```

Записав очередной байт, нужно указать в переменной check, что следующий вводимый байт ASCII-кода будет представлять первую тетраду выводимого байта; для этого обнуляем эту переменную. Как обычно, это делается опосредованно через регистр:

```

166: 30C0           xor     al,al
168: A2xxxx         mov     [check],al
16B: EB9B           jmp     108

```

В этом месте должен размещаться код для обработки случая первой тетрады (метка FIRST). Но сюда ссылается команда по адресу 150h, и если сделать ссылку на адрес 16Dh, получится инструкция '74 1B', а 1Bh у нас тоже "запрещен". Придется сослаться на следующий адрес 16Eh, а по адресу 16Dh опять поставить пор:

```

16D: 90             nop
16E: B104           mov     cl,04
170: D2E4           shl     ah,cl
172: 8826nnnn       mov     byte ptr [place2],ah
176: FE0Ennnn       dec     byte ptr [check]
17A: EB8C           jmp     108

```

Инструкция по смещению 176 делает отметку в переменной check, что следующий вводимый байт будет соответствовать второй тетраде выходного байта. Вообще-то, check представляет собой простую логическую переменную, и решение делается на основе проверки ее на предмет 0 - не 0. Можно было бы использовать инструкцию 'inc byte ptr [check]', но, к несчастью, при этом в коде

появляется запрещенный символ 06. Приходится вместо inc использовать dec; в нашем случае это совершенно не играет роли. Вот и все - здесь конец программы, инструкция возврата и метка EXIT:

```
17C: C3                ret
```

Программа заняла 7Dh = 125 байтов. Переменные, как мы уже говорили, можно разместить и за пределами файла, "виртуально"; т.е. можно было бы использовать адрес 17Dh. Но, как вы понимаете :), сочинение такого опуса и его подгонка заняла не один час, и для облегчения себе жизни, чтобы не пересчитывать каждый раз хотя бы адреса переменных, я отвел им места "с запасом" - для place, place2 и check соответственно 180h, 181h и 182h. Теперь можно вставить эти адреса, а также вычислить смещения и подставить их вместо 'nnnn' в инструкции. Окончательно мы получим такой код:

```
31 C0 A3 80 | 01 A3 82 01 | B4 3F 31 DB | 31 C9 41 BA
80 01 CD 21 | 72 66 85 C0 | 74 62 8A 26 | 80 01 80 FC
30 72 E5 80 | FC 39 77 05 | 80 EC 30 EB | 1D 80 FC 41
72 D6 80 FC | 46 77 05 80 | EC 37 EB 0E | 80 FC 61 72
C7 80 FC 66 | 77 C2 80 EC | 57 90 8A 1E | 82 01 84 DB
74 1C F6 DC | 28 26 81 01 | B4 40 31 DB | 43 89 D9 BA
81 01 CD 21 | 72 16 30 C0 | A2 82 01 EB | 9B 90 B1 04
D2 E4 88 26 | 81 01 FE 0E | 82 01 EB 8C | C3
```

Но это еще не все - надо все это перевести в десятичную систему, и если вы проделаете это самостоятельно, то поймете, насколько ценный инструмент мы на самом деле создаем :). Но раз уж я проделал эту работу, не стану жадничать и приведу готовый вариант:

```
049 192 163 128 | 001 163 130 001 | 180 063 049 219 | 049 201 065 186
128 001 205 033 | 114 102 133 192 | 116 098 138 038 | 128 001 128 252
048 114 229 128 | 252 057 119 005 | 128 236 048 235 | 029 128 252 065
114 214 128 252 | 070 119 005 128 | 236 055 235 014 | 128 252 097 114
199 128 252 102 | 119 194 128 236 | 087 144 138 030 | 130 001 132 219
116 028 246 220 | 040 038 129 001 | 180 064 049 219 | 067 137 217 186
129 001 205 033 | 114 022 048 192 | 162 130 001 235 | 155 144 177 004
210 228 136 038 | 129 001 254 014 | 130 001 235 140 | 195
```

Это не избавляет вас от необходимости вручную набивать числа. Если кто-то успел позабыть, как это делается ;), я напому. В консольном окне (либо после загрузки с подготовленной дискеты) набирается команда:

```
copy con txt2bin.com
```

Затем вводятся все приведенные выше числа (десятичные), при вводе каждого числа нажимается и удерживается клавиша Alt. В случае ошибки можно просто стереть символ клавишей "Backspace" и ввести заново. Набрав все, нажимаем Ctrl+Z (или F6) и Enter - файл готов. Программа в принципе может работать и в интерактивном режиме, хотя это не очень удобно. Основное ее предназначение - использование в перенаправлениях, типа:

```
txt2bin < input.txt > output.bin
```

В качестве редактора для создания файла input.txt используется все та же внутренняя команда оболочки copy:

```
copy con input.txt
```

Только на этот раз мы набираем не Alt-коды, а нормальный текст, вернее, текстовое представление 16-ричного кода. Непременно нужно создать тестовый файл со всеми возможными 16-ричными значениями и подать его на вход нашего редактора, чтобы проверить, что все работает нормально:

```
00 01 02 03 04 05 06 07 08 09 0a 0b 0c 0d 0e 0f
. . . и т.д. до
f0 f1 f2 f3 f4 f5 f6 f7 f8 f9 fa fb fc fd fe ff
```

Эта программа далеко не шедевр, возможно, в ней есть большие и малые ляпусы, тем не менее, она удовлетворяет главному требованию - она работает, и работает универсально. Я проверял ее

для DOS 6.22, Windows 98 SE/Me/2000 Server/XP Pro/2003 Server, а также загрузочных дисков, созданных Windows 98 и Me. Во всех случаях при наборе Alt-кодов возражений со стороны интерпретатора не поступало :), а сама полученная программа успешно конвертировала тестовый файл со всеми возможными значениями в соответствующий бинарный файл. Кстати, если вы владеете слепой печатью на правом цифровом блоке, можно поспорить, каким образом программа набирается быстрее - с помощью Alt-кодов или традиционным образом, через написание исходного текста на ассемблере с последующим построением (если брать суммарное время, боюсь, сравнение окажется не в пользу ассемблера).

Еще одно замечание по поводу возможности ошибок, трудности программирования в машинных кодах и т.д. и т.п. В ходе подготовки материала я набирал этот числовой блок в Alt-кодах раз двадцать, и ни разу не сделал ни единой ошибки. Между тем в ассемблерных проектах, даже коротких, практически всегда хоть мелкая, но найдется какая-нибудь ошибка. Так что в мире программирования гуляет под видом истин множество предрассудков; развеять их могли бы люди, имеющие собственный опыт (например, создания больших проектов прямо в машинных кодах. Но у кого такой опыт есть? Ведь это считается безумием!) Но если бы мы ограничились тестированием лишь с небольшим пробным файлом, это было бы так, забавой, не более. Кто знает, может, там прячется какая-нибудь хитрая ошибка? Поэтому нужно преобразовать в ASCII-представление какой-нибудь рабочий бинарный файл, затем конвертировать его с помощью нашей утилиты и посмотреть, будет ли исходный файл работать после такого двойного преобразования. Я приведу готовый листинг утилиты, созданной с помощью MASM32, для преобразования бинарных файлов в текстовое 16-ричное представление. Не привожу полного описания, т.к. здесь нет ничего особенного; к тому же в исходном тексте есть комментарии.

```
.386
.model flat,stdcall
option casemap:none
include \masm32\include\windows.inc
include \masm32\include\user32.inc
include \masm32\include\kernel32.inc
include \masm32\include\comdlg32.inc
includelib \masm32\lib\user32.lib
includelib \masm32\lib\kernel32.lib
includelib \masm32\lib\comdlg32.lib

.data
f OPENFILENAME <76,0,0,offset msk,0,0,1,offset file1,256,0,0,\
  offset file2,offset src,\
  OFN_FILEMUSTEXIST OR OFN_HIDEREADONLY OR OFN_PATHMUSTEXIST,\
  0,0,0,0,0,0>
ttl db "Bit2txt",0
err1 db "Opening source file failed",0
err2 db "File reading failed",0
err3 db "File writing failed",0
err4 db "Opening destination file failed",0
msk db "All files",0,"*.*",0,0
src db "Select source file",0
dst db "Select destination file",0
defext db "txt",0

.data?
hf1 dword ? ; дескрипторы файлов
hf2 dword ?
read dword ? ; число прочитанных байтов
wrtn dword ? ; число записанных байтов
file1 db 256 dup(?) ; буфер для имени и пути файла
file2 db 256 dup(?) ; каталог файла-источника
file3 db 256 dup(?) ; каталог файла-назначения
buf db 16 dup(?) ; буфер чтения
buf2 db 50 dup(?) ; буфер записи

.code
start:
```

```

invoke GetWindowsDirectory,offset file2,256
invoke GetCurrentDirectory,256,offset file3
invoke GetOpenFileName,offset f
.if eax==0 ; Файл не выбран
jmp stop
.else
invoke CreateFile,offset file1,GENERIC_READ,0,0,\
OPEN_EXISTING,0,0
mov hf1,eax
.if eax==INVALID_HANDLE_VALUE
invoke MessageBox,0,offset err1,\
offset ttl,MB_ICONERROR
jmp stop
.endif
; Первый (бинарный) файл открыт - выбираем второй (для записи ASCII)
lea eax,f
assume eax:ptr OPENFILENAME
mov [eax].lpstrTitle,offset dst
; поместить в текущий каталог
mov [eax].lpstrInitialDir,offset file3
; расширение по умолчанию - txt
mov [eax].lpstrDefExt,offset defext
; флаги для чтения и записи различны
mov [eax].Flags,OFN_OVERWRITEPROMPT OR OFN_HIDEREADONLY \
OR OFN_PATHMUSTEXIST OR OFN_EXTENSIONDIFFERENT
assume eax:nothing
invoke GetSaveFileName,offset f
.if eax!=0 ; Выбран или создан файл для записи
invoke CreateFile,offset file1,GENERIC_WRITE,\
0,0,CREATE_ALWAYS,0,0
mov hf2,eax
.if eax!=INVALID_HANDLE_VALUE
; успешно открыты оба файла;
; начинаем цикл чтения-записи (bin->ASCII)
m0:
invoke ReadFile,hf1,offset buf,16,offset read,0
; каждый раз читаем строку в 16 байтов
.if eax==0 ; ошибка чтения - выход
invoke MessageBox,0,offset err2,offset ttl,MB_ICONERROR
jmp m4
.endif
mov ebx,read
cmp ebx,0 ; конец файла (прочитано 0 байт) -
je m4 ; выход из цикла
inc ebx
shl ebx,1 ; в EBX вычисляем число байтов, которые нужно будет
add ebx,read ; записать = (число прочитанных байтов * 3 + 2)
; (на каждый двоичный байт - 2 байта ASCII-кода +
; 1 пробел + CRLF в конце строки)
cld
lea esi,buf ; буфер чтения <= 16
lea edi,buf2 ; буфер записи <= 50
mov cl,4 ; для сдвига
m1:
lodsb
mov ah,al
and ah,0fh ; младшая тетрада
cmp ah,10 ; преобразуем значение в ASCII-код:
jb m2 ; если < 10, добавить лишь 30h;
add ah,7 ; если >=10, добавить 37h (A-F)
m2:
add ah,30h
shr al,cl ; теперь старшая тетрада -
cmp al,10 ; все аналогично
jb m3
add al,7
m3:
add al,30h
stosw ; сохранить сразу 2 байта (AH+AL=AX)
mov al,20h ; добавить пробел для форматирования
stosb

```

```

dec read ; прочитанный буфер закончен?
jnz m1 ; нет - продолжить преобразование
mov ax,0a0dh ; да - добавить CRLF для форматирования
stosw ; (начинаем новую строку в 16 байтов)
invoke WriteFile,hf2,offset buf2,ebx,offset wrtn,0
; записать подготовленный буфер buf2
.if eax==0 || ebx!=wrtn ; ошибка записи
invoke MessageBox,0,offset err3,offset ttl,MB_ICONERROR
jmp m4
.endif
jmp m0 ; прочитать очередные 16 (или сколько осталось) байтов
m4:
.else ; CreateFile==INVALID_HANDLE_VALUE
invoke MessageBox,0,offset err4,offset ttl,MB_ICONERROR
.endif ; CreateFile
invoke CloseHandle,hf2
.endif ; GetSaveFileName
invoke CloseHandle,hf1
.endif ; GetOpenFileName
stop:
invoke ExitProcess,0
end start

```

Это небольшое приложение со стандартным графическим интерфейсом, если можно так его назвать: в первом стандартном диалоговом окне открытия файла надо указать исходный бинарный файл, во втором - указать имя нового (или существующего, если он будет переписываться) файла, в который будет осуществлена запись преобразованного в текстовое шестнадцатеричное представление содержимого первого файла.

Первой "жертвой" теста, как водится, стал calc.exe. После "двойного преобразования" он работал великолепно. Но интересно ведь попробовать какой-нибудь большой файл, этак в несколько мегов. После некоторых поисков мне удалось на своей системе найти такой файл, им оказался исполняемый модуль AcrobatReader 6.0 (AcroRd32.exe) в 7,3 Мб. Преобразованный в текстовый вид, он занял 22,8 Мб. Представляете, подать такое примитивной com-программке размером в 125 байт? Даже страшно как-то - вдруг задавит своей массой! Тем не менее, я проделал и это. Конечно, пришлось подождать пару-другую минут (что ж вы хотите :)). И полученный после такого преобразования монстр запустился! Правда, отсутствовало меню и еще что-то по мелочи. Но это я отношу уже к тому, что файл был вырван из контекста сложной системы, перенесен в какой-то новый каталог, а там была куча записей в реестре и т.п. В общем, если бы была лажа с преобразованием, файл не запустился бы вообще.

Поэтому можно считать, что первый инструмент в борьбе за виртуальное выживание у нас уже есть. Я не думаю, что надо заучивать эти 125 байтов наизусть, как "Отче наш". Уловив общую суть, а главное, просмотрев этот путь и узнав, где какие подвохи попадают, можно при необходимости его воспроизвести. А дальше уже дело собственных знаний. С помощью этого инструмента можно создавать все те вещи, которые создаются с помощью debug'a (и о которых я начал писать в статьях ["Приложение Windows голыми руками"](#) и ["Dll в машинных кодах"](#)). А это значит, что можно практически "с нуля" создавать любые нужные приложения (при необходимости используя наш принцип спецназа: используем простые утилиты для создания более сложных, а последние - для создания еще более сложных и т.д. - до выполнения боевой задачи). Фактически, это равносильно созданию этих утилит прямо в Alt-кодах. Более "голый" способ может быть разве что создание загрузочной дискеты с помощью самодельного электромагнита :). Лично я покопаюсь еще в этом направлении, уж больно интересные вещи можно здесь обнаружить.

Подрубаем эмулятор PocketPC к eVC и ActiveSync

Автор: Broken Sword

Дата: 2004-07-10

«Любая девочка с персиками когда-то превратится в бабушку с курагой»

Далее речь пойдет о том, как не имея на руках физического девайса можно благополучно отлаживать/запускать свою/чужую прогу, используя eVC и эмулятор из PPC SDK 2003. Я долго парился (насколько мне известно - не я один) с этой задачей и, наконец, совершенно случайно все заработало. Очень помогла статья CJCraft's Blog – фактически в ней все правильно изложено, однако опущены некоторые ключевые моменты, которые и удалось вычислить bruteforce-ом. Для чего вообще нужен эмулятор? Если есть реальный девайс – то он и не нужен. Не знаю как на COM-порте, но чисто визуально скорость отладки/запуска через USB на реальном устр-ве и на эмуляторе мало чем отличается (хотя по заверениям MS это не так). Поэтому, если вы только планируете заняться написанием прог под WinCE, а девайса по пресловутым причинам нету, то эмулятор конечно является единственным вашим спасением. Итак, приступим.

Для начала полный перечень прог, которые понадобятся для сопряжения среды разработки eVC с эмулятором. Всех их можно «спокойно» слить с микрософтовского сайта (www.microsoft.com J). Здесь представлены те версии, с которыми у меня все 100% работает, поэтому желательно заиметь именно эти версии, т.к. были затыки (см. дальше) именно на этой почве. По умолчанию предполагается, что действие происходит в XP+SP1.

Must have:

1. MS eMbedded Visual C++ 4.0 vs SP3
2. PocketPC 2003 SDK (Nov. 2003)

Необязательными, но для связи с ActiveSync необходимыми являются следующие вещицы:

1. MS ActiveSync 3.7.1
2. Microsoft .NET Framework 1.1 SDK (Nov. 2003)

Все остальные необходимые проги (например, Platform Manager, необходимые для работы вышеперечисленных и идут с ними в комплекте, без них они попросту не станут, поэтому разберетесь). Итак, перво-наперво ставим eVC 4.0. Сразу предупреждаю: eVC со вторым сервиспаком глючит по черному – виснет при открытии любого проекта, страдает эпилептическими припадками и обмороками, до эмулятора дело с ним вообще никогда не дойдет, поэтому ОБЯЗАТЕЛЬНО ставьте третий сервиспак и не морочьте голову. Сразу вслед за eVC ставим PocketPC 2003 SDK. В моей версии SDK сразу были встроены три эмулятора – два телефонных и один КПК-шный. Если вам попадется какая-то обрезанная версия, то отдельно образ нужного эмулятора можно слить с пресловутого сайта.

Теперь переходим к настройкам. Существует множество (где-то в районе пяти) способов работы eVC с эмулятором, включая даже такие уникальные, как закороченные COM-порты, но мы не будем извращаться и выберем самый быстрый и единственный прямой путь – через тернии TCP/IP.

Настройка сети.

По идее работа с TCP/IP предполагает наличие физической Ethernet-среды (т.е. наличие сетевой карты), однако в MS придумали такой хитрый способ, как Loopback Adapter (или, как его окрестили наши умельцы – «Адаптер Microsoft замыкания на себя»). Этот самый адаптер эмулирует наличие сетевой карты. Итак, если у вас нет сетевой карты, то вы обязательно должны установить этот самый адаптер (Настройка – Установка оборудования – Сетевые платы – Microsoft – Адаптер

Microsoft замыкания на себя). Далее лезем в Пуск – Настройка – Сетевые подключения – Мастер установки сети. Я не буду пошагово описывать весь процесс, но в конце всех манипуляций в сетевых подключениях у вас должно появиться «Подключение по локальной сети». Нажимаем на нем правой кнопкой, выбираем свойства. Внимательно смотрим на список протоколов – отмечаем птичкой Virtual PC Emulated Ethernet Switch (остальные отмечены по умолчанию). Все, больше ничего не трогаем, закрываем окно. Теперь довольно опасный шаг – необходимо отключить всю файрволльную систему защиты на вашей машине (позже можете разрешить необходимый порт), т.е. убрать подальше все ZoneAlarm-ы и т.п. На всякий случай нужно также убрать все имеющиеся беспроводные соединения.

Если же вдруг у вас есть сетевуха, то настройте ее аналогично адаптеру замыкания.

Сводим эмулятор с eVC

Здесь важно выполнить все по порядку не упуская ни одного шага:

1. Закрываем все что открыто (включая eVC). Запускаем eVC.
2. Заходим в Tools – Configure Platform Manager.
3. Выделяем Pocket PC 2003 -> Pocket PC 2003 Emulator и жмем Properties.
4. В качестве транспорта (Transport) выбираем «TCP/IP Transport for Windows CE».
5. Жмем Configure справа от транспорта и обязательно ставим птичку на Use Fixed Address (при использовании loopback адаптера оно должно выдать что-то типа 169.254.XX.XXX).

Больше ничего здесь не трогаем.

6. В качестве сервера (Startup Server) выбираем «Emulator Startup Server».
7. Жмем Configure справа от сервера, в Communication-Ethernet выбираем «Virtual Switch». В качестве Image выбираем Pocket PC 2003 для КПК или два других для смартфонов. Значения всех Serial и Parallel портов установите в none.
8. Закрываем окно и жмем Test. Видим красивый скин КПК. Ждем (долго). Если все ОК – вы должны услышать фанфары. (Несчастный писк при загрузке CE в эмуляторе – это еще не фанфары, он всегда будет).

Все. Теперь можно запускать/отлаживать проги из среды eVC. Если же фанфар не было то дальше можете не читать.

Сводим эмулятор с ActiveSync

Здесь все еще проще. Ставим ActiveSync (на всякий случай уточню - не ниже той версии, что указана выше) и запускаем его. Заходим в File -> Connection Settings. Ставим галочку на Allow Network (Ethernet)... Теперь как бы девятым шагом (см. 8 предыдущих выше, т.е. эмулятор уже запущен) проделываем следующее: нажимаем Пуск -> Программы -> Microsoft Pocket PC 2003 SDK -> Pocket PC 2003 SDK, вываливаемся в командное окно. Набираем: EmuASCfg.exe /PPC. Обламываемся J. Вот тут то нам и нужен Microsoft .NET Framework 1.1 SDK (Compact Framework не проканает). Ставим. Опять пробуем: EmuASCfg.exe /PPC. Ждем. После ...complete можно закрывать окно. Теперь впервые переходим шарудить в эмулятор. Заходим в Start -> ActiveSync и жмем Sync. Дальше ActiveSync обнаружит новый КПК и все пойдет по стандартному сценарию – создаете Partnership и можете инсталлировать/копировать/удалять/синхронизировать файлы через ActiveSync, вообще работаете с эмулятором как с обычным КПК.

Запускать и отлаживать проги на эмуляторе вы сами разберетесь как – там не будет никаких проблем, т.е. если были фанфары на восьмом шаге, то можете расслабиться.

Наконец-то все. Надеюсь, ничего не упущено (именно малейшее упущение может испортить всю картину). Если у кого не получится – прошу в комментарии, будем вместе искать и добивать.

P.S. Вообще-то эмулятор по идее был бы очень полезен в реверсе/взломе чужих прог (по уже изложенной выше причине тормознутости на реальных устройствах или за неимением оных). Но

дело в том, что в PPC-шном эмуляторе из SDK 2003 стоит x86 проц Intel Pentium II (надеюсь, в следующих версиях они как-то очень напрягутся и сделают полную эмуляцию хотя бы ARM-а, представляю насколько это сложная задача, однако, вполне осуществимая). Т.е. даже если удастся найти версию нужной проги под x86 (что также весьма сомнительно) и подправить все необходимым образом – мы практически ничего не добьемся. Версия проги под ARM или MIPS будет выглядеть совершенно по- другому и нам придется проделывать всю работу заново.

Кстати, тут сам собой всплывает интересный вывод – эмулятор годится только для создания кейгенов J, и то слава Богу.

Интервью с Privalov'ым, создателем FASM

Автор: Aquila / WASM.RU

Дата: 2004-08-12

- Почему вы решили написать свой ассемблер?

В 1998 году я писал свою собственную операционную систему и мне был нужен хороший инструмент для этого. Тогда я пользовался Turbo Assembler, но в нём не хватало некоторых возможностей, которые мне были нужны - мне очень не нравилось жёстко задавать инструкции с помощью DB-директив. Поэтому я начал проект по созданию собственного ассемблера. Я также сделал его независимым от операционной системы, чтобы можно было легко спортировать в свою OS и в дальнейшем проводить всю разработку там. Я также написал портативный редактор с подсветкой синтаксиса в тех же целях, он назывался "flat editor", и исходники от него должны где-то валяться.

Позже я потерял интерес к разработке этой OS и, к превеличайшему сожалению, даже потерял исходники от последней версии (с работающими портами flat assembler'a и flat editor'a). Но в бэкапах у меня была досовская версия flat assembler'a, и в 2000 году я решил зарелизить её в интернет, просто потому что подумал, что кому-то она может оказаться полезной для разработки OS. И когда я обнаружил, что интерес к flat assembler гораздо больше, чем казалось вначале, я начал улучшать его и адаптировать для других целей, пока он не стал достаточно универсальным средством.

- Сколько времени у вас заняло написание первой версии FASM? Во-первых, пожалуйста, обратите внимание, что это был уже второй раз, когда я писал ассемблер. Первый, созданный в 1998 году, был написан без малейшего представления, как надо писать компилятор или ассемблер. Но таким образом я получил опыт, позволивший мне в 1999 году начать FASM и закончить первую довольно быстро. Я начал этот проект 23 марта, а первая версия с полной поддержкой набора инструкций .386 и способная компилировать саму себя, была готова в начале мая. Так что это заняло у меня около месяца. **- FASM очень удобен, но при этом не стоит ни копейки. Не жалеете ли вы, что приходится давать его бесплатно?**

Я не думаю, что он был бы очень популярен, если бы он был коммерческим продуктом, особенно, если учесть, что сейчас гораздо меньше ассемблерщиков, чем было раньше, поэтому сомневаюсь, что это дало бы мне сколько-нибудь серьёзный доход. Хотя я подумывал о том, чтобы выпустить 64-х битную версию fasm как коммерческий продукт, если только в этом будет достаточный интерес. Я должен признаться, что у меня не достаёт мотивации работать над 64-х битной версией FASM бесплатно.

(Впоследствии Privalov решил не делать 64-х битную версию коммерческой.)

- Будут ли добавлены высокоуровневые конструкции (встроенный PROC, .IF/.WHILE и т.д.) в FASM? Мне всегда хотелось держать синтаксис fasm'a очень простым и основанным на небольшом количестве директив, которые бы позволяли реализовывать другие фишки вроде макросов. И хотя директивы fasm ещё недостаточно могущественны, чтобы реализовать некоторые очень сложные высокоуровневые конструкции, в общем случае этих директив более чем достаточно, чтобы реализовать практически любую возможность, каким-либо боком относящуюся к ассемблеру. И если кому-то действительно нужно больше, я бы посоветовал использовать какой-либо компилятор с fasm в качестве постпроцессора (back-end). **- Будет ли FASM добавлена оптимизация, как формальная (например, замена lea на mov, замена одних опкодов на более короткие), так и неформальная (например, перегруппировка инструкций, выкидывание ненужного кода)?** Нет, на мой взгляд, даже эта формальная оптимизация слишком противоречива, чтобы помещать

её в ассемблер. Тем не менее, практически для всего из вышеперечисленного можно сделать соответствующий макрос. - **Макроязык FASM нетривиален и отличается от MASM/TASM. Вы планируете добавить режим совместимости?**

Ну, он всё же немного похож на тот, который использовался в режиме "ideal" TASM'a, так как когда я проектировал FASM, я во всю пользовался TASM'ом. Все элементарные возможности макроязыка очень похожи, единственная разница, что в fasm'e содержимое макроса заключается в фигурные скобки. Но даже это можно изменить с помощью fasm'овской директивы "fix".

А более продвинутые возможности макроязыка мне пришлось проектировать с учётом уже существующей архитектуры FASM, вот почему они стали такими, какие они есть. Я никогда не думал о добавление какого бы то ни было "режима совместимости".

- **Будет ли 64-х битный режим добавлен в FASM?** Он находится в разработке, но сейчас немного заморожен, так как я работаю над многими другими вещами в то же самое время. Также, как я уже сказал выше, я подумывал о том, чтобы продолжить его разработку только в виде коммерческого проекта. - **Позволяет ли архитектура FASM адаптировать его для поддержки других процессоров (например, GameBoy :))?** Да, я старался сделать архитектуру fasm такой, чтобы подобную адаптацию можно было сделать достаточно легко (что всё равно будет кросс-ассемблированием, так как сам fasm написан под x86), однако если вы захотите использовать инструкции, синтаксис которых очень сильно отличается от Интеловских, вам придётся сделать довольно большие изменения в модуле парсинга, который ответственен за отделение и распознавание все синтаксических частей каждой строки исходного кода. - **Планируете ли вы добавить поддержку других OS в FASM?**

Да, FASM должен без особых проблем работать на любой системе, предназначенной для процессора, совместимого с IA32. Я недавно сделал версию, которая скомпилирована в объектный код, линкующего стандартную C-библиотеку, что должно позволить прилинковывать его к любому exe на любой операционной системе, имеющего такую C-библиотеку.

Также я планировал сделать версию для OS/2, так как некоторые люди просили её, но у меня просто никогда не было этой системы и у меня нет никакой информации, как сделать такой порт.

- **Самый глупый вопрос: мёртв ли ассемблер? :)** Исходя из того, что это интервью имеет место быть, мы можем сказать, что нет. Однако многое изменилось со времён "золотого века" ассемблера, и я могу сказать, что он мёртв в некоторых аспектах. Десять лет назад мнение о языке ассемблера, которое вы могли услышать от среднего программиста, заключалось в том, что ассемблер - это что-то вроде высшего искусства, требующего очень глубокого знания и большого опыта, но дающего вам огромное могущество. Я думаю, что это было что-то вроде программёрской мечты получить умения высокого уровня с помощью ассемблера. А сегодня вы скорее всего услышите что-то вроде следующего: "Ассемблер? Разве это не пустая трата времени?"- **Что вы думаете о .NET и IL?** Идея промежуточного кода на самом деле очень стара и хотя она имеет много преимуществ для HLL-кодеров (особенно в тех языках, которые должны быть "мудрее чем программист", но гораздо более стабильными благодаря этому, в некоторых случаях очень важно давать программисту МЕНЬШЕ силы - вот хотя бы механизмы защиты членов классов являются примером такого подхода). Но сейчас очевидно, что такие программы никогда не смогут побить хорошо написанный нативный машинный код в плане качества. Поэтому похоже, что это просто тенденция к ещё большему разделению языков программирования на две основные области: одна, в которой более важна стабильность, и другая, где более важно качество и сила. Конечно, программисты на ассемблере - это ядро второй группы. - **Что вы думаете об open-source проектах?** Очень хорошо видеть исходный код программы, которую вы используете, так как вы также можете убедиться, что внутри нет никакого опасного кода. И возможность использовать код в ваших собственных проектах тоже очень хорошо, хотя в случае с GPL мне кажется, что палка немного перегнута, так как она заставляет использовать ваш проект также под GPL. Поэтому я

предпочитаю гораздо более простую лицензию в стиле BSD. - **Знаете ли вы о демосцене? Есть ли у вас любимые группы, демки или интро?** Я интересовался этой темой в старые времена, но сейчас все контакты в этой области потеряны. Что касается любимых демо, то это "Second Reality" от Future Crew. - **Ваше мнение о вирусной сцене? Действительно ли она умерла и хорошо это или плохо?** У меня никогда не было никаких контактов с такой сценой, фактически я даже никогда не осознавал её существования. - **Ваше мнение о крэк-сцене и исследовании программ?**

Мне бы не следовало оправдывать reverse engineering, так как это сейчас незаконно во многих странах, хотя по-моему мнение у вас всегда должно быть право реверсировать программу, стоящую на вашем компьютере - у вас просто должно быть право знать, что вы запускаете, так же как вы имеет право знать, что вы едите, когда покупаете какую-либо еду. Но в обоих случаях происходят определённые манипуляции, позволяющие концерная держать свои секреты под замком (я имею ввиду что-нибудь вроде рецепта Кока-колы).

Конечно, в большинстве случаев реверсинг используется для преодоления программной защиты - и снова, у вас должно быть это право, если вы купили программное обеспечение, например, если вы хотите сделать программу, которой вы владеете, более лёгкой в использовании - единственное, что должно быть незаконным - это копирование её другим.

Это было моё мнение насчёт проблем с легальностью, что касается самого reverse engineering и крэкинга, я должен признаться, что делал подобные вещи довольно часто в собственных целях во времена DOS, и на мой взгляд, это может дать огромный опыт с ассемблером и ведёт к получения знаний действительно высокого уровня. Вероятно, среди крэкеров можно найти лучших ассемблерщиков.

- **Каким вы видите будущее программирования?** Программирование сейчас разделяется на разные области, и мне кажется, что в будущем они станут ещё более специализированным, так как действительно универсального языка не существует, и сомневаюсь, что когда-нибудь такой появится. - **Где вы живёте и почему?** Я живу в Польше, потому что я поляк. Конкретно сейчас я в Кракове, где находится мой университет. - **Ваше любимое блюдо :).** Ох, это трудный вопрос. Скажем... курица с картошкой :). - **Ваши любимые книги и/или жанры книг.**

Мне всегда нравилось читать, много читать - и я люблю разные жанры, но SF и фэнтези - это жанры, которые читал больше всего. Поэтому такие книги занимают большую часть списка моих любимых книг, хотя он постоянно меняется. Некоторые из моих любимых книг следующие: "Большой, маленький" Джона Кроули, "Хроники Томаса Ковенанта" Стивена Р. Дональдсона, "Понедельник начинается в субботу" и "Пикник на обочине" Стругацких, многие книги Филиппа К. Дика и Курта Воннегута и другие.

А один из моих любимых авторов книг, не относящихся к SF&F, - это Эрих Мария Ремарк.

- **Вам нравятся компьютерные игры?** Я играл много несколько лет назад, когда игры выходили в основном под DOS. Поэтому все мои любимые игры дотовские, я упомяну только "Betrayal at Krondor" и "Privateer", но это только вершина айсберга. - **Вы верите в Бога или другое высшее существо?** Да, я верующий католик. - **Пробовали ли вы когда-нибудь осознанные сновидения?** Я делал это довольно часто на протяжении многих лет, иногда получая действительно большой контроль (даже изменения всей сцены действия), но затем мне это немного наскучило от этого, и теперь это происходит гораздо реже. И только недавно я узнал, что этот феномен имеет собственное название и даже обсуждается в интернете. - **Нравится ли вам аниме или нет и почему?**

Я не могу ответить на этот вопрос просто потому, что никогда особенно не сталкивался с этим явлением.

Interview with Privalov the author of FASM

Автор: Aquila / WASM.RU

Дата: 2004-08-12

- Why did you decided to code your own assembler?

I was writing my own operating system back in 1998 and I needed a good tool for this. These days Turbo Assembler was my favourite, but still lacked some features I needed - I hated to hardcode instructions with DB directives. So I have started my own assembler project and I also made it OS-independent, so I could easily port it into my own OS and do all the development from there. I have also written a portable editor with syntax highlighting for the same purpose, it was called "flat editor", I still have its sources somewhere.

Later I have lost the interest in continuing that OS, and unfortunately I have even lost the sources for the last version (with working ports of flat assembler and flat editor). But I still had the DOS version of flat assembler on backups, and in 2000 I've decided to release it in the internet, just because I thought someone might find it useful for OS development. And when I found out there was more interest in it than I initially thought, I started improving it and adapting it for other purposes also - finally it became quite an universal tool.

- How many time you had been coding first version of FASM? First of all, please note that it was the second time I was writing an assembler. The first one I have written in 1998 without any clue how to write any compiler or assembler. But I gained this way the experience that allowed me to begin in 1999 the flat assembler project in a planned way and finish the first working version quite fast. I have started this project at March 23rd and the first version with complete 386 instruction set implemented and capable of recompiling itself was ready in the beginning of May. So it all took me about one month. **- FASM is very convenient but for all that is absolutely free. Don't you regret having to give it to everybody for free?**

I don't think it would get very popular if it was commercial product, especially are there are much fewer assembly language programmers than used to be, and so I doubt it would give me any serious profit. Though I was considering making a 64-bit version of fasm as a commercial product, if only there was enough interest in it. I must confess that I lack the motivation to work on the 64-bit fasm for free.

(Lately Privalov decided against turning 64-bit version of fasm commercial way.)

- Will high-level constructions (built-in PROC, .IF/.WHILE etc.) be added into FASM? I always wanted to keep the fasm's syntax very simple and based on just few powerful directives that would allow implementing any other features as an custom "upper layer" for assembly language, like macroinstruction packages. And though fasm's directives might still not powerful enough to implement some very sophisticated HLL-like constructions, they seem to be generally more than enough to implement any customization that is still somewhat related to assembly language. And when someone really needs more, I might suggest using some compiler with fasm as a back-end. **- Will instruction optimisation, both formal (f.e. replacing lea with mov, replacing opcodes with shorter ones) and non-formal (f.e. instruction regrouping, cutting unnecessary code), be added into FASM?** No, even those formal optimisations are too controversial to put them into the assembler in my opinion. However, you can make an optimising macros for most of them. **- FASM Macrolanguage is non-trivial and different from MASM/TASM. Are you planning to add compatability mode?**

Well, they are still a bit similar to those from TASM's ideal mode, as when I was desiging fasm, I was programming in TASM's ideal all the time. All the elementar macro features all very similar, the only difference is that in fasm you use braces to enclose the macro contents. But even this may be changed with the fasm's "fix" directive.

And the more advanced macro features I had to design considering already existing fasm's internals, and that's why they become what they are. I actually never thought of adding any "compatibility mode".

- Will 64-bit mode be added to FASM? It's in development, but currently a bit sustained, as I'm working on many other things at the same time. Also, as I said above, I'm considering that I will continue its development only as a commercial project. **- Is architecture of FASM allow to adapt it to other processors (f.e. GameBoy :))?** Yes, I was trying to make fasm's architecture in a way that would allow such adaptations easy (that would be cross-assembling anyway, since fasm itself is written with x86 mnemonics), but if you would like instructions of syntax much different from the Intel's one, you would have to make quite large changes to parser module, which is the one responsible for separating and recognizing all the syntactic parts of each source line. **- Are you planning to add support of other OSes to FASM?**

Yes, fasm should be able to run on any system based on the IA32-compatible processor. I have recently made a version that is compiled to object code linking to the standard C library that should be linkable to working executable on any operating system that has such C library.

Also I was planning to make some version for OS/2, since there were some people requesting it, but I simply never had this system and I don't have enough knowledge about it to make such port.

- The most stupid question: is assembler dead? :) From the fact that this interview is taking place, we can simply tell: it's not. However much has changed since the "gold" assembly language days and I would say it IS dead in some aspects. Ten years ago the opinion about the assembly language you would hear from the average programmer was that it's something like the highest art, requiring very deep knowledge and experience, but giving you incredible power. I think it was even a kind of common programmers' dream to gain a high skills in programming with assembler. And today you are most likely to hear something like: "Assembler? Isn't it a waste of time?". **- What do you think about .NET and IL?** The idea of intermediate code is actually very old and though it has many advantages for HLL coders (especially for the languages that need to be "wiser than a programmer", but extremely stable thank to it, in some cases it's very important to give LESS power to the programmer - well, even the classing protected mode mechanisms are an example of such approach). But it should be obvious that such programs will never be able to beat well written native machine code in terms of performance. So it seems to be just a tendention to higer specialization, splitting programming languages into two main areas: one where the stability is most important, and one where the performance and power is most important. Of course assembly programmers are the core of the second one. **- What do you think about open-source projects?** It's good to see the source of program you're using, as you can also use it to make sure there is no any dangerous code inside. And an option to reuse code in your own projects is a very good thing, too - but I think it's a bit too strong in case of GPL, as it forces you to release such a projects open-source under GPL. That's why I prefer much simpler BSD-style license. **- Do you know about demo scene? Have you favourite groups, demos or intros?** I used to know about it in the old days, but now I have lost any contact. As for the favourite demos, my favourite of all the time is the Future Crew's "Second Reality". **- Your opinion about virii scene? Is it really dead and is it good or not? :)** I never had any contact with such scene, in fact I didn't even really realize its existence. **- Your opinion about crack and software research scene?**

I should not advocate any reverse engineering, since it is now illegal in many countries, though my opinion is that you should always have right to reverse a program you've got on the computer - you just should have right to know what you're running, in the same way as the right to know what you are eating when you bought some food. But in both cases it has been manipulated to allow concerns keep their secrets hidden (I mean something like the recipe for Coca Cola).

Of course reversing in most cases is used to break the software protection - and, again, you should have right to do it, when you have bought the software, for example just to make the software you own easier to use - it's only copying it to the others that is and should be illegal.

So it was my opinion about legality problems, as for the reverse engineering and cracking itself, I must confess I was doing such things quite frequently for my own use in DOS days, and in my opinion doing it gives you a great experience with assembly language and leads to gaining really high quality skills. Perhaps the best assembly programmers would be found among the crackers.

- What future of programming do you see? The programming is now splitting into different areas, and I think in future they all will become even more specialized, as there are no really universal programming languages and I doubt there ever will be any. **- Where do you live and why?** I live in Poland, as I'm Polish. Right now in the city of Krakow, where is my university placed. **- Your favourite dish :).** Oh, this is a hard question. Let's say... some chicken with potatoes. :- **- Your favourite books and/or book genres.**

I always loved reading, a lot of reading - and I like many different genres, but SF and Fantasy were usually the genres I was reading the most. So also such books would occupy the largest part of my favorites list, though this list is constantly changing. Some of my favourites are: "Little, Big" by John Crowley, "Chronicles of Thomas Covenant" by Stephen R. Donaldson, "Monday begins on Saturday" and "Roadside picnic" by A. and B. Strugatsky, many books of Philip K. Dick and Kurt Vonnegut, etc.

And one of my favourite authors other than SF&F ones is Erich Maria Remarque.

- Do you like computer games? I used to play a lot some years ago, when the games were still released mainly for DOS. So all my favourite games are the DOS ones, I will just mention "Betrayal at Krondor" and "Privateer" but this is just the top of the iceberg. **- Do you believe in God or other superbeings?** Yes, I am active Roman Catholic. **- Have you ever tried lucid dreaming?** I was doing it very frequently for a years, with sometimes really much amount of control (even changing all the scenery), But then I became a bit bored with it and now it happens much less frequently. And only recently I have realized that this phenomenon has its own name and is even discussed in the internet. **- Do you like anime or not and why?**

I cannot simply say that I like it or not, just because I almost don't know it. I never had much contact with it.

Как написать AddIn для WinAsm Studio

Автор: shoo

Дата: 2005-02-05

"Дай порулить!"
(Неизвестный автор)

исходные тексты (находится в архиве wasm_files.zip: files/0358/SimAddIn.zip)

0. Предисловие.

Каждому в глубине души хочется иметь "свой" редактор. Однако добавить в IDE новую возможность значительно проще, чем разработать ее целиком. Основным препятствием для этого (кроме лени) является необходимость изучения работы IDE и интерфейса для AddIn-ов. Сами же разработчики IDE заинтересованы в такой помощи по понятным причинам, и стремятся упростить это узкое место. Данная статья призвана пролить некоторый свет на этот вопрос пользователям WinAsm Studio и базируется на файлах **WinAsm.chm**, **WAAddIn.inc**, анализе кода других AddIn-ов, в том числе примеров, а также на небольшом собственном опыте. Естественно, она не является ни полным переводом, ни исчерпывающим пособием.

В настоящее время я использую версию WinAsm-а 4.0.3, однако большинство из написанного действительно и для версий 3.x.x и даже более ранних. Необходимо заметить, что Антонис Киприянов, автор WinAsm-а не только постоянно совершенствует свою IDE, но и развивает интерфейс для AddIn-ов, внемля просьбам трудящихся.

Автор WinAsm Studio: Antonis Kyprianou.

Составитель WinAsm.chm: Jim Giordano.

Сайт программы: <http://www.winasm.net>

1. Что есть AddIn и как его использовать в WinAsm.

AddIn- это подключаемый код, предназначенный для расширения функций редактора и размещенный в отдельной dll. Для того, чтобы WinAsm его нашел, dll нужно разместить в подкаталоге AddIns домашнего каталога WinAsm-а. Под "подключением" AddIn-а имеется ввиду загрузка и инициализация соответствующей dll. Для того, чтобы загрузить AddIn, необходимо открыть менеджер AddIn-ов (Add-Ins Manager) в меню Add-Ins. Выбрав нужный AddIn в списке, отметить чекбокс "Loaded/Unloaded" чтобы его загрузить. Также можно отметить чекбокс "Load On StartUp", тогда AddIn будет загружаться автоматически при загрузке WinAsm. Для своего функционирования AddIn должен экспортировать функции, часть из которых обязательна. WinAsm, в свою очередь, предоставляет AddIn-у указатели на внутренние структуры с важными переменными, а также выполняет ряд сервисов, обрабатывая дополнительные сообщения в оконной процедуре главного окна. Основной поток сообщений, получаемых окнами WinAsm-а, пропускается вначале через соответствующие процедуры загруженных AddIn-ов, при этом AddIn может не только выполнить свой код, получив соответствующее сообщение, но и прекратить его дальнейшую обработку, вернув **TRUE** в **eax**, полностью подменив, таким образом, некоторую встроенную функцию своим кодом.

2. Что нужно для написания AddIn-a.

Исходник AddIn-a представляет из себя обычный проект dll и включает в себя, как минимум:

- **addin.asm** - исходный текст.
- **addin.def** - файл, в котором определяются имена экспортируемых функций.
- **WAAaddin.inc**- файл, в котором определены структуры, используемые в WinAsm, и ряд констант, в том числе и дополнительные сообщения, обрабатываемые контролем окна редактора **CodeHi.dll**.

WAAaddin.inc регулярно обновляется вместе с появлением новых функций в WinAsm-е. Он обширно откомментирован и снабжен примерами использования некоторых возможностей. Чтобы быстро создать шаблон нового AddIn-a, можно вызвать мастер новых проектов Ctrl+N, и в разделе "Bare Bone" выбрать тип проекта "AddIn". Можно также использовать исходные тексты примеров или рабочих AddIn-ов, так как практически все они ими снабжены.

Также для написания AddIn-a удобно сразу же в имя результирующего бинарного файла включить путь к папке с AddIn-ами. При компиляции необходимо учитывать, что она возможна только тогда, когда AddIn выгружен. Для обхождения этого неудобства можно использовать другой AddIn - Enhanced Add-Ins Manager - улучшенный менеджер AddIn-ов (автор - Mario Vilas), который при компиляции автоматически выгружает текущий AddIn, а после нее - загружает. Однако он не всегда это делает успешно, если при компиляции произошла ошибка. При написании AddIn-a необходимо быть готовым к тому, что WinAsm из-за него рухнет, поэтому не торопитесь включать для него опцию "Load On StartUp". Если же это произошло при изменении существующего AddIn-a, который загружается вместе с WinAsm-ом и рушит его, отключить его автоматическую загрузку можно убрав строку с названием соответствующей dll из раздела [ADDINS] файла winasm.ini (расположен в домашнем каталоге WinAsm-a). WinAsm позволяет запускать себя в "защищенном от AddIn-ов" режиме при удержании F8, однако это оказалось малоэффективным при старте его из разных файловых менеджеров, которые сами обрабатывают нажатие F8. Возможно, в дальнейшем это будет исправлено. Для отладки AddIn-ов можно использовать как vkdebug, идущий в составе пакета masm32, так и любые другие отладчики, запуская под ними WinAsm, загружая AddIn и переходя к отладке. Для отладки я обычно использую следующий макрос, исключающий сообщение об ошибке при запуске WinAsm-a с AddIn-ом вне отладчика:

```
deb      macro
    local nodeb
    push eax
    invoke IsDebuggerPresent
    test eax,eax
    pop eax
    jz nodeb
    int 3
nodeb:
endm
```

3. Механизм загрузки, выгрузки и настройки AddIn-a.

Возможны следующие варианты обращения к dll AddIn-a:

А. Получение сведений об AddIn-е менеджером AddIn-ов

- AddIn не загружен:

```
DllEntry (DLL_PROCESS_ATTACH)
GetWAAaddinData
DllEntry (DLL_PROCESS_DETACH)
```

- AddIn загружен:

```
GetWAddInData
```

Б. Настройка AddIn-а из менеджера AddIn-ов

- AddIn не загружен:

```
DllEntry (DLL_PROCESS_ATTACH)
WAddInConfig
DllEntry (DLL_PROCESS_DETACH)
```

- AddIn загружен:

```
WAddInConfig
```

В. Загрузка AddIn-а:

```
DllEntry (DLL_PROCESS_ATTACH)
WAddInLoad
```

Г. Выгрузка AddIn-а:

```
WAddInUnload
DllEntry (DLL_PROCESS_DETACH)
```

4. Обязательные процедуры.

А. DllEntry

Это стандартная процедура DllMain, она вызывается при загрузке/выгрузке динамической библиотеки. В AddIn-е используется для получения хэндла dll при загрузке.

```
DllEntry Proc hInst:HINSTANCE, reason:DWORD, reserved1:DWORD
```

hInst- хэндл модуля dll; **reason**- параметр, определяющий причину вызова данной функции, в нашем случае может принимать следующие значения: **DLL_PROCESS_ATTACH**- выполняется загрузка библиотеки в память процесса, **DLL_PROCESS_DETACH**- выполняется выгрузка библиотеки из памяти процесса; **reserved1** - он и в Африке reserved1.

Процедура **DllEntry** присутствует в шаблоне и обычно не нуждается в модификации.

Б. GetWAddInData

```
GetWAddInData Proc lpFriendlyName:PTR BYTE, lpDescription:PTR BYTE
Invoke lstrcpy, lpDescription, Offset szDescription
Invoke lstrcpy, lpFriendlyName, Offset szFriendlyName
RET
GetWAddInData EndP
```

Простая процедура, вызываемая **WinAsm** для получения удобочитаемых названий и описаний AddIn-ов для последующего их отображения в списке менеджера. Также присутствует в шаблоне. В модификации нужны текстовые переменные **szFriendlyName** и **szDescription**, содержащие соответственно имя и краткое описание AddIn-а, выводимые менеджером AddIn-ов в списке.

Например, в AddIn.Inc записываем:

```
szFriendlyName DB "Простой AddIn",0
szDescription DB "Этот AddIn позволяет читать и "
DB "вставлять слово под курсором.",0
```

В. WAddInLoad

Процедура, вызываемая при инициализации AddIn-а.

```
WAddInLoad Proc pWinAsmHandles:DWORD, features:PTR DWORD
```

В качестве параметров передаются указатели на структуру с важнейшими хэндлами **HANDLES**(pWinAsmHandles) и на структуру с разными переменными features (содержит номер версии WinAsm-a, указатели на списки процедур загруженных AddIn-ов, на переменную с хэндлом дочернего окна редактора ресурсов и на список меток редактора ресурсов). Эти структуры подробнее описаны в **WAAAddIn.inc**.

В этой процедуре **AddIn** может скопировать в свои переменные такие важные значения, как хэндл главного окна WinAsm-a (**HANDLES.hMain**), хэндл клиентской части главного окна (**HANDLES.hClient**), хэндл главного меню (**HANDLES.hMenu**), и многие другие. Конечно же, можно запомнить и сам указатель на структуру WinAsmHandles, однако нельзя сказать, что это удобней.

Также в этой процедуре AddIn может произвести собственную инициализацию: выделить память, добавить свои опции в меню и т.п. Значение, возвращаемое этой процедурой, свидетельствует об успешности инициализации AddIn-a, и в случае неудачи (eax=TRUE) AddIn будет выгружен.

Г. WAAAddInUnload

Данная процедура не имеет параметров и вызывается при выгрузке AddIn-a. Здесь AddIn должен освободить выделенную ему память, удалить свои опции меню, и т.п. То есть, восстановить status quo.

5. Необязательные процедуры.

Для использования необязательных процедур AddIn-a в WinAsm их названия необходимо вписать (или декомментировать) в разделе EXPORTS def-файла проекта.

А. WAAAddInConfig

Процедура настройки AddIn-a. Вызывается кнопкой из менеджера AddIn-ов и имеет те же параметры, что и процедура WAAAddInLoad. Удобна для настройки AddIn-ов, не создающих свои опции меню или окна, а также позволяет настраивать AddIn-ы, которые в данный момент не загружены.

Б. Процедуры обработки сообщений окон:

FrameWindowProc- получает все сообщения, посылаемые главному (MDI) окну WinAsm.

ChildWindowProc- получает все сообщения, посылаемые всем дочерним окнам WinAsm.

ProjectExplorerProc- получает все сообщения, посылаемые окну менеджера проекта.

OutWindowProc - получает все сообщения, посылаемые окну вывода результатов компиляции.

все эти процедуры имеют одинаковый формат, соответствующий формату процедуре окна:

```
Procname Proc hWnd:DWORD uMsg:UINT, wParam:WPARAM, lParam:LPARAM
```

Перед тем, как процедура окна WinAsm обработает сообщение сама, это сообщение будет пропущено через все соответствующие процедуры AddIn-ов. Если какая-либо процедура AddIn-a вернула eax=TRUE, дальнейшая обработка данного сообщения прекращается. При этом следует заметить, что если два AddIn-a обрабатывают одинаковое сообщение, то первенство будет согласно загрузки, а порядок загрузки AddIn-ов пока не регламентировался. После обработки сообщения WM_COMMAND самим WinAsm-ом всем процедурам FrameWindowProc дополнительно посылается сообщение **WAE_COMMANDFINISHED** с теми же параметрами, что дает возможность выполнить код после обработки команды редактором (а не перед).

6. Работа с меню.

Вероятно, AddIn-у потребуется создать свою опцию меню. Для этого могут понадобиться либо хэндл главного меню, либо хэндл одного из существующих выпадающих меню. Их можно взять из структуры HANDLES и вложенной структуры POPUPMENUS, их соответствие понятно из названий:

```
POPUPMENUS STRUCT
    hFileMenu      HWND
    hEditMenu      HWND
    hViewMenu      HWND
    hProjectMenu   HWND
    hFormatMenu    HWND
    hDialogMenu    HWND
    hMakeMenu      HWND
    hSetActiveBuildMenu HWND    субменю в меню "Make"
    hToolsMenu     HWND
    hAddInsMenu    HWND
    hWindowMenu    HWND
    hHelpMenu      HWND
    hNewFileMenu   HWND    субменю в меню "File"
    hConvertMenu   HWND    субменю в меню "Format"
POPUPMENUS ENDS
```

Например, если мы решили создать новую опцию в выпадающем меню "Add-Ins", то его хэндл мы можем получить так (в процедуре WAAAddInLoad):

```
mov ebx,pWinAsmHandles ; загружаем в ebx указатель на структуру с хэндами
push [ebx].HANDLES.POPUPMENUS.hAddInsMenu ; получаем хэндл субменю "Add-Ins"
pop hSubM ; сохраняем хэндл в нашей переменной
```

Далее, для создания новой опции меню нам потребуется уникальный идентификатор, который можно получить, послав сообщение WAM_GETNEXTMENUID главному окну:

```
invoke SendMessage,hMain,WAM_GETNEXTMENUID,0,0
mov MenuID,eax
```

Теперь можно создавать свою опцию меню:

```
invoke InsertMenu,hSubM,MenuID,MF_BYCOMMAND or MF_OWNERDRAW,MenuID,\
                                     ADDR menu_caption
```

При выгрузке AddIn-а (в процедуре WAAAddInUnload) следует удалить свою опцию меню:

```
invoke DeleteMenu,hSubM,MenuID,MF_BYCOMMAND
```

Хэндл главного окна нужно взять из структуры HANDLES. Там же можно получить и хэндл главного меню:

```
...
push [ebx].HANDLES.hMain ; получаем хэндл главного окна
pop hMain
push [ebx].HANDLES.hMenu ; получаем хэндл меню
pop hMenu
...
```

Иногда AddIn может обрабатывать существующие команды меню, тогда создавать свои идентификаторы не нужно. Встроенные идентификаторы перечислены в WAAAddIn.inc. Обработав таким образом некоторый идентификатор и вернув в eax значение TRUE AddIn может полностью подменить соответствующую функцию WinAsm-а своей. AddIn может также заставлять WinAsm выполнять нужные функции, посылая встроенные идентификаторы главному окну WinAsm-а.

7. Создание плавающих окон (docking windows).

Для создания такого рода окон необходимо использовать внутренние сообщения главному окну WinAsm-а, описанные в разделе "Docking windows" файла WAAAddIns.inc:

WAM_CREATEDOCKINGWINDOW - создать окно

WAM_GETCLIENTRECT - получить размеры клиентской области окна
WAM_DESTROYDOCKINGWINDOW - удалить окно

Также для создания такого окна необходимо будет заполнить структуру **DOCKINGDATA**, описанную там же. **8. Работа с окном редактора.** Вероятно, **AddIn**-у потребуется обратиться к окну редактора для анализа/модификации редактируемого текста. Для этого вначале необходима небольшая рутинная последовательность:

```
...  
; получаем хэндл активного дочернего окна  
invoke SendMessage,hClient,WM_MDIGETACTIVE,0,0  
; если таковое отсутствует - выходим...  
test eax,eax  
jz wp_exit  
; получаем указатель на структуру с данными окна  
invoke GetWindowLong,EAX,0  
; получаем из структуры хэндл контрола ричедита  
mov eax,[eax].CHILDDATA.hEditor  
mov hEditor,eax  
...
```

Также во многих случаях желательно убедиться в том, что окно видимо при помощи функции **IsWindowVisible**. Теперь можно работать с окном редактора. Сообщения, которые поддерживает контрол, перечислены в **WAAddIn.inc** и состоят как из ряда стандартных сообщений, так и дополнительных, которые начинаются на **CHM_** (CodeHi Message). Стандартные сообщения в **WAAddIn.inc** закомментированы, так как они уже описаны в **windows.inc**. Смысл большинства сообщений понятен из названия.

Например, получим слово, находящееся под курсором:

```
invoke SendMessage,hEditor,CHM_GETWORD,128,offset BufferForWord
```

Также в структуре **CHILDDATA** можно найти другие полезные данные: имя файла с полным путем, внутренний тип файла и некоторые другие. Эта структура также описана в **WAAddIn.inc**.

9. Работа с акселераторами.

Для добавления своих клавиатурных акселераторов можно воспользоваться хэндлом таблицы акселераторов, указатель на него имеется в структуре **HANDLES**: **HANDLES.phAcceleratorTable**. Работа с таблицей акселераторов показана на примере (неидеальном, но работающем) в демонстрационном **AddIn**-е **AcceleratorDemoAddIn**.

10. Хранение настроек.

Для настройки **AddIn**-а можно использовать как процедуру **WAAddInConfig**, так и другие методы. Преимущество первого способа заключается в том, что процедура настройки конфигурации может быть вызвана у незагруженного **AddIn**а, или в том случае, когда у **AddIn**-а нет своих окон, что избавляет от необходимости создавать специальные опции меню только для настройки **AddIn**-а. Соблюдения порядка ради автором **WinAsm**-а предлагается хранить настройки **AddIn**-а в файле **WAAddIns.ini**, находящегося в каталоге **AddIns** вместе с **AddIn**ами. Для хранения своих настроек желательно создать свою секцию, соответствующую названию **AddIn**-а, где и хранить значения переменных. Запись/чтение переменных производится при помощи функций **WritePrivateProfileString** и **GetPrivateProfileString**.

11. Дополнительные возможности для разработки AddIn-ов.

На сайте **WinAsm Studio** меется пакет WAAAddInLib, разработанный Mario Vilas (QvasiModo), в котором часть рутинного кода оформлено в виде процедур, что позволяет ускорить написание AddIn-ов. Пакет также регулярно обновляется. **12. Пожелания.**

Удачи всем!

Александр Журенков (aka shoo)

Создание заплаток на ассемблере FASM

Автор: Smile

Дата: 2006-06-21

Введение

Аналогичным англоязычным названием является - "patch", но использовать я буду отечественный вариант "заплата".

Рассматривать мы будем создание заплаток для бинарных файлов и исполняемых файлов формата PE. Способ подходит как для изменения нескольких байт, так и для создания подпрограмм реализующих дополнительную функциональность.

Основной инструмент - **FlatAssembler**. Компилятор генерирует очень компактный код и предоставляет абсолютный контроль над размещением данных.

Дополнительная особенность - это ужаснейший препроцессор.

Также потребуется программа, которая запишет изменения.

Принцип работы достаточно прост, все изменения описываются в исходном файле, после компиляции выходной файл и вывод препроцессора используются программой правки для внесения изменений.

Исходный файл с описанием изменений ничем не отличается от обыкновенной программы на ассемблере, единственное, блок кода/данных нужно заключать в макрокоманды `__setorg/__mkdiff`. Первая определяет положение в файле, по которому будет записан блок, а вторая определяет конец блока.

Бинарные файлы

При изменении бинарного файла, положение установленное макрокомандой `__setorg` в точности совпадает с позицией в реальном файле.

```
; Файл 1.asm
; Определить строку "Hello" начиная с 8-го байта

include 'patchit.inc' ; Определение макрокоманд

__setorg 8             ; Устанавливаем позицию в файле
db 'Hello' ; Определяем данные
__mkdiff              ; Принять изменения
```

Также можно использовать ассемблер и изменять значения счетчика команд.

Например, нам требуется изменить *.com программу так, чтобы при запуске управление передовалось нашей подпрограмме.

```
; Файл 2.asm
; Изменение инструкций *.com программы

include 'patchit.inc'

; Изменить первую инструкцию
__setorg 0
org $100 ; адрес по которому загружается программа
jmp $120 ; передать управление по адресу $120
__mkdiff

; Подпрограмма выводит на консоль строку текста
__setorg $20 ; переместить положение в файле
org $120 ; установить адрес
mov ah,13h
```



```

mov al,1
xor bh,bh
mov bl,07h
xor dx,dx
mov bp,msg_text
mov cx,msg_size
int $10
ret
msg_text: dw 'Hello sailor!'
msg_size = $ - msg_text
__mkdiff

```

В дополнение можно рассказать о нескольких особенностях использования.

Изменять небольшие участки удобно макрокомандой `__mkchng`, она заменят набор команд `__setorg/__mkdiff` если требуется изменить всего одну строку.

```

; Файл 3.asm
; Пример использования __mkchng

include 'patchit.inc'

__mkchng 10, <db 'Hello'>
; Заменяется на
; __setorg 10
; db 'Hello'
; __mkdiff

```

Для контроля выхода за пределы файла можно использовать макродирективу `__maxorg`, при этом необходимо указать размер файла непосредственно в байтах, либо полный или относительный путь к файлу, тогда макрос сам вычислит размер. Превышение этого значения приведет к ошибке компиляции.

```

; Файл 4.asm
; Пресечение выхода за границу файла

include 'patchit.inc'
__maxorg 2 ; максимальный размер 2 байта
__mkchng 0, <db 1,2,3> ; error 'выход за границу файла'

; Аналогичный способ если доступен файл
__maxorg '1.asm'
__mkchng 10000,<db 'ups!> ; error 'выход за границу файла'

```

Файлы формата PE

Правка исполняемых файлов особенно утомительный процесс, так как значение адресов и смещений запущенной программы не соответствует их положению в файле.

И для того чтобы узнать позицию инструкции в файле(RVA) по адресу памяти(VA), нужно произвести вычисление вида:

$$RVA_секции + VA_базы - VA_инструкции$$

Используя макроопределения можно достаточно реально воспроизвести ситуацию, когда при написании кода вам необходимо знать только адрес в памяти(VA), тогда как программа, используя информацию о файле, сама вычислит действительный адрес инструкции(RVA).

Используя макроопределения ситуация упрощается, устанавливать положение маркера можно по виртуальному адресу.

Все, что потребуется - это используя макродирективу `__format_pe` указать основные характеристики - базовый адрес загрузки и секции. Описание секции состоит из трех параметров "VA RA N". Где, VA - относительный адрес секции в памяти, RA - позиция секции в файле, N - реальный размер секции в байтах.

Узнать значения можно постмотрев заголовков файла в отладчике или специальной программой для работы с файлами формата PE(я использовал PeID).

```
; Файл 5.asm
; Пример изменения PE файла

include 'patchit.inc'

__format_pe $400000,\           ; Базовый адрес загрузки (ImageBase)
    $1000, $400, $200,\       ; Секция кода '.code'
    $2000, $600, $200         ; Секция данных '.data'

; Адреса импортируемых функции
_MessageBoxA = $403044
_ExitProcess = $40303C

__setorg $401000
push 0
push dword caption
push dword message
push 0
call dword [_MessageBoxA]
push 0
call dword [_ExitProcess]
__mkdiff

__setorg $402000
message db 'The file was patched by unknown hacker',0
caption db 'Attention!'
__mkdiff
```

Теперь положение в файле вычисляется по виртуальному адресу. Выход за пределы секции контролируется и вызывает ошибку компиляции.

Заключение

Программа правки и примеры находятся в приложении к статье. В качестве дополнительного описания могу посоветовать только исходный код, он достаточно небольшой.

Хочется поблагодарить товарища Tomasz Grysztar за отличный ассемблер, кодеров с wasm.ru и flatassembler.net/board

Исходники (находится в архиве `wasm_files.zip: files/0404/patchbyfasm.zip`) к статье.

Сайт автора - www.assert.i8.com.

Новогоднее поздравление с 2007 годом

Автор: wasm.ru

Дата: 2006-12-26

Уважаемые воины дзена! Заканчивается 2006 год, который был богат для **wasm.ru** событиями и изменениями, поэтому стоит подвести некоторые итоги того пути, который мы ~~проехали~~ прошли за это время.

Самым главным, несомненно, стал переезд сайта на выделенный сервер. Это значительным образом расширило доступные возможности и заложило основу для дальнейшего развития сайта. Это стало первым выполненным пунктом СПРС - стратегического плана развития сайта, нацеленного на ~~захват мира и господство над человечеством~~ его улучшение, усовершенствование и создание новых сервисов на благо ~~админов~~ пользователей.

Вторым важным шагом стала смена форума, воспринятая поначалу неоднозначно. Некоторые ~~еретики~~ пользователи обращали внимание на недостатки как в ~~кривом дизайне~~ визуальном оформлении, так и в плане юзабилити, однако с их помощью за относительно короткий срок подавляющее количество недочётов было исправлено, и стали очевидны достоинства нового форума: расширенные возможности (среди которых можно отметить ~~введёная спустя 3 года~~ долгожданная система личных сообщений) и ориентированность на больших размеров комьюнити, которое значительно выросло за последние несколько лет.

Вслед за форумом были сделаны некоторые изменения в движке: поправлены некоторые баги и добавлен ряд фиечей~~, хотя вообще движок пора уже давно переписать~~.

Нельзя и не упомянуть о том, что вторая половина года ознаменовалась изрядным пополнением контента сайта: обновления практически во всех разделах сайта появлялись с завидной регулярностью. Мы хотели бы поблагодарить всех тех, кто любезно предоставил нам свои статьи, источники и инструменты ~~(рубанок и стамеска пришлось очень кстати)~~.

Важное изменение произошло и в команде **wasm.ru**: в неё ~~въехал~~ вошёл **dead_body**, который взял на себя почётную миссию обновления разделов исходников.

Ещё несколько хотелось бы сказать относительно года будущего. В нём мы ~~собираемся пинать GUI~~ не собираемся останавливаться на достигнутом. Если ~~нас не заломает~~ всё сложится удачно, то в 2007 году будут реализованы остальные пункты ~~программы КПСС~~ СПРС, среди которых есть ~~захват мира и господство над человечеством~~ дальнейшее наполнение сайта контентом, усовершенствование движка и введение новых сервисов и подпроектов.

Но самое главное, пусть этот год станет удачным для вас. Желаем вам духовного роста и просветления ~~(и чтобы без похмелья)~~, быстрого кода и безглючных программ!

_~~Совет директоров~~ Команда **wasm.ru** _

26. Другие статьи

Swift или Objective-C, или о скелетах в яблочном склепе

Автор: Aquila

Дата: 2015-10-04

Что случилось

Вы, наверное, слышали, что корпорация **Apple**, эти задорные наглецы, недавно выпустила новый язык программирования «**Swift**» с таким же названием, как у [другого языка программирования](#)? По всему миру школьники средних классов на улицах, в переулках и подворотнях обсуждают, когда **Swift** вытеснит **Objective-C**.

Но что взять со школьника средних классов? Сегодня он учит природоведение и «Основы православной культуры», а завтра «Основы исламского шариата» и ОБЖ. Всё вокруг него меняется словно в калейдоскопе: предметы, языки, технологии. Ему 14 лет и у него в запасе еще 5-10 лет, чтобы валять дурака и учить затейливую нелепицу, придуманную в Минобре или Эппл.

Однако многие их нас уже не школьниками или, как минимум, не школьники средних классов. Нас не проведёшь.

Скелеты языков в яблочном склепе

Swift — это не первая попытка Apple создать свой, самый настоящий язык программирования.

В 1987 году компания выпустила [HyperTalk](#) — высокоуровневый язык для начинающих программистов, который могла бы использовать любая домохозяйка.

```
on mouseDown
  answer file "Please select a text file to open."
  if it is empty then exit mouseDown
  put it into filePath
  if there is a file filePath then
    open file filePath
    read from file filePath until return
    put it into cd fld "some field"
    close file filePath
    set the textStyle of character 1 to 10 of card field "some field" to bold
  end if
end mouseDown
```

И где теперь **HyperTalk**? Ушёл в Вальгаллу 1998, когда его изобретатель покинул компанию.

В 1990 Эппл посещает гениальная идея заменить Mac OS объектно-ориентированной ОС, и совместно с IBM они начинают разработку [Taligent](#), для которой предназначался язык программирования [ScriptX](#). Точнее, это была целая мультимедийная платформа — примерно как Macromedia Flash. Но и это не помогло, и она покинула этот бренный мир.

Затем для своего наладонника **Newton** Эппл создала [NewtonScript](#). Вы, наверное, уже догадались, что вместе с Newton'ом NewtonScript отправился в край вечной охоты к Ньютону и Копернику.



Параллельно эппловцы пытались поработить мир путём создания ЯП [Dylan](#), на котором можно было бы программировать и под Newton и под Mac OS.

```
define function factorial (n :: <integer>) => (n! :: <integer>)
  case
    n < 0    => error("Can't take factorial of negative integer: %d\n", n);
    n = 0    => 1;
    otherwise => n * factorial(n - 1);
  end
end;
```

Сброшен за борт ещё раньше, чем NewtonScript.

Однако от навязчивых идей сложно отказаться. В 2008 Apple выпускает [MacRuby](#), однако активное развитие прекращается в 2011, когда главный разработчик проекта покидает компанию.

Наконец, в 2014 предпринята ещё одна попытка создать замену Objective-C. С большой помпой и беснованиями престарелых школьников выпущен Swift.

Что из этого получится

Apple достигла немалых успехов в производстве мультимедийных коробок и прямоугольников. Однако её таланты в этой области оказываются губительными для её языков программирования. С философией апгрейда и развития, которая заключается в том, чтобы выкинуть старое устройство и заменить его новым, сложно создать ЯП, который окажется таким же долгоживущим, как Objective-C. Поэтому мы должны спрашивать не когда Swift вытеснит Objective-C, а когда Swift отправится к праотцам.

Ответ зависит от того, когда этот юноша покинет Apple:



Это Крис Латтнер, создатель Swift.

Следите за ним.

О разновидностях языков программирования

Автор: Aquila

Дата: 2015-11-08

Есть много разных классификаций языков программирования. Появилась она и у меня.

Шахтёрские. У этих языков нет громкой миссии «Спасти Мир и Человечество (или Мир от Человечества)», они не претендуют на то, чтобы быть самыми модными, и уже забыли о том, чтобы быть «дружелюбными к новичку», они не обязательно красивы, эlegantны или изящны. У них только одно назначение – решать задачи и проблемы в своей области, и служить средством добычи зарплаты для их практикующих. Пример – **PHP** и **Java**. **Дзенские.** Практикующие эти языки преследуют своей целью просветление. Освобождение от шаблонов и оков проявленного мира программирования, сощезание истинной природы бытия и осознание своей природы будды. Примеры – **C**, **Ассемблер** и **Forth**. **Мессианские.** Адепты этих языков претендуют на то, что у них миссия от божественных сил спасти программирующее человечество от заблуждений, грехов и ошибок в коде. Когда-то таким языком был Java, провозгласивший одной из заповедей нового культа греховность беззнаковых целых чисел и святость объектной ориентации, но со временем стал настолько опрофанел, что превратился в один из инструментов кодеров-шахтёров.

Как и в случае с обычными религиями на одного искреннего мессию приходится толпа шарлатанов. Распознать их легко: неотъемлемым свойством языка, где есть святость, является гомоиконность. Святой язык — это гомоиконный язык. Если же вам будет проповедовать о блаженстве гетероиконных языков — гоните еретиков прочь! Когда Бог говорил с людьми, он говорил на **Лиспе** со вставками ассемблера. **Шаманские.** Если предыдущие языки претендуют на связь с Верхним миром, то создатели этих заключают договор с сущностями из Нижнего. Именно этим объясняется то, что C++ способен решить практически любую задачу, но никто не понимает, каким образом это происходит. **Головоломки-ловушки.** Вы помните цикл фильмов «Восставшие из ада»? В них главные герои находят коробочку-головоломку, и когда они находят способ, как её открыть, за ними приходят inferнальные сущности и утаскивают в глубины ада. Некоторые считают, что **Хаскелл** — это что-то вроде такой коробочки. Ведь неслучайно, что его разрабатывают в Microsoft Research. **Хипстерские.** Представьте себе язык, в который без особенной системы понатыкали «модных» фиш и раскрутили так, что множество хипстеров абсолютно уверены, что он скоро поработит весь мир. «Я тоже на нём программирую», — говорит хипстер, — «ведь в нём есть pattern matching!» Задачей такого языка является создавать «движуху», «тренды», дать возможность почувствовать, что находишься «на краю прогресса», безопасно попробовать функциональное программирование и так далее.

Список категорий, конечно, неполон. Можно вспомнить, например, эlegantные (**Objective-C**) и мифологические (**Cobol**) языки, но близится вечер и пришло время завершить эту воскресную постиллу.

WebAssembly, или заговор четырёх

Автор: Aquila

Дата: 2015-11-29

Вестерновый салун где-то на восточно-европейских просторах. Несколько олдфагов и миддлфагов играют в тexasский холдем опкодами x86-процессора. Пара ньюфагов пытаются скомпилировать OneHalf под iOS. Неожиданно дверь распаивается, входит самурай с 64-х битной катаной на поясе и кладёт на стол постер «Разыскивается».

«**WASM Wanted!**» — гласит большая надпись.

— Мы снова в деле, котаны, — говорит самурай.

Великолепная четвёрка

Долгое, очень долгое время практически единственным языком программирования для страниц, загружающихся в браузере, был **Javascript**. Здесь можно вспомнить о Java-апплетах и Flash-виджетах, используя которые бесстрашные программисты пытались превзойти всевозможные ограничения, но все эти технологии имели ограниченное и не всегда удобное применение. Основным способом оживить свою страницу — заставить идти снег или бегать котика из одного угла в другой — оставался и остаётся Javascript. Для него было написано множество библиотек и фреймворков, но всё же в различных кругах росло недовольство.

— Желаем программировать на нормальном языке! — роптали одни.

— Почему всё такое медленное? — негодовали другие.

— Хотим компилировать! — восклицали третьи.

Наконец, их мольбы были услышаны. **Google**, **Microsoft**, **Mozilla** и группа инженеров, стоящая за Webkit (**Apple**, как обычно, делает загадочный вид), объединились, чтобы создать стандарт, который откроет новую эпоху в области web-программирования.

Они решили назвать его **WebAssembly**, а сокращённо — попросту **wasm**.

Что это такое

Проблемы, которые были озвучены — сложность программирования на других языках и низкая скорость выполнения, производители браузеров пытались решить по отдельности. Mozilla разрабатывала asm.js — подмножество JavaScript, которое, с одной стороны, исполняется обычным браузером, а с другой, браузер, поддерживающий этот стандарт (Firefox), способен выполнить его гораздо быстрее, чем обычный JavaScript. Также asm.js задумывался как решение проблемы с программированием на других языках — чтобы он мог служить целью компиляции других языков. Google предложил своё решение — Native Client, в котором бы исполнялся запесоченный для безопасности скомпилированный код C и C++. Однако ни то, ни другое решение не смогло завладеть умами людей и разработчиков.

WebAssembly — это новая попытка разработать новый стандарт, который решит старые проблемы и будет принят всеми основными участниками. Объединились производители основных браузеров, поэтому есть надежда, что в этот раз она окажется успешной.

Что из себя представляет WebAssembly, главный репозиторий которого [находится здесь](#)? Это не совсем привычный нам байткод. WebAssembly имеет формат **AST** — абстрактного синтаксического дерева, которое может быть представлено как в бинарном виде — основном для распространения, так и в текстовом, соответствующим двоичному.

Вот пример WebAssembly в человекочитаемой форме.

```
(module
  (func $return_i32 (result i32)
    (unreachable))
  (func $return_f64 (result f64)
    (unreachable))

  (func $if (param i32) (result f32)
    (if_else (get_local 0) (unreachable) (f32.const 0)))

  (func $block
    (block (i32.const 1) (unreachable) (i32.const 2)))

  (func $return_i64 (result i64)
    (return (i64.const 1))
    (unreachable))

  (func $call (result f64)
    (call $return_i32)
    (unreachable))

  (export "return_i32" $return_i32)
  (export "return_f64" $return_f64)
  (export "if" $if)
  (export "block" $block)
  (export "return_i64" $return_i64)
  (export "call" $call)
)
```

Лисперам должно понравиться. Конечно, компилироваться `wasm` будет в бинарную форму.

План захвата мира

В первую очередь, WebAssembly рассчитан на web-платформу. Вот как определяется эта цель:

Проектируется, чтобы запускаться внутри и хорошо интегрироваться с существующей веб-платформой:

- сохранять безверсионную, протестированную и обратносовместимую историю Веб;
- запускаться в той же семантической вселенной, что и JavaScript;
- позволять синхронизированные вызовы из и в JavaScript;
- соблюдать такие же политики безопасности;
- иметь доступ к функциональности браузера через такие же Web API, которые доступны в JavaScript;
- определить человекоредатируемый форма, конвертируемый в и из двоичного для поддержки функциональности просмотра исходников.

Однако в долгосрочной перспективе под властью `wasm` должны оказаться и другие платформы: сервера в датацентрах, IoT-устройства, мобильные и десктопные приложения и т. д., и т. п.

Для программистов самое интересное — это то, для генерации приложений в WebAssembly можно будет использовать не JavaScript, а другие языки. Первым, для кого делается поддержка, является C++. Ветераны указатели и шаблонов снова впереди!

Эпилог

Будущее покажет, перерастёт ли это попытка в лавину. Если да – веб-разработка (прямая и обратная) стоит на пороге больших перемен и становится ближе к традиционной.

Програмирование на fasm под Win64 часть 1

"Ассемблер, отладчик, IDE"

Автор: qwe8013

Дата: 2016-01-15

Я начинаю цикл статей по ассемблеру fasm. Возможно у вас есть вопрос: “Зачем в 21 веке нужен ассемблер?”. Я бы ответил так: Конечно, знание ассемблера не обязательно, но оно способствует пониманию, во что превращается ваш код, как он работает, это позволяет почувствовать силу. Ну и в конце концов: Писать на ассемблере просто приятно (ну по крайней мере небольшие приложения).

Так что надеюсь, что мои статьи будут вам полезны.

Где взять fasm?

Собственно тут: <http://flatassembler.net/download.php>

На этой странице Томаш Грыштар(создатель fasm-a) выкладывает последнюю версию ассемблера. Там есть версии для DOS, Linux, Unix и Windows, нам нужна для Windows. В скачанном архиве находятся следующие компоненты:

- fasm.exe – собственно сам ассемблер
- fasmw.exe – IDE (среда разработки)
- fasm.pdf – документация
- папка source – исходники fasm-a (написан на самом себе)
- папка include – папка с заголовками, импортами, и т.д.
- папка examples – примеры программ на fasm-e

Содержимое fasm.pdf дублирует 2 раздела документации “flat assembler 1.71 Programmer’s Manual” и “Windows programming” отсюда: <http://flatassembler.net/docs.php>

IDE (среда разработки)

Перед тем, как писать программы нужно определиться, в чём их писать. Для fasm-a существуют разные IDE, например: fasmw.exe(находится в архиве с fasm-ом), RadAsm, WinAsm Studio, Fresh, ... Выбирайте, какая вам больше по вкусу. Сразу скажу, что IDE из поставки fasm-a обладает минимальным количеством фичей, так что я бы рекомендовал использовать альтернативную IDE. Я, например, использую RadAsm 3.0, его можно взять здесь: <https://fbedit.svn.sourceforge.net/svnroot/fbedit/RadASM30/Release/RadASM.zip> К статье приложен файл Fasm.ini, там выбрана чёрная тема, добавлены x64 регистры и добавлена подсветка для большего числа инструкций. Можете поставить его вместо Fasm.ini по умолчанию, только исправьте в нём пути к папке с fasm-ом в 6 и 7 строках.

Отладчик

Писать программы это — хорошо, но нужно находить и исправлять баги, для этого нужен отладчик. Существуют разные отладчики способные отлаживать 64-битный код например: WinDbg, fdbg, x64dbg. Я использую WinDbg.

Далее я скажу пару слов о WinDbg. Скачать WinDbg можно с сайта Microsoft. При первом открытии WinDbg производит ужасающее впечатление, но не пугайтесь, нужно просто правильно настроить workspace. Добавлять разные окна можно с помощью меню “View”: от пункта “Command”(то, что показано по умолчанию) до “Processes and Threads”. Чтобы менять цвета оформления в том же меню “View” щёлкните по пункту “Options...”. Кроме окна с командами, которое есть по умолчанию нам понадобятся следующие окна:

1. “Disassembly” – окно дизассемблера, чтобы видеть код.
2. “Registers” – регистры, о них в следующей статье.
3. “Memory” – для просмотра участков памяти.
4. И ещё один “Memory” – для стека, поставьте в нём поле “Virtual” в значение rsp, а поле “Display format” в “Pointer and symbol”, о стеке я расскажу позже.

В принципе можете не заморачиваться и взять мой workspace, приложенный к статье “ссылка”, в WinDbg “File”->“OpenWorkspace in File...”

Это — всё, что я хотел рассказать в первой части.

[Файлы к статье](#)

Поправка к первой части

Автор: qwe8013

Дата: 2016-01-15

Прошу прощения, забыл прикрепить файлы к статье:
<http://files.webfile.ru/766ad9aea4835a4127480f80c4e98a85>

Програмирование на `fast` под Win64 часть 2

"Системы счисления, память, регистры"

Автор: qwe8013

Дата: 2016-01-15

В данной статье мы коснёмся основ без которых программирование на ассемблере невозможно, я расскажу кое-что о системах счисления, коснусь организации памяти и рассмотрю пустую программу.

Системы счисления

Прежде всего я хотел бы остановиться на способах записи целых чисел. Начнём с примера: запись 32167 означает следующее:

$$32167 = 3 \cdot 10^4 + 2 \cdot 10^3 + 1 \cdot 10^2 + 6 \cdot 10^1 + 7 \cdot 10^0 \quad (1)$$

То есть любое число можно представить в виде суммы:

$$\sum_{k=0}^N a_k \cdot 10^k$$

Где a_k – цифра от 0 до 9, N – количество цифр в записи числа. Такая запись числа называется записью в десятичной системе счисления. В принципе, в вышеупомянутой сумме 10 можно заменить на любое другое натуральное число, это число будет называться основанием системы счисления. Например в восьмеричной системе счисления число представляется в виде суммы:

$$\sum_{k=0}^N a_k \cdot 8^k$$

В данном случае требуется не 10 цифр, а 8, от 0 до 7. Обычно, если требуется указать систему счисления, её основание записывается как нижний индекс справа от числа (основание записывается в десятичной системе счисления), например:

$$76647_{(8)} = 7 \cdot 8^4 + 6 \cdot 8^3 + 6 \cdot 8^2 + 4 \cdot 8^1 + 7 \cdot 8^0 \quad (2)$$

в свою очередь:

$$76647_{(8)} = 32167_{(10)}$$

то есть сумма (1) равна сумме (2).

В системе счисления с основанием n существует n цифр, например в десятичной – десять, в восьмеричной – восемь. Если же нужно использовать систему счисления с основанием большим десяти, то придётся добавлять новые цифры, например в шестнадцатеричной системе счисления 16 цифр: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F. Цифры A-F, равны 10-15 соответственно. Чтобы лучше понять рассмотрим несколько примеров:

\$\$

$$A_{(16)} = 10_{(10)}$$

$$F_{(16)} = 15_{(10)}$$

$$A3_{(16)} = 10 \cdot 16^1 + 3 \cdot 16^0 = 163_{(10)}$$

$$F2E_{(16)} = 15 \cdot 16^2 + 2 \cdot 16^1 + 14 \cdot 16^0 = 3886_{(10)}$$

\$\$

Подробнее почитать о системах счисления можно в [википедии](#)

Что касается fastm-a, то в нём числа можно записывать в 4 разных системах счисления: в двоичной, в восьмеричной, в десятичной и в шестнадцатеричной. Чтобы записать число в десятичной системе счисления можно либо не указывать систему счисления вообще, либо дописать к числу справа букву “d”, например: 32167 или 32167d. Чтобы записать в восьмеричной нужно дописать к числу справа букву “o”, например: 32167o равно числу 13431. Чтобы записать в двоичной нужно дописать справа букву “b”, например: 100110b равно числу 38. Записать число в шестнадцатеричной системе счисления можно несколькими способами: можно дописать слева последовательность “0x”, можно дописать слева “\$” и можно дописать справа букву “h”, но в последнем случае число должно начинаться с одной из десятичных цифр. Пример: 0xFE=\$FE=0FEh=254, запись FEh приведёт к ошибке.

Компьютер работает с числами в двоичной системе счисления. Один двоичный знак называется битом, соответственно число состоящее из n двоичных знаков называется n-битным (или n-разрядным). Например число 11010010b – 8-ми битное. Группа из 8 бит называется байтом(byte), из 2-х байт – словом(word), из 4-х байт – двойным словом(double word), из 8-и байт – четверённым словом(quad word), и т.д.

Организация памяти

Каждая программа (её код и данные) располагается в оперативной памяти. Оперативная память состоит из ячеек размером в 1 байт. Для того чтобы обращаться к этим ячейкам, каждой из них сопоставлено некоторое число, называемое адресом (в нашем случае — 64-битное) (см. рис. 1).



рис. 1

Адреса принято записывать в 16-ричной системе счисления. Множество адресов вместе с сопоставленными им ячейками памяти называется адресным пространством. В адресном пространстве не каждому адресу сопоставлена какая-то ячейка памяти. Существует физическое адресное пространство, в нём каждой ячейке оперативной памяти сопоставлен единственный адрес. В целях безопасности сделано так, что у каждого процесса адресное пространство своё, т.е. процессу доступна не вся оперативная память компьютера, а только некоторая её часть. Адресное пространство процесса называется виртуальным. Одной и той же ячейке памяти в разных адресных пространствах могут быть сопоставлены разные адреса.

У виртуальной памяти есть атрибуты доступа: запись, чтение, исполнение. Например, если память не доступна для записи, то попытка записать в неё приведёт к ошибке, если она не доступна для исполнения, то попытка исполнения кода, находящегося там приведёт к ошибке, и т.д.

Ещё хотелось бы добавить, что когда говорят “указатель на что-то”, то имеется ввиду виртуальный адрес, с которого начинается что-то.

Общий вид исходника

Начну с того, что коснусь структуры исполняемого файла.

Исполняемый файл Windows состоит из нескольких частей:

1. Заголовок – обязателен
2. Секции, их может быть любое количество

У каждой секции есть имя (состоит из 8 ANSI символов) и атрибуты, в них входят атрибуты доступа к памяти.

Теперь рассмотрим ~~HelloWorld~~ простейшую программу на fasm-e:

```
format PE64 console;формат файла, подсистема
entry start;точка входа

include 'win64w.inc';заголовки

section '.code' readable executable code;секция кода

proc start;процедура

    nop
    nop
    nop

    ret
endp ;конец процедуры

section '.data' readable writeable data; секция данных
dd ?;данные
```

Первой строчкой указывается формат файла(в нашем случае PE64 – это 64-битный исполняемый файл Windows PE-Portable Executable) и подсистема console(это значит что при запуске нашей программы у неё будет создана консоль).

Вторая строчка `entry start` указывает точку входа, это — место с которого начинает исполняться код нашей программы.

`include 'win64w.inc'` – подключение некоторых заголовочных файлов.

Слово `section` обозначает начало новой секции, после него идёт имя в кавычках, а затем атрибуты.

Секцию кода мы поместили как доступную для чтения и исполнения, но не для записи, атрибут “code” указывает на то, что в данной секции находится код, он не обязателен, без него всё будет нормально работать.

Секция данных доступна для чтения и записи, атрибут “data” указывает на то, что в данной секции находятся данные, он, опять же, не обязателен. В принципе можно создавать любое количество секций(даже с одинаковыми именами) и класть код и данные в любые из них, главное, чтобы то, что предназначено для исполнения находилось в исполняемой памяти, а то, что для записи находилось в памяти доступной для записи.

В секции данных находится загадочная строка `dd ?`, сейчас расскажу, что она означает.

Объявление данных

Для объявления данных существует ряд директив, изображённых в следующей таблице:

Размер (в байтах)	Объявление данных
1	db,file
2	dw,du
4	dd
6	dp,df
8	dq
10	dt

Чтобы объявить данные размером в байт нужно написать `db <значение>`, например для объявления байта со значением `0xAB` служит строка `db 0xAB`, остальные директивы предназначены для объявления данных большего размера. Если вместо значения после директивы стоит “?”, это означает, что вам безразлично, какое там будет значение. Следует так-же отметить, что директива `dt` не предполагает указания целых чисел, а предполагает числа с плавающей точкой. С помощью этих директив можно объявить несколько блоков данных подряд, например: `db 120,10,?` объявляет 3 байта подряд: сначала со значением 12(в восьмеричной системе счисления), далее 10, далее – безразлично. Строка `dd 32167,12` объявляет 2 двойных слова со значениями 32167 и 12 соответственно. Чтобы заполнить участок памяти одним значением используется слово “`dup`”, например строка `dd 12 dup (10)` объявляет 12 двойных слов со значением 10, строка `dd 5 dup (3,6)` объявляет 5 пар двойных слов со значениями 3 и 6 соответственно.

Что касается директив `db` и `du`, они в качестве значения могут принимать строки (в кавычках), например: строка `db 'Hello world!'`, объявляет строку в формате ANSI с нулём на конце, а `du 'Hello world!'`, объявляет такую-же строку только в формате юникод (UTF-16).

Директива `file` принимает в качестве значения имя файла (`file <имя файла>`), после компиляции в этом месте программы будет находиться содержимое файла `<имя файла>`. Чтобы в коде можно было обращаться к определённым данным нужно дать им какое-нибудь имя. Данные с именем мы будем называть переменными. Имя может состоять из латинских букв, цифр и знака подчёркивания (хотя буквы могут быть не только латинские, но и любые другие, но я настоятельно не рекомендую их использовать). Имя пишется перед директивой объявления данных, например так: `qwe dd 32167`, в данном случае мы объявили переменную с именем “`qwe`”, размером 4 байта, у которой начальное значение равно 32167.

Регистры

Перед тем, как подходить к писанию кода необходимо обсудить регистры. Всего есть 16 регистров общего назначения (вообще регистров больше, но общего назначения – 16). Имена 8-и из них: `rax,rcx,rdx,rbx,rsi,rdi`, все остальные называются `r8-r15`, нижние 32 бита регистров называются: `eax,ecx,edx,ebx,esp,ebp,esi,edi`, остальные `r8d-r15d`. Нижние 16 бит называются `ax,cx,dx,bx,sp,bp,si,di`, остальные `r8w-r15w`. Нижние 8 бит называются `al,cl,dl,bl,spl,bpl,sil,dil`, остальные `r8b-r15b`. Так же можно обращаться верхним 8-и битам нижних 16 бит первых 4-х регистров по именам: `ah,ch,dh,bh`. Так же есть регистр – указатель инструкций `rip`, и регистр флагов `rflags`. `rip` всегда указывает на следующую инструкцию (которая идёт после той, которая выполняется), регистры изображены на рис. 2

RAX		RCX		RDX		RBX		64 бита
EAX		ECX		EDX		EBX		32 бита
AX		CX		DX		BX		16 бит
AH AL		CH CL		DH DL		BH BL		по 8 бит

RSP		RBP		RSI		RDI		Rx	64 бита
ESP		EBP		ESI		EDI		RxD	32 бита
SP		BP		SI		DI		RxW	16 бит
SPL		BPL		SIL		DIL		RxB	8 бит

рис. 2

Собственно программа

Если вы скомпилируете программу приведённую выше, то обнаружите при запуске, что она ничего не делает, потому что она действительно ничего не делает! Выше я говорил про точку входа, она начиналась со строки `proc start`, и закончилась на `endp`, что представляют собой эти макросы (`proc` и `endp`), я пока говорить не буду, скажу лишь что объявив таким образом процедуру к ней можно обращаться из любого места программы. Внутри процедуры “start” трижды написана команда “`por`”, так вот:

`por` — команда, которая ничего не делает.

Далее я хотел бы ввести ещё одну команду:

`int3` (без пробела) — команда точки останова. Если программа находится под отладчиком, то как только исполнение доходит до этой инструкции программа останавливается, что будет видно в отладчике. Если же программа не находится под отладчиком, то выполнение этой инструкции немедленно приводит к ошибке.

Попробуйте добавить `int3` перед 3-мя `por`-ами в приведённой в статье программе, скомпилируйте и откройте отладчиком, вы увидите что-то вроде того, что изображено на рис. 3.

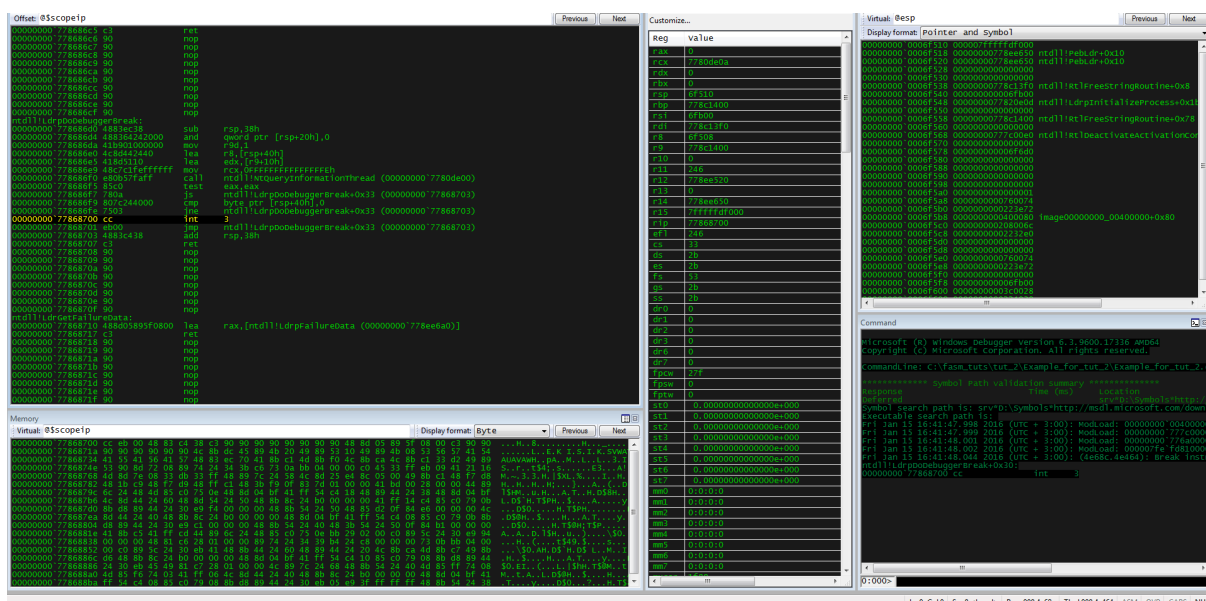


рис. 3

Это — точка останова внутри системного кода. Продолжите выполнение программы (в WinDbg — F5), после чего вы увидите в окне дизассемблера что-то вроде этого:

```
push rbp
mov rbp, rsp
int3
nop
nop
nop
leave
ret
```

“push rbp\mov rbp, rsp” и “leave\ret” — это пролог и эпилог процедуры соответственно, позже я объясню, что это и зачем.

Нажмите несколько раз “step over” (в WinDbg – F10) или “step into” (в WinDbg – F8) и вы увидите как отладчик переходит от одной инструкции к другой.

На этом – всё, в следующей статье я рассмотрю основные машинные инструкции и расскажу несколько подробнее про содержимое этих окон в WinDbg, если вы используете другой отладчик, то разница, в общем, небольшая.

Программирование на fasm под Win64 часть 3 "Окна WinDbg, метки, основные команды"

Автор: qwe8013

Дата: 2016-01-28

В данной статье я расскажу об окнах в WinDbg, и рассмотрю некоторые машинные инструкции.

Окна в WinDbg

Для начала в одном из окон "memory" в поле "Virtual" напишите "@rsp" (без кавычек), и там же в поле "Display format" поставьте значение "Pointer and Symbol", это — окно стека, что это я расскажу позже. В общем у меня окно WinDbg выглядит как на рис.1.

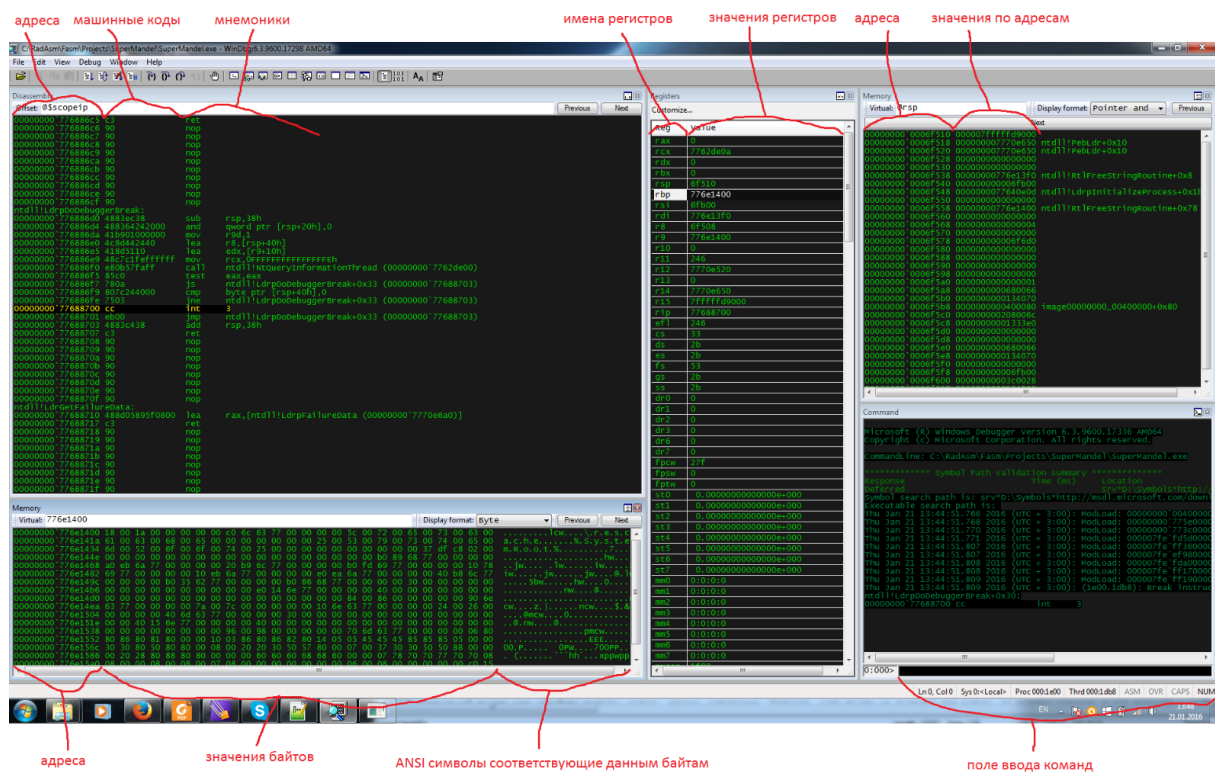


Рис. 1

В окне "disassembly" вы видите 3 колонки: адрес, машинный код и мнемоника. Адрес — это адрес начала инструкции. Обратите внимание: Адреса в WinDbg (как и в других отладчиках) записываются в шестнадцатеричной системе счисления. Далее идёт колонка в которой показан машинный код инструкции (тоже в шестнадцатеричной системе счисления). И последняя колонка — мнемоника, т.е. запись инструкции на языке ассемблера.

В окне "Registers" 2 колонки: левая — названия регистров, правая — их значения. Обратите внимание: значения регистров представлены либо как шестнадцатеричные целые, либо как десятичные числа с плавающей точкой (вещественные).

В окне "memory", где я сказал указать в поле "Virtual" значение "@rsp", есть 2 колонки: левая — адрес, правая — значение типа qword по данному адресу.

В оставшемся окне “memory” — 3 колонки: адрес, значения байтов (шестнадцатеричные) по данному адресу и символы ANSI, соответствующие данным значениям байтов (ANSI — как раз однобайтовая кодировка). Можете попереключать поле “display format” и посмотреть, разные форматы отображения данных. Поле “Virtual” показывает, с какого адреса начать отображение памяти, адрес может включать и регистры, числа в этом поле считаются шестнадцатеричными.

Метки

Надо было сказать в предыдущих статьях: точка с запятой — это знак комментария, всё что есть в строке после неё ассемблер проигнорирует, многострочных комментариев нет.

Метка — это символьное имя (идентификатор) позволяющий обращаться к данной области программы.

Чтобы поставить метку в коде нужно написать её имя и поставить двоеточие, например:

```
proc start

    nop
        label1:; <--Метка
        nop
    ret
endp
```

В прошлой статье я показывал, как можно объявлять переменные, например: `Variable dd ?` Так вот, имя переменной — тоже метка, оно отличается от метки поставленной с помощью двоеточия тем, что оно предполагает, что данные, на которые она указывает будут интерпретированы как данные соответствующего размера. Так же имя метки может начинаться с точки, такая метка называется локальной. При ассемблировании к имени локальной метки добавляется слева имя ближайшей сверху нелокальной метки, например:

```
proc start
;код 1
Lb1:
;код 2
.locLb1:
;код 3
ret
endp
```

В данном примере полное имя метки `.locLb1` — “`Lb1.locLb1`”, любая ссылка из кода 2 и 3 на метку по имени “`.locLb1`”, на самом деле ссылается на “`Lb1.locLb1`”. Если же попытаться сослаться на эту метку из кода 1 по имени “`.locLb1`”, `fasn` будет искать метку с полным именем “`start.locLb1`”, не найдёт её и сообщит об ошибке. Чтобы сослаться на метку “`.locLb1`” из кода 1 нужно писать полное имя метки: “`Lb1.locLb1`”.

Основные команды

Теперь можно приступать к программированию. В прошлой статье я говорил об инструкции `int3`, используйте её чтобы остановить отладчик в том месте, где вам нужно. Общий вид программы будет примерно такой:

```
format PE64 console ;формат - PE 64 бита файла, подсистема
entry start ;точка входа (место, с которого начинается выполнение кода программы)

include 'win64w.inc';заголовки
section '.code' readable executable code;секция кода
```

```

proc start ;начало процедуры

    int3
    ;<--Здесь будет располагаться ваш код.
    ret
endp ;конец процедуры

section '.data' readable writeable data; секция данных
dd ?;данные

```

Первая инструкция, которую я бы хотел обсудить называется `mov`. У этой инструкции два явных операнда (операнд — это то, что подаётся на вход инструкции, явный операнд — это операнд, который вы указываете в коде). В качестве операндов у этой инструкции может выступать константа, память или регистр. Записывается как `mov <op1>, <op2>`, эта команда копирует данные из <op2> в <op1>, сразу замечу, что <op1> не может быть константой (не только для “`mov`”) и вообще большинство (не все) инструкций с двумя явными операндами кладут результат в первый операнд, рассмотрим пример:

```
mov rax, 0x32167
```

данная инструкция записывает число `0x32167` в регистр `rax`, посмотрите в отладчике, как при исполнении этой инструкции значение в регистре `rax` меняется на `0x32167`. Можно вместо `rax` поставить любой регистр, главное, чтобы количество двоичных разрядов константы справа было меньше либо равно количеству разрядов в регистре, например конструкция `mov al, 0x1FF` приведёт к ошибке, т.к. в числе `0x1FF` больше восьми бит. Стоит также обратить внимание на то, что запись в регистр размера одного байта или слова не влияет на остальную часть регистра, но если вы пишете в регистр размером в 32 бита, то верхняя часть соответствующего 64-битного регистра обнуляется. Поэкспериментируйте, а потом двигайтесь дальше.

Так же у инструкции `mov` оба операнда могут быть регистрами, в этом случае значение копируется из одного регистра в другой, количество двоичных разрядов в обоих регистрах должно совпадать. Ну, и наконец одним из операндов этой инструкции может быть память, это надо рассмотреть поподробнее.

Следующие правила записи адреса верны для всех инструкций, которые принимают память в качестве одного из операндов.

Когда ссылаются на память, адрес заключают в квадратные скобки: `[]`. Адрес может выглядеть следующим образом:

1. `[reg1+reg2*2^n +offs32]`
2. `[rip+offs32]`
3. `[offs64]`

В первом случае `reg1` и `reg2` — регистры общего назначения, `n` может принимать значения от 0 до 3, `offs32` — 32-битное число со знаком. Во втором случае `offs32` — тоже 32-битное со знаком, прибавляется к регистру `rip`, замечу, что если вы сошлётесь на глобальную переменную в своём коде по имени, то `fasn` закодирует адрес именно таким способом. Третий случай есть не для всех инструкций и подразумевает ссылку на память по непосредственному адресу, так можно закодировать например инструкции:

```

mov al,[mem]

mov ax,[mem]

mov eax,[mem]

mov rax,[mem]

mov [mem],al

```

```
mov [mem],ax
mov [mem],eax
mov [mem],rax
```

Правда почему-то fasm отказывается ассемблировать эти инструкции с 3-м типом адреса, в документации они есть. И последнее на счёт адреса, у каждой инструкции может быть не более одного явного операнда, ссылающегося на память.

Рассмотрим простой пример:

```
format PE64 console
entry start

include 'win64a.inc'

section '.code' readable executable code

proc start

    int3
    mov eax,[Variable2];Запись значения Variable2 в регистр eax
    mov [Variable1],eax;запись значения регистра eax в переменную Variable1

    ret
endp

section '.data' readable writeable data
Variable1 dd ?
Variable2 dd 210h
```

В этом примере значение Variable2 копируется в Variable1. Чтобы посмотреть в отладчике значения Variable1 и Variable2 скопируйте адрес из инструкции mov в дизассемблере в поле “Virtual” окна “memory” и поставьте “display format” в “long hex”.

Далее я бы хотел рассмотреть пять инструкций общего назначения, которые так же имеют 2 явных операнда, все они записывают результат в <op1>:

add <op1>,<op2> — складывает <op1> и <op2>, если результат не вмещается в <op1>, тогда старшие биты отбрасываются

and <op1>,<op2> — применяет к каждому биту операцию логическое “и”

or <op1>,<op2> — применяет к каждому биту операцию логическое “или”

sub <op1>,<op2> — вычитает из <op1> <op2>, отрицательный результат записывается в [дополнительном коде](#)

xor <op1>,<op2> — применяет к каждому биту логическое “исключающее или”

Кроме них так же полезны инструкции:

xchg <op1>,<op2> — меняет местами значения <op1> и <op2>

lea reg,[memory] — считает адрес [memory] и записывает его значение в регистр reg, обратите внимание, что данная команда не выполняет реального обращения к памяти, а просто считает адрес

not <op1> — применяет логическое “не” к <op1>

Ну, и последняя инструкция о которой я расскажу в этой статье — это pop <op1> операндом этой инструкции может быть либо регистр, либо память, она ничего не делает и в случае если операнд — память обращения к данной памяти не происходит.

Попробуйте использовать эти инструкции и посмотрите в отладчике, к каким результатам приводит их исполнение.

На этом пока всё.

Програмирование на `asm` под Win64 часть 4 "флаги, прыжки, условия"

Автор: qwe8013

Дата: 2016-02-14

В данной статье мы рассмотрим некоторые биты регистра флагов, условные прыжки и ещё некоторые условные инструкции.

Регистр флагов

В предыдущих статьях я говорил о регистре флагов "rflags", он 64-х битный. Регистром флагов он называется, потому что содержит флаги. Флаг — это число длиной в 1 бит, если значение флага равно 1, то говорят, что этот флаг установлен, если — 0, то сброшен. Часть битов регистра флагов зарезервирована (т.е. не используется), часть битов относится к системным флагам, их мы рассматривать не будем. В регистр флагов можно непосредственно записывать значение (правда системные флаги нам менять не дадут), так же они устанавливаются или сбрасываются при выполнении некоторых инструкций.



рис. 1

Доступные нам флаги показаны на рис. 1, рассмотрим некоторые из них:

Carry Flag (флаг переноса)

При сложении 2-х целых чисел результат может не вписаться в операнд-приемник (операнд, в который происходит запись результата), причём не хватать будет 1-го бита, так вот, если требуется дополнительный бит для хранения результата, тогда этот флаг будет установлен, в противном случае — сброшен. Для вычитания: при вычитании из меньшего большего установлен (без знака), иначе — сброшен.

Используется в без знаковой арифметике.

Операции `and`, `or`, `xor` всегда сбрасывают этот флаг.

Zero Flag (флаг нуля) Установлен, если результат операции равен нулю, сброшен в противном случае. **Sign Flag (флаг знака)** Равен значению старшего значащего бита результата (знакового бита). Используется в арифметике со знаком. **Overflow Flag (флаг переполнения)**

Устанавливается, если старший (знаковый) бит результата отличается от значения этого бита для обоих исходных чисел, сбрасывается в противном случае.

Из приведённых в предыдущей статье инструкций на флаги влияют: add, and, or, sub, хог. Инструкции mov, xchg, lea не влияют на флаги. Я приведу ещё несколько инструкций влияющих на флаги:

inc <op1> — увеличивает op1 на единицу, операндом может быть как регистр, так и память.

dec <op1> — уменьшает на единицу op1, аналогично inc.

cmp <op1>, <op2> — сравнивает op1 и op2, по факту просто вычитает из op1 op2, выставляет флаги, а результат вычитания никуда не записывает.

test <op1>, <op2> — аналог and, но не записывает никуда результат операции.

У вас наверно возникает разумный вопрос: “А зачем эти флаги нужны?”, ответ таков: “Для выполнения условных инструкций.” Условные инструкции — это инструкции результат выполнения которых зависит от некоторых флагов в регистре флагов.

Прыжки

Первыми условными инструкциями которые мы здесь рассмотрим будут условные прыжки. Сначала рассмотрим безусловный прыжок: jmp <метка>, в коде выглядит примерно так:

```
;код
label::метка
;код
jmp label
;код
```

после выполнения данной инструкции продолжает исполняться не тот код, который идёт после jmp, а тот, который идёт после метки. Условные прыжки выполняются только если выполнено определённое условие, в противном случае выполняется код идущий сразу после инструкции прыжка. Записываются они следующим образом: j<условие> <метка> здесь я приведу некоторые из них:

jz или je — выполняются при ZF=1

jne или jnz — выполняются при ZF=0

ja — выполняется, если CF=0 и ZF=0

jb или jc — выполняются, если CF=1

jae или jnb — выполняются, если CF=0

jbe — если CF=1 или ZF=1

jl — SF<>OF

jnl — SF=OF

jle — ZF=1 или SF<>OF

jg — ZF=0 и SF=OF

Чтобы лучше понять их значения рассмотрим несколько примеров.

```
format PE64 console
entry start

include 'win64a.inc'

section '.code' readable executable code

proc start

    int3
    mov eax,3;помещаем в регистр eax число 3
    mov edx,4;в edx - 4
    cmp eax,edx;сравниваем
```

```
je .equal;прыгнуть, если равны
```

```
nop  
nop  
nop
```

```
.equal:
```

```
ret  
endp
```

В данном случае прыжок не будет выполнен, т.к. при вычитании 4 из 3 нуля не будет, а соответственно ZF=0, в общем по-этому инструкция и называется je, т.е. jump equal, прыгнуть, если равно. Если в этом примере вместо je поставить jb или jl, то прыжок будет осуществлён, потому что эти мнемоники означают прыжок, если меньше. jb отличается от jl только тем, что jb подразумевает сравнение без знаковых чисел, а jl — знаковых. В связи с этим ещё один пример:

```
format PE64 console  
entry start  
  
include 'win64a.inc'  
  
section '.code' readable executable code  
  
proc start  
  
    int3  
    mov eax,-3;помещаем в регистр eax число -3  
    mov edx,4;в edx 4  
    cmp eax,edx;сравниваем  
    jl .equal;прыгнуть, если меньше  
  
    nop  
    nop  
    nop  
  
    .equal:  
  
    ret  
endp
```

В данном случае прыжок будет выполнен, т.к. -3 меньше, чем 4, но если вы поставите там jb, то -3 надо интерпретировать как 0xFFFFFFFF, что больше 4-х, и поэтому прыжок не будет выполнен.

Для прыжков приведённых выше:

jz или je — прыгнуть если равно
jne или jnz — прыгнуть если не равно
ja — прыгнуть если больше (без знака)
jb или jc — прыгнуть если меньше (без знака)
jae или jnb — прыгнуть если больше либо равно (без знака)
jbe — прыгнуть если меньше либо равно (без знака)
jl — прыгнуть если меньше (со знаком)
jnl — прыгнуть если не меньше (со знаком)
jle — прыгнуть если меньше либо равно (со знаком)
jge — прыгнуть если больше (со знаком)

С помощью прыжков можно написать аналог оператора “if” в языках высокого уровня:

```
cmp eax,edx  
je .l1  
;код 1  
jmp .EndCond
```

```
.lbl1:
```

```
;код 2
```

```
.EndCond:
```

В данном примере если `eax=edx` выполнится код 2, если нет, то код 1.

Так же с помощью прыжков можно организовать цикл:

```
mov eax,10
.loop:
```

```
;код
```

```
dec eax
cmp eax,0
jne .loop
```

Здесь код будет выполняться пока `eax` не равно нулю, инструкция `dec eax` уменьшает `eax` на единицу, поэтому код в цикле выполнится 10 раз, замечу, что инструкцию `cmp eax,0` можно убрать т.к. инструкция `dec eax` выставит флаг нуля, когда `eax` станет равно нулю.

Для иллюстрации использования прыжков рассмотрим пример:

объявим в секции данных массив: `Array dd 12,32,1,0,17`

сейчас я приведу программу, которая ищет в этом массиве максимальное число (без знака) и оставляет результат в регистре `eax`, было бы не плохо, если бы вы сначала попытались написать программу самостоятельно, а потом смотрели дальше.

Программа:

```
format PE64 console
entry start

include 'win64a.inc'

section '.code' readable executable code

proc start

    int3
    mov ecx,5;количество итераций цикла - 5
    xor eax,eax;положим в eax ноль
    lea rdx,[Array];rdx - указатель (адрес) на массив
    .loop:

    cmp [rdx],eax;сравниваем очередное значение из массива со значением в eax
    jbe .NotGreater;если значение из массива меньше либо
    ;равно значению в eax делаем прыжок

    mov eax,[rdx];если всё-таки больше,
    ;записываем в eax большее значение

    .NotGreater:

    add rdx,4;увеличиваем rdx, чтобы он указывал на следующий элемент массива
    dec ecx
    jnz .loop;если не прошло 5-и итераций возвращаемся к началу цикла

    ;здесь в eax будет значение 32
    ret
```

```

endp

section '.data' readable writeable data
Array dd 12,32,1,0,17

```

Для того, чтобы использовать числа со знаком нужно вместо `jbe .NotGreater` поставить `jle .NotGreater` и вначале вместо нуля положить в `eax` -2147483648 (наименьший `dword` со знаком).

Ещё один пример, который я хотел рассмотреть, это — сортировка пузырьком, здесь я приведу пример программы сортирующей по возрастанию массив `Arr dd 0x45,0x65,0x85,0x12,0x432,0x65,0x21,0x09,0x860,0x52,0x46`, но было бы не плохо, если бы вы попытались написать программу самостоятельно, а потом смотрели бы мою реализацию. Алгоритм такой:

1. присваиваем переменной `sorted` значение истина (один)
2. для каждого `i` от нуля до (длины массива-2) делаем шаг 3
3. если `i`-ый элемент массива больше (`i+1`)-го, то меняем эти элементы местами и присваиваем переменной `sorted` значение ложь (ноль)
4. если `sorted`=ложь, то возвращаемся к пункту 1, если истина, то заканчиваем выполнение цикла.

```

format PE64 console
entry start

include 'win64a.inc'

section '.code' readable executable code

proc start

    int3

    .GreatLoop:

        mov al,1;al=истина
        mov ecx,10;в массиве 11 элементов, поэтому цикл должен повториться 10 раз
        lea rdx,[Arr];rdx=указатель(адрес) массива
        .loop:

            mov r8d,[rdx]
            cmp [rdx+4],r8d;сравниваем два соседних элемента
            jae .NotGreater;если следующий больше либо равен предыдущему,
                ;тогда ничего не делаем

            ;если меньше, тогда
            mov al,0;al=ложь
            mov r9d,[rdx+4];меняем местами соседние элементы
            mov [rdx],r9d
            mov [rdx+4],r8d

        .NotGreater:

            add rdx,4;rdx указывает на следующий элемент
            dec ecx
            jnz .loop

            test al,al;эквивалентно cmp al,0
            je .GreatLoop;если al=ложь, тогда продолжаем работу

        ret
    endp

section '.data' readable writeable data
Arr dd 0x45,0x65,0x85,0x12,0x432,0x65,0x21,0x09,0x860,0x52,0x46

```

На этом рассмотрение прыжков я хочу закончить.

Команды set<условие> и stov<условие>

Команда set<cond> принимает в качестве единственного операнда либо однобайтовый регистр, либо один байт в памяти. Если условие выполнено, она в этот байт записывает значение 1, если — нет, то 0. Например инструкция sete al запишет в регистр al значение 1, если установлен ZF, и 0, если не установлен.

Инструкция stov<cond> имеет 2 операнда, первый приемник — только 16, 32 или 64 разрядный регистр, второй либо регистр, либо память такого же размера. Команда записывает в первый операнд значение второго, если условие выполнено, в противном случае — ничего не делает. Например инструкция stovge eax,[variable] запишет значение [variable] в регистр eax, если флаг SF=OF, иначе не сделает ничего.

Попробуйте заменить некоторые прыжки в предыдущих двух примерах на только что рассмотренные 2 инструкции, я приведу алгоритм сортировки:

```
format PE64 console
entry start

include 'win64a.inc'

section '.code' readable executable code

proc start

    int3

    .GreatLoop:

        mov al,1;al=истина
        mov ecx,10;в массиве 11 элементов, поэтому цикл должен повториться 10 раз
        lea rdx,[Arr];rdx=указатель(адрес) массива
        .loop:

            mov r8d,[rdx]
            mov r9d,[rdx+4]
            cmp r9d,r8d;сравниваем два соседних элемента

            stovb r8d,[rdx+4];если следующий меньше предыдущего,
            stovb r9d,[rdx];тогда меняем местами значения

            setnb ah;обратите внимание, что здесь стоит не setnb al,
            and al,ah;а эта связка, это нужно для того чтобы если
            ;al=0, тогда оно не сможет принять значение 1

            mov [rdx],r8d
            mov [rdx+4],r9d

            add rdx,4;rdx указывает на следующий элемент
            dec ecx
            jnz .loop

            test al,al;эквивалентно cmp al,0
            je .GreatLoop;если al=ложь, тогда продолжаем работу

        ret
    endp
section '.data' readable writeable data
```

```
Arr dd 0x45,0x65,0x85,0x12,0x432,0x65,0x21,0x09,0x860,0x52,0x46
```

команды **sbb** и **adc**

Данные команды предназначены для того, чтобы работать с целыми числами больше 64 бит. Команда **adc** действует аналогично **add**, но помимо этого прибавляет к результату значение флага **CF**. **sbb** аналогична **sub**, но вычитает из результата значение **CF**. Например для вычитания 128 битных чисел можно написать:

```
mov rax,[Var1]
sub [Var2],rax
mov rax,[Var1+8]
sbb [Var2+8],rax
...
Var1 dq 1,2
Var2 dq 2,2
```

Для сложения вместо **sub** и **sbb** — **add** и **adc**.

На этом я хотел бы закончить статью.

Програмирование на `fasn` под Win64 часть 5 "стек, процедуры, соглашения о вызове"

Автор: qwe8013

Дата: 2016-04-03

В данной статье мы рассмотрим, что такое стек, процедуры, макросы для объявления процедур и как их вызывать.

Стек — область памяти, в него можно добавлять данные и извлекать их, добавление и извлечение организовано по правилу "последним вошёл, первым вышел".

Приведу пример: если добавить в стек подряд числа 1, 2, 3, то извлекаются они в порядке: 3, 2, 1. Для помещения в стек значения служит команда `push <op>`, в качестве операнда может выступать регистр общего назначения или память, причём размер должен быть либо 2 байта, либо 8 байт. Вытащить из стека значение можно командой `pop <op>`, в качестве операнда тут опять же может выступать либо регистр, либо память. Для примера напишем программу, которая перевернёт массив задом на перед, вообще-то для этого действия стек не нужен и более оптимально было-бы обойтись без стека, но в качестве демонстрации — подойдёт.

```
format PE64 console
entry start

include 'win64a.inc'

section '.code' readable executable code

proc start

    int3

    mov ecx,5;ecx - счётчик
    lea rax,[Arr];rax указывает на исходный массив

    .PushArr;;цикл из 5-и итераций

        push qword [rax];помещаем очередной элемент массива в стек

        add rax,8;передвигаем указатель на следующий элемент массива
        dec ecx;вычитаем единицу из счётчика
        jnz .PushArr;если не весь массив обработан, возвращаемся к следующей итерации

    mov ecx,5;ecx - счётчик
    lea rax,[ArrDst];rax указывает на конечный массив

    .PopArr;;цикл из 5-и итераций

        pop qword [rax];достаём из стека элемент и записываем его в конечный массив

        add rax,8;передвигаем указатель на следующий элемент массива
        dec ecx;вычитаем единицу из счётчика
        jnz .PopArr;если не весь массив обработан, возвращаемся к следующей итерации

    ret
endp

section '.data' readable writeable data
Arr dq 0x32167,0x456ab,0x76bd,0xa657f,0x213fe;исходный массив
ArrDst dq 5 dup (?);конечный массив
```


Как я говорил в одной из предыдущих статей, можно писать и читать регистр флагов, делается это инструкциями `pushf` и `popf`, без явных операндов.

`pushf` — добавляет значение регистра флагов в стек, `popf` достаёт значение со стека и записывает в регистр флагов, соответственно, писать в регистр флагов можно так:

```
push SomeValue
popf
```

Из написанного выше понятно, что можно класть значения в стек и вынимать их, а можно ли обратиться не к верхнему значению в стеке? Можно. На вершину стека указывает регистр `rsp`, когда вы добавляете значение в стек, значение этого регистра уменьшается, и туда, куда указывает `rsp` записывается значение, когда вы извлекаете значение из стека, извлекается значение, на которое указывает `rsp`, а потом значение `rsp` увеличивается. Соответственно, чтобы обратиться к произвольному значению из стека надо написать что-то в духе: `mov rax, [rsp+8]` — здесь в регистр `rax` записывается значение из стека.

Процедуры

Процедура (или функция) — это подпрограмма, к которой можно обращаться из разных мест вашей программы, она представляет собой такой же машинный код, как и остальная программа, в её начале ставится метка. Для того, чтобы передать управление процедуре используется команда `call <op>`, где в качестве операнда выступает либо адрес (метка), либо регистр (указывающий на метку), либо память (в которой располагается адрес метки).

Эта команда передаёт управление по указанному адресу как и `jmp`, но кроме этого она ещё кладёт в стек адрес возврата, т.е. адрес следующей за `call` команды.

Заканчивается процедура инструкцией `ret` или `ret imm16`, где `imm16` обозначает 2-х байтовое целое непосредственное значение (т.е. не регистр и не память, а значение закодированное в инструкции). Эта инструкция достаёт из стека адрес возврата и передаёт на него управление, `imm16` — это количество байт, которое будет вытащено из стека помимо адреса возврата, т.е. просто увеличится значение регистра `rsp`.

Кроме инструкции `ret` также есть инструкция `retf`, но мы не будем её рассматривать.

Также, как возможно вам известно из языков высокого уровня, у каждой процедуры есть локальные переменные, к которым можно обращаться только из этой процедуры. Локальные переменные создаются в стеке, т.е. значение регистра `rsp` уменьшается и в получившемся пространстве появляется свободное место для хранения переменных, перед выходом из процедуры (перед командой `ret`) регистр `rsp` должен быть возвращён в исходное состояние. Рассмотрим пример:

```
format PE64 console
entry start

include 'win64a.inc'

section '.code' readable executable code

Procedure:;начало процедуры
sub rsp,16;выделяем 16 байт для локальных переменных

mov [rsp],rax;записываем значение rax в локальную переменную
mov [rsp+8],rdx;rdx - в другую

mov rcx,[rsp];читаем значение одной из переменных в rcx
xor rcx,[rsp+8];ксорим rcx со значением другой локальной переменной

add rsp,16;возвращаем rsp на место
ret;выходим из процедуры

proc start
```

```

int3
call Procedure;вызываем процедуру

ret
endp

```

Сразу скажу, что обращаться к стеку можно только по адресам большим текущего значения `rsp`, так что не рекомендуется обращаться по адресам `[rsp-xx]`

Удобно для обращения к локальным переменным использовать второй другой регистр: `rbp`. Это будет выглядеть примерно так:

```

format PE64 console
entry start

include 'win64a.inc'

section '.code' readable executable code

Procedure;;начало процедуры
push rbp;сохраняем значение rbp
mov rbp,rsp;rbp указывает на стек
sub rsp,16;выделяем 16 байт для локальных переменных

mov [rbp-16],rax;записываем значение rax в локальную переменную
mov [rbp-8],rdx;rdx - в другую

mov rcx,[rbp-16];читаем значение одной из переменных в rcx
xor rcx,[rbp-8];ксорим rcx со значением другой локальной переменной

mov rsp,rbp;возвращаем rsp на место
pop rbp;восстанавливаем значение rbp
ret;выходим из процедуры

proc start

int3
call Procedure;вызываем процедуру

ret
endp

```

Вместо связки команд `mov rsp,rbp / pop rbp` можно использовать команду `leave`, без явных операндов.

Обращаться к переменным по адресам `[rbp-xx]` не очень удобно, хотелось бы обращаться к ним по именам. Для этого нам нужны несколько макросов для использования которых мы подключаем `win64a.inc`.

Макросы следующие:

```

proc
endp
ret
locals
endl

```

Макрос `proc` объявляет начало процедуры, имя после этого макроса будет именем метки начала процедуры. Например если вы напишете `proc SomeProc`, то обращаться к ней следует так: `call SomeProc`. Макрос `endp` объявляет конец процедуры, для каждого объявления процедуры обязательно должен присутствовать `endp`. `ret` означает возврат из процедуры, т.е. там, где в процедуре вы поставите `ret` `fasn` поставит что-то вроде `mov rsp,rbp / pop rbp / ret`. Обратите внимание, что `endp` и `ret` — разные вещи: первый указывает конец процедуры, а второй — только возврат, их может быть несколько, рассмотрим пример:

```

format PE64 console
entry start
include 'win64a.inc'

```

```

section '.code' readable executable code

proc SomeProc;Объявляем процедуру

    cmp [SomeData],1000;сравниваем SomeData с 1000
    ja .Lb11;если SomeData больше 1000 - прыгаем на Lb11

    add [SomeData],200;если SomeData меньше либо равно 100, то прибавляем к нему 200

    ret;возврат из процедуры

.Lb11:

    sub [SomeData],200;если SomeData больше 1000, то вычитаем из него 200

    ret;возврат из процедуры

endp;процедура закончена

proc start

    int3
    mov [SomeData],32167
    call SomeProc;вызываем процедуру

    nop

    ret
endp

section '.data' readable writeable
SomeData dd ?

```

В процедуре из этого примера два возврата, и после каждого исполнение процедуры заканчивается и управление передаётся в процедуру “start” на команду nop.

Что касается макросов locals и endl, то они нужны для объявления локальных переменных. Делается это следующим образом: в начале процедуры вызывается макрос locals, после него объявляются переменные по тем же правилам, что и глобальные переменные, и после них вызывается макрос endl, приведу пример:

```

format PE64 console
entry start

include 'win64a.inc'

section '.code' readable executable code

proc Procedure;начало процедуры
locals
Var1 dq ?
Var2 dq ?
endl

    mov [Var1],rax;записываем значение rax в локальную переменную
    mov [Var2],rdx;rdx - в другую

    mov rcx,[Var1];читаем значение одной из переменных в rcx
    xor rcx,[Var2];ксорим rcx со значением другой локальной переменной

    ret;возврат из процедуры
endp;процедура закончена

proc start

    int3
    call Procedure;вызываем процедуру

    ret
endp

```

К локальным переменным данной процедуры можно обращаться только между макросами `proc` и `endp`, соответственно в разных процедурах могут находиться локальные переменные с одинаковыми именами, и это не будет создавать проблем. Только помните: использовать указатели на локальные переменные можно только до возврата из процедуры, после него нельзя считать осмысленными значения на которые ссылаются эти указатели.

Соглашения о вызове

К этому моменту мы научились объявлять процедуры и их локальные переменные с помощью макросов, но помимо этого процедуры могут принимать параметры и возвращать значения. Для этого существуют соглашения о вызове. Соглашение о вызове это — способ передачи параметров в функцию и возврата из неё значений. В принципе, на ассемблере, вы можете изобрести какие угодно соглашения о вызове, и их использовать, но для того, чтобы вызывать процедуры из чужого кода нужно заранее договориться о способе передачи параметров и возврате значений, для этого существуют стандартные соглашения о вызове. Если погуглить, то можно выяснить, что в 32-битном режиме есть ряд соглашений о вызове: `stdcall`, `fastcall`, `cdecl`, и т.д. В 64-битном режиме же принято одно соглашение о вызове и заключается оно в следующем:

1. Первые 4 параметра передаются через регистры, если это — указатели или целые числа то передаются они в следующем порядке:

1-rcx, 2-rdx, 3-r8, 4-r9. Если это — числа с плавающей точкой, то они передаются через регистры `xmm0-xmm3` (о них я расскажу в последующих статьях).

При этом в стеке вызывающая функция выделяет место для хранения этих параметров.

2. Следующие параметры передаются через стек.

3. После выполнения процедуры стек очищает от параметров вызывающая функция.

4. Возвращаемое значение функция записывает в `rax`.

5. После вызова функции должны остаться неизменными регистры: `r12-r15`, `rdi`, `rsi`, `rbx`, `rbp`, остальные регистры общего назначения могут после вызова измениться.

Приведу пример вызова функции `SomeFunc` с 6 параметрами:

```
...
sub rsp,6*8;выделяем место в стеке для 6-и параметров

mov rcx,Val1;первый параметр
mov rdx,Val2;второй параметр
mov r8,Val3;третий параметр
mov r9,Val4;четвёртый параметр
mov [rsp+4*8],Val5;пятый - уже через стек
mov [rsp+5*8],Val6;шестой - тоже
call SomeFunc;вызываем функцию
;в регистре rax - возвращённое значение

add rsp,6*8;освобождаем стек от параметров
...
```

В принципе так мы больше писать не будем, потому что существуют макросы `fastcall` и `invoke`, отличаются они тем, что если в `fastcall` стоит инструкция `call Func`, то в `invoke` стоит `call [Func]`. Только что приведённый код можно переписать в следующем виде:

```
...
fastcall SomeFunc,Val1,Val2,Val3,Val4,Val5,Val6
...
```

Для того, чтобы внутри процедуры обращаться к параметрам по именам нужно перечислить их в макросе `proc` после имени процедуры, например:

```
...
proc SomeProc,Param1,Param2,Param3,Param4,Param5,Param6
...
```

К параметрам можно обращаться также, как к локальным переменным – по именам. ВАЖНО: В НАЧАЛЕ ИСПОЛНЕНИЯ ПРОЦЕДУРЫ В ПЕРВЫХ 4-Х ПАРАМЕТРАХ ЛЕЖИТ МУСОР, ОБРАЩАТЬСЯ К НИМ СЛЕДУЕТ ЧЕРЕЗ РЕГИСТРЫ! Приведу пример объявления процедуры с 2-мя параметрами, в следующем коде процедура “Reverse” переворачивает массив байтов заданной длины.

```
format PE64 console
entry start

include 'win64a.inc'

section '.code' readable executable code

proc Reverse,Arr,ArrLen

;rcx-Arr - указатель на массив
;rdx-ArrLen - длина массива
;обратите внимание, что к параметрам идёт обращение через регистры,
;т.к. по адресам Arr и ArrLen лежит мусор, но если нужно сохранить
;начения этих переменных можно записать их туда:
;mov [Arr],rcx
;mov [ArrLen],rdx

lea rax,[rcx+rdx-1];rax - указатель на массив

shr rdx,1;дулим rdx на 2
jz .exit; если в массиве меньше двух элементов - на выход

.loop:

mov r8b,[rcx];меняем местами значение по адресу [rcx]
xchg r8b,[rax]; и [rax]
mov [rcx],r8b

inc rcx;увеличиваем rcx
dec rax;уменьшаем rax
dec rdx;уменьшаем счётчик
jnz .loop

.exit:
ret
endp

proc start
locals
Arr1 db 3,2,1,6,7
Arr2 db 9,8,7,6,5,4,3,2,1,0
endl

int3
lea rcx,[Arr1]
fastcall Reverse,rcx,5;вызываем функцию для первого массива

lea rcx,[Arr2]
fastcall Reverse,rcx,10;вызываем функцию для второго массива

ret
endp
```

Скажу ещё пару слов о выделении памяти в стеке и передаче параметров.

```

00000000`0040106b cc          int     3
00000000`0040106c 488d4df0      lea     rcx, [rbp-10h]
00000000`00401070 4883ec20      sub     rsp, 20h
00000000`00401074 48c7c205000000 mov    rdx, 5
00000000`0040107b e880ffff      call   image00000000_00400000+0x1000 (00000000`00401000)
00000000`00401080 4883c420      add     rsp, 20h
00000000`00401084 488d4df5      lea     rcx, [rbp-08h]
00000000`00401088 4883ec20      sub     rsp, 20h
00000000`0040108c 48c7c20a000000 mov    rdx, 0Ah
00000000`00401093 e868ffff      call   image00000000_00400000+0x1000 (00000000`00401000)
00000000`00401098 4883c420      add     rsp, 20h
00000000`0040109c 4889ec        mov     rsp, rbp
00000000`0040109f 5d            pop     rbp
00000000`004010a0 c3            ret     c3

```

Рис. 1

Если вы взгляните на код процедуры start (см. Рис.1), то увидите, что для вызова процедур постоянно выделяется и освобождается память в стеке (выделено красным), чтобы это оптимизировать нужно вызывать макросы frame и endf в начале и в конце процедуры соответственно, при их использовании память выделяется в начале процедуры и освобождается в конце, пример:

```

proc start
  locals
  Arr1 db 3,2,1,6,7
  Arr2 db 9,8,7,6,5,4,3,2,1,0
endf
frame

  int3
  lea rcx, [Arr1]
  fastcall Reverse, rcx, 5; вызываем функцию для первого массива

  lea rcx, [Arr2]
  fastcall Reverse, rcx, 10; вызываем функцию для второго массива

endf
ret
endp

```

Из всего выше описанного очевидно, что использование регистра rbp не обязательно, и это можно исправить, достаточно в программе вставить следующие строки:

```

prologue@proc equ static_rsp_prologue
epilogue@proc equ static_rsp_epilogue
close@proc equ static_rsp_close

```

Они заставят использовать вместо rbp только rsp, и rbp тогда можно использовать в своих целях. ОБРАТИТЕ ВНИМАНИЕ:

1. ИСПОЛЬЗОВАТЬ ТОЛЬКО RSP БУДУТ ПРОЦЕДУРЫ СТОЯЩИЕ ПОСЛЕ ЭТИХ СТРОК
2. В ЭТОМ РЕЖИМЕ НЕ НУЖНО ИСПОЛЬЗОВАТЬ frame и endf

Чтобы Вернуть обратно rbp нужно вставить строки:

```

prologue@proc equ prologuedef
epilogue@proc equ epilloguedef
close@proc equ

```

На этом — всё.

Програмирование на `fasn` под Win64 часть 6 "DLL, API, импорт"

Автор: qwe8013

Дата: 2016-04-17

Так как на этом сайте нет функции прикрепления файлов к статье, но можно прикреплять изображения, я залию сюда [rarjpeg](#) с файлами к первой статье и к этой, поскольку заливать на сторонние хостинги не охота.

<http://wasm.ru/pictures/ed/e8/817895b8-05d7-4c81-9cd5-cb76997c1df6.jpg>



Начало

В данной статье я не буду рассказывать о машинных инструкциях, а расскажу немного об импорте и выводе на консоль, тут будет долгожданный HelloWorld.

Что такое DLL?

DLL – Dynamic Link Library (Динамически подключаемая библиотека). Она представляет собой исполняемый файл (PE). С точки зрения структуры заголовков исполняемого файла DLL почти не отличается от EXE. Впрочем, в данной статье я не буду описывать заголовки PE, а ограничусь лишь упоминанием таблицы импорта. Сразу скажу, что у DLL есть точка входа, в DLL хранят ресурсы и код, таким образом программа, загружая к себе в адресное пространство DLL, получает доступ ко всему что в ней хранится. Сейчас нам важен код, и далее мы поговорим о нем. Для того чтобы получить доступ к коду, нужно узнать, где именно в памяти хранится нужный нам код. Для этого в DLL существует таблица экспорта, она содержит информацию о соответствии между определёнными смещениями в DLL и их именами или номерами. Имена хранятся в формате ANSI. В принципе, по смещению заданному именем (или номером) может находиться как код функции, так и данные, но обычно там – код.

API

API – application programming interface (интерфейс программирования приложений). В API входят системные функции, которые вызывает программа при необходимости. Все эти функции хранятся в системных DLL-библиотеках и указаны там в таблицах экспорта. Сразу оговорюсь, что при вызове этих функций значение регистра `rsp` должно быть кратно 16, если вы вызываете функции не используя стандартные макросы это необходимо учитывать, со стандартными макросами об этом можно не беспокоиться, если конечно вы не “дёргаете” самостоятельно `rsp`.

Импорт

Импорт функции из библиотеки(DLL) подразумевает загрузку этой DLL в адресное пространство процесса с последующим использованием этой функции. Импорт бывает статическим и динамическим. Статический импорт подразумевает загрузку DLL и получение указателей на функции при старте процесса, и освобождение DLL при его завершении. Всё это делает операционная система. Динамический импорт заключается в том, что DLL вы можете загружать и выгружать когда угодно, и получать указатели на функции когда угодно.

Статический импорт

Статический импорт осуществляется с помощью таблицы импорта, в которой записаны имена библиотек и имена функций, импортируемых из них. Начинается таблица импорта в коде с 2-х слов: `data import`, а заканчивается `end data`. Внутри таблицы импорта нужно написать, какие библиотеки нам нужны, и какие функции мы будем импортировать. Делается это примерно так (слеш в конце строки означает, что эту и следующую строку надо интерпретировать как одну):

```
data import
library kernel32, 'kernel32.dll', user32, 'user32.dll'
import kernel32, \
    CreateFileA, 'CreateFileA', \
    CloseHandle, 'CloseHandle'

import user32, \
    MessageBoxA, 'MessageBoxA'
end data
```


В данном примере мы сначала объявляем, какие библиотеки нам нужны и каждому имени библиотеки ставим в соответствие идентификатор (идентификатор – это набор символов, с помощью которого можно обращаться к соответствующим объектам, например: имя переменной – идентификатор, имя метки – тоже, и т.д.): kernel32 – для kernel32.dll, user32 – для user32.dll. Потом с помощью макроса `import` мы перечисляем функции, которые хотим импортировать из данной библиотеки. Первый аргумент этого макроса - идентификатор из `library`, а дальше идут пары: имя функции для использования из кода, и имя функции в кавычках, как оно записано в таблице экспорта. Для примера рассмотрим программу, которая выводит на консоль долгожданную строку “Hello world!”:

```
format PE64 console; формат -- 64 бита, подсистема -- console
entry start;точка входа -- start

include 'win64a.inc';заголовки с нужными макросами

section '.code' readable executable code;секция кода

proc start;точка входа
local s;локальные переменные
temp dd ?;переменная, в неё будет записано количество символов при вводе и выводе
endl
frame

invoke GetStdHandle,STD_OUTPUT_HANDLE;получение описателя вывода
lea rdx,[Text]
lea r9,[temp]
invoke WriteConsoleA,rax,rdx,14,r9,0;вывод на консоль

invoke GetStdHandle,STD_INPUT_HANDLE;получение описателя ввода
lea rdx,[temp]
lea r9,[temp]
invoke ReadConsoleA,rax,rdx,1,r9,0;читаем с консоли до нажатия Enter

invoke ExitProcess,0;завершаем процесс
endf
ret;возврат из процедуры
endp

section '.data' readable writeable data;секция данных
data import;начало таблицы импорта
library kernel32,'kernel32.dll';импортируем только из kernel32.dll
import kernel32, \;функции:
    GetStdHandle,'GetStdHandle',\
    WriteConsoleA,'WriteConsoleA',\
    ReadConsoleA,'ReadConsoleA',\
    ExitProcess,'ExitProcess'
end data
Text db 'Hello world! ',13,10;текст для вывода
```

Сразу скажу, что без строки `include 'win64a.inc'` нельзя использовать макросы, которые тут применяются. Здесь мы объявляем одну локальную переменную `temp` для записи туда ненужных для нас данных. Дальше мы вызываем функцию `GetStdHandle` и первым (и единственным) параметром к ней идёт `STD_OUTPUT_HANDLE`. `STD_OUTPUT_HANDLE` – это константа. Константы объявляются так: имя константы=значение, в данном случае: `STD_OUTPUT_HANDLE = -11`, пока что я не буду на этом заострять внимание. Сама же функция `GetStdHandle` возвращает (в `rax`) хендлы ввода, вывода или ошибок. Хендл (или описатель) — это некоторое “магическое число” нужное для доступа к некоторым системным ресурсам. В этой программе мы один раз получаем хендл вывода, и в другой раз — хендл ввода, эти хендлы можно спользовать сколько угодно раз в программе. После получения описателя вывода мы сразу передаём его в функцию [WriteConsoleA](#). Она выводит на консоль текст и имеет параметры:

1. Хендл вывода
2. Указатель на строку в формате ANSI

3. Количество символов для вывода
4. Указатель на переменную типа dword, в которую будет записано число выведенных символов
5. должен быть 0

Заметьте, здесь я положил в `rdx` значение второго параметра, поэтому макрос `invoke` не добавит дополнительных обращений к `rdx`. Важно: если вы используете для передачи параметров в функцию регистры `rcx, rdx, r8, r9`, тогда вы должны передавать `rcx` как первый параметр, `rdx` – второй, `r8` – третий, `r9` – четвёртый. Никогда не делайте так:

```
lea rcx,[SomeData]
invoke SomeFunc,FirstParam,rcx,...
```

Это может привести к ошибкам.

Далее получаем хендл вывода и передаём в функцию [ReadConsoleA](#). Она читает введённую строку из консоли, у неё аналогичные параметры. Только первый параметр — хендл ввода, а второй — указатель на область памяти, в которую будут записаны прочитанные с консоли символы. Как вы могли заметить, я передал в качестве указателя на строку адрес той же переменной `temp` и передал в качестве длины входящей строки 1 символ. Я это сделал, т.к. ни количество введённых символов, ни введённая строка нас в данном случае не интересуют. В конце вызывается функция `ExitProcess`, она завершает процесс с кодом завершения, передаваемым в качестве параметра, поэтому вызов `ret` в конце процедуры не обязателен.

И тут нужно сказать про директиву `include...` Она... Как бы это... Ну, вон там вот... В общем, она вставляет в исходник содержимое файла, имя которого следует после неё. Например строка `include 'win64a.inc'` вставляет в исходник содержимое файла “win64a.inc”, он входит в стандартную поставку `fasn-a` в папке “INCLUDE”. Если вы его откроете, то увидите ещё кучу разных `include-ов`.

Вообще-то говоря, можно было бы использовать и юникодную версию функций, просто вместо `A` в конце имени функции нужно ставить `W`, и `Text` объявить не как `db`, а как `du`, да и у `ReadConsoleW` последний параметр не обязан быть нулём.

Очевидно, что постоянно прописывать все импортируемые функции не удобно, поэтому в стандартной поставке `fasn-a` есть файлы в `INCLUDE\API`, которые облегчат нам жизнь. Просто вместо макроса `import` вставляете `include 'api\kernel32.inc'`, аналогично и со всеми библиотеками для которых есть инклюды в этой папке:

```
data import;начало таблицы импорта
library kernel32,'kernel32.dll'
include 'api\kernel32.inc'
end data
```

Помимо макроса `import` в этих инклюдах используется макрос `api`. Этот макрос предназначен для функций, у которых есть 2 варианта: ANSI и Unicode. У ANSI функций в конце названия стоит буква `A`, у Юникод — `W`. Так вот, макрос `api` позволяет пользоваться функцией не указывая `A` или `W`. Работает это так: если в начале файла стоит `include 'win64a.inc'`, то все обращения к `ReadConsole` (например) будут заменены на `ReadConsoleA`, а если стоит `include 'win64w.inc'`, то `ReadConsoleW`. Пример той же программы, только – Юникод:

```
format PE64 console
entry start

include 'win64w.inc'

section '.code' readable executable code

proc start
    locals
        temp dd ?
    endl
    frame
```

```

invoke GetStdHandle,STD_OUTPUT_HANDLE
lea rdx,[Text]
lea r9,[temp]
invoke WriteConsole,rax,rdx,14,r9,0

invoke GetStdHandle,STD_INPUT_HANDLE
lea rdx,[temp]
lea r9,[temp]
invoke ReadConsole,rax,rdx,1,r9,0

invoke ExitProcess,0
endf
ret
endp

section '.data' readable writeable data
data import;начало таблицы импорта
library kernel32,'kernel32.dll'
include 'api\kernel32.inc'
end data
Text du 'Hello world! ',13,10

```

У меня есть программа для генерации import-инклюдников. Её вместе с моими инклудами я прикреплю к статье, правда после создания инклюдника иногда возникают 2 проблемы:

1. В разных инклудах совпадают функции, соответственно, где-то нужно их удалить или закомментировать.
2. Имена функций совпадают со словами, используемыми `fasn`-ом, надо менять имя функции !!! НЕ ИМЯ В КАВЫЧКАХ, А ИМЯ ИДЕНТИФИКАТОРА!!

Есть так же импорт по ординалам: это импорт не по имени функции, а по номеру, но он в документированном API не встречается... вроде бы.

Динамический импорт

Динамический импорт осуществляется в явном виде программой. Для начала нужно либо загрузить нужную библиотеку, либо получить указатель на уже загруженную. Чтобы получить указатель на уже загруженную библиотеку надо использовать функцию [GetModuleHandle](#) (A/W) с единственным параметром — указателем на имя библиотеки, она вернёт адрес, где расположена библиотека, либо же 0 в случае ошибки. В WinApi (не в Native API) в качестве строк используются строки с нулем на конце, т.е. последний символ должен быть нулевым. Чтобы загрузить библиотеку надо использовать функцию [LoadLibrary](#) (A/W), у которой тоже 1 параметр: имя библиотеки, эта функция возвращает либо указатель на загруженную библиотеку, либо 0 в случае ошибки. На каждый вызов `LoadLibrary` в вашем коде должен присутствовать вызов `FreeLibrary`, освобождающий память, в которой хранится библиотека. Ну и наконец, чтобы найти нужную функцию, используется функция `GetProcAddress` с 2-мя параметрами: первый — адрес загрузки библиотеки, второй — указатель на строку в формате ANSI с именем функции. Обратите внимание, что нет функций `GetProcAddress(A/W)`, есть только одна, и она принимает строку в ANSI. Пример:

```

format PE64 console
entry start

include 'win64a.inc'

section '.code' readable executable code

proc start
    locals
        temp dd ?
    endl
    frame
    lea rcx,[Msvcrt_name]

```

```

invoke LoadLibrary,rcx;загружаем библиотеку msvcrt.dll
mov [hMsvcrt],rax

lea rdx,[printf_name]
invoke GetProcAddress,[hMsvcrt],rdx;получаем адрес функции printf
mov [printf],rax

lea rcx,[K32_name]
invoke GetModuleHandle,rcx;получаем адрес kernel32.dll

lea rdx,[WriteConsole_name]
invoke GetProcAddress,rax,rdx;получаем адрес функции WriteConsole
mov [WriteConsole],rax

;Дальше всё то же самое, как и в предыдущем примере, за исключением вызова printf

invoke GetStdHandle,STD_OUTPUT_HANDLE
lea rdx,[Text]
lea r9,[temp]
invoke WriteConsole,rax,rdx,19,r9,0

lea rcx,[Text1]
invoke printf,rcx;вызов printf

invoke GetStdHandle,STD_INPUT_HANDLE
lea rdx,[temp]
lea r9,[temp]
invoke ReadConsole,rax,rdx,1,r9,0

invoke FreeLibrary,[hMsvcrt];освобождаем память

invoke ExitProcess,0
endf
ret
endp

section '.data' readable writeable data
data import;начало таблицы импорта
library kernel32,'kernel32.dll'
import kernel32,\
    GetModuleHandle,'GetModuleHandleA',\
    LoadLibrary,'LoadLibraryA',\
    FreeLibrary,'FreeLibrary',\
    GetProcAddress,'GetProcAddress',\
    GetStdHandle,'GetStdHandle',\
    ReadConsole,'ReadConsoleA',\
    ExitProcess,'ExitProcess'
end data
Text db 'WriteConsole used',13,10
Text1 db 'printf used',13,10,0

printf_name db 'printf',0
WriteConsole_name db 'WriteConsoleA',0
Msvcrt_name db 'msvcrt.dll',0
K32_name db 'kernel32.dll'

hMsvcrt dq ?;адрес загрузки msvcrt.dll
printf dq ?;адрес printf
WriteConsole dq ?;адрес WriteConsole

```

Скажу так-же, что если загрузить одну и ту же библиотеку n раз, тогда не будет n копий этой библиотеки, а будет одна, но освободится она только после n вызовов FreeLibrary.

На этом, как говорится, “полномочия этой статьи — всё”.

Програмирование на `fasn` под Win64 часть 7 "структуры, `bmp`, `FPU`"

Автор: qwe8013

Дата: 2016-07-16

В общем после долгого перерыва, связанного с моими проблемами с учёбой я продолжу.

Я хотел посвятить данную статью полностью `FPU`, но просто делать вычисления на мой взгляд довольно скучно, поэтому здесь будет рассмотрен формат `bmp`, функции записи в файл и частично `FPU`.

Прикреплённые файлы

Так как до сих пор невозможно прикреплять к статьям файлы (или я не понял как это делать), то вы полюбуетесь, как доктор Феррагут с командой отважно борются с опасным вирусом. [rarjpeg](#)

Иллюстрация из исходной публикации недоступна.

Структуры

Мы уже рассмотрели, как объявлять простые типы данных (`db`, `dw`, etc.), но иногда нужно использовать в качестве

единицы данных что-то более сложное. Например, может быть необходимо хранить массив (некоторое количество

поряд идущих данных одинакового типа) цветов, цвет можно задать тройкой чисел, поэтому каждому цвету

должна соответствовать структура из 3-х чисел. Вообще-то в `fasn`-е структура объявляется директивой `struc`,

но мы будем пользоваться макросом `struct`, т.к. он несколько удобнее. И так, объявление типа структура

начинается со слова `struct`, после него идёт имя типа, завершается определением словом `ends`.
Внутри структуры

поля объявляются как обычные переменные, пример:

```
struct MY_STRUCT  
  
    Field1 dd ?  
    Field2 dq ?  
    Field3 POINT  
  
ends
```

В данном примере объявлена структура с именем "`MY_STRUCT`" со следующими полями: `Field1` – `dword`(4 байта),

`Field2` – `qword`(8 байт) и `Field3` – `POINT`. `POINT` – тоже структура, она объявлена в стандартных заголовках `fasn`-а.

переменные типа структуры объявляются так-же как и переменные простых типов: `MyVariable MY_STRUCT` – это

объявление переменной с именем “MyVariable” и с типом “MY_STRUCT”. Чтобы обратиться к определённому полю

структуры нужно написать имя переменной, точка, имя поля, для наглядности приведём пример:

```
struct MY_STRUCT

    Field1 dd ?
    Field2 dq ?
    Field3 POINT

ends

proc start
    frame

    mov eax,[MyVariable.Field1];Чтение значения из первого поля переменной в регистр eax
    mov [MyVariable.Field2],rdx;Запись значения регистра rdx во второе поле переменной
    mov ecx,[MyVariable.Field3.x];Чтение из третьего поля переменной, оно - структура, поэтому
        обращаемся
    ;к её полю

    endf
    ret
endp

MyVariable MY_STRUCT
```

Значение переменных-структур можно заранее инициализировать, например, если написать так:

MyVariable MY_STRUCT 1,2,<36h,39h>, то будет объявлена переменная со значениями:

MyVariable.Field1=1,

MyVariable.Field2=2, MyVariable.Field3.x=36h, MyVariable.Field3.y=39h.Получить размер структуры

можно используя слово sizeof.<Имя структуры>, например следующая конструкция записывает в регистр eax

размер структуры MY_STRUCT: mov eax,sizeof.MY_STRUCT. И последнее, что я хотел бы рассказать

про структуры – это как обращаться к структуре, имея указатель на неё в регистре: надо просто прибавить

имя структуры, точка, имя поля, например mov rax,[rdx+MY_STRUCT.Field2], в данном примере в регистр rax

записывается второе поле определённой выше структуры, на экземпляр которой указывает регистр rdx.

Пара слов о записи в файл

Для того, чтобы работать с файлом (как и с любым объектом в системе) нужно сначала получить его описатель.

Описатель (или хендл, англ. handle) – это некоторое “магическое” число, нужное для доступа к разным системным ресурсам, в данном

случае – к файлу. Файлу, как и любому другому системному объекту, может соответствовать несколько хендлов,

которые могут находиться в разных процессах. Для открытия или создания файла предназначена функция

[CreateFile](#), её

прототип на си следующий:

```
HANDLE WINAPI CreateFile(
    _In_      LPCTSTR          lpFileName,
    _In_      DWORD            dwDesiredAccess,
```

```

_In_      DWORD          dwShareMode,
_In_opt_ LPSECURITY_ATTRIBUTES lpSecurityAttributes,
_In_      DWORD          dwCreationDisposition,
_In_      DWORD          dwFlagsAndAttributes,
_In_opt_  HANDLE          hTemplateFile
);

```

Параметры:

1. lpFileName – имя файла
2. dwDesiredAccess – флаги доступа к файлу (см. ниже)
3. dwShareMode – флаги общего доступа к файлу (см. ниже)
4. lpSecurityAttributes – дескриптор безопасности, пока мы не будем использовать его
5. dwCreationDisposition – условия создания файла (см. ниже)
6. dwFlagsAndAttributes – атрибуты файла
7. hTemplateFile – мы не будем его использовать

dwDesiredAccess

Флаги, которые указывают права доступа к объекту через данный описатель
Обычно используются 2 флага:

1. GENERIC_READ – доступ на чтение
2. GENERIC_WRITE – доступ на запись

Флаги – это по сути отдельные биты, т.е. в байте 8 бит, и соответственно байт может содержать 8 флагов, каждый флаг может быть либо установлен (значение 1), либо сброшен (значение 0).

dwShareMode

Какой доступ к файлу могут запросить все остальные (флаги).
Флаги:

1. FILE_SHARE_READ – могут открывать на чтение
2. FILE_SHARE_WRITE – могут открывать на запись
3. FILE_SHARE_DELETE – могут помечать на удаление

dwCreationDisposition

Возможные значения (не флаги, только одно значение):

1. CREATE_ALWAYS – если файл существует, то будет перезаписан, если – нет, то создан
2. CREATE_NEW – создать, если файл ещё не существует, если существует, то вызов завершится ошибкой
3. OPEN_ALWAYS – если файл существует, то он будет открыт, если – не существует, то будет создан
4. OPEN_EXISTING – если файл существует, то открыть, если – нет, то ошибка
5. TRUNCATE_EXISTING – если файл существует, то перезаписать, если – нет, то ошибка

Если функция завершила успешно, то она возвращает описатель файла, если – ошибкой, то возвращает
INVALID_HANDLE_VALUE(-1).

Когда работа с файлом закончена нужно закрыть дескриптор, передав его в качестве единственного параметра функции CloseHandle.

Для записи в файл используется функция [WriteFile](#)

Её прототип на си:

```
BOOL WINAPI WriteFile(  
    _In_      HANDLE      hFile,  
    _In_      LPCVOID     lpBuffer,  
    _In_      DWORD       nNumberOfBytesToWrite,  
    _Out_opt_ LPDWORD     lpNumberOfBytesWritten,  
    _Inout_opt_ LPOVERLAPPED lpOverlapped  
);
```

Параметры:

1. hFile – дескриптор файла
2. lpBuffer – указатель на область памяти, которую нужно записать в файл
3. nNumberOfBytesToWrite – количество байт, записанных в файл
4. lpNumberOfBytesWritten – указатель на переменную dword (4 байта), в которую будет записано количество записанных байт в файл
5. lpOverlapped – нужно для асинхронной записи, пока это мы использовать не будем

Так же существует понятие как файловый указатель – это смещение в файле, начиная с которого будут

записаны/прочтены данные при записи/чтении файла. При записи/чтении файла этот указатель сдвигается на

размер записанных/прочтённых данных. Для того, чтобы его менять есть функция [SetFilePointer](#)

Здесь мы её использовать не будем.

Формат bmp-файлов

В начале файла идёт структура под названием BITMAPFILEHEADER:

```
struct BITMAPFILEHEADER  
    bfType      dw ?  
    bfSize      dd ?  
    bfReserved1 dw ?  
    bfReserved2 dw ?  
    bfOffBits   dd ?  
ends
```

Это первая структура bmp-файла, значения полей:

1. bfType – содержит сигнатуру 'BM', обозначающую, что перед нами bmp-файл
2. bfSize – размер файла в байтах
3. bfReserved1 – зарезервировано
4. bfReserved2 – зарезервировано
5. bfOffBits – смещение данных непосредственно о цвете пикселей относительно начала файла

Сразу после BITMAPFILEHEADER идёт структура BITMAPINFOHEADER, на самом деле существует несколько версий данной структуры, мы будем использовать одну из них.

```
struct BITMAPINFOHEADER  
    biSize      dd ?  
    biWidth     dd ?  
    biHeight    dd ?  
    biPlanes    dw ?  
    biBitCount  dw ?
```



```

    biCompression    dd ?
    biSizeImage      dd ?
    biXPelsPerMeter  dd ?
    biYPelsPerMeter  dd ?
    biClrUsed        dd ?
    biClrImportant   dd ?
ends

```

Значения полей следующие:

1. biSize – размер данной структуры в байтах (определяет её версию)
2. biWidth – ширина изображения в точках
3. biHeight – высота изображения в точках
4. biPlanes – должно быть 1
5. biBitCount – количество бит на точку, в нашем случае 24
6. biCompression – в нашем случае – BI_RGB

Остальные поля для нас не важны и их можно заполнить нулями.

Далее идут данные о цвете. Цвет представляется в виде смеси 3-х компонент: синей, зелёной и красной.

Цвет каждой точки кодируется 3-мя байтами (24 бита), первый — синяя компонента цвета, второй — зелёная, а третий — красная.

Соответствующая структура:

```

struct RGBTRIPLE

    rgbtBlue db ?;синий
    rgbtGreen db ?;зелёный
    rgbtRed db ?;красный

ends

```

Цвета точек хранятся горизонтальными линиями (строками): сначала нижняя линия слева направо, потом на точку повыше, и т.д. Причём размер линии в байтах должен быть кратен 4, соответственно в конце каждой линии может оказаться несколько ненужных байт.

В качестве примера к статье в прикреплённом архиве находится папка “Grad”, в ней программа создающая рядом с собой файл “test.bmp”, в котором цвет синий сверху плавно перетекает в чёрный снизу.

Иллюстрация из исходной публикации недоступна.

В этой программе есть такие строки:

```

LINE_LENGTH equ 1023;длина линии в пикселях
LINE_LEN_BYTES equ (((LINE_LENGTH*3)+3) and (not 3));длина линии в байтах (с учётом выравнивания)
NUM_OF_LINES equ 1023;количество линий

```

Слово “equ” здесь используется для объявления констант, везде ниже этих строк все вхождения того, что слева от “equ” будут заменяться на то, что справа от “equ”, например вхождение LINE_LENGTH будет заменино на 1023, в этой статье я не буду останавливаться на этом.

Числа с плавающей точкой

Существуют разные способы хранения вещественных чисел в памяти компьютера, в том числе — с плавающей точкой.

На формате этих чисел я останавливаться не буду, об этом можно прочитать [здесь](#)

Единственно, скажу что процессор понимает числа с плавающей точкой 3-х размеров:

1. 4 байта – одинарная точность
2. 8 байт – двойная точность
3. 10 байт – расширенная точность

Объявить их можно следующим образом:

```
dd 0.5; размер - 4 байта  
dq 2.3E-5; размер - 8 байт  
dt 5f; размер - 10 байт
```

С первым примером и так всё ясно, запись во втором означает

$2.3 \cdot 10^{-5}$

т.е. перед “E” ставится любое число с плавающей точкой, а после – целая степень десяти, на которую надо умножить это число. В последнем примере буква “f” в конце означает, что это – не целое число, а число с плавающей точкой, так-же можно написать так: `dt 5.0`. Т.к. число с плавающей точкой может принимать только конечное число значений, то результаты операций с ними не точны, это стоит учитывать.

FPU

FPU – Floating Point Unit. В давние времена процессоры не умели работать с числами с плавающей точкой,

для этого был создан сопроцессор (FPU), который с ними работать умел, с тех пор инструкции, которые он

выполнял называются инструкциями FPU, ещё этот набор инструкций называется набором x87. Сейчас, конечно,

всё находится в одном процессоре. Многие могут сказать, что инструкции x87 устарели и гораздо удобнее и

быстрее работать с SSE, но раз эти инструкции не выкинули из 64-х битов, то я про них расскажу.

У FPU есть 8 регистров общего назначения `fpr0-fpr7`, регистр состояния (status word) – аналог регистра флагов, регистр контроля (control word) и tag word.

Обращение к регистрам общего назначения осуществляется через имена `st(0)-st(7)`, в `fasm-е` они называются: `st0-st7`, работа с этими регистрами осуществляется

по принципу стека, т.е. когда вы записываете что-нибудь в регистр, значения в регистрах смещаются (`st6->st7`, `st5->st6`, `st4->st5`, `st3->st4`, `st2->st3`, `st1->st2`,

`st0->st1`), после чего ваше значение записывается в `st0`. Соответственно, когда вы читаете значение, вы получаете значение из `st0`, а затем значения регистров

смещаются в обратную сторону (на самом деле при этих смещениях значения на перебрасываются из регистра в регистр, просто регистры переименовываются, но это для нас не существенно).

В данной статье я не буду касаться регистров status word, control word и tag word.

Инструкции FPU

Рассмотрим команды, добавляющие число в стек FPU:

1. `fld1` – добавляет единицу
2. `fldl2e` – добавляет двоичный логарифм от `e`
3. `fldl2t` – добавляет двоичный логарифм десяти

4. `fldlg2` – добавляет десятичный логарифм двух
5. `fldln2` – добавляет натуральный логарифм двух
6. `fldpi` – добавляет число пи

7. `fldz` – добавляет ноль

8. `fild <op>` – единственный операнд – память, откуда загружается целое число (со знаком) и конвертируется в число с плавающей точкой, размер операнда может быть 16, 32 или 64 бита

9. `fld <op>` – единственный операнд – либо память, либо один из регистров `st0-st7`, загружает число с плавающей точкой 32, 64, 80 бит

Команды, вынимающие значение из стека:

10. `fst <op>` – копирует значение с вершины стека (оставляя его там) в `<op>`, `<op>` — `st`-регистр либо память (только 32 и 64 бита)

11. `fstp <op>` – вынимает значение со стека в `<op>`, сдвигает значения в стеке, `<op>` — `st`-регистр либо память (тут можно так же и 80 бит)

12. `fist <op>` – аналогично `fst <op>`, только округляют значение до целого (как – зависит от значения в FPU control word), `<op>` — только память 16 или 32 бита

13. `fistp <op>` – аналогично `fstp <op>`, только округляют значение до целого (как – зависит от значения в FPU control word), `<op>` — память 16, 32 или 64 бита

14. `fisttp <op>` – то же, что и `fistp <op>`, только округляет к нулю

Команды сложения:

15. `fadd st0,sti` – прибавляет к регистру `st0` значение регистра `sti` (то есть `i`-го регистра) например `fadd st0,st2`

16. `fadd sti,st0` – прибавляет к регистру `sti` значение регистра `st0`

17. `fadd <mem>` – прибавляет к регистру `st0` число с плавающей точкой из памяти (4 или 8 байт)

18. `faddp sti,st0` – прибавляет к регистру `sti` значение регистра `st0`, а затем выталкивает из этого стека верхнее значение (`st0`)

19. `fiadd <mem>` – прибавляет к регистру `st0` целое со знаком из памяти (2 или 4 байта)

Для инструкций вычитания всё аналогично, просто вместо “add” нужно поставить “sub” (ну и вычитается из первого операнда второй),

так же есть инструкции вычитания `fsubr`, они похожи на `fsub`, но если результат операции `fsub` – это “a-b”, то здесь будет “b-a”.

Инструкции умножения `fmul` так же аналогичны `fadd`, а `fdiv` и `fdivr` (деление) аналогичны вычитанию.

Инструкция `fsqrt` заменяет значение `st0` на квадратный корень от `st0`.

Инструкция `fxch sti` меняет местами значения регистров `st0` и `sti`.

Инструкция `fabs` заменяет значение `st0` его модулем.

И ещё одна из важнейших инструкций FPU – это `fnop`, она ничего не делает.

Попробуйте поиграться с этими инструкциями и посмотреть в отладчике, что получится. А когда закончите рассмотрите ещё 2 примера Circle и Lemniscate.

В них появляется функция `TransformCoordForw` она делает преобразование координат из координат картинки в рабочие координаты. Рабочими я назвал координаты, в которых удобнее производить вычисления, в этих примерах начало рабочих координат находится в центре, и обе координаты принимают значения от -1 до 1.

Преобразование идёт по следующей формуле:

$$X = \frac{X' \cdot Width - CenterX}{Width}$$

где X – рабочая координата, X' – координата картинки, $Width'$ – ширина картинки в точках, $Width$ – ширина картинки в рабочих координатах,

$CenterX$ – смещение центра по оси X относительно левого нижнего угла в рабочих координатах. Для Y – аналогично.

В комментариях к исходникам вместо X' написано $_X$.

В примере Lemniscate есть ещё одна функция LemniscateFunction, дело в том, что лемнискату Бернулли можно задать следующим образом:

$$F(x,y)=0$$

так вот, LemniscateFunction – это и есть функция F.

Иллюстрация из исходной публикации недоступна.

Иллюстрация из исходной публикации недоступна.

Кстати, здесь у лемнискаты края обрезаны, попробуйте это исправить!

[Вот](#) статья про лемнискату, там есть ссылки на другие кривые. На этом пока всё.

Как уменьшить размер программы в Windows x64

Автор: Miki___

Дата: 2016-07-19

В качестве «подопытного кролика» возьмем простейшее приложение выводящее на экран фразу "Win64 Assembly is Great!" написанное на диалекте MASM

```
OPTION DOTNAME
option casemap:none
include c:\masm64\include\temphls.inc
include c:\masm64\include\win64.inc
include c:\masm64\include\kernel32.inc
includelib c:\masm64\lib\kernel32.lib
include c:\masm64\include\user32.inc
includelib c:\masm64\lib\user32.lib
OPTION PROLOGUE:none
OPTION EPILOGUE:none
.data
MsgCaption      db 'Win64 Iczelion's lesson #2: MessageBox',0
MsgBoxText      db 'Win64 Assembly is Great!',0
.code
WinMain proc
    sub rsp,28h
    invoke MessageBox, NULL, &MsgBoxText, &MsgCaption, MB_OK
    invoke ExitProcess,NULL
WinMain endp
end
```

Собирается bat-файлом со следующим содержанием

```
cls
очистить экран

set masm64_path=c:\masm64\
здесь можно указать какой-нибудь другой путь к пакету masm64

set filename=tut_02a
здесь можно указать другое название asm-файла

%masm64_path%bin\ml64 /c %filename%.asm
%masm64_path%bin\link /SUBSYSTEM:WINDOWS
^

/entry:WinMain %filename%.obj
del %filename%.obj
удалить программный "мусор"
```

Обратите внимание на символ выделенный красным цветом. Одним из условий удобочитаемости текста программы является его ширина: все строки должны уместиться на экране — это примерная ширина текста в 80 символов. А при написании bat-файла основное требование — запись команды в одну строку. Использование символа «^» позволяет устранить это противоречие. Символ «^» экранирует конец строки — это позволяет разместить содержимое длинной строки на экране в нескольких строках, а система, как и прежде, будет считать их одной длинной строкой.

В результате компиляции мы получили tut_02a.exe с размером 2560 байт. При просмотре внутренностей exe-файла программой hiew32 видно, что 90% содержимого нули. Возникает законный вопрос, а как уменьшить размер программы, но чтобы при этом не терялась ее функциональность?

Шаг первый — уменьшение размера выравнивания сегментов У программы link.exe есть ключ **/ALIGN:размер** Ключ **/ALIGN[MENT]** указывает компоновщику на необходимость выравнивания сегментов в исполняемом файле на границу, кратную значению **размер**. **Размер** — это число

равное двум возведенное в целую степень (20=1, 21=2, 22=4, 23=8, ... до 216=65536 включительно). Если об этом ключе не упоминать, то выравнивание равно 29=512 байт. Посмотреть значение выравнивания можно при помощи редактора бинарных файлов hiew32 в заголовке программы в поле «File alignment» (Открываем наш exe программой hiew32, нажимаем F4 (Mode) и выбираем Hex, нажимаем F8 (Header) в поле «File alignment» видим число 200 hex=512 dec).

Кроме ключа /ALIGN добавим в файл makeit.bat еще некоторые ключи компиляции и линковки, которые позволят нам уменьшить количество служебной информации в asm-файлах.

- Добавление ключа компиляции /Cp — «сохранить регистр символов во всех идентификаторах» позволяет не писать каждый раз в тексте ассемблерного файла «option casemap:none».
- Добавление ключа компиляции /I:путь — «установить путь для включаемых файлов» позволяет нам не указывать каждый раз полный путь к inc-файлам.
- Ключ /LIBPATH:путь говорит линкеру, где находятся библиотеки импорта, что позволяет нам не указывать каждый раз полный путь к lib-файлам.

Так выглядит содержимое файла makeit.bat до изменений:

```
cls
set masm64_path=c:\masm64\
set filename=tut_02a
%masm64_path%bin\ml64 /c %filename%.asm
%masm64_path%bin\link /SUBSYSTEM:WINDOWS ^
/entry:WinMain %filename%.obj
del %filename%.obj
```

а так после:

```
cls
set masm64_path=c:\masm64\
set filename=tut_02a
%masm64_path%bin\ml64
/Cp
/c
/I"%masm64_path%Include"
%filename%.asm
%masm64_path%bin\link /SUBSYSTEM:WINDOWS
/LIBPATH:"%masm64_path%Lib"
^

/ALIGN:256
/entry:WinMain %filename%.obj
del %filename%.obj
```

Содержимое файла tut_02a.asm после изменений

```
OPTION DOTNAME
include temphls.inc
include win64.inc
include kernel32.inc
includelib kernel32.lib
include user32.inc
includelib user32.lib
OPTION PROLOGUE:none
OPTION EPILOGUE:none
.data
MsgCaption      db 'Win64 Iczelion''s lesson #2: MessageBox',0
MsgBoxText      db 'Win64 Assembly is Great!',0
.code
WinMain proc
    sub rsp,28h
    invoke MessageBox, NULL, &MsgBoxText, &MsgCaption, MB_OK
    invoke ExitProcess,NULL
WinMain endp
end
```

при компиляции получаем сообщение

LINK : warning LNK4108: /ALIGN specified without /DRIVER; image may not run

размер файла tut_02a.exe меняется с 2560 на 1536 байт и, не смотря, на предупреждение **image may not run** файл tut_02a.exe благополучно запускается

Продолжаем эксперимент, изменяя значение /ALIGN:размер, и наблюдаем за уменьшением размера файла tut_02a.exe:

/ALIGN:размер	Заголовок	Код	Данные	Импорт	Общий размер tut_02a.exe
Ключ не указан	1024	512	512	512	2560 байт
размер = 256	768	256	256	256	1536 байт
размер = 128	640	128	128	256	1152 байт
размер = 64	576	64	64	192	896 байт
размер = 32	576	64	64	192	896 байт
размер = 16	576	48	64	192	880 байт
размер = 8	—	—	—	—	Ошибка при создании файла

похоже, что в этом направлении мы достигли предела и от исходной программы в 2560 байт пришли к программе размером в 880 байт, но согласитесь, что и это не так уж плохо!

Шаг второй — объединяем сегмент кода и сегмент данных

Наша программа использует два сегмента, сегмент кода и сегмент данных, посмотрите внимательно через hiew32 — между этими сегментами прослойка из нулей, от которых мы и пытаемся избавиться. А помните во времена DOS'a можно было создавать COM-файлы, которые в единственном сегменте содержал и код, и стек, и данные? А нельзя ли и здесь создать, что-то подобное?

Оказывается можно — при помощи ключа **/SECTION** можно присвоить секции кода с атрибутами «выполнять» и «читать» атрибуты секции данных «читать» и «писать».

Вносим изменения в текст программы

```
OPTION DOTNAME
include temphls.inc
include win64.inc
include kernel32.inc
includelib kernel32.lib
include user32.inc
includelib user32.lib
OPTION PROLOGUE:none
OPTION EPILOGUE:none
.code
MsgCaption      db 'Win64 Iczelion's lesson #2: MessageBox',0
MsgBoxText      db 'Win64 Assembly is Great!',0
WinMain proc
    sub rsp,28h
    invoke MessageBox, NULL, &MsgBoxText, &MsgCaption, MB_OK
    invoke ExitProcess,NULL
WinMain endp
end

cls
set masm64_path=\masm64\
set filename=tut_02a
%masm64_path%bin\ml64 /Cp /c /I"%masm64_path%Include" %filename%.asm
%masm64_path%bin\link /SUBSYSTEM:WINDOWS /LIBPATH:"%masm64_path%Lib" ^

/SECTION:.text,w
/ALIGN:16 /entry:WinMain %filename%.obj
del %filename%.obj
```

Опции линкера /SECTION

Опция командной строки компоновщика **link.exe /SECTION:name,[!]{DEKPRSW}[,ALIGN=#]** позволяет принудительно назначать атрибуты секциям PE-файла. Для секции можно задать один или несколько атрибутов. Следует задавать все атрибуты, которые должна иметь секция; если какой-либо знак атрибута не указан, то его бит будет отключен. Если не указан атрибут **R**, **W** или **E**, то существующее состояние чтения, записи или исполнения остается неизменным. Чтобы инвертировать атрибут, перед его символом указывают знак «!». С помощью параметра **ALIGN=#** можно задать значение выравнивания для конкретной секции. Значения знаков атрибутов приведены в следующей таблице.

Буква	Атрибут	Значение	Перевод
D	Discardable	Marks the section as discardable	Секция помечается как выгружаемая
E	Execute	The section is executable	Секция является выполняемой
K	Cacheable	Marks the section as not cacheable	Секция помечается как некешируемая
P	Pageable	Marks the section as not pageable	Секция помечается как секция без страничной организации
R	Read	Allows read operations on data	Допускаются операции чтения данных
S	Shared	Shares the section among all processes that load the image	Секция совместно используется всеми процессами, загружающими образ
W	Write	Allows write operations on data	Допускаются операции записи данных

В данном случае секции с именем «.text» (содержащей код программы) и уже имеющей атрибуты R (доступна для чтения) и E (исполнимая), устанавливается атрибут W (доступна для записи)

```
/SECTION:.text,W
```

В результате получаем tut_02a.exe размером 848 байт.

Заголовок	Код и данные	Импорт	Общий размер
544	112	192	848 байт

Шаг третий — алгоритмическая оптимизация

При старте нашего приложения в x64dbg видно, что на вершине стека лежит адрес функции *kernel32.RtlExitUserThread*, а именно ее вызывает функция *ExitProcess* для завершения приложения, поэтому возникает вопрос, а необходим ли нам в *этой программе* вызов функции *ExitProcess*? Заменяем вызов *ExitProcess* на команду *retn*, которая извлечет из стека адрес функции *RtlExitUserThread* и передаст на нее управление.

```
OPTION DOTNAME
include temphls.inc
include win64.inc
include user32.inc
```



```

includelib user32.lib
OPTION PROLOGUE:none
OPTION EPILOGUE:none
.code
MsgCaption      db 'Win64 Iczelion''s lesson #2: MessageBox',0
MsgBoxText      db 'Win64 Assembly is Great!','0
;-----
WinMain proc
    sub rsp,28h
    invoke MessageBox, NULL, &MsgBoxText, &MsgCaption, MB_OK
    add rsp,28h
    retn
WinMain endp
end

```

Заголовок	Код и данные	Импорт	Общий размер
544	112	112	768 байт

Шаг четвертый — «хирургический» — ампутируем у файла «хвост».

hiew32 показывает, что в хвосте нашего файла, сразу за строкой «user32.dll» целых шестнадцать байт содержащих нули.

```

.400002D0: 00 00 00 00-00 00 00 00-E2 01 4D 65-73 73 61 67      7@Messag
.400002E0: 65 42 6F 78-41 00 75 73-65 72 33 32-2E 64 6C 6C      eBoxA user32.dll
.400002F0:
00 00 00 00-00 00 00 00-00 00 00 00-00 00 00 00

```

А не удалить ли их нам вручную? Сказано — сделано! В far'е нажимаем F4 (открываем файл на редактирование), нажмем Ctrl+End и переходим в конец файла. Далее, нажимая на Backspace, удаляем лишнее, жмем на F2 и сохраняем изменения в файле, нажимаем на F10 и выходим из файла — размер tut_02a.exe стал 748 байт. Нажимаем на него и запускаем tut_02a.exe — запускается нормально! Стоп-стоп! 768-748=20 байт, а должно быть 16! Смотрим в hiew32 — о, боже!, мы отрезали вместе с нулями от «user32.dll» кусок «.dll», но файл-то всё равно работает! Запомним на будущее, что от DLL достаточно только названия, а точку и расширение файла система подставит сама.

Заголовок	Код и данные	Импорт	Общий размер
544	112	92	748 байт

Шаг пятый — очередная алгоритмическая оптимизация

Как устроен вызов функции в Win64? Сначала формируется стековый кадр (*stack frame*), то есть, область памяти в стеке, через которую передаются аргументы и другие "опции", затем устанавливаются аргументы и вызывается инструкция *call*.

Win64 ABI требует, чтобы стековый кадр был выровнен на 16 байт. Это ключевой момент, от которого следует отталкиваться во всех последующих рассуждениях.

При входе в любую функцию значение регистра RSP не кратно 16, оно смещено на 8 байт адресом возврата, который "положила" в стек инструкция *call*. Если поставить точку останова на любую 64-битную функцию, то при входе в нее можно увидеть, что значение RSP всегда заканчивается на 8. Вот поэтому пролог функции в Win64 делает что-нибудь в стиле "sub rsp, 28h" — это и есть выравнивание стека на 16. Именно так, как пишут авторы многочисленных книг, то есть, до вызова других функций и до формирования стековых кадров для них.

Но это еще не все. Все тот же Win64 ABI требует, чтобы на время вызова в стеке было зарезервировано 32 байта под нужды вызываемой функции. Эта область в разных источниках называется по-разному: "register space", "home area" и так далее. Предполагалось, что в этой памяти вызываемая функция сможет временно хранить значения регистров RCX, RDX, R8 и R9. Но

нужен ли "home area" для функции MessageBox?

Меняем *sub rsp,28h* на *sub rsp,18h* и *add rsp,28h* на *add rsp,18h* (соблюдаем условие кратности 16 содержимого RSP), компилируем, линкуем, запускаем на выполнение. Программа работает и нормально завершается.

Меняем *sub rsp,18h* на *sub rsp,8* и *add rsp,18h* на *add rsp,8* компилируем, линкуем, запускаем на выполнение. Программа работает и нормально завершается.

Эквивалент *sub rsp,8* — команда *push reg*, какой из регистров при этом мы будем использовать не важно, главное, что размер команды всего один байт! Эквивалент *add rsp,8* — команда *pop reg*. Компилируем, линкуем, запускаем на выполнение. Программа работает и нормально завершается.

Заголовок	Код и данные	Импорт	Общий размер
544	96	112	752 байт

Шаг шестой — уменьшаем DOS-stub.

Код нашей программы начинается с 220h=544-го байта. Всё, что выше — это заголовок нашего EXE-файла — вот бы его уменьшить! Заголовок нашего файла состоит из двух частей. От строки «MZ» до строки «PE» заголовок DOS (0C8h=200 байт) и от строки «PE» до 220h=544-го байта PE-заголовок (344 байт). Адрес строки «PE» содержится в DOS-заголовке в двойном слове по смещению 3Ch.

Начнем с уменьшения DOS-стаба. Возьмем hiew32.exe и создадим с его помощью вот такой файл

```
.000000:  4D 5A 00 00 00 00 00 00 00 00 00 00 00 00 00 00 MZ
.000010:  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
.000020:  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
.000030:  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
```

Присвоим ему имя stubby.exe, сохраним в папке bin и будем использовать в качестве stub-программы. В очередной раз внесем изменения в файл makeit.bat.

```
cls
set masm64_path=\masm64\
set filename=tut_02a
%masm64_path%bin\ml64 /Cp /c /I"%masm64_path%Include" %filename%.asm
%masm64_path%bin\link /SUBSYSTEM:WINDOWS /LIBPATH:"%masm64_path%Lib" ^

/STUB:%masm64_path%bin\stubby.exe
/SECTION:.text,W /ALIGN:16 /entry:WinMain %filename%.obj
del %filename%.obj
```

Заголовок	Код и данные	Импорт	Общий размер
480	96	112	688 байт

Шаг седьмой — используем ключи /LARGEADDRESSAWARE, /BASE:адрес и 32-разрядную адресацию

Приложение, работающее в Win64 может работать с адресами размер которых свыше 2 гигабайта. В 64-разрядных компиляторах для этого используется параметр **/LARGEADDRESSAWARE** В 32-разрядных компиляторах включен параметр **/LARGEADDRESSAWARE:NO** Параметр **/BASE** задает базовый адрес программы, переопределяя заданное по умолчанию расположение EXE-файла (для 32-разрядных Win = 0x400000, для 64-разрядных 0x140000000) или DLL-файла (для 32-разрядных = 0x10000000). Сначала операционная система пытается загрузить программу по указанному или используемому по умолчанию базовому адресу. Если по этому адресу недостаточно места, то система перемещает программу в памяти. Для предотвращения перемещения используется параметр **/FIXED**.

Если указанный адрес не кратен 64 Кб, компоновщик выдает ошибку. При необходимости можно также указать размер программы, чтобы компоновщик мог выдать предупреждение в случае, если программа не помещается в заданные таким образом пределы.

```
.0000000140000220: 55          push rbp
.0000000140000221: 33C9       xor ecx,ecx
.0000000140000223: 488D153C000000 lea rdx,[00000001400001E0] ;'Win64 Assembly is Gret!'
.000000014000022A: 4C8D050E000000 lea r8,[00000001400001F9] ;'Win64 Iczelion's lesson #2:
    MessageBox'
.0000000140000231: 4883C428   xor r9d,r9d
.0000000140000234: FF1553000000 call MessageBoxA
.000000014000023A: 5D         pop rbp
.000000014000023B: C3         retn
```

Вносим изменения в файл makeit.bat

```
cls
set masm64_path=\masm62\
set filename=tut_02a
if exist %filename%.exe del %filename%.exe
if exist %filename%.obj del %filename%.obj
%masm64_path%bin\ml64 /Cp /c /I"%masm64_path%Include" %filename%.asm
%masm64_path%bin\link /SUBSYSTEM:WINDOWS /LIBPATH:"%masm64_path%Lib" ^

/LARGEADDRESSAWARE:NO /BASE:0x400000
/STUB:%masm64_path%bin\stubby.exe ^
/SECTION:.text,w /ALIGN:16 /entry:WinMain %filename%.obj
```

Вносим изменения в asm-файл

```
OPTION DOTNAME
include temphls.inc
include win64.inc
include user32.inc
includelib user32.lib
OPTION PROLOGUE:none
OPTION EPILOGUE:none
.code
MsgBoxText      db 'Win64 Assembly is Great!',0
MsgCaption      db 'Win64 Iczelion's lesson #2: MessageBox',0
WinMain proc
    push rbp
    mov edx,offset MsgBoxText
    mov r8d,offset MsgCaption
    invoke MessageBox,NULL,,MB_OK
    pop rbp
    retn
WinMain endp
end

.00400220: 55          push rbp
.00400221: BAE0014000  mov edx,0004001E0 ;'Win64 Assembly is Gret!'
.00400226: 41B8F9014000 mov r8d,0004001F9 ;'Win64 Iczelion's lesson #2: MessageBox'
.0040022C: 33C9       xor ecx,ecx
.0040022E: 4883C428   xor r9d,r9d
.00400231: FF1509000000 call MessageBoxA
.00400237: 5D         pop rbp
.00400238: C3         retn
```

Заголовок	Код и данные	Импорт	Общий размер
480	96	112	688 байт

Вносим изменения в asm-файл

```
OPTION DOTNAME
include temphls.inc
include win64.inc
include user32.inc
includelib user32.lib
OPTION PROLOGUE:none
```

```

OPTION EPILOGUE:none
.code
MsgBoxText      db 'Win64 Assembly is Great!',0
MsgCaption      db 'Win64 Iczelion''s lesson #2: MessageBox',0
WinMain proc
    push rbp
    mov edx,offset MsgBoxText
    lea r8d,[rdx+sizeof MsgBoxText];r8=offset MsgCaption
    invoke MessageBox,NULL,,MB_OK
    pop rbp
    retn
WinMain endp
end

.00400220: 55                push rbp
.00400221: BAE0014000        mov edx,0004001E0 ;'Win64 Assembly is Gret!'
.00400226: 448D4219          lea r8d,[rdx+019] ;'Win64 Iczelion's lesson #2: MessageBox'
.0040022A: 33C9              xor ecx,ecx
.0040022C: 4883C428          xor r9d,r9d
.0040022F: FF1509000000      call MessageBoxA
.00400235: 5D                pop rbp
.00400236: C3                retn

```

Заголовок	Код и данные	Импорт	Общий размер
480	96	112	688 байт

Шаг восьмой — сократим размеры заголовка и импорта

Так как код программы и данные нам сократить не удастся — поэтому сократим размеры заголовка и импорта PE-файла. Нам потребуется создать bin-файл, заголовок которого мы напишем самостоятельно. Для этого требуются ассемблеры, которые могут создавать бинарные файлы (NASM, FASM). За основу бинарного файла возьмем содержимое предыдущего tut_02a.exe размером 688 байта

```

format binary as 'exe'
include 'win64a.inc'
struc dbs [data]
{
    common
    . db data
    .size = $ - .
}

IMAGE_DOS_SIGNATURE      equ 5A4Dh
IMAGE_NT_SIGNATURE       equ 00004550h
PROCESSOR_AMD_X8664      equ 8664h
IMAGE_SCN_CNT_CODE       equ 00000020h
IMAGE_SCN_MEM_WRITE      equ 80000000h
IMAGE_SCN_MEM_READ       equ 40000000h
IMAGE_SCN_CNT_INITIALIZED_DATA equ 00000040h
IMAGE_SUBSYSTEM_WINDOWS_GUI equ 2
IMAGE_NT_OPTIONAL_HDR64_MAGIC equ 20Bh
IMAGE_FILE_RELOCS_STRIPPED equ 1
IMAGE_FILE_EXECUTABLE_IMAGE equ 2
IMAGE_BASE               equ 0x400000
align1                   equ 0x10
use64
org 0

;-----DOS-stub-----

Signature      dw IMAGE_DOS_SIGNATURE
times 0x3A db 0
NewExe_offset  dd ntHeader
times 0x48 db 0

;-----PE-заголовок-----
ntHeader      dd IMAGE_NT_SIGNATURE

```

```

; 'PE'
; image_header----Файловый заголовок

Machine          dw PROCESSOR_AMD_X8664
; Тип центрального процессора

Count_of_section  dw 2
; Количество секций

TimeStump        dd 0
; Информация о времени, когда был собран данный PE-файл

Symbol_table_offset dd 0
; Указатель на размер отладочной информации

Symbol_table_count dd 0
; Указатель на COFF-таблицу символов PE-формата

Size_of_optional_header dw section_table-optional_header
; Размер опционального заголовка

Characteristics    dw IMAGE_FILE_RELOCS_STRIPPED or \
IMAGE_FILE_EXECUTABLE_IMAGE
; Атрибуты файла
; -----Стандартные поля NT

optional_header:
Magic_optional_header dw IMAGE_NT_OPTIONAL_HDR64_MAGIC
; Состояние отображаемого файла

Linker_version_major_and_minor dw 9
; Содержат версию линковщика, создавшего данный файл

Size_of_code       dd Import_Table-begin
; Суммарный размер секций кода

Size_of_init_data   dd 0x70
; Суммарный размер инициализированных данных

Size_of_uninit_data dd 0
; Суммарный размер неинициализированных данных

entry_point        dd start
base_of_code        dd begin

; -----Дополнительные поля NT-----

image_base         dq IMAGE_BASE
section_alignment   dd align1
file_alignment      dd align1
OS_version_major_minor dw 5,2
image_version_major_minor dd 0
subsystem_version_major_minor dw 5,2
Win32_version       dd 0
size_of_image       dd end_import
size_of_header       dd begin
checksum            dd 0
subsystem           dw IMAGE_SUBSYSTEM_WINDOWS_GUI
DLL_flag            dw 8000h
; IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE

Stack_allocation    dq 0x100000
Stack_commit        dq 0x1000
Heap_allocation      dq 0x100000
Heap_commit         dq 0x1000
loader_flag         dd 0
number_of_dirs       dd 16
export_RVA_size      dq 0
import_RVA_size      dd _import,user32_table2-_import
; _import,0x3C;end_import-import
resource            dd 0,0

```

```

exception      dd 0,0
security       dd 0,0
fixups_        dd 0,0
debug          dd 0,0
description    dd 0,0
MIPS_GP        dd 0,0
TLS            dd 0,0
Load_config    dd 0,0
Bound_import   dd 0,0
import_table1  dd Import_Table,_import-Import_Table
;0x10

delay_import   dd 0,0
com_runtime    dd 0,0
reserved       dd 0,0

;-----

section_table   dq '.text'
.virtual_size   dd a0-begin
;0x57

.virtual_address dd begin
.Physical_size  dd Import_Table-begin
.Physical_offset dd begin
.Relocations_and_Linenumbers dq 0
.Relocations_and_Linenumbers_count dd 0
.Attributes     dd IMAGE_SCN_MEM_WRITE or IMAGE_SCN_CNT_CODE
;0x80000020
;-----

section_table1  dq '.rdata'
.virtual_size   dd a1-Import_Table
;0x62

.virtual_address dd Import_Table
.Physical_size   dd end_import-Import_Table
.Physical_offset dd Import_Table
.Relocations_and_Linenumbers dq 0
.Relocations_and_Linenumbers_count dd 0
.Attributes      dd IMAGE_SCN_MEM_READ or IMAGE_SCN_CNT_INITIALIZED_DATA
;0x40000040
;-----данные и код-----

begin:
MsgBoxText db 'Win64 Assembly is Great!',0
MsgCaption db "Win64 Iczelion's lesson #2: MessageBox",0
start:
    push rbp
    xor ecx,ecx
    mov edx,MsgBoxText+IMAGE_BASE
    lea r8d,[rdx+MsgBoxText.size]
    xor r9d,r9d
    call [MessageBox]
    pop rbp
    retn
a0:
    times (($ - begin) mod align1) db 0

;выравнивание сегмента кода и данных до значения равного align1
;-----секция импорта-----

Import_Table:
user32_table:
MessageBox dq _MessageBox,0
_import:
dd user32_table2,0,0,user32_dll,user32_table
dd 0,0,0,0,0
user32_table2:
dq _MessageBox,0
_MessageBox db 0xE2,1,"MessageBoxA",0

```

```

user32_dll db "user32.dll",0
a1:
times (($ - Import_Table) mod align1) db 0
end_import:

```

Заголовок	Код и данные	Импорт	Общий размер
480	96	112	688 байт

сократим DOS-stub до 64 байт, уменьшив таблицу разделов, по умолчанию количество разделов должно быть равным 16, нам же требуется только секция импорта. В таблице разделов секция импорта идет сразу после секции экспорта.

```

format binary as 'exe'
include 'win64a.inc'

struc dbs [data]
{
    common
    . db data
    .size = $ - .
}

IMAGE_DOS_SIGNATURE     equ 5A4Dh
IMAGE_NT_SIGNATURE      equ 00004550h
PROCESSOR_AMD_X8664     equ 8664h
IMAGE_SCN_CNT_CODE      equ 00000020h
IMAGE_SCN_MEM_WRITE     equ 80000000h
IMAGE_SCN_MEM_READ      equ 40000000h
IMAGE_SCN_CNT_INITIALIZED_DATA equ 00000040h
IMAGE_SUBSYSTEM_WINDOWS_GUI equ 2
IMAGE_NT_OPTIONAL_HDR64_MAGIC equ 20Bh
IMAGE_FILE_RELOCS_STRIPPED equ 1
IMAGE_FILE_EXECUTABLE_IMAGE equ 2
IMAGE_BASE              equ 0x400000
align1                  equ 0x10
IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE equ 8000h
use64
org 0

;-----DOS-stub-----

Signature      dw IMAGE_DOS_SIGNATURE
times
0x3A
db 0
NewExe_offset  dd ntHeader

;times 0x48 db 0

;-----PE-заголовок-----

ntHeader      dd IMAGE_NT_SIGNATURE
; 'PE'
;image_header----Файловый заголовок

Machine      dw PROCESSOR_AMD_X8664
;Тип центрального процессора

Count_of_section dw 2
;Количество секций

TimeStamp    dd 0
;Информация о времени, когда был собран данный PE-файл

Symbol_table_offset dd 0
;Указатель на размер отладочной информации

Symbol_table_count dd 0
;Указатель на COFF-таблицу символов PE-формата

```

```

Size_of_optional_header dw section_table-optional_header
;Размер опционального заголовка

Characteristics          dw IMAGE_FILE_RELOCS_STRIPPED or \
IMAGE_FILE_EXECUTABLE_IMAGE
;Атрибуты файла
;-----Стандартные поля NT

optional_header:
Magic_optional_header   dw IMAGE_NT_OPTIONAL_HDR64_MAGIC
;Состояние отображаемого файла

Linker_version_major_and_minor dw 9
;Содержит версию линковщика, создавшего данный файл

Size_of_code            dd Import_Table-begin
;Суммарный размер секций кода

Size_of_init_data       dd 0x70
;Суммарный размер инициализированных данных

Size_of_uninit_data     dd 0
;Суммарный размер неинициализированных данных

entry_point            dd start
base_of_code           dd begin

;-----Дополнительные поля NT-----

image_base             dq IMAGE_BASE
section_alignment      dd align1
file_alignment         dd align1
OS_version_major_minor dw 5,2
image_version_major_minor dd 0
subsystem_version_major_minor dw 5,2
Win32_version          dd 0
size_of_image          dd end_import
size_of_header         dd begin
checksum               dd 0
subsystem              dw IMAGE_SUBSYSTEM_WINDOWS_GUI
DLL_flag               dw IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE
Stack_allocation       dq 0x100000
Stack_commit           dq 0x1000
Heap_allocation        dq 0x100000
Heap_commit            dq 0x1000
loader_flag            dd 0
number_of_dirs         dd
(section_table-export_RVA_size)/8

export_RVA_size        dq 0
import_RVA_size        dd _import,user32_table2-_import
;_import,0x3C;end_import-import

;resource              dd 0,0
;exception             dd 0,0
;security              dd 0,0
;fixups_               dd 0,0
;debug                dd 0,0
;description           dd 0,0
;MIPS_GP               dd 0,0
;TLS                  dd 0,0
;Load_config           dd 0,0
;Bound_import          dd 0,0
;import_table1         dd Import_Table,_import-Import_Table
;delay_import          dd 0,0
;com_runtime           dd 0,0
;reserved              dd 0,0

;-----

section_table          dq '.text'

```



```

.virtual_size      dd a0-begin
;0x57

.virtual_address   dd begin
.Physical_size     dd Import_Table-begin
.Physical_offset   dd begin
.Relocations_and_Linenumbers dq 0
.Relocations_and_Linenumbers_count dd 0
.Attributes        dd IMAGE_SCN_MEM_WRITE or IMAGE_SCN_CNT_CODE
;0x80000020
;-----

section_table1     dq '.rdata'
.virtual_size      dd a1-Import_Table
;0x62

.virtual_address   dd Import_Table
.Physical_size     dd end_import-Import_Table
.Physical_offset   dd Import_Table
.Relocations_and_Linenumbers dq 0
.Relocations_and_Linenumbers_count dd 0
.Attributes        dd IMAGE_SCN_MEM_READ or IMAGE_SCN_CNT_INITIALIZED_DATA
;0x40000040
;-----данные и код-----

begin:
MsgBoxText  db 'Win64 Assembly is Great!',0
MsgCaption  db "Win64 Iczelion's lesson #2: MessageBox",0
start:
    push rbp
    xor ecx,ecx
    mov edx,MsgBoxText+IMAGE_BASE
    lea r8d,[rdx+MsgBoxText.size]
    xor r9d,r9d
    call [MessageBox]
    pop rbp
    retn
a0:
    times (($ - begin) mod align1) db 0

;выравнивание сегмента кода и данных до значения равного align1
;-----секция импорта-----

Import_Table:
user32_table:
MessageBox  dq _MessageBox,0
_import:
dd user32_table2,0,0,user32_dll,user32_table
dd 0,0,0,0,0
user32_table2:
dq _MessageBox,0
_MessageBox  db 0xE2,1,"MessageBoxA",0
user32_dll  db "user32.dll",0
a1:
times (($ - Import_Table) mod align1) db 0
end_import:

```

Заголовок	Код и данные	Импорт	Общий размер
296	96	112	504 байт

Что можно сократить в секции импорта? Секция импорта содержит две одинаковые таблицы (Import LookUp Table и Import Address Table), содержащие ссылки на названия импортируемых функций. Можно ли обойтись только таблицей Import Address Table? Удаляем Import LookUp Table и обновляем ссылки на user32_table2

```

format binary as 'exe'
include 'win64a.inc'

struc db [data]

```

```

{
    common
    . db data
    .size = $ - .
}

IMAGE_DOS_SIGNATURE     equ 5A4Dh
IMAGE_NT_SIGNATURE       equ 00004550h
PROCESSOR_AMD_X8664      equ 8664h
IMAGE_SCN_CNT_CODE       equ 00000020h
IMAGE_SCN_MEM_WRITE      equ 80000000h
IMAGE_SCN_MEM_READ       equ 40000000h
IMAGE_SCN_CNT_INITIALIZED_DATA equ 00000040h
IMAGE_SUBSYSTEM_WINDOWS_GUI equ 2
IMAGE_NT_OPTIONAL_HDR64_MAGIC equ 20Bh
IMAGE_FILE_RELOCS_STRIPPED equ 1
IMAGE_FILE_EXECUTABLE_IMAGE equ 2
IMAGE_BASE               equ 4000000h
align1                   equ 10h
IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE equ 8000h
use64
org 0

;-----DOS-stub-----

Signature      dw IMAGE_DOS_SIGNATURE
times 0x3A db 0
NewExe_offset  dd ntHeader

;-----PE-заголовок-----

ntHeader      dd IMAGE_NT_SIGNATURE
; 'PE'
;image_header----Файловый заголовок

Machine       dw PROCESSOR_AMD_X8664
;Тип центрального процессора

Count_of_section dw 2
;Количество секций

TimeStump     dd 0
;Информация о времени, когда был собран данный PE-файл

Symbol_table_offset dd 0
;Указатель на размер отладочной информации

Symbol_table_count dd 0
;Указатель на COFF-таблицу символов PE-формата

Size_of_optional_header dw section_table-optional_header
;Размер опционального заголовка

Characteristics dw IMAGE_FILE_RELOCS_STRIPPED or \
IMAGE_FILE_EXECUTABLE_IMAGE
;Атрибуты файла
;-----Стандартные поля NT

optional_header:
Magic_optional_header dw IMAGE_NT_OPTIONAL_HDR64_MAGIC
;Состояние отображаемого файла

Linker_version_major_and_minor dw 9
;Содержат версию линковщика, создавшего данный файл.

Size_of_code      dd Import_Table-begin
;Суммарный размер секций кода

Size_of_init_data dd 0x70
;Суммарный размер инициализированных данных
Size_of_uninit_data dd 0

```

;Суммарный размер неинициализированных данных

```
entry_point      dd start
base_of_code     dd begin
```

;-----Дополнительные поля NT-----

```
image_base       dq IMAGE_BASE
section_alignment dd align1
file_alignment   dd align1
OS_version_major_minor dw 5,2
image_version_major_minor dd 0
subsystem_version_major_minor dw 5,2
Win32_version    dd 0
size_of_image    dd end_import
size_of_header   dd begin
checksum         dd 0
subsystem       dw IMAGE_SUBSYSTEM_WINDOWS_GUI
DLL_flag        dw IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE
Stack_allocation dq 0x100000
Stack_commit     dq 0x1000
Heap_allocation  dq 0x100000
Heap_commit     dq 0x1000
loader_flag     dd 0
number_of_dirs   dd (section_table-export_RVA_size)/8
export_RVA_size  dq 0
import_RVA_size  dd _import,0x3C
;_import,0x3C;end_import-import
;-----
```

```
section_table     dq '.text'
.virtual_size     dd a0-begin
;0x57

.virtual_address  dd begin
.Physical_size    dd Import_Table-begin
.Physical_offset  dd begin
.Relocations_and_Linenumbers dq 0
.Relocations_and_Linenumbers_count dd 0
.Attributes       dd IMAGE_SCN_MEM_WRITE or IMAGE_SCN_CNT_CODE
;0x80000020
;-----
```

```
section_table1    dq '.rdata'
.virtual_size     dd end_import-Import_Table
;0x62

.virtual_address  dd Import_Table
.Physical_size    dd end_import-Import_Table
.Physical_offset  dd Import_Table
.Relocations_and_Linenumbers dq 0
.Relocations_and_Linenumbers_count dd 0
.Attributes       dd IMAGE_SCN_MEM_READ or IMAGE_SCN_CNT_INITIALIZED_DATA
;0x40000040
;-----данные и код-----
```

```
begin:
MsgBoxText db 'Win64 Assembly is Great!',0
MsgCaption db "Win64 Iczelion's lesson #2: MessageBox",0
start:
```

```
    push rbp
    xor ecx,ecx
    mov edx,MsgBoxText+IMAGE_BASE
    lea r8d,[rdx+MsgBoxText.size]
    xor r9d,r9d
    call [MessageBox]
    pop rbp
    retn
```

```
a0:
    times (($ - begin)mod align1) db 0
;выравнивание сегмента кода и данных до значения равного align1
```

```

;-----секция импорта-----

Import_Table:
user32_table:
MessageBox dq _MessageBox,0
_import:
dd 0,0,0,user32_dll,user32_table
dq 0,0,0
_MessageBox db 0xE2,1,"MessageBoxA",0
user32_dll db "user32"
end_import:

```

и как в четвертом шаге удаляем 16 нулевых байтов в конце файла и окончание ".dll".

Заголовок	Код и данные	Импорт	Общий размер
296	96	80	472 байт

сокращаем DOS-stub на 16 байтов (3Ah-2Eh-4=10h), смещение `e_lfanew` оказывается внутри PE-заголовка. Так как у программы со стабом короче 64 байт смещение от начала файла 3Ch (поле `e_lfanew`) попадает уже внутрь PE-заголовка, то нужно, чтобы он не попал на поле PE-заголовка имеющее критическое значение при загрузке файла. Помещаем указатель на ntHeader в поле `Symbol_table_offset`. При размещении в нем числа 30h получаем работоспособное приложение.

```

format binary as 'exe'
include 'win64a.inc'

struc db [data]
{
    common
    . db data
    .size = $ - .
}

IMAGE_DOS_SIGNATURE equ 5A4Dh
IMAGE_NT_SIGNATURE  equ 00004550h
PROCESSOR_AMD_X8664 equ 8664h
IMAGE_SCN_CNT_CODE   equ 00000020h
IMAGE_SCN_MEM_WRITE   equ 80000000h
IMAGE_SCN_MEM_READ    equ 40000000h
IMAGE_SCN_CNT_INITIALIZED_DATA equ 00000040h
IMAGE_SUBSYSTEM_WINDOWS_GUI equ 2
IMAGE_NT_OPTIONAL_HDR64_MAGIC equ 20Bh
IMAGE_FILE_RELOCS_STRIPPED equ 1
IMAGE_FILE_EXECUTABLE_IMAGE equ 2
IMAGE_BASE equ 400000h
align1 equ 10h
IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE equ 8000h
use64
org 0

;-----DOS-stub-----

Signature dw IMAGE_DOS_SIGNATURE

times 0x2E db 0

;NewExe_offset dd ntHeader

;-----PE-заголовок-----

ntHeader dd IMAGE_NT_SIGNATURE
;'PE'

;image_header----Файловый заголовок

Machine dw PROCESSOR_AMD_X8664
;Тип центрального процессора
Count_of_section dw 2

```

```

;Количество секций

TimeStump      dd 0
;Информация о времени, когда был собран данный PE-файл

Symbol_table_offset dd ntHeader
;Указатель на размер отладочной информации

Symbol_table_count dd 0
;Указатель на COFF-таблицу символов PE-формата

Size_of_optional_header dw section_table-optional_header
;Размер опционального заголовка

Characteristics    dw IMAGE_FILE_RELOCS_STRIPPED or \
IMAGE_FILE_EXECUTABLE_IMAGE
;Атрибуты файла
;-----Стандартные поля NT

optional_header:
Magic_optional_header dw IMAGE_NT_OPTIONAL_HDR64_MAGIC
;Состояние отображаемого файла

Linker_version_major_and_minor dw 9
;Содержат версию линковщика, создавшего данный файл

Size_of_code      dd Import_Table-begin
;Суммарный размер секций кода

Size_of_init_data  dd 0x70
;Суммарный размер инициализированных данных

Size_of_uninit_data dd 0
;Суммарный размер неинициализированных данных

entry_point       dd start
base_of_code       dd begin

;-----Дополнительные поля NT-----

image_base        dq IMAGE_BASE
section_alignment  dd align1
file_alignment     dd align1
OS_version_major_minor dw 5,2
image_version_major_minor dd 0
subsystem_version_major_minor dw 5,2
Win32_version      dd 0
size_of_image      dd end_import
size_of_header     dd begin
checksum           dd 0
subsystem          dw IMAGE_SUBSYSTEM_WINDOWS_GUI
DLL_flag           dw IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE
Stack_allocation   dq 0x100000
Stack_commit       dq 0x1000
Heap_allocation     dq 0x100000
Heap_commit        dq 0x1000
loader_flag        dd 0
number_of_dirs     dd (section_table-export_RVA_size)/8
export_RVA_size    dq 0
import_RVA_size     dd _import,0x3C

;-----

section_table      dq '.text'
.virtual_size      dd a0-begin
.virtual_address    dd begin
.Physical_size     dd Import_Table-begin
.Physical_offset    dd begin
.Relocations_and_Linenumbers dq 0
.Relocations_and_Linenumbers_count dd 0
.Attributes        dd IMAGE_SCN_MEM_WRITE or IMAGE_SCN_CNT_CODE

```

```

;-----
section_table1      dq '.rdata'
.virtual_size       dd end_import-Import_Table
.virtual_address    dd Import_Table
.Physical_size      dd end_import-Import_Table
.Physical_offset    dd Import_Table
.Relocations_and_Linenumbers dq 0
.Relocations_and_Linenumbers_count dd 0
.Attributes         dd IMAGE_SCN_MEM_READ or IMAGE_SCN_CNT_INITIALIZED_DATA

;-----данные и код-----

begin:
MsgBoxText  db 'Win64 Assembly is Great!',0
MsgCaption  db "Win64 Iczelion's lesson #2: MessageBox",0
start:
    push rbp
    xor ecx,ecx
    mov edx,MsgBoxText+IMAGE_BASE
    lea r8d,[rdx+MsgBoxText.size]
    xor r9d,r9d
    call [MessageBox]
    pop rbp
    retn
a0:
    times (($ - begin) mod align1) db 0

;выравнивание сегмента кода и данных до значения равного align1
;-----секция импорта-----

Import_Table:
user32_table:
MessageBox  dq _MessageBox,0
_import:
dd 0,0,0,user32_dll,user32_table
dq 0,0,0
_MessageBox  db 0xE2,1,"MessageBoxA",0
user32_dll  db "user32"
end_import:

```

Заголовок	Код и данные	Импорт	Общий размер
280	96	80	456 байт

сокращаем DOS-stub еще на 16 байтов (2Eh-1Eh=10h), смещение *e_ifanew* оказывается в поле *Size_of_code*. По одним источникам — это поле используется для первичного отведения памяти под приложение. По другим — не используются вообще. Если поместить туда число 20h, то практическая проверка показывает, что приложение с таким stub'ом работает нормально.

```

format binary as 'exe'
include 'win64a.inc'

struc db [data]
{
    common
    . db data
    .size = $ - .
}

IMAGE_DOS_SIGNATURE      equ 5A4Dh
IMAGE_NT_SIGNATURE       equ 00004550h
PROCESSOR_AMD_X8664      equ 8664h
IMAGE_SCN_CNT_CODE       equ 00000020h
IMAGE_SCN_MEM_WRITE      equ 80000000h
IMAGE_SCN_MEM_READ       equ 40000000h
IMAGE_SCN_CNT_INITIALIZED_DATA equ 00000040h
IMAGE_SUBSYSTEM_WINDOWS_GUI equ 2
IMAGE_NT_OPTIONAL_HDR64_MAGIC equ 20Bh
IMAGE_FILE_RELOCS_STRIPPED equ 1

```

```

IMAGE_FILE_EXECUTABLE_IMAGE equ 2
IMAGE_BASE equ 0x400000
align1 equ 0x10
IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE equ 8000h
use64
org 0

;-----DOS-stub-----

Signature dw IMAGE_DOS_SIGNATURE

times 0x1E db 0

;-----PE-заголовок-----

ntHeader dd IMAGE_NT_SIGNATURE
; 'PE'
; image_header----Файловый заголовок

Machine dw PROCESSOR_AMD_X8664
; Тип центрального процессора

Count_of_section dw 2
; Количество секций

TimeStump dd 0
; Информация о времени, когда был собран данный PE-файл

Symbol_table_offset dd 0
; Указатель на размер отладочной информации

Symbol_table_count dd 0
; Указатель на COFF-таблицу символов PE-формата

Size_of_optional_header dw section_table-optional_header
; Размер опционального заголовка

Characteristics dw IMAGE_FILE_RELOCS_STRIPPED or \
IMAGE_FILE_EXECUTABLE_IMAGE
; Атрибуты файла
; -----Стандартные поля NT

optional_header:
Magic_optional_header dw IMAGE_NT_OPTIONAL_HDR64_MAGIC
; Состояние отображаемого файла

Linker_version_major_and_minor dw 9
; Содержат версию линковщика, создавшего данный файл

Size_of_code dd ntHeader
; Суммарный размер секций кода

Size_of_init_data dd 0x70
; Суммарный размер инициализированных данных

Size_of_uninit_data dd 0
; Суммарный размер неинициализированных данных

entry_point dd start
base_of_code dd begin

; -----Дополнительные поля NT-----

image_base dq IMAGE_BASE
section_alignment dd align1
file_alignment dd align1
OS_version_major_minor dw 5,2
image_version_major_minor dd 0
subsystem_version_major_minor dw 5,2
Win32_version dd 0
size_of_image dd end_import

```

```

size_of_header      dd begin
checksum            dd 0
subsystem           dw IMAGE_SUBSYSTEM_WINDOWS_GUI
DLL_flag            dw IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE
Stack_allocation    dq 0x100000
Stack_commit        dq 0x1000
Heap_allocation      dq 0x100000
Heap_commit         dq 0x1000
loader_flag         dd 0
number_of_dirs       dd (section_table-export_RVA_size)/8
export_RVA_size      dq 0
import_RVA_size      dd _import,0x3C

;-----

section_table        dq '.text'
.virtual_size        dd a0-begin
.virtual_address      dd begin
.Physical_size       dd Import_Table-begin
.Physical_offset     dd begin
.Relocations_and_Linenumbers dq 0
.Relocations_and_Linenumbers_count dd 0
.Attributes          dd IMAGE_SCN_MEM_WRITE or IMAGE_SCN_CNT_CODE

;-----

section_table1        dq '.rdata'
.virtual_size         dd end_import-Import_Table
.virtual_address      dd Import_Table
.Physical_size        dd end_import-Import_Table
.Physical_offset     dd Import_Table
.Relocations_and_Linenumbers dq 0
.Relocations_and_Linenumbers_count dd 0
.Attributes          dd IMAGE_SCN_MEM_READ or IMAGE_SCN_CNT_INITIALIZED_DATA

;-----данные и код-----

begin:
MsgBoxText  db 'Win64 Assembly is Great!',0
MsgCaption  db "Win64 Iczelion's lesson #2: MessageBox",0
start:
    push rbp
    xor ecx,ecx
    mov edx,MsgBoxText+IMAGE_BASE
    lea r8d,[rdx+MsgBoxText.size]
    xor r9d,r9d
    call [MessageBox]
    pop rbp
    retn
a0:
    times (($ - begin) mod align1) db 0

;выравнивание сегмента кода и данных до значения равного align1
;-----секция импорта-----

Import_Table:
user32_table:
MessageBox  dq _MessageBox,0
_import:
dd 0,0,0,user32_dll,user32_table
dq 0,0,0
_MessageBox  db 0xE2,1,"MessageBoxA",0
user32_dll  db "user32"
end_import:

```

Заголовок	Код и данные	Импорт	Общий размер
264	96	80	440 байт

сокращаем DOS-stub до 16 байт, смещение *e_ifanew* оказывается в поле *base_of_code*. При размещении в нем числа 10h получаем работоспособное приложение.

В начале секции импорта убрали байты для выравнивания.

```
format binary as 'exe'
include 'win64a.inc'

struc dbs [data]
{
    common
    . db data
    .size = $ - .
}

IMAGE_DOS_SIGNATURE      equ 5A4Dh
IMAGE_NT_SIGNATURE       equ 00004550h
PROCESSOR_AMD_X8664      equ 8664h
IMAGE_SCN_CNT_CODE       equ 00000020h
IMAGE_SCN_MEM_WRITE      equ 80000000h
IMAGE_SCN_MEM_READ       equ 40000000h
IMAGE_SCN_CNT_INITIALIZED_DATA equ 00000040h
IMAGE_SUBSYSTEM_WINDOWS_GUI equ 2
IMAGE_NT_OPTIONAL_HDR64_MAGIC equ 20Bh
IMAGE_FILE_RELOCS_STRIPPED equ 1
IMAGE_FILE_EXECUTABLE_IMAGE equ 2
IMAGE_BASE               equ 0x400000
align1                   equ 0x10
IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE equ 8000h
use64
org 0

;-----DOS-stub-----

Signature      dw IMAGE_DOS_SIGNATURE

times 0xE db 0

;-----PE-заголовок-----

ntHeader      dd IMAGE_NT_SIGNATURE
; 'PE'
;image_header----Файловый заголовок

Machine       dw PROCESSOR_AMD_X8664
;Тип центрального процессора

Count_of_section dw 2
;Количество секций

TimeStump     dd 0
;Информация о времени, когда был собран данный PE-файл

Symbol_table_offset dd 0
;Указатель на размер отладочной информации

Symbol_table_count dd 0
;Указатель на COFF-таблицу символов PE-формата

Size_of_optional_header dw section_table-optional_header
;Размер опционального заголовка

Characteristics dw IMAGE_FILE_RELOCS_STRIPPED or \
IMAGE_FILE_EXECUTABLE_IMAGE
;Атрибуты файла
;-----Стандартные поля NT

optional_header:
Magic_optional_header dw IMAGE_NT_OPTIONAL_HDR64_MAGIC
;Состояние отображаемого файла
Linker_version_major_and_minor dw 9
```

```

;Содержат версию линковщика, создавшего данный файл

Size_of_code      dd Import_Table-begin
;Суммарный размер секций кода

Size_of_init_data  dd 0x70
;Суммарный размер инициализированных данных

Size_of_uninit_data dd 0
;Суммарный размер неинициализированных данных

entry_point       dd start

base_of_code       dd ntHeader

;-----Дополнительные поля NT-----

image_base        dq IMAGE_BASE
section_alignment  dd align1
file_alignment     dd align1
OS_version_major_minor dw 5,2
image_version_major_minor dd 0
subsystem_version_major_minor dw 5,2
Win32_version      dd 0
size_of_image      dd end_import
size_of_header     dd begin
checksum          dd 0
subsystem          dw IMAGE_SUBSYSTEM_WINDOWS_GUI
DLL_flag           dw IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE
Stack_allocation   dq 0x100000
Stack_commit       dq 0x1000
Heap_allocation    dq 0x100000
Heap_commit        dq 0x1000
loader_flag        dd 0
number_of_dirs     dd (section_table-export_RVA_size)/8
export_RVA_size    dq 0
import_RVA_size    dd _import,0x3C

;-----

section_table      dq '.text'
.virtual_size      dd a0-begin
.virtual_address   dd begin
.Physical_size     dd Import_Table-begin
.Physical_offset   dd begin
.Relocations_and_Linenumbers dq 0
.Relocations_and_Linenumbers_count dd 0
.Attributes        dd IMAGE_SCN_MEM_WRITE or IMAGE_SCN_CNT_CODE

;-----

section_table1     dq '.rdata'
.virtual_size      dd end_import-Import_Table
.virtual_address   dd Import_Table
.Physical_size     dd end_import-Import_Table
.Physical_offset   dd Import_Table
.Relocations_and_Linenumbers dq 0
.Relocations_and_Linenumbers_count dd 0
.Attributes        dd IMAGE_SCN_MEM_READ or IMAGE_SCN_CNT_INITIALIZED_DATA

;-----данные и код-----

begin:
MsgBoxText db 'Win64 Assembly is Great!',0
MsgCaption db "Win64 Iczelion's lesson #2: MessageBox",0
start:
    push rbp
    xor ecx,ecx
    mov edx,MsgBoxText+IMAGE_BASE
    lea r8d,[rdx+MsgBoxText.size]
    xor r9d,r9d

```

```

    call [MessageBox]
    pop rbp
    retn

;-----секция импорта-----

Import_Table:
user32_table:
MessageBox dq _MessageBox,0
_import:
dd 0,0,0,user32_dll,user32_table
dq 0,0,0
_MessageBox db 0xE2,1,"MessageBoxA",0
user32_dll db "user32"
end_import:

```

Мы удалили таблицу, которую используют для импорта по ординалам. Внимание, вопрос — А зачем нам в таком случае ординалы перед названием функций? Обнуляем ординалы и задаем себе следующий вопрос — Если ординал нулевой — нужен ли нулевой байт на конце строки? Удаляем нуль-терминаторы, а заодно и нулевые байты, которые делали адреса названий функции кратными двум, так как импортируемые функции теперь у нас заканчиваются двумя нулевыми байтами (а это расточительно!) — помещаем в поле ординала последний символ названия функции или названия dll и нуль-терминатор

количество секций делаем равным 0, удаляем секцию '.rdata' и '.text', размещаем импорт в секции кода.

уменьшаем DOS-stub до 4 байт. смещение *e_lfanew* оказывается в поле *file alignment*. section alignment=file alignment=4 уменьшение импорта

```

format binary as 'exe'
include 'win64a.inc'

struc dbs [data]
{
    common
    . db data
    .size = $ - .
}

IMAGE_DOS_SIGNATURE equ 5A4Dh
IMAGE_NT_SIGNATURE equ 00004550h
PROCESSOR_AMD_X8664 equ 8664h
IMAGE_SCN_CNT_CODE equ 00000020h
IMAGE_SCN_MEM_WRITE equ 80000000h
IMAGE_SCN_MEM_READ equ 40000000h
IMAGE_SCN_CNT_INITIALIZED_DATA equ 00000040h
IMAGE_SUBSYSTEM_WINDOWS_GUI equ 2
IMAGE_NT_OPTIONAL_HDR64_MAGIC equ 20Bh
IMAGE_FILE_RELOCS_STRIPPED equ 1
IMAGE_FILE_EXECUTABLE_IMAGE equ 2
IMAGE_BASE equ 0x400000
IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE equ 8000h
use64
org 0

;-----DOS-stub-----

Signature dw IMAGE_DOS_SIGNATURE,0

;-----PE-заголовок-----

ntHeader dd IMAGE_NT_SIGNATURE
; 'PE'
;image_header----Файловый заголовок

Machine dw PROCESSOR_AMD_X8664
;Тип центрального процессора

```

```

Count_of_section    dw 0
;Количество секций

TimeStamp           dd 0
;Информация о времени, когда был собран данный PE-файл

Symbol_table_offset dd 0
;Указатель на размер отладочной информации

Symbol_table_count  dd 0
;Указатель на COFF-таблицу символов PE-формата

Size_of_optional_header dw begin-optional_header
;Размер опционального заголовка

Characteristics      dw IMAGE_FILE_RELOCS_STRIPPED or \
IMAGE_FILE_EXECUTABLE_IMAGE
;Атрибуты файла
;-----Стандартные поля NT

optional_header:
Magic_optional_header dw IMAGE_NT_OPTIONAL_HDR64_MAGIC
;Состояние отображаемого файла

Linker_version_major_and_minor dw 9
;Содержат версию линковщика, создавшего данный файл

Size_of_code        dd Import_Table-begin
;Суммарный размер секций кода

Size_of_init_data   dd 0x70
;Суммарный размер инициализированных данных

Size_of_uninit_data dd 0
;Суммарный размер неинициализированных данных

entry_point         dd start
base_of_code        dd begin

;-----Дополнительные поля NT-----

image_base          dq IMAGE_BASE

section_alignment   dd 4
file_alignment      dd ntHeader

OS_version_major_minor dw 5,2
image_version_major_minor dd 0
subsystem_version_major_minor dw 5,2
Win32_version       dd 0
size_of_image       dd end_import
size_of_header      dd begin
checksum            dd 0
subsystem           dw IMAGE_SUBSYSTEM_WINDOWS_GUI
DLL_flag            dw IMAGE_DLLCHARACTERISTICS_TERMINAL_SERVER_AWARE
Stack_allocation    dq 0x100000
Stack_commit        dq 0x1000
Heap_allocation     dq 0x100000
Heap_commit         dq 0x1000
loader_flag         dd 0
number_of_dirs      dd (begin-export_RVA_size)/8
export_RVA_size     dq 0
import_RVA_size     dd _import,end_import-import

;-----
;section_table      dq '.text'
;.virtual_size      dd a0-begin
;.virtual_address   dd begin
;.Physical_size     dd Import_Table-begin
;.Physical_offset   dd begin
;.Relocations_and_Linenumbers dq 0

```

```

; .Relocations_and_Linenumbers_count dd 0
; .Attributes dd IMAGE_SCN_MEM_WRITE or IMAGE_SCN_CNT_CODE
;-----
; section_table1 dq '.rdata'
; .virtual_size dd end_import-Import_Table
; .virtual_address dd Import_Table
; .Physical_size dd end_import-Import_Table
; .Physical_offset dd Import_Table
; .Relocations_and_Linenumbers dq 0
; .Relocations_and_Linenumbers_count dd 0
; .Attributes dd IMAGE_SCN_MEM_READ or IMAGE_SCN_CNT_INITIALIZED_DATA

;-----данные, код и импорт-----

begin:
MsgBoxText db 'Win64 Assembly is Great!',0
MsgCaption db "Win64 Iczelion's lesson #2: MessageBox",0
start:
    push rbp
    xor ecx,ecx
    mov edx,MsgBoxText+IMAGE_BASE
    lea r8d,[rdx+MsgBoxText.size]
    xor r9d,r9d
    call [MessageBox]
    pop rbp
    ret

;-----секция импорта-----

Import_Table:
user32_table:
MessageBox dq _MessageBox
_import:
dd 0,0,0,user32_dll,user32_table,0
user32_dll db "user32"
dd 0
_MessageBox db 0,0,"MessageBoxA"
end_import:

```

Заголовок	Код и данные	Импорт	Общий размер
156	87	55	298 байт

Мораль

Так путем нехитрых преобразований можно на порядок уменьшить размер exe-файла с 2560 байт до 298 байт



© Miki_ 2016

Уроки Iczelion'a для Win x64. Урок первый

Автор: Miki___

Дата: 2016-07-28

Где взять 64-разрядные компилятор или линкер?

64-разрядные ассемблерный компилятор и линкер фирма MicroSoft не распространяет отдельно, но **ml64.exe** и **link.exe** можно получить бесплатно (пожалуйста, прочитайте и соблюдайте "Лицензионное соглашение"), если скачать комплект C++ компиляторов от Microsoft. Вам потребуется:

- либо [Windows Driver Kits Version 7.1.0 \(WDK\)](#);
- либо [MICROSOFT WINDOWS SOFTWARE DEVELOPMENT KIT FOR WINDOWS 7 and .NET FRAMEWORK 4 \(SDK\)](#).

После установки **WDK** **ml64.exe** и **link.exe** содержатся в папках **C:\WinDDK\7600.16385.1\bin\x86\amd**.

После установки **SDK** **ml64.exe** и **link.exe** содержатся в папках **C:\Program Files\Microsoft Visual Studio 10.0\VC\bin\x86_amd64**.

Обычно используют 64-битный **link** из папки **amd64**, а **ml64** уже 32-битный из папки **x86_amd64**. В папке **amd64** есть 64-битная версия **ml64**, но при работе с большим количеством макросов он почему-то зависает или просто медленно работает. В конце концов выбор за Вами.

На сайте [dsmhelp.narod.ru](#) можно найти набор **lib**- и **inc**- файлов для создания 64-разрядных приложений на ассемблере, а также кучу полезных макросов.

Создание первой программы

Создаем каталог **masm64**, а внутри него подкаталоги **bin**, **include**, **lib**, **examples**. Подкаталоги **include** и **lib** заполняем содержимым взятым с сайта [dsmhelp.narod.ru](#).

В каталог **bin** помещаем:

- **cvtres.exe**
- **link.exe**
- **ml64.exe**
- **msobj80.dll**
- **mispdb80.dll**
- **msvc80.dll**
- **msvc90.dll**
- **msvcr80.dll**
- **msvcr90.dll**
- **rc.exe**
- **rcdll.dll**

При помощи **hiew32** создаем файл **stubby.exe**:

```
.000000:  4D 5A 00 00 00 00 00 00  00 00 00 00 00 00 00 00  MZ
.000010:  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00
```

```
.000020:  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00
.000030:  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00
```

И помещаем **stubby.exe** в каталог **bin**. Также в папке **bin** создадим файл **asm.bat**:

```
cls <--- очистка экрана
set filename=%~n1 <--- имя asm-файла без расширения
set %masm64_path%=d:\MASM64\ <--- здесь путь к папке masm64, у Вас он может быть другим
if exist %filename%.exe del %filename%.exe <--- если существует exe-файл с таким же именем -
удаляем его
%masm64_path%\bin\ml64 /Cp /c /I"%masm64_path%Include" %filename%.asm <--- компиляция
%masm64_path%\bin\link /SUBSYSTEM:WINDOWS /LIBPATH:"%masm64_path%Lib" ^ <--- линковка
/LARGEADDRESSAWARE:NO /BASE:0x400000 /STUB:%masm64_path%\bin\stubby.exe ^
/SECTION:.text,w /ALIGN:16 /entry:WinMain %filename%.obj
del %filename%.obj <--- удаляем "программный мусор"
```

Перед сохранением **asm.bat** удалите комментарии помеченные красным цветом.

Создаем ассоциации асемблерных файлов с файлом asm-bat

Щелкаем по ярлычку "Мой компьютер" -> "Сервис" -> "Свойства папки" -> "Типы файлов" -> "Создать" -> "ОК" -> "Обзор" -> "d:\MASM64\bin\asm.bat" -> "Открыть".

Или более дзенный вариант:

Жмем на кнопку "Пуск" -> "Выполнить", запускаем **regedit**:

- в ветке **HKEY_CLASSES_ROOT** -> **Application** создаем раздел **asm.bat** -> **shell** -> **open** -> **command**;
- в ветке **HKEY_CURRENT_USER** -> **Software** -> **Microsoft** -> **Windows** -> **CurrentVersion** -> **Explorer** -> **FileExts** создаем раздел **.asm**.

«скелет» ассемблерного файла для x64

```
.data
; здесь инициализированные данные для программы
.data?
; здесь неинициализированные данные для программы
.code
; здесь текст программы
end
```

В отличие от ассемблерного файла для 16-/32-разрядного программирования:

1. отсутствует упоминание модели и типа CPU;
2. "вход" в программу определяется не по последней строчке в asm-файле **"end <метка>"**, а по опции в bat-файле **"/entry:<метка>"**.

Первая программа

Создадим в папке **Include** файл **win64a.inc** и поместим в него следующие строки:

```
OPTION DOTNAME
include temphls.inc
include win64.inc
include kernel32.inc
include user32.inc
includelib kernel32.lib
includelib user32.lib
```

```
OPTION PROLOGUE:rbpFramePrologue
OPTION EPILOGUE:none
IMAGE_BASE equ 400000h
```

Набираем в блокноте текст нашей программы и сохраняем ее с именем **tut_02.asm** в каталоге **samples**:

```
include win64a.inc
.code
WinMain proc
    push rbp
    mov edx,offset MsgCaption
    lea r8,[rdx+sizeof MsgCaption]
    invoke MessageBox,NULL,,,MB_OK
    pop rbp
    retn
WinMain endp
;--данные-----
MsgCaption      db "Iczelion's tutorial #2a",0
MsgBoxText      db "Win64 Assembly is Great!",0
end
```

Кликаем по файлу **tut_02.asm** и получаем файл **tut_02.exe**. Запускаем файл **tut_02.exe** на исполнение и любуемся на творение своих «шаловливых ручек».

© Miki___ 2016